

// ZWIRE //

zwire

v0.6.56

THE COMPANION

the Blink base · the rebrand · the HUD · the fork

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTAPP

Contents

Before You Begin	4
Chapter 1 — The Pitch	6
Chapter 2 — Why a Real Blink Base	8
Chapter 3 — The Architecture	10
Chapter 4 — The Base	13
Chapter 5 — The Runtime Rebrand	16
Chapter 6 — The New-Tab HUD	18
Chapter 7 — The Internal-HUD Extension: The Injection Model	21
Chapter 8 — The Eight Schemes and the Sync Bus	25
Chapter 9 — Vim on Every Page	31
Chapter 10 — tmux in the Browser	34
Chapter 11 — The Powerline Statusbar	39
Chapter 12 — The Command Palette	42
Chapter 13 — Fuzzy Find	45
Chapter 14 — The HUD Internal Pages	48
Chapter 15 — The Native Host	52
Chapter 16 — The Terminal	55
Chapter 17 — zgui-core, the Shared Toolkit	58
Chapter 18 — zpwrchrome, the Power-Tool	60
Chapter 19 — The Launcher and State Layout	63
Chapter 20 — The CDP Verification Layer	66
Chapter 21 — The Source Fork	70
Chapter 22 — Dev Loop, Testing, and CI	77
Chapter 23 — Packaging and Self-Contained Installs	80
Chapter 24 — Cross-Platform, Updating, and Operations	83

Chapter 25 — The Trust Boundaries and the Threat Model	86
Chapter 26 — The Performance Budget	88
Chapter 27 — Extending the HUD	91
Chapter 28 — The Keyboard Arbitration Model	93
Chapter 29 — The Six Boundaries and Their Transports	96
Chapter 30 — Living Inside MV3	99
Chapter 31 — The Storage Schema and Its Ownership Rules	102
Chapter 32 — When a Hop Fails	105
Chapter 33 — The Cost of Reuse	108
Chapter 34 — The Lineage: The Cyberpunk Idiom and the zpwr Fleet	111
Chapter 35 — Positioning: zwire Among the Custom Browsers	113
Chapter 36 — Failure Modes and Recovery	115
Chapter 37 — Text Input, Composition, and the Keyboard That Is Not Yours	118
Chapter 38 — Living on the Open Web	122
Chapter 39 — Accessibility and the Neon Surface	126
Chapter 40 — Two Browsers, One Host, One Theme File	130
Chapter 41 — Debugging a System With Six Boundaries	133
Chapter 42 — Incognito, and the Extension That Is Not There	137
Chapter 43 — The Dead Zones	140
Chapter 44 — Zoom, Density, and the Geometry That Is Not Yours	142
Chapter 45 — The Long Session	145
Chapter 46 — What Leaves the Machine	147
Chapter 47 — Teardown, Portability, and the Second Machine	150
Chapter 48 — Version Numbers as a Contract	153
Chapter 49 — Lifecycle Hooks: stryke on Browser Events	156
Chapter 50 — Output Triggers: Regex on What the Page Renders	159
Chapter 51 — Pane Pipelines: A Dataflow Edge Between Two Panes	161
Chapter 52 — The Audio Dashboard	163
Chapter 53 — The Later HUD Pages	165
Appendix A — File and Directory Map	167

Appendix B — The State Directory (\$ZWIRE_STATE)	169
Appendix C — Keybinding Reference	170
Appendix D — The Eight-Scheme Palette	171
Appendix E — Launch Flag Reference	173
Appendix F — The Chrome WebUI Token Map	174
Appendix G — The Native-Messaging Protocol	177
Appendix H — The Seeded Keyword Commands	178
Appendix I — The Storage Keys	180
Appendix J — The Rust Host Modules	181
Appendix K — The Launcher and Build Scripts	182
Appendix L — The CDP Verification Probes	183
Appendix M — The zpwchrome Manifest	184
Appendix N — The Fork Patch Targets	185
Appendix O — The Three Extension Manifests	186
Appendix P — The Content-Script Injection Order	187
Appendix Q — The Fuzzy Matcher	188
Appendix R — Visual Effects and Readline	189
Appendix S — The System-Stats Sample	190
Appendix T — The Palette Destinations	191
Appendix U — The Command Bus	192
Appendix V — The Frecency Model	193
Appendix W — The Extension Messaging Graph	194
Appendix X — The zgui-core Toolkit Surface	195
Appendix Y — The HUD Internal Pages	196
Appendix Z — The Dev-Loop Scripts	197
Appendix AA — Three Paths, End to End	198
Appendix AB — Glossary	199
Appendix AC — The Statusbar Segments	200
Appendix AD — Grounding and Edition	201

Before You Begin

This is the book about **zwire** — the one you read in a chair, front to back, to understand not only *what* **zwire** does but *why* it is shaped the way it is. It is not the README, which enumerates every switch, path, and patch and which you reach for when you need one fact fast. This book is the other thing: the continuous account of the browser — from the unbranded Blink snapshot it starts from to the neon toolbar the source fork compiles — and of the deliberate line it draws between what a runtime rebrand can do and what only a patched Chromium can.

zwire is a **Chromium/Blink browser forked into a tiling cyberpunk HUD**. It is not a WebView wrapper and not a new engine: it is a real Blink base, patched at the seams a compiled fork can reach and extended with a keyboard-driven workspace. The repository states the intent on its own front page — [\[FORKED CHROMIUM · TILING HUD · CYBERPUNK\]](#), with the one-line creed the rest of this book unpacks: “*Not Chrome with a skin — a real Blink engine turned into a keyboard-driven, tiling, cyberpunk workspace.*” It carries a tmux-style tiling overlay, a `⌘K` command palette, durable sessions, a browser-wide audio engine, and the **zpwchrome** power-tool preloaded against a dedicated profile so it never touches your system Chrome.

The reason **zwire** needs a *real* Blink engine, rather than a WebKit shell like Tauri or Servo, is concrete and it drives every decision downstream: the **zpwchrome** extension is Manifest V3 and depends on `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, and a service-worker background, with `minimum_chrome_version: 127`. None of that runs anywhere but a genuine Chromium engine. The base is therefore a plain, **unbranded Chromium snapshot** — no Google logo, no “for automated testing” stripe — which also happens to retain the `--load-extension` switch that branded Chrome removed in version 137. That single retained switch is what lets **zwire** preload its extensions at all; the book returns to it in Chapter 2.

zwire ships two build paths, and the whole shape of the project is the boundary between them. The **runtime rebrand** (the default, no compile) applies HUD colors, the HUD new-tab, the internal-page skin, and your extensions on top of a prebuilt Chromium base — tab shapes stay stock, because a Chrome theme extension can change colors and nothing else. The **source fork** (`fork/`, which compiles Chromium) removes that limit: it restyles the native chrome itself — tab shapes, UI fonts, the neon toolbar, native menus and dialogs, and a browser-wide audio engine — through a series of twenty-five authored patches against a pinned Chromium tag. This book is honest about which capabilities live on which side of that line, because the line *is* the design.

The whole thing is two commands to stand up and one to run:

```
git clone --recurse-submodules https://github.com/MenkeTechnologies/zwire.git
cd zwire
scripts/install.sh          # fetch base + link `zwire` on PATH + rebrand (macOS)
zwire                       # launch with the HUD new tab
```

This is fast-moving software: the edition string on the cover tracks `zwire/package.json (v0.6.56` at the time of writing), and where a feature needs the compiled fork, or a native host must be installed separately, this book marks it rather than rounding up. **zwire** is a MenkeTechnologies project, built on the Chromium project’s engine, and this book is grounded in that code.

How to read this book

The book runs bottom-up. The first chapters are the pitch and the justification — what **zwire** is and why it must be Chromium. Then it descends the stack: the base, the rebrand, the new tab, and the large internal-HUD

extension, which gets a chapter per feature it layers on every page (the eight schemes, vim, the tmux overlay, the powerline statusbar, the command palette, the fuzzy find, the HUD internal pages, the native host, and the terminal). After the internal HUD come the power-tool submodule, the launcher, and the CDP verification probes, then the source fork, the dev-and-CI loop, the packaging, and the operations runbook. The last five chapters cover the automation and content layer the HUD grew on top of that base — stryke lifecycle hooks, output triggers, pane pipelines, the audio dashboard over the browser-wide DSP engine, and the later HUD pages. The reference appendices at the back cover the file map, the state directory, the full keybinding tables, the eight-scheme palette (dark and light), the launch flags, the Chrome WebUI token map, the native-messaging protocol, the seeded keyword commands, the storage keys, the Rust host modules, the launcher and build scripts, the CDP probes, the power-tool manifest, and the fork patch targets.

Every claim in the chapters is grounded in a file in the [zwire](#) repository, and where the book says “the code comments say,” it is quoting the source. Let’s start.

Chapter 1 – The Pitch

A browser is the least likely thing on the stack to be rewritten, and zwire does not rewrite it. This chapter states what zwire *is* — not “a browser from scratch,” but Chromium **owned** at the seams a rebrand and a patch series can reach: a real Blink engine wearing the cyberpunk HUD, carrying the author’s extensions, on a profile that never collides with system Chrome.

What zwire is – a rebrand and an extension host, not a new engine

The lazy framing is “a custom browser,” which usually means either a WebView wrapper (WebKit, no MV3) or a fresh engine (a decade of work nobody finishes). zwire is neither. It takes a real Chromium and changes the two things that matter for a daily driver — how it looks and what it runs — while inheriting the engine, the security model, and the update cadence for free.

The mechanics are unglamorous on purpose. `bin/zwire` is a shell script. It reads one path (`~/.zwire/base.path`, written when the base was fetched), and it `execs` that binary against a dedicated profile with a fixed set of extensions loaded. There is no daemon, no supervisor, no packaged Electron runtime, no bundled Node. The “browser” is a Chromium snapshot Google itself builds and hosts; zwire’s contribution is the profile, the extension set, the icon, the name, and — in the fork — twenty-five C++ patches. Everything else is Blink, unmodified, doing what Blink does. That is the entire trick, and it is why zwire exists in kilobytes of shell and JavaScript while a from-scratch browser would be millions of lines of C++ nobody finishes.

Two consequences follow immediately, and they run through the whole book. First, zwire tracks upstream Chromium *for free*: a security fix that lands in the Blink tree is one `scripts/fetch-base.sh` away, because the base is a stock snapshot, not a frozen fork of one. (The fork path pays for its native restyle by giving that up — Chapter 21 is honest about the rebase cost.) Second, every capability zwire adds on the runtime path is an *extension* or a *launch flag* — nothing patched, nothing recompiled — so the runtime rebrand degrades gracefully: delete an extension directory and the launcher simply skips it, and you still have a working browser.

What the rebrand buys, and what only the fork buys

A Chrome theme extension can repaint the frame, the toolbar background, and the tab strip — and nothing else. Tab *shape*, the UI *font*, the toolbar *border*, the omnibox field *geometry*: those are native C++ in Chromium’s Views layer, not exposed to any extension API. This is not a zwire limitation; it is the boundary Chromium draws around what a theme is allowed to touch. The runtime rebrand accepts that boundary and does everything it can reach on the right side of it — which turns out to be a great deal, because zwire pushes the color and content surface much further than a theme extension normally does:

- **Colors** it changes through the theme layer and, more aggressively, through the internal-HUD content script that skins `chrome://` pages directly.
- **Content** it owns completely: the new tab is a full HUD page (`chrome_url_overrides`), and several internal pages (`chrome://extensions`, `chrome://history`, `chrome://bookmarks`, `chrome://settings`, `chrome://version`) are redirected to HUD-styled replacements served from the extension.

- **Behavior** it layers on with content scripts: vim keybindings on every page, a tmux-style tiling overlay, a powerline statusbar, a `⌘K` command palette, a fuzzy find bar that replaces Chrome's own, and an xterm terminal over a real PTY.

What stays out of reach on the runtime path is precisely the native Views chrome: the pill-shaped tabs, the system UI font, the rounded omnibox. The `fork/` path compiles a patched Chromium to reach those — trapezoid HUD tabs, a Share Tech Mono UI font, a neon under-toolbar line, HUD-colored frame, and the eight HUD schemes wired into the DevTools Theme dropdown. `zwire` ships *both* paths rather than forcing the ~100 GB Chromium checkout on everyone: the rebrand is the daily driver, and the fork is there for whoever wants the whole browser in the HUD and will pay the build-and-rebase cost. The boundary between them is not a wart to be apologized for — it is the map of what a browser lets you change without becoming its maintainer.

Why it never touches system Chrome

`zwire` runs against `~/.zwire/profile` — a dedicated `--user-data-dir` — and never your system Chrome profile. This is not politeness; it is a hard requirement of loading unpacked extensions with elevated flags. The launcher passes `--load-extension`, `--extensions-on-chrome-urls`, and `--test-type`; you do not want those flags, those specific unpacked extensions, or the developer-mode banner they raise anywhere near the profile that holds your real logins, cookies, and saved passwords. A separate user-data-dir keeps the two worlds from ever sharing state: `zwire`'s bookmarks, history, sessions, and extension metadata live under `~/.zwire/profile`, and Chrome's live where they always did.

The isolation goes one level deeper than the profile. The extensions the launcher loads are staged into a per-user `~/.zwire/ext/` copy at launch, not run in place from the `.app` bundle, because Chromium writes extension `_metadata/` indexes into the extension directory and every user on the machine needs their own writable copy. And the native-messaging host — the bridge that powers the HUD terminal, the system-stats stream, and the scheme sync — is installed *profile-relative*, into `~/.zwire/profile/NativeMessagingHosts/`, so it is wired for `zwire` and invisible to system Chrome. The whole design keeps `zwire` a guest that brought its own everything: its own profile, its own extension copies, its own native host, its own icon and name. Nothing it does can be felt by the browser you already had.

The two tiers, in the launcher's own words

The whole shape of the project is stated at the top of `bin/zwire` itself, and it is worth reading before the chapters take the stack apart: “Chromium, rebranded, with the `zpwrcchrome` power-tool + cyberpunk new-tab + the internal-HUD skin preloaded. Runtime rebrand: a plain Chromium snapshot binary launched against a dedicated profile with a fixed extension set. Nothing to compile (the optional whole-chrome HUD lives in `fork/`).” Two tiers, named in the first four lines of the launcher: a runtime rebrand that compiles nothing and reaches everything an extension and a launch flag can reach, and an optional source fork that recompiles Chromium to reach the native chrome. The parenthetical is the honest boundary the rest of the book draws — “the optional whole-chrome HUD lives in `fork/`” — and it is why the same `bin/zwire` launches both: the launcher reads only `base.path`, so it does not care whether that path points at a downloaded snapshot or a compiled fork. The runtime tier is the daily driver, the fork tier is the whole-browser HUD for whoever will pay the build cost, and the seam between them is a design decision, not a limitation to apologize for.

Chapter 2 — Why a Real Blink Base

This chapter is the technical justification for the whole project: the exact set of Manifest V3 capabilities `zpwchrome` requires, why each one rules out the non-Chromium alternatives, and why the base must be *unbranded* Chromium rather than Google Chrome or Chrome for Testing.

The MV3 capability set `zpwchrome` requires

`zpwchrome` declares `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, a service-worker background, and `minimum_chrome_version: 127`. Each is load-bearing, and the combination exists only on a genuine Blink engine. Take them one at a time:

- `userScripts` is Manifest V3's supervised replacement for the old arbitrary-code injection that `tabs.executeScript` used to allow. It lets an extension register user-authored scripts that run in page context. It is a Chromium-only API — it does not exist in Safari's WebExtensions surface, and it has no analogue in a WebView, which exposes no extension model at all.
- `declarativeNetRequestWithHostAccess` is the MV3 network-rules engine: the extension ships declarative match/modify rules that the browser applies in its own network stack, rather than round-tripping every request through a JavaScript listener. The `WithHostAccess` variant grants the dynamic, host-scoped rules `zpwchrome` needs. This is the modern content-blocking and request-rewriting substrate, and it is implemented in Chromium's network service — not reachable from a wrapper that only owns a WebView. (zwire's own `hud-internal` extension uses the plain `declarativeNetRequest` form for its frame-busting rule set, discussed in Chapter 10.)
- `nativeMessaging` is the bridge to a local process: the extension launches a registered native binary and speaks a length-prefixed JSON protocol over its stdio. `zpwchrome`'s `pass` integration and segmented downloads need it to talk to a native host; zwire's own HUD host (Chapter 15) uses the same mechanism for the terminal and the system-stats stream. Native messaging requires the browser to read a host manifest from a known directory and spawn the process — something only a full browser (not a WebView) does.
- `webRequest` is the observe/block network API. Even in MV3, where the *blocking* form is constrained in favor of declarativeNetRequest, the observational surface remains, and `zpwchrome` declares it.
- **A service-worker background** is the MV3 background model: no persistent background page, an event-driven worker that the browser wakes on demand and suspends when idle. Every piece of zwire that coordinates across tabs — the scheme bridge, the `⌘K` command relay, the terminal open-state — is written for this suspend-any-time worker, and it only exists inside a Chromium extension runtime.
- `minimum_chrome_version: 127` is the extension declaring, in its own manifest, that it will not load on anything older than Chromium 127. That is not a preference; it is a floor. It rules out not only WebKit and Servo but any Chromium old enough to predate the APIs above.

The point is cumulative. Any *one* of these might be worked around; the *set* can only be satisfied by a current, genuine Chromium/Blink engine running a real extension runtime. Laid against the alternatives:

Capability need	WebView (Tauri)	Servo	Safari WebExtensions	Chromium
userScripts	no ext model	no	no	yes
declarativeNetRequest	no	no	partial	yes
nativeMessaging	no (own IPC)	no	limited	yes
service-worker bg	no ext model	no	yes	yes
--load-extension	n/a	n/a	n/a	yes (unbranded)

That is why zwire is a Chromium rebrand and not a Tauri app, a Servo experiment, or a Safari extension. The engine is not an implementation detail zwire happened to pick — it is the one requirement everything else hangs from.

Why unbranded Chromium, not Chrome or Chrome for Testing

The base is a plain Chromium snapshot from the public snapshot index — no Google logo, no “for automated testing” banner. This narrows the field to exactly one build, and it is worth seeing why the two obvious alternatives are both wrong.

Google Chrome is the branded build, and branded Chrome removed the `--load-extension` command-line switch in version 137. That switch is the entire mechanism by which zwire preloads its extensions on launch — the launcher’s `--load-extension="$LOAD"` line. Without it there is no way to load the HUD new-tab, the internal-HUD skin, and `zpwchrome` unpacked from disk at startup. So branded Chrome is disqualified by a removal that is specific to the *branded* build; the upstream Chromium engine still honors the switch.

Chrome for Testing is Google’s dedicated automation build, and it keeps `--load-extension` — but it wears a permanent “for automated testing” info-bar stripe across the top of every window, because that is its entire purpose: a build you cannot mistake for a user’s real browser. That stripe is exactly what you do *not* want on a daily driver you have gone to the trouble of rebranding with a cyberpunk icon and name. So Chrome for Testing is disqualified by the banner.

Plain unbranded Chromium — the continuous-build snapshot Google hosts at storage.googleapis.com/chromium-browser-snapshots — threads the needle: it is a real Chromium/Blink build with no Google product branding *and* no testing banner, and it retains `--load-extension`. `scripts/fetch-base.sh` resolves the correct platform id (`Mac_Arm`, `Mac`, `Linux_x64`, `Win_x64`) and, by default, the latest snapshot revision from the `LAST_CHANGE` marker; pass an explicit revision as the one argument to pin an exact build. The resolved revision is recorded so a launch is reproducible. Chapter 4 opens the fetch mechanics; the takeaway here is that the choice of base is not aesthetic — it is the only build that satisfies both constraints at once.

The manifest is the evidence

This chapter’s argument is not abstract; it is written down in the power-tool’s `manifest.json`, and it is worth pointing at the exact lines. `zpwchrome` declares `manifest_version: 3`, `minimum_chrome_version: 127`, and nineteen permissions, four of which — `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, and `webRequest` — are the ones that exist only on a current Chromium engine running a real extension runtime. The `minimum_chrome_version` line is not a preference the launcher could talk it out of; it is a hard floor the browser itself enforces, so any base older than Chromium 127, and every non-Chromium engine, refuses to load the extension at all. The internal-HUD manifest adds its own load-bearing declarations — `declarativeNetRequest` for the frame-busting rule, `developerPrivate` and `management` for the extensions page, `settingsPrivate` for the settings page, `system.display` for the worker’s window math — none of which a WebView exposes and none of which Safari’s WebExtensions surface fully implements. The whole of Chapter 2 reduces to a claim you can check by reading two manifests: the *set* of capabilities these extensions require is satisfiable on exactly one engine, and that engine is a genuine, current Chromium. Everything else about zwire is downstream of that single, verifiable fact. (Appendix M is the full manifest.)

Chapter 3 – The Architecture

zwire is seven cooperating layers. This chapter names each one and its single responsibility before the later chapters open them up: **Base**, **Rebrand**, **Theme**, **New tab**, **Internal HUD**, **Power-tool**, and **Launcher**, with the optional **Fork** underneath them all.

The seven layers, top to bottom

Read the stack from the engine up:

- **Base** — a pinned, plain Chromium snapshot downloaded by `scripts/fetch-base.sh` into `~/.zwire/base`, with its resolved binary path recorded in `~/.zwire/base.path` and its revision in `~/.zwire/base.version`. This is the engine, unmodified. Everything else is skin and cargo loaded onto it. On the fork path this same slot holds the compiled patched Chromium instead; the launcher does not care which, because it reads only `base.path`. (Chapter 4.)
- **Rebrand** — `scripts/rebrand-macos.sh`, the in-place bundle patch that makes the Dock, `⌘-Tab` switcher, and menu bar show “zwire” with the cyberpunk icon. It sets `CFBundleName/CFBundleDisplayName`, installs the `.icns`, deletes the key that would override it, re-signs the bundle ad-hoc, and re-registers it with LaunchServices. On Linux there is no bundle to patch — the launcher name *is* the brand — so this layer is macOS-only. (Chapter 5.)
- **Theme** — `theme/`, a Chrome theme extension that maps the HUD palette onto frame, toolbar, and tabs (colors only). It is present in the tree but, deliberately, **not loaded by the launcher** — the reason is the whole point of the palette pipeline and gets its own section below.
- **New tab** — `newtab/`, a `chrome_url_overrides.newtab` extension: the full HUD new tab, with a neon clock, an omnibox, quick-launch tiles, and its fonts vendored locally so it renders offline. (Chapter 6.)
- **Internal HUD** — `extensions/hud-internal`, the MV3 content-script suite that skins Chrome’s own internal pages, redirects several of them to HUD replacements, and layers vim keys, a tmux overlay, a powerline statusbar, the `⌘K` palette, a fuzzy find bar, and a terminal onto every page — plus the floating eight-scheme picker and its native host. This is the biggest layer, and Chapters 7 through 16 open it one feature at a time.
- **Power-tool** — `extensions/zpwrchrome`, the Manifest V3 power-tool, loaded as a **submodule** rather than vendored, so zwire tracks it upstream instead of freezing a copy. (Chapter 18.)
- **Launcher** — `bin/zwire`, the single entry point that assembles the flags, ensures the native host, makes the profile, and `execs` the base. (Chapter 19.)
- **Fork** (optional, underneath) — `fork/`, the source build that compiles a patched Chromium so the native chrome itself lives in the HUD. It is not a layer the runtime path touches; it is an alternate *base*. (Chapter 21.)

The layering has a property worth naming: every layer above the base is optional and independently removable. Delete `theme/` (already unloaded), delete `newtab/`, delete the `zpwrchrome` submodule — the launcher includes only the extension directories that actually contain a `manifest.json`, so a missing layer degrades to a plainer browser rather than a broken one. That graceful-skip rule is in the launcher itself, and it is what lets a fresh clone without submodules still run.

Why the static theme is present but not loaded

The `theme/` extension maps the HUD palette onto frame, toolbar, and tabs — but the launcher does not load it, and the launcher says so in a comment where it builds the extension list. Two things own the chrome palette ahead of it, and a static theme would fight both:

1. **The fork's native color mixer (patch 0002).** On the fork path, the cyberpunk palette is forced onto frame, toolbar, tab background/foreground, and the omnibox directly in Chromium's `chrome_color_mixer.cc`, keyed to the active scheme. That is the authoritative source of the chrome colors when the fork is in use.
2. **The internal-HUD skin.** On the runtime path, the `hud-internal` content scripts drive the scheme, and the scheme is mirrored through a native file so the whole HUD — pages, statusbar, palette — follows one picker.

A Chrome custom theme extension applies **last** in Chromium's color pipeline: it is designed to win, so the user's chosen theme is what they see. If `zwire` loaded `theme/`, it would override both the native mixer and the scheme-driven skin, pinning the chrome to one static palette and defeating the eight-scheme picker. So the theme extension is kept in the tree — it is a correct, working Chrome theme, useful as a reference and loadable by hand — but excluded from the launcher's default set precisely because it is *too* authoritative. Understanding that ordering is understanding the whole palette pipeline: the scheme picker is the source of truth, the native mixer and the content-script skin are its two renderers, and a static theme is deliberately kept out of the way.

One source of truth, four renderers

It is worth stating the palette architecture as a whole here, because it is the shape every later chapter reconnects to. There is exactly one source of truth for the active scheme — the native file `~/.zwire/hud-scheme`, owned by the host — and there are four renderers that all reconcile to it. The internal-HUD content scripts read a mirror of it in `chrome.storage.local` (`zb_scheme`) and style every ordinary page and every skinned `chrome://` page from it. The new-tab extension, a different origin that cannot share that storage, polls the native file directly and styles itself. The power-tool receives it over `runtime.sendMessage` and colors its own surfaces. And on the fork path, the compiled color mixer reads the same native file and colors the native chrome. Four renderers, one file, no primary — pick a scheme anywhere and the file changes, and every renderer follows within its own refresh. The reason the static `theme/` extension is excluded is now precise: it would be a *fifth* renderer that applies last and ignores the file, pinning the chrome to one palette and breaking the picker for the other four. Keeping it in the tree but out of the launcher is not an oversight; it is the one place a working Chrome theme would actively fight the architecture.

The layering has a second property the launcher enforces: every layer above the base is independently removable. The extension list is built by including only directories that contain a `manifest.json`, so a missing `zpwchrome` submodule, a deleted `newtab`, or an uninitialized checkout simply drops that layer and the browser runs with what remains. The native host is ensured but never required. Nothing above the stock engine is load-bearing for launch — which is why a fresh clone without submodules still opens a working browser, just a plainer one.

The layers at a glance

The seven layers, their one file or directory, and the chapter that opens each:

layer	where	responsibility	ch
Base	\$ZWIRE_STATE/base + base.path	the stock (or forked) engine	4
Rebrand	scripts/rebrand-macos.sh	Dock name + icon (macOS only)	5
Theme	theme/ (NOT loaded)	colors-only Chrome theme	3
New tab	newtab/	the HUD new-tab override	6
Internal HUD	extensions/hud-internal/	skin + vim + tmux + palette + find + statusbar + terminal	7-16
Power-tool	extensions/zpwichrome/ (submod)	the MV3 power-tool	18
Launcher	bin/zwire	assemble flags, exec the base	19
Fork	fork/ (optional base)	compiled whole-chrome HUD	21

Every row above the base is optional and independently removable; the launcher reads only `base.path`, so it neither knows nor cares whether the base is a snapshot or a compiled fork, and it loads only the extension layers that actually carry a manifest. That single indifference is what lets one launcher serve both build tiers and lets a partial checkout still run.

Chapter 4 – The Base

The engine layer: how `scripts/fetch-base.sh` resolves and pins a Chromium snapshot, where it lands under `~/.zwire/base`, and how `base.path` / `base.version` record what was resolved so a launch is reproducible.

Fetching and pinning a snapshot

`scripts/fetch-base.sh` is the whole of the base layer, and it is a small, readable script. Its job is to turn “the host I am running on” into “the correct Chromium snapshot binary on disk,” and to record what it resolved.

It begins by mapping the host to a snapshot platform. `uname -s/uname -m` select one of four rows:

```
Darwin/arm64      -> Mac_Arm    chrome-mac.zip  chrome-mac
Darwin/x86_64     -> Mac        chrome-mac.zip  chrome-mac
Linux/x86_64 | amd64 -> Linux_x64  chrome-linux.zip chrome-linux
MINGW/MSYS/CYGWIN -> Win_x64    chrome-win.zip  chrome-win
```

Each row names the snapshot platform id, the zip filename Google publishes for it, and the subdirectory that zip extracts to. Anything else exits with an “unsupported platform” message rather than guessing.

The revision comes next. With no argument, the script fetches the `LAST_CHANGE` marker for the platform — the latest continuous-build revision — and uses that. Pass a revision as the single argument and it pins that exact build instead:

```
scripts/fetch-base.sh      # latest snapshot (LAST_CHANGE)
scripts/fetch-base.sh 1656770 # pin an exact revision
```

This is the pinning story in full. There is no lockfile and no separate manifest: the “pin” is the revision string you pass, and the record of what you got is the `base.version` file the script writes. Pinning matters because Chromium snapshots are continuous — the latest changes hour to hour — and a reproducible `zwire` means knowing exactly which Blink you are running. For a machine you want to keep stable, fetch a revision once and never pass no-argument again until you choose to roll forward.

For total control there is a bypass. `bin/zwire` honors a `ZWIRE_BASE` environment variable pointing at any executable Chromium binary, checked before it consults `base.path`. That lets you point `zwire` at a Chromium you built yourself, a distro package, or a second snapshot, without going through `fetch-base.sh` at all — useful when developing the fork, where you want the launcher’s extension loading but your own compiled base.

What lands on disk

The download itself is a `curl` of `storage.googleapis.com/chromium-browser-snapshots/<PLAT>/<REV>/<ZIP>` into `~/.zwire`, an `unzip` into `~/.zwire/base`, and a delete of the zip. Then the script resolves the actual executable inside the extracted tree, because the path differs per platform:

- On macOS it finds the Chromium binary at `.../Chromium.app/Contents/MacOS/Chromium` inside the extracted `chrome-mac`.
- On Linux the binary is `.../chrome-linux/chrome`.

- On Windows/MSYS it is `.../chrome-win/chrome.exe`.

With the binary located, the script writes exactly two small files and nothing else:

```
~/.zwire/base.path      the absolute path to the resolved binary (bin/zwire reads this)
~/.zwire/base.version   the revision, as "r<REV>"                (reproducibility)
~ record)
```

`base.path` is the contract with the launcher: `bin/zwire` reads that one line to find the engine. `base.version` is the human-and-tooling record of which snapshot is installed. The `base/` directory holds the extracted browser itself — roughly 325 MB, the number the dev-loop scripts cite when they warn about what a `nuke` re-downloads. That is the entire on-disk footprint of the base layer — one directory and two one-line files — and it is deliberately flat so that a base upgrade is just “run `fetch-base.sh` again,” which overwrites the extracted tree and rewrites the two files. (On macOS an upgrade also means re-running the rebrand, because the fetch replaces the freshly-patched bundle with a stock one; Chapter 5 explains why.)

The script, exactly

`fetch-base.sh` is small enough to walk in full. Its own header names the base it targets: “a plain Chromium continuous-build snapshot (Google-hosted): a real Chromium/Blink build with NO ‘Google Chrome’ product branding and NO ‘for automated testing’ banner ... Unbranded Chromium also retains the `--load-extension` switch.” It maps `uname -s/-m` to a `PLAT/ZIP/SUBDIR` triple through the four-row `case` of Chapter 3, then, with no revision argument, `curls` the platform’s `LAST_CHANGE` sentinel from `storage.googleapis.com/chromium-browser-snapshots/<PLAT>` to learn the latest continuous-build number; a revision passed as the single argument pins that build instead. The download is a `curl ... --progress-bar` of `<BASE_URL>/<REV>/<ZIP>` into the state dir, an `unzip -q -o` into `$ZWIRE_STATE/base` (after an `rm -rf` of any prior extraction), and a delete of the zip.

Then it resolves the executable, which differs per platform: on macOS a `find` for the `Chromium` binary inside `*Chromium.app/Contents/MacOS/*`, on Linux `<SUBDIR>/chrome`, on MSYS `<SUBDIR>/chrome.exe`. It writes the two contract files — `printf '%s\n' "$BIN" > base.path` and `printf 'r%s\n' "$REV" > base.version` — and prints where it installed. That `r`-prefix on `base.version` is the marker Chapter 19 reads to tell a snapshot (`r...`) from a compiled fork (`fork-...`). The `~325 MB` figure the dev-loop scripts cite for the extracted browser is not hardcoded in this script; it lives in `localinstall.sh` and `nuke.sh` as the size those operations warn about re-downloading.

Snapshots are not releases, and what pinning actually buys

It is worth being precise about what a continuous-build snapshot *is*, because the word “version” does two different jobs in this book and confusing them makes the pinning story look sloppier than it is. A Chrome release — the thing with a four-part version like the tag the fork pins — is a branch that was cut, stabilized, and shipped on a schedule. A snapshot is not that. It is a build of the Chromium trunk at one revision, produced continuously, with no stabilization step and no promise that any particular revision is better than the one before it. The revision number is a monotonic counter over commits, not a version in any semantic sense: a higher number means later, and nothing else.

That has three practical consequences the base layer is designed around. First, “latest” is a moving target measured in hours, not weeks, which is why `LAST_CHANGE` is a *sentinel* the script reads rather than a version the script knows. Second, there is no release-notes surface to consult, so the only meaningful statement about which Blink you are running is the number itself — hence `base.version`, written on every fetch, as the one durable record. Third, and least obvious: because a snapshot is a trunk build, it can carry a regression that a released Chrome would never have shipped, and the correct response is not to wait for a fix but to fetch a

different revision. Pinning is therefore not a conservatism knob; it is the mechanism by which you *roll back*, and it is the reason the script takes a revision as its single argument rather than hiding the choice behind a flag.

The rollback procedure is the pinning procedure run backwards, and it is worth stating because nothing else in the tree does it for you. There is no lockfile, no manifest of known-good revisions, and no history of what was installed before — `base.version` holds exactly one number, the current one. So a machine you want to be able to roll back must record the number somewhere the fetch does not overwrite, which in practice means noting it before an upgrade. That is a deliberate thinness rather than an oversight: a version database would be a second source of truth about the base, and the base layer's whole design premise is that there is exactly one — the binary on disk, described by two one-line files beside it.

The three ways a base gets there, and why the launcher cannot tell

The launcher reads `base.path` and nothing else, and that indifference is what lets three quite different things occupy the same slot. A snapshot fetched by the script is the common case. A compiled fork installed by the fork's packaging step is the second, and it writes the same two files with a `fork--`-prefixed version so the *only* on-disk evidence of which tier you are running is that prefix. The third is the environment override, an executable path in `ZWIRE_BASE` consulted ahead of `base.path`, which bypasses the fetch mechanism entirely and points the launcher at any Chromium you like — a distro package, a second snapshot, or an incremental fork build you have not packaged yet.

The third case is the one that earns its keep during fork development. A fork build produces a browser in the build output directory long before anyone wants to install it over the working base, and the override lets you run that build with the full extension set, the real profile, and the real native host, without disturbing the snapshot the same machine still uses as its daily driver. That is a genuinely different workflow from “install and hope,” and it exists because the launcher's base resolution is a precedence rule — an executable override, then the recorded path, then a clear error — rather than a lookup in a single place.

The error case is worth one sentence of its own, because it is the only fatal condition in the entire launcher. If neither the override nor the recorded path resolves to an executable, the launcher prints the exact command that fixes it and exits non-zero. Every other missing component in the system degrades; the base does not, because there is nothing left to run.

The upgrade's second-order effects

An upgrade is `fetch-base.sh` again, and the script's behavior on an existing install is worth spelling out because two of its effects surprise people. The fetch removes the prior extraction before unzipping, so the base directory is replaced rather than merged — which is correct (a merged Chromium tree of two revisions would be a genuinely dangerous artifact) and which means anything you put inside the base directory is gone. And on macOS the fresh extraction is a stock, Chromium-signed bundle, so every one of the rebrand's four edits is undone at once: the name, the icon, the deleted icon-name key, and the ad-hoc seal. The rebrand is not stored anywhere the fetch could preserve; it is a patch applied to a bundle the fetch just replaced.

What the upgrade does *not* touch is equally important and is why an upgrade is safe to run casually. The profile lives outside the base directory, so bookmarks, history, sessions, and every extension's stored state survive untouched. The per-user staged extension copy is rebuilt at the next launch from the source directories, so it needs no migration. The native host manifest points at a binary outside the base tree, so it is unaffected. And the scheme and the shared preferences live in the fleet-wide store, not under the install root, so the browser comes back up in the same colors it went down in. The base is the one layer where “replace it entirely” is the correct upgrade strategy, and the layout above it is arranged so that replacing it costs one macOS-only command.

Chapter 5 – The Runtime Rebrand

The no-compile path. On macOS `scripts/rebrand-macos.sh` patches the base bundle's Dock name to `zwire`, swaps in the cyberpunk `.icns`, deletes `CFBundleIconName`, and re-signs ad-hoc so macOS honors the change. This chapter explains why each of those steps is required and why the rebrand must be re-run after a base upgrade.

The Dock name, the icon, and the ad-hoc re-sign

The rebrand is an in-place edit of the base bundle under `~/zwire/base` — it never touches a system browser install. The script reads `base.path`, walks up to the `.app` directory (stripping `/Contents/MacOS/*`), and patches the bundle's `Info.plist` and `Resources`. Four operations, each of which is there for a specific macOS reason:

1. **The name.** `CFBundleName` and `CFBundleDisplayName` are set to `zwire` via `PlistBuddy` (Set, falling back to Add if the key is absent). These are what the Dock, the `⌘-Tab` switcher, and the menu bar read. Change them and the app stops calling itself “Chromium.”

2. **The icon, under a new filename.** The cyberpunk `branding/zwire.icns` is copied into the bundle's `Resources` as `zwire.icns`, and `CFBundleIconFile` is pointed at `zwire`. It is installed under a *new* filename rather than overwriting Chromium's own icon so it does not collide with the original icon's LaunchServices association — a fresh name is a fresh association.

3. **Deleting `CFBundleIconName`.** This is the non-obvious step, and the script comments it. Modern Chromium ships its icon as `AppIcon` inside a compiled `Assets.car` and references it through `CFBundleIconName`, which takes **precedence** over the `.icns` named in `CFBundleIconFile`. If you set `CFBundleIconFile` but leave `CFBundleIconName` in place, macOS keeps showing the compiled Chromium icon and your `.icns` is ignored. So the script deletes `CFBundleIconName`, forcing macOS to fall back to `zwire.icns`. Without this deletion the icon swap silently does nothing.

4. **The ad-hoc re-sign.** The base bundle is ad-hoc signed. Editing `Info.plist` and `Resources` invalidates that signature, and once the signature is invalid macOS treats the icon and name change as *tampering* and reverts to the cached original. The fix is to re-seal the bundle: `codesign --force --sign - "$APP"` re-signs it ad-hoc (the nested frameworks and helper apps keep their own valid seals, so only the top level needs re-signing). This is what makes macOS *accept* the change rather than roll it back. The script warns but does not fail if the re-sign fails, because a browser with a stale icon is still a working browser.

Finally it nudges LaunchServices to refresh its icon/name cache for this bundle (`lsregister -f "$APP"`) and *touches* the bundle so the change is noticed. The result: `~/zwire/base/Chromium.app` now presents as “zwire” with the cyberpunk icon, everywhere macOS shows an app's identity, without a single line of Chromium recompiled.

The order matters and the script encodes it: patch the plist and resources **first**, re-sign **second**, re-register **third**. Re-signing before the edits would seal the wrong state; re-registering before re-signing would cache a bundle macOS still considers tampered.

When the icon cache lingers: bust and nuke

macOS caches Dock and `⌘-Tab` icons aggressively by bundle, so occasionally the old Chromium icon or name lingers after a rebrand even when every step above ran. The dev-loop has a dedicated escape hatch for this: `scripts/bust.sh` (“cache-bust”) re-seals and re-registers *both* the base bundle and any `/Applications/zwire.app` wrapper with LaunchServices, forcing the cyberpunk icon and the “zwire” name to refresh immediately. `scripts/nuke.sh` is the heavier option — it removes the wrapper and the ~325 MB base, re-fetches from scratch, rebrands, and reinstalls, while preserving your profile. These exist because the LaunchServices cache is the one part of the rebrand macOS does not always honor on the first try, and a browser whose Dock icon is wrong reads as broken even when it is not.

Why an upgrade forces a re-run, and why Linux needs none

An *upgrade* forces a re-run of the rebrand: a fresh `fetch-base.sh` unzips a stock, Chromium-signed bundle over the top of the patched one, so every one of the four edits above is gone and must be reapplied. `scripts/install.sh` chains the rebrand after the fetch on macOS for exactly this reason, and the README’s update recipe ends with `rebrand-macos.sh` for the same one.

On Linux an application’s identity — the launcher entry, the taskbar name, the window title — comes from how it was invoked and from a `.desktop` file, not from metadata embedded in the binary. So on Linux the launcher *name itself* is the brand: the user runs `zwire`, and the process and window read `zwire` because that is what started them. There is no in-place bundle rewrite because there is no bundle to rewrite, and `install.sh` only invokes `rebrand-macos.sh` when `uname -s` is `Darwin`. “Rebrand” therefore means two different concrete things on the two platforms — a bundle patch on macOS, a launcher name on Linux — and the project does not pretend otherwise.

The four operations, as commands

The rebrand is four `PlistBuddy` and `codesign` operations in a fixed order, and the script’s comments give the reason for each. It resolves the `.app` by stripping `/Contents/MacOS/*` off `base.path`, then a `set_key` helper that tries `Set` and falls back to `Add ... string` writes `CFBundleName` and `CFBundleDisplayName` to `zwire`. It copies `branding/zwire.icns` into the bundle’s `Resources` under a new filename and points `CFBundleIconFile` at it, then deletes `CFBundleIconName` — the non-obvious step whose comment explains that modern Chromium references its icon as `AppIcon` inside a compiled `Assets.car` through that key, which “takes PRECEDENCE over the `.icns` in `CFBundleIconFile`,” so the key must go for the custom icon to win. It re-seals the bundle with `codesign --force --sign - "$APP"` (warning but not failing on error, “nested frameworks/helpers keep their own valid seals”), and finally re-registers with `lsregister -f` and `touches` the bundle. The order is load-bearing and the script encodes it: name and resources first, re-sign second, re-register third — re-signing before the edits would seal the wrong state, and re-registering before re-signing would cache a bundle macOS still considers tampered. `scripts/bust.sh` runs exactly the re-seal-and-re-register half against both the base bundle and any `/Applications/zwire.app` when the LaunchServices cache is stubborn.

Chapter 6 — The New-Tab HUD

`newtab/` is a `chrome_url_overrides.newtab` extension: the full HUD new tab — a neon clock, an omnibox, quick-launch tiles — with its fonts vendored locally so it renders offline and never phones a font CDN. This chapter walks the page, its palette module, and the scheme sync that keeps it in step with the rest of the HUD.

The page and its locally-vendored fonts

The new tab is a small MV3 extension whose whole purpose is one override: `chrome_url_overrides.newtab` points at `newtab.html`. When `zwire` opens a new tab, Chromium serves that page instead of Google's. The manifest pins a stable extension id via its embedded `key` (id `gpoep...gieb`), declares the `nativeMessaging`, `tabs`, and `history` permissions the palette needs, and lists its own files as web-accessible so they load in page context — including from the `tmux` overlay, which frames this page directly.

`newtab.js` is the page's logic, and it is deliberately dependency-free:

- **The clock.** A one-second `setInterval` writes a zero-padded `HH:MM:SS` time and an uppercased `WEEKDAY, MON DD, YYYY` date into the page. No library, just `Date` and `padStart`.
- **The quick-launch tiles.** A default set (GitHub, DuckDuckGo, MDN, crates.io, Hacker News, the docs site) renders as neon tiles, each a two-letter glyph plus a label. The set is overridable at runtime through a `zb.tiles localStorage` key holding an array of `{label, url}` objects, so the tiles are yours to edit without touching the extension.
- **The omnibox.** The page's own search field decides, per input, whether you typed a URL or a query. `looksLikeUrl` is a small heuristic: a scheme (`https://...`) is a URL; whitespace means a search; a bare `host.tld[/path]` or the literal `localhost` is a URL. A URL navigates directly (adding `https://` if you omitted the scheme); anything else goes to the search engine, default DuckDuckGo, overridable via a `zb.engine localStorage` key with a `%s` placeholder.
- **Reclaiming focus.** This is the page's least obvious and most necessary trick. On its own new tab, Chrome parks the text cursor in the `browser` omnibox and defeats the page's `<input autofocus>`, so the page never receives a single keystroke — which would kill both typing and the `⌘K` palette until you clicked in. `newtab.js` calls `q.focus({preventScroll:true})` in JavaScript, which moves `activeElement` to the page field where the attribute could not, and retries it across the first animation frames and again whenever the tab is re-shown. That is why the HUD new tab is typeable the instant it appears.

The fonts are **vendored locally** under `newtab/fonts/`, not linked from a CDN, and this is a design choice with two payoffs. First, the new tab renders correctly **offline** — the display face is present on disk, so a HUD new tab on a plane looks exactly like one on a fiber connection. Second, the page never makes a request to a font host on every new tab, which is both a privacy property (no third party sees a beacon each time you open a tab) and a latency one (no network round-trip in the render path of the most-opened page in the browser). The `fonts/LICENSE` file records the vendored faces' licensing. Vendoring costs a few hundred kilobytes in the extension; it buys an offline-correct, beacon-free, instant new tab, and for the page you open more than any other that trade is obviously right.

Palette, schemes, and scheme-sync

The new tab is a *different extension* from `hud-internal`, which means the global `⌘K` command palette that `hud-internal` injects into ordinary pages cannot reach it — a content script from one extension does not run in another extension's page. So `newtab/palette.js` carries its own copy of the same `ZGui.palette`, built on the shared `zgui-core` fuzzy engine, and because it runs as an extension page with the `tabs` permission it can call `chrome.tabs` directly with no worker round-trip. The palette it builds is substantial: dozens of `chrome://` internal pages (extensions, settings, history, downloads, flags, gpu, net-internals, and a long tail of the `-internals` diagnostics pages), the sub-pages inside `chrome://settings`, one entry per color scheme, plus your frecent history (scored by visit count and recency over the last 90 days) and your open tabs, all fuzzy-filtered in one list.

There is a wrinkle in how the palette opens, and it is the same `⌘K`-reservation problem as focus. On the new tab, Chrome reserves `⌘K` at the browser level, so no page `keydown` listener can catch it. The `hud-internal` background worker owns `⌘K` as a `chrome.commands` shortcut; when the active tab is the new tab it sends the new-tab page a `zwireOpenPalette` message (permitted because the new-tab manifest lists the HUD extension in `externally_connectable`), and `palette.js` toggles the palette in response. On an *ordinary* page `⌘K` and `:` are caught directly. So the palette opens the same way everywhere; the plumbing under it differs because the new tab is a privileged surface Chrome guards more tightly.

The colors are kept in step through `schemes.js`, `palette.js`, and `scheme-sync.js`. `schemes.js` is the canonical table of the eight HUD schemes — its full contents are Chapter 8 and Appendix D. `scheme-sync.js` keeps the new tab following the one scheme the user picked anywhere: because separate extensions do not share `chrome.storage`, the scheme travels through the native file the HUD host writes, and the new tab reads it so switching the scheme in the internal-HUD picker recolors the new tab too. `tmux-pane.js` adds a new-tab surface for the tiling overlay. The through-line of the chapter: the new tab is its own extension, so everything shared with the rest of the HUD — the palette, the eight schemes, the current scheme — has to be *re-carried* here and *re-synced* through the one channel two extensions can share, the native file.

The three support scripts

Behind the page sit three small scripts that make the new tab a full HUD citizen despite being its own extension. `palette.js` is the new tab's copy of the `⌘K` palette, and because it runs as an extension page with the `tabs` permission it calls `chrome.tabs` directly with no worker round-trip — its list carries the same forty-odd `chrome://` destinations, the nineteen settings sub-pages, one entry per scheme, and, appended asynchronously, your open tabs and your frecent history scored by the same 90-day formula the worker uses. It opens the same three ways the ordinary palette does; on the new tab specifically it responds to the worker's `zwireOpenPalette` message, permitted because the new-tab manifest lists the HUD worker in `externally_connectable`.

`scheme-sync.js` is how the colors follow. It cannot share `chrome.storage` with the internal HUD, so instead it polls the native host every 1.5 seconds (`{cmd: 'get'}` to `com.zwire.hud`), applies the returned scheme's variables to `:root`, and applies the returned UI preferences the same way — toggling the `no-scanlines/no-vignette/no-neon-glow/no-anim` classes (an effect is *on* when its class is *absent*, matching the shared effects toggle) and setting light or dark from `ui.light`. `tmux-pane.js` is the forwarder that lets the new tab be a `tmux` pane: when the page is framed as a pane (its window name is `zbtmux`), the overlay lives in the parent frame and this script catches the prefix and command keys and posts them up, mirroring the pane-side half of `zbtmux.js` so the tiling overlay can frame the new tab as reliably as any site.

Why the focus trick needs four attempts

The focus-reclaim is worth one more sentence than Chapter 6 gave it, because the retries are not superstition. Chrome parks the cursor in the *browser* omnibox and overrides the page's `autofocus` after the page

has already run, so a single `focus()` call at load loses a race it cannot see. `newtab.js` therefore calls `q.focus({preventScroll:true})` immediately, again on the first animation frame, and again at 60 and 200 milliseconds, and once more on every `visibilitychange` when the tab becomes visible — covering the load race, the post-layout override, and the “switched back to an already-open new tab” case. It also binds a capture-phase `keydown` directly on the field that swallows `⌘K` (stopping the macOS beep and keeping the field from seeing the key) and calls the palette opener, so the palette works even in the instant before focus has fully settled.

Web-accessible, so the overlay can frame it

One manifest detail earns the new tab its place in the tmux overlay. Fresh tmux panes open the zwire new tab, but `chrome://newtab` cannot be put in an `iframe` — `chrome://` pages are un-embeddable by design — so the overlay frames the new-tab *extension's* page directly, at its stable `chrome-extension://gpoep...gieb/newtab.html` URL. That only works because the new-tab manifest lists its own files as `web_accessible_resources` for `<all_urls>`, which is what lets another page load them in an `iframe`. It is the same page `chrome://newtab` redirects to, reached by its real extension URL instead of the un-framable alias. The manifest's embedded `key` pins that URL to a stable id across installs, so the overlay's hardcoded new-tab address never goes stale — a small manifest choice that is load-bearing for the tiling overlay two chapters later.

Chapter 7 – The Internal-HUD Extension: The Injection Model

`extensions/hud-internal` is the largest layer on the runtime path, and it is worth understanding its *shape* before the feature chapters open each script. The extension is an MV3 background worker plus a carefully ordered stack of content scripts, a set of HUD replacement pages, a native host, and the shared `zgui-core` library. This chapter is the map; Chapters 8–16 are the territory.

The manifest as a wiring diagram

The manifest declares a broad permission set — `storage`, `nativeMessaging`, `management`, `tabs`, `webNavigation`, `history`, `downloads`, `bookmarks`, `sessions`, `alarms`, `notifications`, `developerPrivate`, `settingsPrivate`, `system.display`, and `declarativeNetRequest` — plus `<all_urls>` host access. Each permission earns a feature: `management/developerPrivate` power the HUD extensions page, `history/bookmarks/downloads/sessions` power the HUD replacements for those internal pages, `system.display` feeds window math, and `declarativeNetRequest` carries the `frame_bust` rule that lets the `tmux` overlay iframe any site.

The content scripts are registered in several blocks, and the *order within a block* is load-bearing because later scripts depend on globals earlier ones set. The canonical block loads, in order: `schemes.js` (defines `window.ZWIRE_HUD` with the palette table), the `zgui-core` `util.js/fzf.js/command-palette.js` (defines `window.ZGui`), then `zkeys.js` (the keybinding registry `window.ZWIRE_KEYMAP`), `cmd-defaults.js`, `zpalette.js`, and `zvim.js`. By the time `zvim.js` runs it can read both the keymap and the palette; by the time the palette runs `ZGui` exists. Every script also guards against double-injection with a `window.__zb*Loaded` flag, because content scripts can be injected more than once across frame and navigation events.

Two treatments for `chrome://` pages, one switch that enables both

`chrome://` pages are privileged, and by default no extension content script runs on them — that is why `zwire` launches with `--extensions-on-chrome-urls`, the switch that lets `hud-internal`'s content scripts match `chrome://*/*` at all. With that switch set, the manifest wires two different treatments for internal pages:

- **Redirect.** For a handful of high-traffic pages — `chrome://extensions`, `chrome://settings`, `chrome://history`, `chrome://bookmarks`, `chrome://version` — a `document_start` content script (`redirect.js`) sends you to a full HUD replacement served from the extension's own `pages/` directory. These are not restyles of Chrome's page; they are purpose-built HUD pages that reimplement the surface using the extension's privileged permissions. Chapter 14 is the tour.
- **Skin.** For every *other* `chrome://` page (the ones without a bespoke replacement — flags, discards, dns, password-manager, and the rest), a `document_start` pair — `schemes.js` then `theme.js` — restyles the page in place with the active scheme's palette. Chapter 8 covers what `theme.js` maps.

The excluded set is exactly the redirected pages plus the new-tab URLs (which the new-tab extension owns), so the two treatments never fight over the same page.

Where each feature lives

The rest of the internal HUD is a set of content scripts and their support files, each of which gets its own chapter:

```
schemes.js      the 8-scheme palette table (window.ZWIRE_HUD)      -> Ch 8
theme.js        the chrome:// WebUI-token skin            -> Ch 8
background.js   the service worker: scheme bridge, tab bus, term flag -> Ch 8,12,16
zvim.js         vim keybindings on every page        -> Ch 9
ztmux-config.js the tmux overlay provider (shared ZGui.tmux WM) -> Ch 10
ztmux-pane.js   the tmux pane-side forwarder (all frames) -> Ch 10
zpowerline.js   the powerline statusbar adapter (shared ZGui.powerline) -> Ch 11
zpalette.js     the global %K command palette          -> Ch 12
zkeys.js        the keybinding registry (single source of truth) -> Ch 9,12
cmd-defaults.js the seeded %K keyword commands          -> Ch 12
zfind.js        the fuzzy find bar (replaces Ctrl+%F)    -> Ch 13
pages/          the HUD internal-page replacements      -> Ch 14
native/zwire-host the native host (scheme/ui/sysinfo/PTY/fs/exec/kv) -> Ch 15
lib/term/       the xterm terminal over the native PTY   -> Ch 16
lib/zgui-core/  the shared cyberpunk UI toolkit (submodule) -> Ch 17
```

That is the whole internal HUD in one view: a manifest that wires an ordered content-script stack against every page, two treatments for `chrome://`, a background worker that coordinates across tabs, a native host for the machine, and a shared toolkit under it. The feature chapters take them in that order.

The seven injection blocks, in order

The manifest's `content_scripts` array is seven blocks, and the split is not cosmetic — each block has its own match set, run timing, and frame scope, chosen so the right script runs on the right surface at the right moment. Read top to bottom:

#	matches	run_at	all_frames	scripts
1	chrome://{5 redirected}	document_start	no	redirect.js
2	chrome://{*/} (minus above)	document_start	no	schemes.js, theme.js
3	http/https/file/chrome://	document_idle	no	schemes.js, fzf.js, zfind.js
4	http/https/file/chrome://	document_start	no	schemes.js, zgui util/fzf/ command-palette, zkeys.js, cmd-defaults.js, zpalette.js, zvim.js
5	http/https/file/chrome://	document_idle	no	schemes.js, powerline.js, zpowerline.js (statusbar)
6	http/https/file	document_idle	yes	ztmux-pane.js (pane forwarder; top frame is ZGui.tmux + ztmux-config.js)
7	http/https/file/chrome://	document_idle	no	xterm + terminal + boot

Three things fall out of the table. Block 1 fires first and only on the five redirected `chrome://` pages, at `document_start`, so the redirect happens before Chrome's own WebUI paints; block 2 skins every *other* `chrome://` page, also at `document_start`, and its exclude set is exactly block 1's match set plus the new-tab URLs, so the redirect and the skin never fight. Block 4 — the keyboard and palette stack — runs at `document_start` and in a fixed internal order, because later scripts read globals earlier ones set: `schemes.js` defines the palette table, the `zgui-core` trio defines `ZGui`, `zkeys.js` defines the keymap, and only then do `zpalette.js` and `zvim.js` load able to read both. Block 6 is the only one with `all_frames: true`, because the tmux overlay must run its forwarder inside every framed pane, not just the top document. Every script also carries a `window.__zb...Loaded` guard, because content scripts can be injected more than once across an SPA's frame and navigation events, and a second injection must be a no-op.

The permission set reads as a feature list once you know which API powers which surface: `management` and

`developerPrivate` back the extensions page and the shortcut editors; `history`, `bookmarks`, `downloads`, and `sessions` back the four list pages; `settingsPrivate` backs the settings page; `system.display` feeds the worker's window-tiling math; `notifications` and `alarms` power the CI monitor; `browsingData`, `tabGroups`, `readingList`, and `power` back the automation bus's data-clearing, tab-grouping, reading-list, and keep-awake verbs; `declarativeNetRequest` carries the single `frame_bust` rule that lets the tmux overlay iframe any site; and `<all_urls>` host access is what lets the content-script stack run everywhere. The `externally_connectable` list holds two ids — the power-tool and the new-tab page — the only extensions permitted to message the worker from outside; the new-tab page uses it to bridge its palette into the automation bus described next.

The automation bus: 109 browser verbs through one executor

The HUD is not only keyboard-driven; it is scriptable. A stryke script — run from a terminal, from a page's ⌘K palette, from the new-tab palette, or from a HUD page — reaches the browser through `zwire-host` and drives it over a semantic action bus. The surface has two halves: 109 `browser.*` verbs that manipulate the browser itself — one branch each in the worker's `execZbCmd` — and the host's own commands, the file, process, job, bus, key-value, PTY, and hook families `zwire-host` dispatches directly.

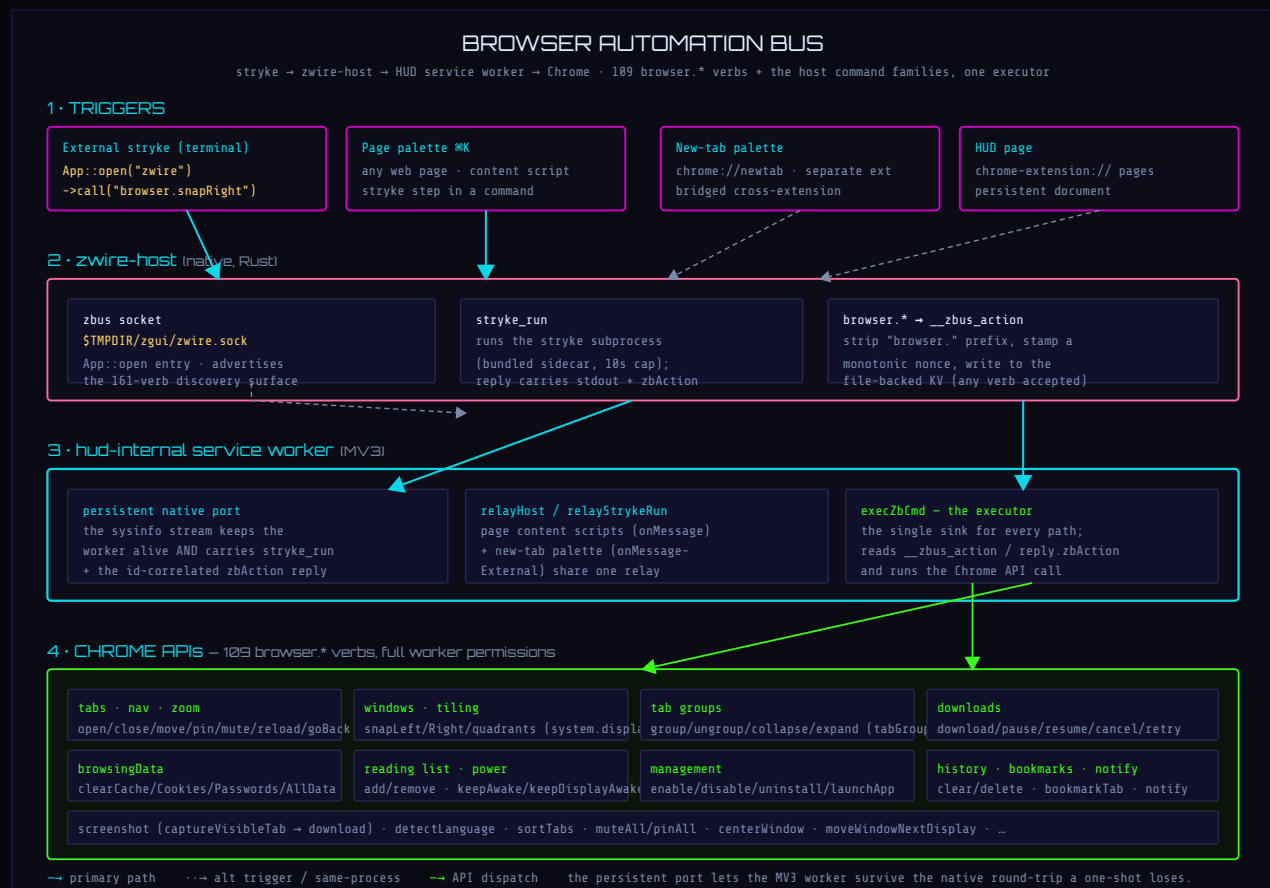


Figure 1: zwire's browser automation bus: a stryke trigger reaches zwire-host, which runs stryke and stamps a browser.* action into a file-backed KV; the MV3 service worker's `execZbCmd`, kept alive across the native round-trip by the persistent sysinfo port, is the single executor that runs the Chrome API call across the 109-verb browser surface.

The path has four stages. A trigger runs stryke code containing `App::open("zwire")->call("browser.<verb>")`. `zwire-host` runs the stryke subprocess and, for any `browser.*` verb, strips the prefix and stamps `{a:<verb>, ...args, _n:<nonce>}` into a file-backed key-value entry — it accepts any verb, so the host never needs the verb list: discovery is advisory, execution is open. The MV3 service worker is the single executor. Its `execZbCmd` reads that action — or the `zbAction` piggybacked on a `stryke_run` reply — and issues the matching Chrome API call, with the worker's full `tabs`, `windows`, `downloads`, `bookmarks`, `history`, `tabGroups`, `browsingData`, `readingList`, and `power` permissions behind it.

The hard part is MV3's service-worker lifecycle. A one-shot native message issued from a page's content script loses its reply, because the worker is torn down during the roughly 200-millisecond native round-trip — external pages saw a zero-percent success rate. The fix is the persistent native port that already streams system stats: it keeps the worker alive and delivers the reply on an already-open channel, correlated by a request id so the browser action fires exactly once. The new-tab page — a separate extension whose content scripts are excluded from `chrome://newtab` and whose storage is isolated from the worker's bus — bridges its palette's host-backed steps into the same executor over `externally_connectable`, so `execZbCmd` stays the one place a browser action is ever run, whichever surface asked for it.

The 109 browser verbs span the browser's own surface: tab lifecycle and navigation, page reload and history motion, per-tab zoom, window state and — using `system.display`'s work area — left/right/quadrant tiling and multi-monitor moves; tab grouping; the full download lifecycle; browsing-data clearing; the reading list; keep-awake; extension and app management; and history, bookmark, and notification actions. Because the executor is one function and the host forwards any verb, adding a capability is a single `execZbCmd` branch plus a single discovery-list entry — the surface grows without a new transport.

Chapter 8 – The Eight Schemes and the Sync Bus

The HUD has eight color schemes — cyberpunk, midnight, matrix, ember, arctic, crimson, toxic, vapor — and every surface in the browser follows the one the user picks: the ordinary-page content scripts, the `chrome://skins`, the separate new-tab extension, the `zpwchrome` power-tool, and (on the fork) the native color mixer. This chapter is the palette table, the token map that applies it to Chrome's own pages, and the cross-extension bus that keeps them all agreeing.

The palette table

`schemes.js` is the canonical scheme table, `window.ZWIRE_HUD`, and it is the same file loaded by every content script and by the new-tab extension — auto-extracted from the same `hud-theme.js` the wider MenkeTechnologies app suite uses, so the browser cannot drift from the apps. It defines three things: an ordered list of twenty CSS custom-property keys (`SCHEME_VAR_KEYS`), the scheme order (`SCHEME_ORDER`), and the schemes themselves (`COLOR_SCHEMES`). Each scheme has a `label`, a one-line `desc`, a dark `vars` set, and a parallel `lightVars` set:

```
cyberpunk: {
  label: 'Cyberpunk', desc: 'Hot pink + cyan (default)',
  vars: {
    '--accent': '#ff2a6d', '--cyan': '#05d9e8', '--magenta': '#d300c5',
    '--green': '#39ff14', '--red': '#ff073a',
    '--text': '#e0f0ff', '--text-dim': '#7a8ba8',
    '--bg-primary': '#05050a', '--bg-card': '#0d0d1a', '--border': '#1a1a3e',
    /* ...20 keys total, plus the glow/dim variants... */
  },
  lightVars: { /* the same 20 keys, tuned for a light background */ },
}
```

The twenty keys are the HUD's whole vocabulary: three accent shades, three cyan shades, two magenta, a green pair, a red, three text weights, four background depths, and two border shades. Every HUD surface — the statusbar, the tmux panes, the palette, the find bar, the internal pages — styles itself entirely in terms of these variables, so recoloring the browser is just swapping which scheme's values are written to `:root`. The eight schemes' full dark-and-light palettes are Appendix D; the point here is that they are *data*, one table, shared by every consumer, with a matched light variant so the HUD works on a light background too.

Applying the palette to Chrome's own pages

`theme.js` is the script that maps the active scheme onto Chrome's internal pages — the `chrome://` pages that have no bespoke HUD replacement. Its comment is careful about what it is *not*: it deliberately drops all the HUD decoration (the grid lattice, CRT scanlines, neon glow, the floating picker) and maps *only* the scheme's palette onto Chrome's WebUI color tokens, plus the mono font, so these pages simply inherit the same theme the rest of the browser uses — no HUD chrome on top. The mechanism is a stylesheet that rewrites Chrome's own CSS custom properties to point at the HUD variables:

```

:root, :host {
  --color-sys-surface: var(--bg-primary) !important;
  --color-sys-on-surface: var(--text) !important;
  --color-sys-primary: var(--cyan) !important;
  --color-sys-secondary: var(--accent) !important;
  --color-sys-outline: var(--border) !important;
  /* ...plus the --cr-fallback-color-* and --cr-*--background-color families... */
}

```

Chrome's WebUI is built on these `--color-sys-*` and `--cr-*` tokens, so overriding them with `!important` recolors the page's own components without touching their markup. The scheme `theme.js` reads the native source of truth (`~/ .zwire/hud-scheme`), fetched via the background worker — a content script cannot call `sendNativeMessage` itself, so it reads the mirrored value the worker maintains.

The sync bus: how five consumers agree on one scheme

The hard part is not styling one surface; it is keeping every surface agreeing on one value across three isolation boundaries — content scripts vs the worker, extension vs extension, and the browser vs the native chrome. `hud-internal` solves it with a two-tier store plus a native file as the ultimate source of truth:

- **Content scripts ↔ worker.** The background worker mirrors the current scheme into `chrome.storage.local` under `zb_scheme`, and the content scripts read and observe that key. The worker uses storage rather than an async native round-trip on purpose, and the code comments say why: an MV3 worker can suspend mid-round-trip, so a `sendResponse` across the native host is unreliable, while a storage write always lands.
- **Extension ↔ extension ↔ native chrome.** The scheme travels through `native/zwire-host` and the scheme file it owns. When a page or the picker sets a scheme, the worker relays it to the host (which atomically writes the file) and writes `zb_scheme`. On startup it seeds `zb_scheme` from the file, which may already hold a scheme chosen in a prior session. The new-tab extension reads the same file (the one channel two extensions can share), and on the fork the native color mixer reads it too.
- **The `zpwchrome` bridge.** Because separate extensions cannot share storage, the worker also pushes the scheme to `zpwchrome` over `runtime.sendMessage`, with a guard (`lastPushed`) so a scheme that came from `zpwchrome` is not echoed straight back into a loop. `zpwchrome` can also pull the current scheme on startup (`zb-scheme-get`) and push a user's pick back (`zb-scheme-set`).

A companion file, `~/ .zwire/hud-ui.json`, carries the shared visual-effect and light-mode preferences (light mode, scanlines, vignette, glow, animation) the same way, since those are per-origin `localStorage` on each page and the native file is the only channel that reaches the other extensions. The whole design is one value with one source of truth (the native file) and several caches that all reconcile to it — which is why picking “matrix” in the floating picker recolors the statusbar, the tmux panes, the find bar, the new tab, the `chrome://` pages, and the power-tool, all at once.

The fleet-wide single source: `zwire-host` and `global.toml`

The per-extension mesh above — `chrome.storage` mirrors plus cross-extension `sendMessage` pushes — is fragile: two surfaces writing at once (a scheme change here, a light toggle there) could desync `newtab`, `zpwchrome`, and the HUD. Newer builds move the *source of truth* off that mesh and onto the native host. The scheme, the light-mode flag, the visual-effect prefs, and any user-defined custom schemes now live in one app-independent file the Rust `zwire-host` owns — `~/ .zwire/global.toml` — read and written by every `zwire-host` client, not just the browser but the wider MenkeTechnologies fleet (Audio-Haxor, `zmax`, `zpw-daw`, and the rest), so a scheme picked in any app repaints all of them. `newtab` and `zpwchrome`

subscribe the host bus directly (a snapshot on connect plus live updates), and the HUD background folds a host theme subscription into its sysinfo port and bridges it to `chrome.storage` with an echo guard — retiring the old push/pull/mirror/seed mesh. This also fixed a real race: two concurrent writes (`{scheme}` and `{ui}`) could clobber each other and silently revert the scheme, so the host now serializes the read-modify-write of `global.toml` behind a cross-process lock (`global.toml.lock`). The legacy `hud-scheme` and `hud-ui.json` files are no longer the bus; the host still writes a plain-text `hud-scheme` projection (and, for light mode, a `hud-light` projection) *beside* `global.toml` purely so the fork's C++ color mixer — which cannot parse TOML at paint time — has a trivial file to read. `zwire-host` also keeps a shared command log, `~/.zwire/hostlog.jsonl`, recording every message it sends and receives (the sysinfo stream included), which the HOST internal page reads back through a `hostlog` command.

Light and dark are part of the same single source. Every HUD surface can toggle light mode — the `⌘K` palette carries a “Toggle light mode” command, the shared command palette from `zgui-core` adds its own light/dark entries, and the app-shell settings modal has a switch — and each writes the light flag through to `global.toml` exactly the way a scheme change does, so light toggled from any surface flows to the one file and every renderer follows. On the fork, the native chrome follows too: the toolbar, tabstrip, and frame read the `hud-light` projection and repaint on launch, so the whole browser, native chrome included, honors one light/dark choice (Chapter 21).

The background worker's full job list

The scheme relay is one of a dozen things `background.js` does. It is a single MV3 service worker — no persistent page, woken by events and suspended when idle — and it is the coordinator every content script leans on for anything a page cannot do itself. Its own header comment names only the first two jobs (relay the scheme to the native host, mirror it into `chrome.storage.local`), but reading the file top to bottom the worker turns out to own nine distinct responsibilities, each wired to its own set of Chromium events:

- **The scheme fan-out** (Chapter 8's subject): `mirror(scheme)` writes `zb_scheme` and calls `pushToZpwr`, which relays to the power-tool over `runtime.sendMessage` with the `lastPushed` echo guard.
- **The native seed on startup**: `seedFromNative` sends `{cmd: 'get'}` to `com.zwire.hud` and mirrors whatever scheme the file already held, so a scheme picked in a previous session survives a browser restart. It is registered on both `onStartup` and `onInstalled`, and called immediately, because the file “may already hold a scheme picked in a prior session or written before launch.”
- **The `⌘K` command owner**: the worker registers `open-palette` as a `chrome.commands` shortcut (`Command+K` on macOS, `Ctrl+K` elsewhere) and routes it to whichever palette matches the active tab — the content-script palette on an ordinary page (`tabs.sendMessage`), the new-tab extension's palette (cross-ext `sendMessage`), or a HUD page's own palette (a runtime broadcast). Its comment spells out why the browser command must be the single owner: the new-tab page reserves `⌘K` at the browser level before any page JS sees it, so a page listener could never open the palette there, but a `chrome.commands` shortcut fires regardless of focus — at the cost of consuming `⌘K` on every tab, which is why the routing lives here.
- **The sysinfo stream** (Chapter 11): a persistent `connectNative` port that sends `sysinfo_start`, writes each `{sys:{...}}` frame into `zb_sys`, and reconnects five seconds after any disconnect. The comment notes a second reason the port is held open: it keeps the MV3 worker itself alive.
- **The per-tab PTY relay** (Chapter 16): the `zwire-pty` connection handler, covered in its own section below.
- **Four storage mirrors**: the worker keeps `zb_tabs` (every open tab), `zb_exts` (enabled extensions, via `chrome.management`), `zb_shortcuts` (every extension's keyboard shortcuts, via `developerPrivate` — `chrome.management` does not expose commands), and `zb_frecent` (a frequency-scored history list) continuously fresh, each wired to the relevant Chromium event (`tabs.onUpdated`, `management.onInstalled`, `history.onVisited`, and so on). These exist because a content-script palette cannot call `chrome.tabs` or `chrome.management` itself, and — the comment is explicit — “an async worker round-trip” from a page is unreliable when the worker may be asleep, so the worker publishes to storage eagerly and the page reads storage directly.

- **The custom-command seed:** `seedCustomCmds` populates the `%%K` keyword-command registry once, guarded by a `zb_cmds_seeded` flag, “merging defaults with any existing user entries” so a user’s edits and deletions are never clobbered. (Chapter 12 and Appendix H.)
- **The CI monitor:** a GitHub-Actions poller on a `chrome.alarms` heartbeat, covered below.
- **A second tiling engine:** a `chrome.windows`-geometry tmux implementation that tiles *real OS windows* as panes, driven from the same `zb_cmd` bus, distinct from the in-page iframe overlay of Chapter 10.

The frequency list is worth pinning down because the same formula appears in three places (the worker, the new-tab palette, and the HUD pages’ `zg-boot.js`): a `chrome.history.search` over the last 90 days, scored $(\text{visitCount} + 2 \cdot \text{typedCount}) / (1 + \text{ageDays} \cdot 0.3)$, `chrome://` URLs filtered out, top 30 kept. A typed visit counts double a clicked one, and the recency divisor decays a site’s score by roughly a third per day of age, so a page you typed yesterday outranks one you clicked ten times last month.

The storage command bus

The recurring problem the worker solves is that an MV3 service worker can be suspended at any instant, and a `runtime.sendMessage` to a sleeping worker is not guaranteed to wake it in time. The HUD’s answer, used by vim, the palette, and the tmux overlay alike, is the **storage command bus**: a content script writes a `zb_cmd` object into `chrome.storage.local`, and `storage.onChanged` is a reliable MV3 wake-up, so the worker always runs the command. The worker’s comment states the contract directly — “content scripts write `zb_cmd` to storage; `storage.onChanged` is a reliable MV3 wakeup (unlike `sendMessage` to a sleeping worker), so palette navigation / tab-switching always executes.”

A `zb_cmd` carries an action field `a` and a monotonic tag `n` (so a repeated identical command still registers as a change), plus any parameters. The worker’s `onChanged` handler dispatches on `c.a` across a broad tab-and-window vocabulary: `open/openTab` (navigate or new tab), `activate` (switch to a tab and focus its window), `newTab/newWindow/duplicateTab/reopenTab`, `pinTab/muteTab`, `closeTab/closeOthers`, `nextTab/prevTab`, `tmux` (handed to the worker’s own tiling engine), and `ping` (a wake-and-refresh that re-runs all four storage mirrors). This is why a vim `x` can close the current tab and a palette selection can switch to another window even though neither the vim script nor the palette script can touch `chrome.tabs`: they write one storage key and the worker, woken by the write, does the privileged work. The same `onChanged` handler also fans a `zb_scheme` write out to the power-tool, so a scheme picked on the Settings page — which writes storage and the native file but never messages the worker — still reaches `zpw:chrome`.

The per-tab PTY relay, keyed by tab

The terminal’s bytes flow through the worker because a content script cannot open a native-messaging port that survives navigation. The worker exposes a `zwire-pty runtime.onConnect` port and keeps a `ptySessions` map **keyed by tab id**, so — the comment says — “the shell SURVIVES page navigation.” Each session holds its own `connectNative` port to `com.zwire.hud`, the content-script port currently attached (or `null`), an output buffer, and a `spawned` flag. Three details make it robust: output produced while no page is attached is buffered (capped at 500 messages) and flushed on reconnect; `pty_spawn` is honored only once per tab (`if (sess.spawned) return`), so a re-injected content script re-attaches to the running shell instead of forking a second one; and a content-script disconnect sets `sess.cs = null` rather than killing the native port, “to keep the shell alive across nav,” while an actual `tabs.onRemoved` tears the native port down. The result is a shell session bound to a browser tab, not to a page — you can click a link, load a new site, and keep typing in the same zsh.

The CI monitor: a poller, an alarm, and a desktop notification

The one worker feature with no page-facing counterpart in the earlier chapters is a continuous-integration monitor. On a `chrome.alarms` heartbeat named `zb-ci-poll` (`periodInMinutes: 10`) the worker runs `pollCi`, which walks your GitHub repositories through the REST API — `/user/repos` with a token, else `/users/<user>/repos` for the default account `MenkeTechnologies` — fetches the latest workflow run per repo, and tallies failing, running, and passing counts into a `zb_ci_status` object the statusbar can show as a dot. The request budget is tuned to GitHub's anonymous cap: the comment “stay under the 60/hr anon cap” sits next to a repo limit that rises from five to twelve only when a token is present. When the failing count rises above the previous poll, and `chrome.notifications` exists, the worker raises a desktop notification titled “CI failing” whose body lists the first four broken repos, drawn with a 16-pixel hot-pink `CI` tile inlined as a base64 SVG “so notifications never depend on an icon file.” This is the only `chrome.alarms` use in the extension, and the only place the HUD reaches out to a network service on its own initiative; the CI page of Chapter 14 is its interactive front end, and the two share the `zb_ci` config key.

The other tmux: tiling real OS windows

There is a second tiling engine in the worker, and it is easy to miss because it shares the `tmux` vocabulary with the in-page overlay of Chapter 10 while doing something entirely different. The overlay tiles *iframes inside one page*; the worker's engine tiles *real Chromium windows* by geometry. Its own comment states the model: “PANES = real browser windows, tiled by `chrome.windows` geometry. WINDOWS (tmux) = named groups of panes; only the active group is visible, the rest are minimized. SESSIONS = the whole thing, persisted to storage.” State lives in a `TMUX` object mirrored to `zb_tmux_state` so it survives a worker eviction, and the tiling area comes from `chrome.system.display`'s primary work area (falling back to 1440×820 when the display API is unavailable).

The engine is driven from the same `zb_cmd` bus (`{a:'tmux', sub:...}`) and its subcommands mirror `tmux`: `split` (down splits into rows, else columns), `navigate`, `zoom`, `closePane`, `newWindow`, `nextWindow/prevWindow`, `selectLayout` (cycling `cols`, `rows`, `main-v`, `grid`), `syncToggle`, `killWindow`, and `retile`. `ArectsFor(n, layout)` helper computes the pixel rectangles — a 60% main pane for `main-v`, a $\lceil\sqrt{n}\rceil$ -column grid otherwise — and `tile` minimizes every window not in the active group, positions the rest, and focuses the active one. Because real windows can be closed or focused outside the engine's knowledge, a `reconcile` step drops closed panes and prunes empty groups on `windows.onRemoved` and `onFocusChanged`, and an `ensureInit` rebuilds the state from `zb_tmux_state` after an eviction, defaulting to “the current window is the sole pane.” The engine even has its own synchronize-panes: a `zbSync` message fans a keystroke out to the active tab of every sibling window in a syncing group. It is a genuine window manager for your browser windows, living in the service worker, distinct from and parallel to the in-page overlay — two answers to “tmux in the browser,” one tiling pages and one tiling windows.

The color science under the eight names

The eight schemes are not eight arbitrary palettes; they are eight *coordinated* ones, and the coordination is what lets a single content script skin an arbitrary `chrome://` page without auditing it first. Each scheme is a small closed set of roles — a background, a raised surface, a foreground, a dimmed foreground, an accent, and a warning — and the skinning layer paints roles, not pixels. That indirection is the reason the same `theme.js` that colors `chrome://history` also colors `chrome://settings` and every HUD internal page: the page asks for `--zw-accent`, not for `#00e5ff`, and the scheme table decides what the accent is this minute. Change the scheme and every surface that referenced a role moves in lockstep, because there was only ever one place the literal color lived.

The dark schemes share a discipline that keeps neon legible against black, which is the hard case a cyberpunk

palette must survive. A saturated cyan or magenta at full value on a near-black field is the classic mistake — it blooms, it strobes on LCD subpixels, and long-form text set in it is unreadable. The schemes avoid it by never using the accent as body text: the accent lives on borders, the active tab, the statusbar segments, the cursor, and the focus ring, where a thin run of high chroma reads as *signal* rather than as a wall of glow, while the body foreground is a high-value near-white or near-cyan held well below full saturation. The eye reads the accent as an edge and the foreground as a plane, and the two never compete. This is why a scheme can be maximal in name — **matrix**, **synthwave**, **cyberpunk** — and still be a working reading surface: the loud color is rationed to the geometry, not spent on the prose.

The light schemes are the harder engineering, and the book is honest that they came later. Inverting a dark cyberpunk palette does not produce a usable light one — the same cyan that reads as signal on black is nearly invisible on white, and a magenta that glows on a dark field turns muddy on a bright one. The light variants therefore re-derive each role against a light ground rather than flipping a value: the background rises to a warm off-white, the accents drop in value and rise in saturation so they still carry against the paper, and the dimmed foreground — the one role most likely to fail a contrast check — is pinned to a value that clears the readable threshold rather than being computed as a fixed alpha over the background, because a fixed alpha is exactly what fails when the background changes from black to white. The fork's native color mixer reads the same role names from the host's **hud-light** projection, so the eight-times-two matrix stays a single source: eight names, a dark and a light derivation of each, one set of roles, and one table that every surface — content script, new tab, fork chrome, and DevTools dropdown — reads through. The palette is not decoration bolted onto the browser; it is a small, closed color system, and the eight names are its presets.

Chapter 9 – Vim on Every Page

`zvim.js` puts real vim keybindings on every page: normal mode by default, suppressed while you are typing in an editable field, with motions, link hints, marks, tab operations, and a per-tab off switch. This chapter is the whole keyboard model, and it pairs with `zkeys.js`, the shared registry that makes every key remappable.

Normal mode, and never stealing keys

The script installs one capture-phase `keydown` listener on the document and decides, per key, whether it owns it. The first rule is restraint: if the active element is editable (an `input`, `textarea`, `select`, or `contentEditable`), vim does nothing except let `Escape` blur the field — so typing in a search box or a comment box is never intercepted. With any of `Alt/Meta/Ctrl` held (except its own combos) it also bows out. And crucially, an *unknown* key is left alone: the listener only calls `preventDefault` when the key maps to a real action, so keys vim does not claim reach the page normally. When it *does* act, it calls both `preventDefault` and `stopImmediatePropagation`, so a single-key site shortcut (a Gmail `j`, say) does not also fire.

Motions, screens, and the z-family

The core motions are the vim ones, each a small `scrollBy/scrollTo`:

- `j/k` scroll down/up, `h/l` scroll left/right (66px steps), `d/u` half-page down/up.
- `gg` (a two-key sequence via the `g` prefix) jumps to the top; `G` to the bottom.
- `H/M/L` jump to the high/middle/low of the *document* — the vim screen motions, mapped here to top/middle/bottom of the page.
- The `z`-family (`zt/zz/zb`, via the `z` prefix) positions a reference element to the top/center/bottom of the viewport. The reference is the focused element if it is real and on-screen, else whatever is at the center of the window — computed with `getBoundingClientRect` and a `scrollBy`.

Two-key sequences are handled with a small pending-key state and a 900ms timeout: pressing `g`, `z`, `y`, `m`, or ``` arms a prefix, and the next key completes it (or the prefix expires). This is how `gg`, `gt/gT` (tab next/prev), `gi` (focus the first input), `yy` (yank the URL to the clipboard), `m<x>` (set a mark), and ``<x>/'<x>` (jump to a mark) all work from single-character keys without colliding.

Link hints

`f` and `F` enter hint mode. The script collects every clickable element in the viewport — `a[href]`, `button`, form controls, `[role=button]`, `[onclick]`, `[tabindex]` — filters out the invisible and off-screen ones, and overlays a short letter label on each (one letter when there are few targets, two-letter combinations when there are many, drawn from a home-row-first alphabet). You type the label; a unique match activates immediately. `f` clicks in place; `F` opens links in a new tab (routed through the background worker). Editable targets get focused rather than clicked, and `Escape` or a no-match cancels. Highlighting the labels is a fixed-position overlay themed in the HUD yellow, with the active filter shown in cyan.

Tabs, history, marks, and the storage bus

Tab operations cannot be done from a content script directly — a page cannot close or switch its own tab — so they route through the background worker over what the code calls the storage command bus: `zvim.js` writes a `zb_cmd` object into `chrome.storage.local`, which reliably wakes the MV3 worker (more reliable than a `runtime.sendMessage` to a suspended worker), and the worker performs the tab action. `J/K` switch tabs, `x` closes, `t` opens, `r` reloads, `[/]` are back/forward. Marks are per-page scroll positions kept in memory. A small toast in the corner confirms actions like a yank or a mark set.

The keybinding registry and remaps

Every default key above lives in `zkeys.js`, `window.ZWIRE_KEYMAP` — the single source of truth, loaded both as a content script (before `zvim/zpalette/zfind`) and by the Keyboard settings page so both render and dispatch from the same list. It is a categorized table: Scroll, Tabs & history, Jump & hints, Palette & find, and a large Tmux category (every post-prefix key), plus global chorded hotkeys (`⌘K`, `⌘F`) and the fixed native prefix. User remaps live in `chrome.storage.local` under `zb_keys` as `{action: key}`; `zvim` builds its `key` → `action` map by merging those over the defaults and rebuilds it live when the key changes. So rebinding `scrollDown` from `j` to `s` in the settings page immediately changes what `zvim` fires — and, because `ztmux.js` reads the same registry, the same mechanism remaps the tmux post-prefix keys too. `Ctrl/⌘+\` toggles vim off for the tab (and back on), for the moments you want the page's raw keys.

The handler, in the order it decides

The whole of `zvim.js` is one capture-phase `keydown` listener on the document — registered with the `true` third argument so it sees keys before the page does — and reading it is reading a sequence of guards in a deliberate order. First, if vim is disabled for the tab, only `Ctrl/⌘+\` (which re-enables it) is honored and everything else returns. Second, hint mode: while link hints are on screen, the listener routes the key to the hint filter before any other rule, so typing a hint label is never mistaken for a motion. Third, the editable guard: inside an `input`, `textarea`, `select`, or `contentEditable` element only `Escape` acts (it blurs the field) and every other key passes through untouched. Fourth, the modifier bailout: any key held with `Alt`, `Meta`, or `Ctrl` is left alone, because those chords belong to the palette, the find bar, and the vim toggle, not to motions. Only after all four does the key reach the prefix state machine and the keymap. The listener is scrupulous about *not* stealing keys: it calls `preventDefault` only when the key maps to a real action — an unknown key gets no interference — and when it does act it calls both `preventDefault` and `stopImmediatePropagation`, whose comment reads “win over site single-key shortcuts,” so a Gmail `j` fires the vim scroll and not Gmail's own handler.

The exact motion amounts are small and fixed: `j/k/h/l` scroll 66 pixels; `d/u` scroll half the viewport height (`innerHeight · 0.5`, rounded); `G` and `L` both jump to the document bottom (they run identical code); `H` to the top; `M` to the vertical middle of the document. The `z`-family positions a *reference element* — the focused element if it is real and on screen, otherwise whatever sits at the center of the window (`elementFromPoint` at the viewport midpoint) — to the top, center, or bottom of the viewport, computed from its `getBoundingClientRect` with a four-pixel margin.

The two-key prefix machine, marks, and yank

Multi-key commands run through one small state variable and a **900-millisecond** timeout. Pressing `g`, `z`, `y`, `m`, or ``` arms that prefix; the next key completes it or, after 900 ms of silence, the prefix simply clears. Under `g`: `gg` scrolls to the top, `gt/gT` switch to the next/previous tab (through the storage bus), and `gi` focuses the first non-hidden input on the page. Under `y`, `yy` copies the page URL to the clipboard and flashes a toast. Under `m`, `m<x>` records the current scroll position into an in-memory `marks` object keyed by the character you pressed;

under ``` (and `'`, which is aliased to it), ``<x>` scrolls back to that saved position. Marks are per-page and live only in memory — they are scroll bookmarks for the current page, not durable named locations. The toast that confirms a yank or a mark is a single fixed-position element in the corner, themed in the HUD cyan on a translucent black, faded out after 1.2 seconds.

Link hints, precisely

`f` and `F` enter hint mode. The script collects everything plausibly clickable — the selector is `a[href], button, non-hidden input, select, textarea, [role=button], [role=link], [onclick],` and `[tabindex]` — then discards anything smaller than two pixels, anything scrolled off screen, and anything with `visibility:hidden, display:none,` or zero opacity. Each survivor gets a label drawn from a home-row-first alphabet (`asdfghjklqwertyuiop...`): one letter while there are 26 or fewer targets, two-letter combinations beyond that. Labels render as fixed-position yellow chips (cyan once you have typed a matching prefix). Typing narrows the set; the moment exactly one label matches, that target activates. `f` clicks in place; `F` routes a link through the storage bus to open in a new tab; an editable target is *focused* rather than clicked; `Escape` or a dead-end keystroke cancels. The hint overlay sits at z-index 2147483645, one below the find bar and palette, so those always draw over it.

One registry, live remaps

Every default key above is data in `zkeys.js` (`window.ZWIRE_KEYMAP`), and `zvim.js` never hardcodes a key — it builds a `key → action` table by walking the registry's categories and letting a user override win over each default. The overrides live in `chrome.storage.local` under `zb_keys` as `{action: key}`, and the script both reads them at load and re-listens on `storage.onChange`, so rebinding `scrollDown` from `j` to `s` on the Keyboard settings page changes what fires on every open page the instant you save it, with no reload. The same registry and the same `zb_keys` key drive the tmux overlay's post-prefix keys, which is why one settings page remaps both. The per-tab off switch (`Ctrl/⌘+\`) flips an in-memory flag rather than a stored one, so it is deliberately transient — a page where you have turned vim off gets its raw keys back, and the next page starts with vim on again.

Chapter 10 – tmux in the Browser

`ztmux.js` is the most ambitious content script in the extension: a real, in-page tiling window manager modeled on tmux and zellij, where a session holds windows which hold panes, the panes are actual webpages rendered in iframes, and you split, zoom, swap, rotate, resize, lay out, copy, paste, and broadcast-type across them — all without a single OS window. At 777 lines it is a small application; this chapter is its model and its command set.

The session/window/pane tree

The whole state is one in-memory object: a session `S` with a list of `windows` and an `active` index. Each window has a binary *tree* of nodes — a `leaf` is a pane (an id, a url, a title), and a `split` has a direction (`row` = side by side, `col` = stacked), a `ratio`, and two children `a/b`. Nesting is arbitrary: a split's child can be another split, so panes tile to any depth. This is the same data model tmux uses internally, rebuilt in JavaScript, and every command is a tree edit:

- `splitPane(dir)` replaces the active leaf, in place, with a new split whose children are the old leaf and a fresh pane.
- `closePane()` finds the active leaf's containing split and promotes its sibling into the split's slot (collapsing the split), killing the window if it was the last pane.
- `swapPane`, `rotatePanes`, and `breakPane` (move a pane out to its own window) are all slot swaps in the tree.
- `resizePane(dir, amt)` walks from the active leaf up to the first ancestor split on the right axis and nudges its `ratio`, clamped to `[0.08, 0.92]`.

A `nodeSlot(root, target)` helper returns the `{parent, key}` such that `parent[key] === target`, and that little abstraction is what makes every structural command a two-line edit rather than a special case.

Panes are iframes, which is why the fork strips X-Frame-Options

Each pane renders a real webpage in an iframe. The catch is that most sites send `X-Frame-Options` or a frame-ancestors CSP that forbids being framed — so `hud-internal` ships a `declarativeNetRequest` rule (`rules/frame_bust.json`) that strips those headers, and on the fork the native `ancestor_throttle.cc` patch (0008) does the same at the engine level. That is the whole reason patch 0008 exists: without it, the tmux overlay could only tile a handful of framing-friendly sites. Fresh panes open the zwire new tab, and because `chrome://newtab` itself cannot be iframed (`chrome://` pages are un-embeddable), the overlay frames the new-tab *extension*'s web-accessible page directly — the same page `chrome://newtab` redirects to.

The overlay runs in *all* frames, and the roles differ. The top frame hosts the overlay UI, the tree, and all the commands. A pane iframe (marked with `window.name='zbtmux'`) runs a thin forwarder: it catches the `Ctrl-b` prefix and the following command key and `postMessages` them up to the top frame, so the prefix works even while your focus is inside a framed page. The prefix is `Ctrl-b` (the real tmux one) or, because the pre-rebuild fork's native split patch used to eat `Ctrl-b`, `⌘-b` (Alt-b) as an alternate nothing else intercepts.

The command set

After the prefix, the keys are the tmux ones, and every one is remappable through the shared registry (Chapter 9). The full set from `zkeys.js`:

```
% split right      " split down      o next pane      ; last pane
h/j/k/l select pane ← ↓ ↑ →      H/J/K/L resize pane ← ↓ ↑ →
z zoom pane        x close pane      { } swap ← →      0 rotate
! break to window  q pane numbers    space cycle layout
F/G/I grid 4/8/16
c new window        n/p next/prev win  Tab last window    w window list
, rename window     . move window      ' go to window      & kill window
m mark / swap       e synchronize panes E toggle this pane in sync group
[ copy mode         ] paste buffer    = choose buffer
s sessions          S save session
r reload pane       t clock           : palette          d detach  ? help
0-9 select window  ←→↑↓ focus by direction (Ctrl/Alt + arrows resize)
```

Layouts (`space`) cycle through even-horizontal, even-vertical, main-vertical, main-horizontal, and tiled — each rebuilt by `buildEven/cycleLayout` from the current leaf list. Directional focus (`navDir`) uses the rendered pane rectangles to pick the nearest pane in a direction, so the arrows move focus spatially like tmux's `select-pane -LRUD`.

Copy mode, paste buffers, and synchronize-panes

Because panes are real scrollable documents, copy mode (`[`) is a genuine scroll-and-select mode inside the focused pane: `j/k/d/u/g/G` scroll, you select text, and `y/Enter` yanks the selection up to the overlay's paste-buffer stack (most-recent-first, capped at 30). `]` pastes the most recent buffer into the focused pane's field, and `=` opens a chooser. Any ordinary copy in a pane is also captured into the stack.

Synchronize-panes (`e`) is the broadcast-typing feature, and it is careful about line editing. When sync is on, a pane forwards each editable keystroke up to the top frame, which relays it to every other pane's focused (or last-focused) field. Plain characters, `Enter`, and `Backspace` forward as-is; the readline combos forward as *semantic tokens* so the broadcast stays correct — `C-w` (kill word) and `C-u` (kill to line start), plus their macOS `⌘/⌘-Delete` twins. Each receiving pane applies the token cursor-aware, deleting the right span and firing a real `input` event so the framed page's own handlers see the change. This is how you type one command into eight terminals-in-tabs at once.

Persistence

The whole tree lives in memory, so a reload of the host page would wipe it. `ztmux.js` snapshots the windows, panes, layout, and paste buffers to per-tab `sessionStorage` (`zbtmux-session-v1`) and restores on re-inject, so the tmux session survives a page reload — the tree, the active window and pane, and the buffers all come back. It is themed entirely with the active scheme's CSS variables, so the tabs, pane borders, dividers, and modals match cyberpunk, ember, matrix, and the rest live. The overlay reserves the bottom 22px for the real powerline statusbar, which is the next chapter.

Resize and layout, as tree math

Resizing walks the tree. `resizePane(dir, amt)` picks an axis from the direction (`left/right` → a `row` split, `up/down` → a `col` split), builds the path from the active leaf up to the root recording which side each descent came from, and nudges the ratio of the *first* ancestor split on that axis, clamped to `[0.08, 0.92]`. The nudge size depends on how you asked: a plain `Ctrl+arrow` resizes by 0.03, `Alt+arrow` by 0.08, and

the remappable `tmux-resize-*` keys by 0.05; dragging a divider with the mouse is the exception, clamped instead to `[0.06, 0.94]` from the pointer's position within the split. The clamp is what keeps a pane from ever collapsing to nothing or swallowing its sibling.

The five layouts (`space` cycles them) are each a fresh tree built from the current flat list of leaves. `even-h` and `even-v` recursively split the list at its median so the panes tile evenly across a row or a column. `main-v` gives the first leaf 60% of the width and stacks the rest in a right-hand column; `main-h` gives the first leaf 60% of the height and lays the rest in a row beneath — the classic tmux main- pane layouts. `tilled` groups the leaves into rows of `√n` columns. There is a separate grid path, too: `tmux-grid-4`, `tmux-grid-8`, and `tmux-grid-16` (bound to `F/G/I` after the prefix) pad the window with blank panes up to 4, 8, or 16 and build a square-ish tile — a one-key way to summon an even wall of panes.

Directional focus (the arrow keys, `navDir`) is geometric, not tree-order. It reads the rendered percentage rectangle of each pane, takes the active pane's center, and among the panes strictly on the requested side (with a one-unit epsilon) picks the one whose center is nearest by straight-line distance. There is no wrap: if nothing lies to the left, the left arrow does nothing. That is the same “nearest pane in a direction” rule tmux's `select-pane -LRUD` uses, rebuilt from the panes' on-screen geometry.

The forwarder protocol, and typing into eight terminals

Because each pane is a cross-origin iframe, the overlay cannot listen for keys inside a pane from the top frame — so the pane runs a thin forwarder and the two frames talk by `postMessage`. Every message from a pane up to the top frame carries a `__zbtmux` tag and one of a small set of shapes: `{prefix:1}` (the `Ctrl-b/⌘-b` prefix was pressed inside the pane, so the top frame should arm), `{cmdKey, ctrl, alt}` (the post-prefix command key), `{palette:1}` (`⌘K` inside a pane, so the palette opens in the top frame and drives the active pane rather than opening a stray tab), `{yank:...}` (a copy-mode selection), and `{synckey:...}` (a broadcast keystroke). The top frame replies downward with `{setSync:...}`, `{copyMode:true}`, `{pasteText:...}`, and `{syncapply:...}`. This is why the prefix works while your cursor is inside a framed site: the pane catches it and hands it up.

Synchronize-panes (`e`) is the most careful piece. When sync is on and you are typing in an editable field, the pane forwards each keystroke up as a `synckey` token, and the top frame relays it to every other pane in the sync group. Plain characters, `Enter`, and `Backspace` travel as themselves; the line-editing combos travel as *semantic* tokens — `Ctrl-w` and `Alt-Backspace` both become `C-w` (kill word), `Ctrl-u` and `⌘-Backspace` both become `C-u` (kill to line start). Each receiving pane applies the token cursor-aware against its focused (or last-focused) field: `C-w` walks back over trailing whitespace and then a run of non-whitespace and deletes that span; `C-u` deletes back to the start of the current line (`lastIndexOf('\n')`); a plain character is spliced in at the caret. Every edit dispatches a real `input` event so the framed page's own handlers see the change. That token discipline is what lets one command typed once land correctly in eight different sites' input fields at once, even when their fields disagree about selection state.

Copy mode, paste buffers, and the chooser

Copy mode (`[`) runs inside the focused pane, because the pane is a real scrollable document. A magenta indicator appears, `j/k` scroll by 64 pixels, `d/u` by half the viewport, `Space/b` by nine-tenths of it, `g/G` jump to the ends, you select text with the mouse, and `y` or `Enter` yanks the selection up to the overlay's paste-buffer stack. The stack is most-recent-first, de-duplicated, and capped at 30 entries; any ordinary copy inside a pane is captured into it too. `]` pastes the most recent buffer into the focused pane's field, and `=` opens a chooser — a modal listing each buffer's first 90 characters, selectable by click or by digit. The buffers persist with the rest of the session.

Two kinds of persistence

The overlay has two separate persistence layers, and they answer two different questions. A per-tab `sessionStorage` snapshot under `zbtmux-session-v1` keeps the live tree, the active window and pane, the open flag, and the paste buffers, so a reload of the host page restores exactly where you were — this is the “survive an accidental refresh” layer, and it migrates an older per-window `sync` boolean to the newer `sync-member-id` array on the way in. The second layer is *named* sessions: `chrome.storage.local` under `zb_tmux_sessions` holds durable, user-named layouts (each a list of windows, each a list of panes with their URLs and titles), created with `S`, listed and loaded with `s`, and managed in full on the Sessions page of Chapter 14. Loading a named session writes a `zb_tmux_load` trigger the live overlay watches, rebuilds every window’s tree from its saved pane list, and re-opens. One layer is a crash-recovery snapshot; the other is a library of layouts you keep.

The rest of the command surface

The window and pane operations round out the set, and every one is a tree edit or a small modal. Windows: `c` new, `n/p/Tab` next/previous/last, `,` rename, `.` move (renumber), `'` go-to by prompt, `w` list, `k` kill, and `0–9` jump. Panes: `q` overlays a large number on each pane (type the digit to focus it, auto-dismissed after three seconds), `m` marks the active pane and swaps it with a previously marked one, `!` breaks a pane out into its own window. `r` reloads a pane by re-assigning its `src`; `t` shows a full-screen clock ticking every second; `?` opens a help card built from the same keymap the settings page renders, so the help can never drift from the actual bindings; `d` detaches (hides) the overlay. The overlay writes a small `zb_tmux` record to storage on every change so the real powerline statusbar — a separate content script in the top frame — can show the window/pane counts and light the prefix indicator without the overlay drawing its own bar.

The pane sandbox and the z-index ladder

Each pane iframe is deliberately sandboxed, and the exact token set is the interesting part. The overlay sets `sandbox="allow-scripts allow-same-origin allow-forms allow-popups allow-popups-to-escape-sandbox allow-modals allow-downloads allow-top-navigation-by-user-activation allow-storage-access-by-user-activation allow-presentation"` — a broad grant so framed sites behave like real tabs, with one pointed omission: it does not grant automatic top-navigation, only the user-activation form. That omission is what defeats JavaScript frame-busting scripts that try to break out by reassigning `top.location` on load; combined with the `frame_bust` header-strip of the earlier section, it is what lets the overlay hold an arbitrary site in a pane without the site either refusing to be framed or escaping the frame. Panes also get `allow='clipboard-read; clipboard-write; fullscreen'` so copy, paste, and fullscreen work inside them.

The overlay’s stacking is a fixed z-index ladder, tuned so the right thing always draws on top: the whole overlay sits at 2147483640, a pane’s click-cover at 2, the drag dividers at 6, the pane-number badges at 40, a pane’s autocomplete at 45, the drag shield at 50, and modals at 60; the copy-mode indicator inside a pane climbs to the maximum 2147483647 so it is never hidden by page content. Every surface is themed from the active scheme by injecting the scheme’s variables into an `#zbtmux` CSS prefix — so the tabs, pane borders, dividers, modals, and the marked-pane accent all recolor live when you switch schemes, with the marked pane drawing an accent border and a ♦ and a synced pane a cyan border. It is a small windowing toolkit’s worth of CSS, all keyed to the same twenty variables the rest of the HUD uses.

Now a shared component: `ZGui.tmux`

The overlay described above began as `ztmux.js`, a bespoke content script living only in `hud-internal`. It has since been promoted into the shared `zgui-core` toolkit as `ZGui.tmux` — the same tiling window

manager, now the common one the house Tauri apps consume too — and the HUD consumes it rather than carrying its own copy. The model, the command set, the copy-mode and synchronize-panes behavior, and the persistence are all unchanged; what moved is *where the code lives*. In the manifest the overlay is now two thin scripts around the shared WM: the top frame loads `ZGui.tmux` and its dependencies (the shared split-pane, modal, toast, and button-bar widgets) plus `ztmux-config.js`, the provider that calls `ZGui.tmux.init` with iframe panes (an address bar, `openEmptyPane/renderPane`, and `applyKey/copyMode/paste` bridged over `postMessage`) and reads prefs from `chrome.storage`; and every frame loads `ztmux-pane.js`, the pane-side forwarder extracted from the old script, which relays the prefix, the synced keystrokes, and yanks up to the top frame and runs copy-mode locally over the pane's page. `zgui-core` v1.0.18 added the external-feed API (`prefix/key/relaySync/yank`) precisely so a cross-origin-iframe consumer like this one can feed the shared WM from outside. The retired `ztmux.js` is left in the tree for reference but is no longer injected.

Chapter 11 – The Powerline Statusbar

`zstatus.js` pins a tmux/powerline statusbar to the bottom of every top-level page. The left side is the HUD's own indicators; the right side is a live system monitor fed by the native host. This chapter is its segments, its powerline geometry, and the stream behind the numbers.

Left and right

The bar sits at `position:fixed; bottom:0; height:22px`, top frame only, themed by the active scheme, in the Share Tech Mono face. Reading left to right:

- **Left:** a `ZW` badge, the `C-b` prefix indicator (lit while a tmux prefix is armed), the current scheme name, the `VIM` status, and `⌘K`.
- **Right:** the machine, streamed from the native host into `chrome.storage`'s `zb_sys` key — CPU, MEM, SWAP, DISK, IO, NET, LOAD, UP (uptime), a temperature, BATT, LAN IP, WAN IP, host name, and a clock.

Each right-side segment is a `<span>` with a `data-*` hook the refresh function writes into: `data-cpu`, `data-mem`, `data-swap`, `data-disk`, `data-io`, `data-net`, `data-load`, `data-up`, `data-temp`, `data-batt`, `data-lip` (LAN), `data-host`, `data-clock`. The refresh formats each from the `zb_sys` object — CPU as a percentage, memory and swap as used/total, disk as a percent, IO and NET as per-second rates with `↑/↓` and `R/W`, battery with a charging-vs-on-battery glyph and a percentage, uptime humanized, and the clock ticking every second.

Powerline chevrons

The bar draws true powerline chevrons, not glyph characters, using CSS borders. Each block is a `.plr` with a triangular `:before` pseudo-element formed from transparent top/bottom borders and a solid side border, so the segments interlock as arrows. Left-of-the-spacer segments point right (`►`) and right-of-the-spacer segments point left (`◄`), and consecutive blocks alternate between two background shades (`s0/s1`) so the chevrons read as distinct steps. This is exactly how a tmux powerline theme is built in the terminal, rebuilt here in CSS so the browser statusbar looks like the shell statusbar the rest of the stack uses.

The stream behind the numbers

The numbers are real because `zwire` is a desktop app with a native host that can read the machine. `zstatus.js` does not sample anything itself; it renders whatever is in `zb_sys` and re-renders when that key changes. The native host fills it: Chapter 15 covers `zwire-host`'s `sysinfo_start` mode, which samples the machine through the `sysinfo` crate on a background thread and streams the result, which the background worker writes into `zb_sys`. So the statusbar is a genuine system monitor — CPU load, memory pressure, network throughput, battery, temperature — pinned to the bottom of every web page, not a decorative mock. The bar is toggled from the `⌘K` palette, and (like everything HUD) recolors instantly with the scheme.

Every segment, and how it is formatted

Each right-hand segment reads one field of the `zb_sys` object the worker writes and formats it with a small, purpose-built formatter, so the numbers read like a tmux theme rather than raw JSON. CPU is a whole-number percent; MEM and SWAP are used-over-total run through a byte-humanizer that steps `B/K/M/G/T` in powers of 1024 and shows one decimal below 100; DISK is a percent; IO is `R<rate> W<rate>` and NET is `↑<rate> ↓<rate>`, both per-second and both humanized the same way; LOAD is the three load averages joined by spaces; UP humanizes uptime as `Xd Yh Zm`; the temperature is `<n>°C` and, when the machine reports no sensor, that segment hides itself entirely rather than showing a dash; BATT prefixes a charging glyph (a bolt ⚡ when plugged, a battery glyph on battery) before the percent; LAN and WAN are the private and public IPs; HOST is the short hostname; and the clock, updated on its own one-second interval independent of the two-second stats stream, shows `MM-DD HH:MM:SS`. Every segment writes into a `data-*` span (`data-cpu`, `data-mem`, and so on down to `data-clock`), so a stats refresh rewrites text in place without rebuilding the bar.

The left side is the HUD's own state, not the machine: a `ZW` badge, the `C-b` prefix indicator, the current scheme name (filled from the scheme table's label), a per-window tmux readout (`i·paneCount`, the active window emphasized, a `⇄` when any window is syncing), a static `VIM` marker, and a clickable `⌘K` that opens the palette. A flexible spacer between the two halves is what splits the bar into a left group and a right group.

Powerline geometry in CSS

The chevrons are real triangles, not glyphs. The 22-pixel bar height sets up an 11-pixel transparent top and bottom border, and each segment carries a pseudo- element with a solid side border that draws the arrow: left-of-spacer blocks put a `border-left` on their `::after` so they point right, right-of-spacer blocks put a `border-right` on their `::before` so they point left, and consecutive blocks alternate between two background shades (`--bg-card` and `--bg-secondary`) so the interlocking arrows read as distinct steps. The layout function finds the spacer, walks left assigning the point-right class with an alternating shade index, walks right assigning the point-left class, and skips the badge, spacer, and prefix block. This is exactly how a tmux powerline theme is assembled in a terminal, rebuilt in CSS so the browser's bottom line matches the shell's.

The `C-b` prefix block does one extra thing: it lights up. When you press the tmux prefix, the block's background flips from dim to the scheme accent for 1.6 seconds and then fades back — a visible confirmation that the prefix armed. Because the statusbar lives only in the top frame while the prefix can be pressed inside a framed pane, the block listens two ways: directly for `Ctrl-b` keydowns in the top frame, and for the overlay's `zb_tmux.armed` storage write, which reaches it reliably across frames. The bar itself is `position:fixed` at the bottom, 22 pixels tall, top-frame only, with a blurred translucent background, and it re-themes from `zb_scheme` and re-renders from `zb_sys/zb_tmux` on every relevant storage change.

Now a shared component: `ZGui.powerline`

Like the tmux overlay, the statusbar has been promoted out of the HUD and into `zgui-core`. The bespoke `zstatus.js` is retired; the bar is now `ZGui.powerline` (shipped in `zgui-core` v1.0.17 as `powerline.js/powerline.css`) — the exact same powerline bar the house Tauri apps (`zoffice`, `zphoto`, `zpdf`) render, so the browser's bottom line and the apps' now come from one implementation. What `hud-internal` ships is a thin adapter, `zpowerline.js`, that feeds `ZGui.powerline.init` from `chrome.storage:zb_sys` becomes the system-stats segments, `zb_tmux` the window/pane segment and the armed prefix lamp, `zb_scheme/zb_ui` the scheme (injected onto `:root` so the shared bar inherits the live theme), and `zb_status` the show/hide toggle. The manifest's top-frame statusbar content script now loads `powerline.js` plus `zpowerline.js` (and `powerline.css`) in place of `zstatus.js`. Every segment, formatter, chevron, and behavior described above is preserved — it is the same bar, sourced once and shared — with `zstatus.js` left in the tree for reference but no longer injected.

The sampling contract, and what the numbers do not mean

A statusbar that shows a number implies a contract about what the number describes, and the bar's two halves make different promises. The left half is HUD state — the armed prefix, the scheme name, the pane readout — and it is exact by construction, because the thing it describes is a value the browser itself holds. The right half is a *measurement* of a machine, and measurements have sampling semantics that a two-character-wide segment cannot convey. Reading the bar correctly means knowing three things it does not say.

The first is that nothing on the right side is sampled by the page. The bar renders whatever is in the stats key and re-renders when that key changes, and the sampling happens in the host on its own thread, at the stream's cadence. There is therefore a number on screen that is always at least one sample old, and possibly much older if the stream has stalled — because a stalled stream produces no storage change, and no storage change means no re-render, which means the bar keeps displaying the last value it received with no visual indication that it has gone stale. That is the correct behavior for a bar with no room for a staleness indicator, and it is worth internalizing when diagnosing: blank segments mean the stream never started, while *frozen* segments mean it started and stopped. Chapter 36 treats the first case; the second looks identical to a working bar on an idle machine.

The second is that the clock and the stats are on different clocks in the literal sense. The clock segment updates on its own one-second interval, independent of the two-second stats stream, so the two halves of the right side are never in step: the time advances twice for every stats refresh. This is deliberate and it is the right call — a clock that only moved when the machine reported its CPU would look broken — but it means the timestamp beside a CPU reading is not that reading's timestamp. If you are correlating a spike on the bar against something else, the clock is not the sample time.

The third concerns the rate segments specifically. Chapter 26 records that network and disk throughput are computed as deltas over the elapsed interval, so the host does no work to maintain a counter between ticks and simply subtracts two readings when a tick fires. A delta over an interval is meaningless without a previous reading, which means the first sample after the stream starts — and the first after any resume — has nothing to subtract from. Whatever such a segment displays at that instant is an artifact of the implementation's choice for the undefined case, not a measurement, and it settles within one interval. The practical rule is that the first rate you see after opening a browser is not a rate.

Two smaller pieces of the contract are worth stating because they are the kind of thing a reader assumes wrongly. The temperature segment hides itself entirely when the machine reports no sensor rather than showing a dash, which is a policy decision with a real consequence for layout: the bar's segment count is machine-dependent, so a screenshot from one machine will not have the same chevron sequence as another. And the memory and swap segments show used over total through a byte humanizer stepping in powers of 1024, which is the terminal convention rather than the storage-vendor one — the same reading a shell tool would print, deliberately, because the whole point of the bar is that it looks like the tmux status line the rest of the stack wears.

The through-line is that the bar is a *monitor*, not a dashboard. A dashboard's job is to be authoritative and it earns that with timestamps, units, and staleness indicators, none of which fit in a 22-pixel strip. A monitor's job is to make a change noticeable in peripheral vision, and for that a value that is one sample old and unlabeled is completely sufficient. Knowing which of the two you are reading is what keeps the bar useful rather than misleading.

Chapter 12 — The Command Palette

The `⌘K` command palette is the HUD's universal launcher: one fuzzy-filtered list that jumps to any internal or native page, switches scheme, runs a keyword search, and surfaces your tabs and frecent history. It exists in **four** implementations, one per execution context, all sharing the same `zgui-core` engine and a single backend-agnostic item source (`palette-cmds.js`, vendored verbatim into each extension so the four can't drift):

- `zpalette.js` — the content-script palette on ordinary `http(s)/chrome://` pages.
- `pages/zg-boot.js` — the palette on the HUD's own extension pages (Settings, Sessions, Host console, and the rest). A web page gets the content-script palette; a HUD page gets this one — both are `hud-internal`, which is why it reads as one but is two.
- `newtab/palette.js` — the New Tab palette (a separate extension).
- `zpwchrome/lib/zpc-palette.js` — the palette on the `zpwchrome` power-tool's own dashboard pages (a second separate extension).

Every surface lists the same rows, including deep-links into `zpwchrome`'s tool pages (Downloads, Pass, Userscripts, Host console, ...). `hud-internal` owns `⌘K` browser-wide as a `chrome.commands` shortcut — a page keydown can never intercept it — and its worker routes each press to the palette matching the active tab. This chapter is what it lists and how it opens.

What the palette lists

Opened with `⌘K` (or `:` on a page, or `o` in vim mode), the palette registers several sources into one fuzzy list:

- **Internal pages** — dozens of `chrome://` destinations, from the everyday (extensions, settings, history, downloads, bookmarks, flags, gpu) to the deep diagnostics tail (`net-internals`, `serviceworker-internals`, `indexeddb-internals`, `webrtc-internals`, `tracing`, `histograms`, and many more), each opening in a new tab.
- **Settings sub-pages** — the “tabs” inside `chrome://settings` (Appearance, Privacy & security, Performance, Search engine, and the rest), so you can jump straight to one.
- **Schemes** — one entry per color scheme, which sets it through the native bridge.
- **Open tabs** — every tab, with its title and URL, switching to it (and focusing its window) on select.
- **Frecent history** — your history scored by visit count and recency over the last 90 days, top matches first.
- **zwire destinations** — the CI-runs page, the app store, the Chrome Web Store.

The engine is `zgui-core`'s `ZGui.palette` over `ZGui.fzf`, so typing filters fuzzily and the matched characters are highlighted. The styling is inlined into the content script (the `.palette-*` CSS) so it works on arbitrary pages, themed by the active scheme's variables.

Keyword commands (cmd-defaults.js)

Beyond jumping to pages, the palette runs *keyword commands* — a short keyword plus an argument that expands into a URL. The background worker seeds a default set on first run only (`cmd-defaults.js` and the `CUSTOM_CMD_SEED` in `background.js`), stored in a user-editable `zb_custom_cmds` array so your edits and deletions are never clobbered. The seeds are the everyday web tools:

```
cg ChatGPT      cl Claude      pp Perplexity    wa WolframAlpha
gm Gmail        gd Google Drive  cal Google Calendar
tr Translate    img Google Images  def Define word
hn Hacker News  li LinkedIn      ...
```

Typing `cg how do futures work` in the palette expands the `{q}` in the ChatGPT template with your query and opens it. Each rule is `{id, icon, label, keyword, type, value}`, where `type` is `url` (a template) or `action`, and the whole set is edited on the HUD Commands page (Chapter 14). This turns the palette into a custom search-engine bar with your own keywords, seeded sensibly and yours to change.

How it opens across surfaces

Opening the palette is the same `⌘K`-reservation dance as focus on the new tab. On an ordinary page, `zpalette.js` catches `⌘K/Ctrl+K` (and `:` when not in a field) directly in a capture-phase listener. On the new tab, Chrome reserves `⌘K` at the browser level, so the `hud-internal` background worker owns it as a `chrome.commands` shortcut and messages the new-tab page to open its own palette. And from *inside* a `tmux` pane, `⌘K` is forwarded up to the top frame so the palette opens there and navigates the pane rather than opening a stray new tab. Three surfaces, three plumbing paths, one palette — because `⌘K` is guarded differently on each and the HUD routes around each guard.

The full source catalog

The list the palette assembles is larger than the summary above suggests. The static half is three tables in `zpalette.js`: a `PAGES` set of the HUD's own replacement pages (Extensions, Settings, History, Bookmarks, CI runs, Shortcuts, Extension shortcuts, Custom commands, System info); a `CHROME` set of roughly forty-four `chrome://` destinations that runs from the everyday (downloads, flags, gpu) down the entire `-internals` diagnostics tail (`net-internals`, `serviceworker-internals`, `indexeddb-internals`, `quota-internals`, `webrtc-internals`, `media-internals`, `histograms`, `tracing`, `ukm`, `predictors`, `memory-internals`, and more), ending at a “chrome://about” entry that lists them all; and a `SETTINGS` set of the nineteen tabs inside `chrome://settings` (Appearance, Privacy, Security, Cookies, Performance, Search engine, and the rest). On top of the static tables the palette registers one entry per scheme, your open tabs (read from `zb_tabs`), your frecent history (from `zb_frecent`), your extensions (each offering a *Tweak* and a *Manage* action), and two external destinations — the Chrome Web Store and the zwire app store. Every page or tab entry routes through the storage command bus so the sleeping worker actually performs the navigation.

Keyword commands and the search providers

Beyond jumping to a destination, the palette runs *keyword commands*. Two provider functions feed the fuzzy list live as you type. The first is a built-in search table: type `crate serde` and it expands crates.io's search URL with `serde`; a bare keyword opens that site's origin. The second is your own `zb_custom_cmds`, seeded once by the worker and edited on the Commands page, and it is richer than a URL opener — a custom command's `type` may be `url` (substitute the typed argument for `{q}` and open it), `shell` (run the argument in the HUD terminal), `js` (evaluate the argument as the body of a function with the typed text bound to `q`), `action` (a built-in browser action), or `scheme` (switch the color scheme). A generic Google/DuckDuckGo fallback is always registered last, marked so the ranking keeps it beneath every real match. So the same input box is a launcher, a custom search-engine bar, a shell one-liner runner, and a tiny JavaScript scratchpad, disambiguated entirely by the leading keyword. The scheme entries and the `cycle color scheme` action both go through the worker's native bridge, so a scheme picked from the palette lands in the same `~/zwire/hud-scheme` file every other surface reads.

The provider tail: shortcuts, extensions, and a Chrome cheatsheet

Three more providers round out what the palette can surface, each drawing on a storage mirror the worker keeps fresh. A shortcut provider reads `zb_shortcuts` — every extension’s keyboard commands, published by the worker via `developerPrivate` — so you can find a binding by name. An extension provider reads `zb_exts` and, per extension, offers a *Tweak* entry (opening `chrome://extensions/?id=...`) and a *Manage* entry, so the palette doubles as an extension launcher. And a platform-aware Chrome-shortcut provider carries a curated cheatsheet of the browser’s own key bindings, keyed on whether the machine is a Mac, so typing part of a native action surfaces its real chord. None of these are static text: they are the worker’s live `zb_*` mirrors turned into palette rows, which is why the palette is never stale — it is a view over the same shared storage the statusbar, the settings pages, and the new tab all read.

Chapter 13 – Fuzzy Find

`zfind.js` hijacks `Ctrl/⌘+F` on every page and replaces Chrome's native find with the HUD's own fuzzy find bar: substring find-in-page with all occurrences navigable, an `zfind` fallback when there is no substring hit, and highlighting that never mutates the page's DOM. This chapter is that one script.

Substring first, fuzzy second

The find bar is a small fixed-position input themed by the active scheme. As you type, it first does a plain substring find-in-page across all text nodes, collects every occurrence as a navigable match, and lets you cycle them (with a count, like Chrome's own “3/12”). When there is *no* substring hit, it falls back to the same `zfind` fuzzy matcher the palette and the sibling apps use (`ZGui.zfind`), so a misremembered or scattered query still finds the closest text. Substring-first is the right default — exact matches are what you usually want — with fuzzy as the safety net rather than the primary mode.

The CSS Custom Highlight API

The interesting engineering choice is how it highlights. Instead of wrapping matches in `<mark>` elements — which mutates the page's DOM, can break the page's own scripts and layout, and is expensive to undo — `zfind.js` uses the CSS Custom Highlight API: it builds `Range` objects for each match and registers them as named highlights that the browser paints via a `::highlight()` pseudo-element, with zero DOM mutation. The page's DOM is never touched; the highlights are a paint-layer overlay the browser manages. The script guards for the API (`'highlights' in CSS`) and no-ops where it is absent, but on the Chromium base it targets, it is present. This is the modern, correct way to highlight find matches, and it is why the HUD find bar can run on complex web apps without corrupting them the way a `<mark>`-wrapping finder would.

Two highlight layers, and the two ranges

The bar keeps two named highlights, not one. All matches are painted underlined in the scheme cyan under the name `zfind`, and the *current* match is painted in the accent color with a translucent background under a second name, `zfind-cur`, which is deleted and rebuilt every time you cycle. Cycling wraps (`Enter` forward, `Shift+Enter` back) and scrolls the current match to the center of the viewport; the counter reads `current/total` exactly like Chrome's own find. Because a `::highlight()` rule cannot read a `var()` off the page's own elements, the script bakes concrete scheme colors into the highlight CSS rather than referencing the HUD variables — a small compromise the code comments call out.

Two implementation choices are worth naming. The match walk uses a `TreeWalker` over text nodes, skipping `SCRIPT/STYLE/NOSCRIPT/TEXTAREA` parents, hidden nodes, and the find bar's own DOM, and it caps at 6000 nodes and 2000 matches so a pathological page cannot hang the search; the fuzzy fallback additionally skips any single text node longer than 300 characters. And the bar's `position:fixed` CSS is injected *synchronously* before the input is focused — the comment explains that focusing an in-flow element would scroll the page down to it, which is the exact “Cmd+F scrolls the page” bug a naïve find bar exhibits. There is no debounce: the search runs on every keystroke, which is affordable precisely because the highlight layer never touches the DOM.

Why replace the native find at all

Replacing `Ctrl/⌘+F` is a larger claim than it first appears, because Chrome's find bar is not a page-level feature — it is browser chrome, implemented in C++, with behaviors no page script gets for free: it searches across the frame tree, it drives the browser's own match highlighting in the scrollbar, and it survives navigation within the bar's own lifetime. Overriding it means giving those up, and it is worth being explicit about what is bought in exchange, because “our find bar is themed” would not justify the trade.

Three things are bought. The first is the fuzzy fallback: the native find is a substring matcher and nothing else, so a query you half-remember returns nothing, and the correct recovery — try a shorter fragment — is a manual binary search. The HUD bar tries substring first, which is the right default because exact matches are what you usually want, and falls back to the same scored matcher the palette uses when the substring finds nothing. That is a strictly additive capability: everything the native bar would have found is still found, in the same order, and the cases it would have failed now return the closest text instead of an empty count.

The second is theme and keyboard continuity. The bar is a HUD surface styled from the active scheme, and its editing behaves like every other HUD input because the shared toolkit installs readline-style line editing globally on text fields. A user who has `Ctrl-W` and `Ctrl-U` in their fingers gets them in the find bar, which the native bar does not offer.

The third is that the bar is *code you can extend*, which the browser's own find bar is not. The count, the cycling, the highlight styling, and the match-collection policy are all in one script, so a change to any of them is a change to a file rather than a patch to Chromium. On the runtime path that distinction is the whole point of the project.

What is given up is real and the book should say so: the HUD bar searches the top document, so text inside a cross-origin iframe on the page is outside its reach in a way the native bar's frame-tree search is not, and it does not paint the scrollbar tick marks the native bar shows. Those are the honest costs of moving the feature from the browser's C++ into a content script.

The walk, its caps, and where it gives up

The matching side is a `TreeWalker` over text nodes, and the interesting engineering is in what it refuses to look at. Parents that are `SCRIPT`, `STYLE`, `NOSCRIPT`, or `TEXTAREA` are skipped, because their text is not text the user is reading; hidden nodes are skipped for the same reason; and the bar's own DOM is skipped so a query never matches the query box. Past those, two hard caps bound the work: at most 6000 nodes walked and at most 2000 matches collected, with the fuzzy fallback additionally skipping any single text node longer than 300 characters.

Those caps are the difference between a find bar and a hang. A modern web application can hold tens of thousands of text nodes, and a search that runs on every keystroke with no debounce — which this one does — would be visibly janky long before it was correct if it walked all of them. The caps are chosen so the common case is unaffected (a document with fewer than 6000 text nodes is fully searched) and the pathological case degrades to a partial result rather than a frozen tab. The honest consequence is that on a very large document the match count is a floor rather than a total, and the fuzzy fallback in particular will not consider the longest paragraphs at all — which is the right trade, since a 300-character-plus node is prose, and fuzzy matching prose returns noise.

The absence of a debounce is a deliberate consequence of the highlight design rather than an oversight. Searching on every keystroke is affordable *because* the highlight layer never touches the DOM: registering a set of ranges as a named highlight costs a paint, not a reflow, and discarding the previous set costs nothing at all. A find bar that wrapped matches in elements would have to insert and remove thousands of nodes per keystroke, which is precisely why such implementations debounce — and why they corrupt pages whose own scripts observe the DOM. The two choices are linked: the highlight API is what makes the no-debounce search viable, and the no-debounce search is what makes the bar feel like it is filtering rather than querying.

Two highlights, one cycle, and the color compromise

The two named highlights are worth one more look because the split is what makes cycling cheap. All matches live in one named highlight painted underlined in the scheme's cyan; the current match lives in a second named highlight painted in the accent with a translucent background. Cycling forward or backward deletes and rebuilds only the second one — a single range — while the first stays exactly as it was. If both were one highlight, every press of **Enter** would rebuild the whole match set to change which member was emphasized, which is the difference between constant work per cycle and work proportional to the match count.

The compromise the code comments call out is the color. A `::highlight()` rule is not matched against the page's own elements, so it cannot read a `var()` off them, which means the highlight CSS cannot reference the HUD's scheme variables the way every other surface does. The script bakes concrete colors into the highlight rules instead. This is the one place in the runtime HUD where a scheme value is duplicated rather than referenced, and the reason it is acceptable is bounded: it affects two rules, both inside one script, both re-emitted when the scheme changes. It is worth flagging precisely because the rest of the HUD is so consistent about painting roles rather than pixels — this is the exception, and it exists because the platform's highlight pseudo-element does not participate in the cascade the way an element does.

Chapter 14 – The HUD Internal Pages

Several of Chrome's internal pages are not *skinned* but *replaced*: a `document_start` redirect (`redirect.js`) sends `chrome://extensions`, `chrome://settings`, `chrome://history`, `chrome://bookmarks`, and `chrome://version` to purpose-built HUD pages under the extension's `pages/` directory, which reimplement the surface using the extension's privileged permissions. This chapter is the tour.

Why replace rather than skin

The `chrome://` pages that get replaced are the ones you interact with, not just read — the extension manager, settings, history, bookmarks. Skinning them (the `theme.js` token map) recolors them but leaves Chrome's own UI and its quirks; replacing them lets the HUD present the *data* in its own cyberpunk components with its own fuzzy filtering and keyboard model. The replacements are real, backed by the manifest's permissions: `management` and `developerPrivate` for the extensions page, `history` for history, `bookmarks` for bookmarks, `downloads` for downloads, `sessions` and `system.display` where needed. They are served from the extension, so they are web-accessible pages the redirect can navigate to.

The pages

From the `pages/` directory, the replacements and HUD-native pages are:

<code>extensions.html</code>	the extension manager (enable/disable/reload/remove), HUD-styled, via <code>management + developerPrivate</code>
<code>settings.html</code>	a HUD settings surface (scheme picker, visual-effect toggles, light mode) writing the <code>hud-ui.json</code> prefs
<code>history.html</code>	history, fuzzy-filterable, via the history permission
<code>bookmarks.html</code>	bookmarks, via the bookmarks permission
<code>downloads.html</code>	downloads, via the downloads permission
<code>readinglist.html</code>	the reading list, via the readinglist permission
<code>version.html</code>	a HUD <code>chrome://version</code> replacement
<code>keys.html</code>	the keybinding editor – renders <code>ZWIRE_KEYMAP</code> and writes remaps to <code>zb_keys</code> (Chapter 9), same registry <code>zvim</code> reads
<code>commands.html</code>	the <code>⌘K</code> keyword-command editor – edits <code>zb_custom_cmds</code> (Chapter 12)
<code>extshortcuts.html</code>	extension keyboard-shortcut management
<code>sessions.html</code>	the tmux session manager (Chapter 10)
<code>ci.html</code>	a zwire CI-runs surface (linked from the palette)
<code>terminal.html</code>	the standalone terminal page (Chapter 16)
<code>host.html</code>	the zwire-host console (Chapter 15)
<code>dashboard.html</code>	one searchable tile grid launching every HUD page and every <code>chrome://</code> internal page (Chapter 53)
<code>hooks.html</code>	stryke scripts bound to browser lifecycle events (Chapter 49)
<code>triggers.html</code>	regex-on-page-output triggers (Chapter 50)
<code>pipes.html</code>	reactive dataflow edges between tiled panes (Chapter 51)
<code>audio.html</code>	the browser-wide audio engine's dashboard (Chapter 52)
<code>notes.html</code>	a folder tree of Markdown notes (Chapter 53)
<code>feeds.html</code>	an RSS/Atom reader (Chapter 53)
<code>translate.html</code>	a translation panel (Chapter 53)
<code>store.html</code>	the in-browser product catalog + first-run welcome (Chapter 53)

Two of these are the HUD's own configuration surfaces and close loops opened earlier in the book: `keys.html` renders the shared keybinding registry and writes remaps into `zb_keys`, the exact key the vim and tmux scripts read, so rebinding a key in the page changes what fires on every page; and `commands.html` edits the `zb_custom_cmds` array the palette's keyword commands come from. The pages share a set of HUD stylesheets (`hud-pages.css`, `ext-page.css`) and helper scripts (`zg-boot.js`, `zg-list.js`, `fzf.js`), so they are one consistent surface, not a dozen snowflakes. `redirect.js` runs at `document_start` so the swap happens before Chrome's own page paints, and the redirected set is exactly the set excluded from the `theme.js` skin so the two never collide.

What each replacement actually does

These are not skins with a `chrome://` page underneath; each is a working reimplementation on a privileged Chrome API, and the API is what makes the page more than cosmetic:

- **Extensions** (`extensions.js`, the largest page at ~350 lines) is a full extension manager on `chrome.developerPrivate` plus `chrome.management`. It lists every extension including disabled and terminated ones, and it can enable and disable, uninstall (with the confirm dialog), reload an unpacked extension, load a new unpacked directory, pack a directory, check for updates, toggle developer mode, pin to the toolbar, set per-extension site access (`ON_CLICK`, `ON_SPECIFIC_SITES`, `ON_ALL_SITES`), flip the incognito/file-access/error- collection toggles, read an extension's on-disk size, and open its service- worker devtools. A `#shortcuts` sub-view edits every extension's keyboard commands in place, suspending Chrome's own shortcut handling while it captures the chord.
- **Settings** (`settings.js`) is built on `chrome.settingsPrivate`, so it renders every Chrome preference the API exposes — grouped by the first dotted segment of the key, each mapped to the right control (a toggle for booleans, a select where the pref advertises allowed values, a number or text field otherwise, a read-only JSON textarea for list and dictionary prefs) and disabled when the pref is policy- controlled. On top of the raw prefs sit the HUD's own Appearance card (the eight- scheme picker and a light-mode toggle) and an Effects card (scanlines, vignette, glow, animations), whose values are pushed to the

native `hud-ui.json` file so the separate new-tab extension can follow them.

- **History, Bookmarks, and Downloads** are thin configurations of a shared filtered-list component (`zg-list.js`) over `chrome.history.search`, `chrome.bookmarks.getTree` (flattened to a folder-path table), and `chrome.downloads.search` respectively, each fuzzy-filterable and each wiring the matching `on*` events for live updates. Downloads opens a completed file on click through `chrome.downloads.open`; the HUD navigation actually points its Downloads entry at native `chrome://downloads`, because the power-tool owns the download manager, which leaves this page a working-but-unlinked alternative.
- **System** (`version.js`) replaces `chrome://version` with a HUD panel that shows the zwire edition, the HUD extension's manifest version, the Chromium version parsed from the user agent, the install and profile paths, and a *live* native-host probe — it sends `{cmd: 'get'}` to `com.zwire.hud` and reports connected with the current scheme and active effects, or “not connected” with the exact error and the setup command to run.

The two configuration surfaces, and two the book had not named

Two replacement pages are the HUD's own control panels and close loops opened earlier: **Keyboard** (`keys.js`) renders the shared `ZWIRE_KEYMAP` registry as editable chips, writes remaps into `zb_keys` (the exact key vim and tmux read), warns when a new binding collides with an existing one, and also edits the tmux prefix chord and arm-timeout that `ztmux.js` obeys; **Commands** (`commands.js`) is the CRUD editor for `zb_custom_cmds`, enforcing unique keywords, supporting all five command types (url/shell/js/action/scheme), and — a nice touch — remembering a deleted default in a `zb_cmds_removed` tombstone list so the worker's seeder never re-adds it.

Two more pages exist that the earlier map did not call out. **Sessions** (`sessions.js`) is a durable manager for the tmux overlay's named layouts: it reads and writes `zb_tmux_sessions`, lets you create, duplicate, rename, and delete sessions and edit their windows and panes, renders a small SVG preview of each layout using the same even-split geometry the overlay uses, exports and imports the whole set as JSON, and “attaches” a session by writing the `zb_tmux_load` trigger the live overlay watches. **CI** (`ci.js`) is a full GitHub Actions dashboard — the interactive front end to the worker's background poller of Chapter 8. It reads a `zb_ci` config (GitHub user, optional token, repo limit), fetches each repository's recent workflow runs through the GitHub REST API with the modern `X-GitHub-API-Version` header, renders them as per-repo accordions with status pills and relative timestamps (repos with a failing or in-flight latest run open by default), and handles rate limiting by reading the `x-ratelimit-reset` header and backing off until the window clears. A token raises the repo budget and switches the repo query from the public account to your full affiliations.

One shell under all of them

Every page mounts through `zg-boot.js` (`window.ZBHUD`), which is why they read as one surface rather than a dozen. It builds the sticky HUD header and the cross-page navigation row, applies the persisted CRT and effect preferences, and installs the same `⌘K` command palette the content scripts carry — registered here with the page's own extra commands plus the shared HUD toggles, nav destinations, custom commands, recent sites, and open tabs. It also owns the pages' half of the scheme bridge: `ZGui.colorscheme.onApply` sends the pick to the native host and mirrors it to `zb_scheme`, and a 1.5-second poll of the native file re-applies an externally chosen scheme so a page left open still follows a scheme picked elsewhere. The three list pages share `zg-list.js`, and the fuzzy matching everywhere is the shared `fzf.js` — so the consistency across a dozen pages is literal shared code, not a dozen parallel implementations.

The custom-command runner, five ways

The palette that `zg-boot.js` installs on every HUD page is the same one the content scripts carry, and it runs custom commands through one dispatcher, `runCustomBoot`, whose five branches are the concrete meaning of the command types the Commands page edits. A `url` command substitutes the typed argument for `{q}` and opens the result in a new tab. A `shell` command hands the argument to `window.zwireTermRun`, which opens the terminal and types it (Chapter 16). A `js` command evaluates the argument as the body of `new Function('q', value)`, with the typed text bound to `q` — a genuine one-line scripting hook, scoped to the page. An `action` command writes a `zb_cmd` onto the storage bus for the worker to perform. And a `scheme` command sets the color scheme through the native host and the storage mirror at once. The same five types resolve identically in the content-script palette, because both call the same runner against the same `zb_custom_cmds` array — so a command you define once behaves the same whether you trigger it from an ordinary page or from a HUD page. It is the last loop the book closes on the palette: one editor, one storage key, one runner, five behaviors, reachable from every surface.

Chapter 15 – The Native Host

`native/zwire-host` is the bridge between the browser and the machine — and, as of this edition, a good deal more than a bridge. It is a single self-contained Rust binary (~500 KB, no Python and no `psutil`, `panic = "abort"` and `opt-level="z"` in release) that began as `zwire`'s Chrome native-messaging host and grew into a **universal local endpoint**: a process any app can ask to read the machine, spawn a shell, crawl the filesystem, run a command, or hold a little state. It ships as its own MIT-licensed submodule under `extensions/hud-internal/native/zwire-host`, with its own README, its own CI, and a 15-module `src/` tree. (Earlier printings of this book described a Python reference host, `hud_host.py`; that file has been removed — the Rust host is now the only one, and where the old text speaks of a “reference,” read it as the historical first cut of the same scheme/stats/PTY path `zwire-host` now owns outright.)

One dispatcher, three transports

The heart of the host is a single **capability router** (`src/session.rs`, whose own doc line reads “the capability router shared by every transport”). Every request is a JSON object dispatched on its `cmd`; every reply is a JSON object; and a request may carry an `id` that is echoed on its reply, so one connection can multiplex many in-flight requests, streams, and terminals at once. Three transports feed that one router (`src/transport.rs`, “the three ways to reach the dispatcher”):

- **Native messaging** (the default, and what `zwire` uses). Chrome spawns the host and speaks length-prefixed JSON on `stdin/stdout`: a 4-byte little-endian `u32` length followed by that many UTF-8 bytes of body. `src/proto.rs` is the whole framing — `write_all(8(data.len() as u32).to_le_bytes())` out, a `read_exact` of four bytes then the body in, and `None` on a short read as the port-closed sentinel.
- **Local-socket daemon** (`zwire-host serve`). The same router, reached over newline-delimited JSON (one object per line) on a **Unix domain socket** (macOS/Linux: `$ZWIRE_HOST_SOCK`, else `$XDG_RUNTIME_DIR/zwire-host.sock`, else `~/.zwire/host.sock`, created `0600` under a `0700` dir because it exposes `exec/fs/pty`) or a **named pipe** on Windows (`\\.\pipe\zwire-host-<user>`). This is what lets `tmux`, `emacs`, shell scripts, and any language talk to the host without a client library — `printf '{"cmd": "sysinfo_once"}\n' | nc -U ~/.zwire/host.sock` is a complete client.
- **In-process library** (`zwire_host` as a crate). The same dispatcher, plus `api::walk` / `api::exec` helpers, so a sibling host can embed it and reuse the crawl/exec/stream machinery rather than reimplement it.

Because all three sit behind the one router, every command in the next section works identically over any of them.

The capability surface

Each capability is one small module under `src/`, and the fifteen of them together are the whole of what the host does:

- **Discovery & state** — `hello` feature-tests the host (`{ok, host, version, os, arch, pid, caps: [...]}`), `hostinfo` returns one-shot machine facts, and a per-app key/value store (`src/store.rs`) persists JSON under `~/.<app>/kv/` via `kv_set/kv_get/kv_merge/kv_del/kv_keys`.
- **System stats** (`src/sysmon.rs`) — `sysinfo_once` for a snapshot, `sysinfo_start/sysinfo_stop` to stream one `{sys: {...}}` frame per interval.

- **Filesystem** (`src/fsops.rs`) — `fs_read/fs_write/fs_append/fs_list/fs_stat/fs_mkdir/fs_rm`, plus `fs_walk`, a recursive crawl with `depth/ext/dirs_only/contains` filters.
- **File watching** (`src/watch.rs`) — `fs_watch` streams `created/modified/removed` events and `fs_tail` streams appended lines (`tail -f`, surviving rotation), each keyed by `id` and cancelled with `watch_stop`.
- **Exec & OS** (`src/exec.rs`, `src/osops.rs`) — `exec` runs a program to completion (base64 `stdout/stderr`), and `open/clipboard_get/clipboard_set/notify` reach the desktop.
- **Background jobs** (`src/jobs.rs`) — `job_start` spawns a long-running command that survives the connection and fires a desktop notification on completion; `job_list/job_result/job_poll` track and drain them.
- **Process tools** (`src/procs.rs`) — `ps`, `kill`, and `which`.
- **Pub/sub bus** (`src/bus.rs`) — `sub/unsub/pub` turn the host into a coordination hub; the daemon itself publishes on the `scheme` and `ui` topics whenever they change, so a subscribed HUD gets **live theme sync without polling**.
- **Host-to-host peering** (`src/peer.rs`) — daemons run with TCP peering to federate the bus across machines (`peers/peer_connect/remotes`), token-gated and single-hop so star and mesh topologies never loop.
- **PTY terminals** (`src/pty.rs`) — the terminal backend, below.

The scheme and UI bridge

The host's original job survives unchanged as the “legacy zwire scheme/ui” command group. A `{cmd:'get'}` returns the host `version` plus the current scheme (defaulting to `cyberpunk`) and the UI prefs; a `{scheme:'matrix'}` message — validated against the eight allowed names — and a `{ui:{...}}` merge each rewrite their HUD file with a write-to-`.tmp`-then-atomic-rename so a concurrent reader (the fork's color mixer, the new-tab poller) never sees a half-written file. These files are the cross-extension bus of Chapter 8; the host is what owns them on disk, and — via the `scheme/ui` pub topics above — what now pushes a change to every subscriber the instant it lands.

The system-stats stream

`sysinfo_start` (with an optional `interval_ms`, floored at 250 ms) enters `src/sysmon.rs`, which samples the machine through the cross-platform `sysinfo` crate — no `psutil`, no `pip install` — and sends one `{sys:{...}}` frame per tick:

<code>cpu</code>	<code>overall CPU %</code>	<code>mem/swap</code>	<code>used / total (bytes)</code>
<code>load</code>	<code>1/5/15 load average</code>	<code>uptime</code>	<code>seconds since boot</code>
<code>disk</code>	<code>root filesystem used / total</code>	<code>batt</code>	<code>pmset percent + plugged (macOS)</code>
<code>net</code>	<code>bytes sent/recvd rate (Δ/dt)</code>	<code>io</code>	<code>disk read/write rate (Δ/dt)</code>
<code>temp</code>	<code>hottest Components sensor</code>	<code>host</code>	<code>short hostname</code>
<code>lip</code>	<code>LAN IP</code>	<code>pip</code>	<code>public IP via api.ipify.org</code>

The one structural change from the old reference matters: the stream runs on its **own background thread**, so a persistent connection stays free for other RPCs while it ticks — a HUD can hold a `sysinfo` stream, a PTY, and a file watch on the same pipe at once, which the single-process Python host could not. Rates (`net`, `io`) are deltas over the elapsed wall-clock interval (`dt`, floored at 0.1 s), so they read as true per-second throughput rather than cumulative counters; the public IP is fetched from `api.ipify.org` and refreshed only periodically so the stream never hammers the lookup; and any sensor a platform lacks (no battery off macOS, no temperature) is simply omitted from the frame rather than crashing it. Everything the statusbar shows (Chapter 11) is one field of this sample.

The PTY relay

`pty_spawn` turns the host into a terminal backend (`src/pty.rs`). It opens a pseudo-terminal through the cross-platform `portable-pty` crate — `native_pty_system().openpty(PtySize{...})` — and runs a program in it: `$SHELL` (falling back to `/bin/sh`) with args defaulting to `["-l"]` for a login shell, at optional `rows/cols` (24×80 default), `cwd`, and `env`. A per-session reader thread pumps the shell's output as `{ev:"output", b64:...}` frames; `pty_write` feeds input, `pty_resize` changes the window size, and `pty_kill` ends it with an `{ev:"exit"}`. The important upgrade over the old single-shell reference is **multiplexing**: sessions are keyed by `id`, so `pty_write/pty_resize/pty_kill` route to the right terminal and one connection can hold several at once (an absent `id` addresses a single default session). It is a genuine login shell in a genuine PTY, driven from a web page — the piece with the most moving parts and the one the CDP probes (Chapter 20) spend most of their assertions verifying. Chapter 16 is the browser end of this pipe.

Packaging, security, and the library

`zwire` wires the host up as a standard Chrome native-messaging host: a manifest named `com.zwire.hud` (`type: "stdio"`) points `path` at the built binary and lists the allowed extension origins in `allowed_origins`, and `scripts/localinstall.sh` builds `zwire-host` and drops that manifest into the browser's `NativeMessagingHosts/` directory when it packages the `.app`. Because the binary is a single static Rust artifact, the consuming bundle has **zero runtime dependencies** — no system Python to be missing, no `psutil` to `pip install`, nothing to break on a fresh machine.

The daemon transport takes its exposure seriously, because it hands out `exec`, `fs`, and `pty`: the Unix socket is created `0600` inside a `0700` directory (owner-only), and inbound TCP peering is gated by a shared `--token` (or `$ZWIRE_HOST_TOKEN`) that a connection must present before anything privileged — local socket clients are trusted and skip it. `sysinfo` and `portable-pty` abstract the OS, so the same source builds for macOS, Linux, and Windows, and CI runs the four canonical polish gates (`fmt`, `clippy -D warnings`, `doc`, `test`) on all three. And because the whole thing is also a library (`zwire_host`), sibling hosts — `zpwichrome`'s among them — embed it to reuse the crawl/exec/stream machinery rather than reimplement it. Chapter 16 is the browser end of the PTY pipe.

Chapter 16 – The Terminal

The HUD ships a real terminal: an `xterm.js` front end (`lib/term/`) whose bytes come from the native host's PTY relay, bridged through the service worker, rendered as an overlay on any page and as a standalone page, and kept open across navigation. This chapter is the browser end of the pipe Chapter 15 opened.

xterm over the service-worker bridge

The terminal is `xterm.js` (`lib/term/xterm.js` + `terminal.js`) plus a boot script. A content script cannot open a native-messaging port that survives page navigation, and the PTY must outlive any single page, so the terminal's bytes flow through the background service worker: the worker holds the `connectNative` port to `com.zwire.hud`, sends `pty_spawn`, and relays `pty_write/pty_resize` down and base64 `output` up. The content-script terminal decodes the base64 into xterm and sends your keystrokes back as `pty_write`. Toggle it with `Ctrl+`` and it drops in as an overlay pane; there is also a standalone `pages/terminal.html`.

Staying open across navigation

A terminal that vanished on every link click would be useless, so `zterm-boot.js` mirrors the overlay's open state into `chrome.storage.local` under `zb_term_open`. Per-origin `localStorage` does not carry across different sites, so the boot script uses the cross-origin storage key instead: a `MutationObserver` on the pane's `active` class records every open/close, a `pagehide` handler snapshots the state right before navigating, and on the next page the overlay re-injects and re-opens automatically if it was explicitly open — and the worker reconnects the same tab's PTY, so the shell session continues across the navigation.

The open flag is deliberately strict and self-clearing. It re-opens only when `zb_term_open === true` (a real boolean, not a stale truthy value) and the pane is not already showing, and the background worker resets it to `false` on browser startup and on extension reload (Chapter 8's worker also owns this), so a stale “open” flag from a previous session never re-pops the terminal on every page of a fresh one. The result is a terminal that follows you from page to page within a session but does not haunt the next one — the browser-side half of a genuine shell, running the same login `zsh` you have in a real terminal, over the PTY the native host forked.

The transport shim and the xterm instance

The terminal front end is not `zwire`-original either; `terminal.js` is the shared `zpwir-embed-terminal` component, extracted verbatim from Audio-Haxor, and its own header says the only `zwire`-specific change is “a transport shim (auto-detects Tauri vs JUCE).” That shim, `TT`, picks a transport at load: a standalone extension page can `connectNative('com.zwire.hud')` directly, while a content-script overlay — which cannot open a native port — takes the `chrome.runtime.connect({name: 'zwire-pty'})` branch to the worker instead. Both branches speak the same four-message protocol (`pty_spawn`, `pty_write`, `pty_resize`, `pty_kill`) and decode the same base64 `output` frames into a `Uint8Array` that xterm writes directly, so the same component runs identically over the two paths.

The xterm instance itself is configured for the HUD: a blinking block cursor, a 13-pixel Hack Nerd Font (falling back through Share Tech Mono and Menlo), a fully transparent background so the scheme shows through, cyan cursor and selection, a ten-thousand-line scrollbar, and the sixteen ANSI colors set explicitly. There is no fit addon — the component measures the rendered cell size from xterm's own render service and computes

rows and columns by hand, re-fitting on a 50-millisecond- debounced `ResizeObserver` and sending each new size down as `pty_resize`. One piece of the component exists solely for release WebKit: because that engine ignores `setAttribute('style', ...)` on the DOM renderer's rows, a monkey-patch replaces xterm's `_addStyle` to set `el.style.color` and `el.style.backgroundColor` directly, so truecolor output actually paints — the same class of WebKit-release quirk the GUI apps in the suite fight constantly.

Docks, Ctrl-letters, and shell one-liners

The overlay is a draggable, dockable pane, not a fixed panel. It carries four dock positions (the corners), snaps to the nearest one on release using pixel-positioned zones (because “CSS calc() with percentages doesn't resolve in release WebView,” the comment notes), and persists its geometry. Two keyboard details make it usable as a real terminal inside a browser. First, `Ctrl+`` toggles it — deliberately not `Ctrl+T`, which is “left to Chrome for new-tab.” Second, when the terminal has focus a capture-phase handler forwards `Ctrl+<letter>` combinations straight to the PTY as their control codes (`Ctrl-A`, `Ctrl-E`, `Ctrl-K`, `Ctrl-U`, `Ctrl-W`), because macOS WKWebView would otherwise swallow them for its own Emacs bindings — so readline editing works in the browser terminal the way it does in a real one. And `zwireTermRun`, the entry point behind the palette's `shell` command type, opens the terminal and then polls for up to ten seconds until the PTY is alive before writing the command, so a one-liner fired the instant the terminal opens is never dropped before the shell exists.

The session's lifetime, and the four ways it ends

A shell inside a browser has a lifetime problem no ordinary terminal has: the thing displaying it — a content script in a document — is destroyed and recreated constantly, by every navigation, while the thing running it must not be. The design's answer is to bind the session to the *tab* rather than to the page, and the consequences of that choice are worth walking, because they explain every otherwise-odd behavior in the terminal.

A session begins when a spawn request reaches the worker for a tab that has none. It is honored once per tab: a second spawn for the same tab returns without doing anything, so a content script re-injected by a navigation re-attaches to the running shell rather than forking a second one behind the same window. This single guard is what prevents the worst failure mode in the feature — two shells writing to one screen, which looks like corruption rather than like a bug.

From there the session ends in exactly four ways, and only one of them is the user quitting. The tab closing tears the native port down, because the tab is the lifetime the session was bound to. The shell exiting sends its exit frame up and the session ends with it. An explicit kill ends it deliberately. And the browser or the host dying ends it by ending the process at one end of the pipe. What is conspicuously *not* on that list is the page navigating: a content-script disconnect nulls the attachment and leaves the native port alive, with output buffering until something re-attaches. That asymmetry — page disconnect is not session end, tab removal is — is the entire mechanism behind “click a link and keep typing in the same zsh.”

The buffer that covers the gap has a bound, and the bound is a real decision. Output produced while nothing is attached is held, capped at 500 messages, and flushed on reconnect. A cap is required because a detached session could produce output indefinitely — a build, a log tail — and an unbounded buffer in the worker is a memory leak with extra steps. The consequence of the cap is that a very chatty command run across a long navigation will lose its earliest output, which is the correct thing to lose: the tail is what the user will read.

Sizing without a fit addon

The terminal computes its own geometry rather than using xterm's fit addon, and the reason is worth stating because it is the same class of problem the rest of the suite fights. A fit addon measures the container and asks the terminal to match it; that requires the container's resolved size to be trustworthy at the moment it is

asked, which in a release web view — with a dock-positioned, draggable overlay whose CSS is resolving at the same time — it frequently is not.

So the component measures the rendered cell size from xterm's own render service and computes rows and columns by hand, re-fitting on a resize observation debounced at 50 milliseconds and sending each resulting size down to the host as a resize request. Two properties fall out. The measurement is taken from the *terminal's own rendering* rather than from the container's computed style, which makes it independent of whatever the surrounding layout is doing. And the debounce means a drag that changes the size continuously produces a handful of resize messages rather than one per animation frame — which matters because each one is a real `ioctl` on a pseudo-terminal and a `SIGWINCH` to the shell, and a shell receiving hundreds of those per second will redraw its prompt hundreds of times.

The same release-web-view pressure explains two other choices the chapter has already named: the dock zones are pixel-positioned because percentage-based `calc()` does not resolve reliably in the release engine, and a monkey-patch replaces xterm's style application so truecolor output actually paints, because that engine ignores the attribute-based path the renderer normally uses. None of these are xterm bugs or web-view bugs in any interesting sense; they are the ordinary cost of running a component built for one rendering environment inside another, and they are isolated to the smallest possible surface — one measurement path, one geometry helper, one patched method — which is what keeps the shared component shared.

Why `Ctrl+`` and not `Ctrl+T`

The toggle chord is a small decision with a large amount of reasoning behind it, and it is a good closing illustration of the arbitration model. `Ctrl+T` is the obvious choice for a terminal and is exactly wrong here: it is Chrome's new-tab accelerator, it is claimed at the browser level, and a page listener cannot reliably take it. Binding it would produce a feature that opened a new tab most of the time and a terminal occasionally, which is worse than not binding it at all.

`Ctrl+`` is unclaimed by the browser, is the chord the same gesture uses in several editors, and — usefully — is a key most web applications do not bind, so the claim-only-what-is-mapped discipline costs almost nothing to honor around it. The terminal's second keyboard claim, the `Ctrl+<letter>` forwarding, is scoped to the moments the terminal has focus precisely so that it does not violate the same discipline: outside the terminal those chords belong to the browser and the page, and the terminal does not want them.

Chapter 17 — zgui-core, the Shared Toolkit

Under the internal HUD sits `lib/zgui-core`, a git submodule that is the shared cyberpunk GUI toolkit for the whole MenkeTechnologies app suite. `zwire` consumes a slice of it — the fuzzy matcher, the command palette, the effect toggles, the fonts — and this chapter is what that library is and why `zwire` loads it from the submodule rather than copying it.

One toolkit, many consumers

`zgui-core` is, in its own words, the core GUI library shared across the app suite (Audio-Haxor, traderview, `ztranslator`, `zpdf`, `zemail`, `zoffice`, and more) — one canonical implementation of the shell, settings, dialog, table, and notification chrome so the many apps stop drifting their own divergent copies. Each component was extracted from an app that had already built it. It exposes a single namespace, `window.ZGui`, and its `webui/` directory holds on the order of 246 component scripts plus their CSS — a filterable card grid of buttons, inputs, charts, cyberpunk effects, and overlays that its own demo page renders.

What `zwire` uses

`zwire` does not load the whole library; it loads the handful of modules its content scripts depend on, in the manifest's canonical block:

- `fzf.js` — `ZGui.fzf`, the `fzf`-style fuzzy matcher with configurable scoring weights and a matched-character highlighter (`<mark class="fzf-hl">`). This is the engine behind the command palette (Chapter 12), the fuzzy find fallback (Chapter 13), and the fuzzy filtering on the HUD internal pages (Chapter 14).
- `command-palette.js` — `ZGui.palette`, the palette shell the `%K` list is built on.
- `util.js` — the shared helpers the above need.
- `fonts.css` + `webui/fonts/` — the shared typeface loader: `@font-face` for Orbitron (display) and Share Tech Mono (body/mono), with the `.ttf` files bundled under SIL OFL, so the HUD's font stack resolves the same way it does in every other app in the suite.
- `fx.js` — `ZGui.fx`, the global cyberpunk effect toggles (CRT scanlines, bezel vignette, neon glow, animations), each flipping a `body.no-*` class the theme reacts to — the mechanism behind the visual-effect prefs in `hud-ui.json`.

Submodule, not copy

`zgui-core`'s own guidance is emphatic: consume it as a git submodule and load the modules directly from the submodule path — *never copy* `webui/*`, because copies go stale and partial-sync. `zwire` follows that: `lib/zgui-core` is a submodule (declared in `.gitmodules`), and the manifest references files under `lib/zgui-core/webui/...` directly. This is the same reuse-not-copy discipline the `zpwchrome` power-tool and the native host follow, and it means an improvement to the shared fuzzy matcher or palette reaches `zwire` with a submodule bump rather than a hand-merge. The HUD's palette and find bar are not `zwire`-original UI code; they are the suite's shared components, wired to `zwire`'s data.

Inside the fuzzy matcher

`ZGui.fzf` is not a substring test; it is a scored fzf-style matcher, and its scoring is the reason the palette and find bar rank the way they do. Its own comment records the lineage — “EXTRACTED VERBATIM from Audio-Haxor” and “shared so the apps stop drifting their own copies.” The core, `fzfMatch(needle, haystack)`, is a case-insensitive subsequence match that, for each possible starting position of the first character, greedily takes the earliest subsequent matches and scores the run: each matched character is worth a base amount, adjacency to the previous match earns a consecutive bonus, and a gap between matches costs a start penalty plus a per-character extension penalty. On top of that come *position* bonuses that encode what humans mean by a good match — a character right after a non-word character scores a word-boundary bonus, a capital right after a lowercase scores a camelCase bonus, and the first matched character’s bonus is multiplied. The default weights are concrete (match 16, gap-start -3, gap-extend -1, boundary 9, non-word 8, camel 7, consecutive 4, first-char multiplier 2) and are themselves tunable and persisted, which is how the whole suite can retune ranking in one place. Above `fzfMatch` sits an extended query parser — space-separated AND terms, `|` for OR groups, and the fzf sigils `!` (negate), `'` (exact), `^` (prefix), `$` (suffix) — and a highlighter that wraps the matched characters in `<mark class="fzf-hl">`, which is the cyan emphasis you see on the matched letters in every list.

The palette shell and the shared readline

`ZGui.palette` is the list host the `%K` overlay is built on, and its ranking is a deliberate three tiers: static items and “strong” provider results are fzf-scored together at the top (ties favoring the static entries), “secondary” reference rows sit in the middle, and “fallback” rows — the generic web search — are pinned last no matter what you type, so a real command is never buried under a search suggestion. One more piece of `util.js` reaches every HUD input: on load it installs Emacs/ readline line-editing globally (`^A/^E` to move, `^W/^K/^U` to kill, `M-b/ M-f` by word, and the rest) and disables autocorrect and spellcheck on text fields. That is why the palette input, the find bar, and every settings field share the same terminal-style editing — it is one installed behavior from the shared toolkit, not a per-field reimplementation.

The visual-effects layer

The cyberpunk *look* — the grid lattice, the CRT scanlines, the bezel vignette, the neon glow — is also `zgui-core`, and it is built on one clean convention worth stating: an effect is *on* when its class is *absent*. `ZGui.fx` maps four effects to four body classes (`no-scanlines`, `no-vignette`, `no-neon-glow`, `no-anim`), and turning an effect off adds its class; the default, with no class present, is everything on. Each toggle persists to its own `zg-fx-...` `localStorage` key, and the Settings page bridges the same four toggles up to the native `hud-ui.json` so the new-tab extension follows.

The CSS behind the effects is unglamorous and worth demystifying, because none of it is magic. The grid lattice is two `linear-gradient` hairlines on a 40-pixel fixed- attachment cell, layered under three faint `radial-gradient` color blooms on the body background. The scanlines are a single `repeating-linear-gradient` on `body::after` — two transparent pixels, two barely-visible cyan pixels, repeated — plus, when the animated variant is on, two injected beam divs that sweep across on twelve- and eighteen-second loops. The vignette is one `radial-gradient` ellipse on `body::before` that is transparent to 58% and then darkens to the edges. And the neon glow is not a filter at all; it is pervasive `text-shadow` and `box-shadow` on themed elements. That last point is why the off switch is so blunt: `body.no-neon-glow *` sets `text-shadow` and `box-shadow` to `none !important`, `body.no-anim *` sets `animation` and `transition` to `none !important`, and the two overlay pseudo-elements are simply `display:none`. The effects are cheap layered CSS with a global kill switch, which is exactly why they can be toggled live without a reflow storm — and why the same look renders identically across every HUD surface that includes the one stylesheet.

Chapter 18 — `zpwrcode`, the Power-Tool

`extensions/zpwrcode` is the Manifest V3 power-tool, loaded as a **submodule** — reused, not copied — so `zwire` tracks upstream instead of forking a snapshot of it. This chapter covers what the extension provides, the native host its `pass` and segmented-download features need, and the cosmetic developer-extensions banner that unpacked loading produces.

Loaded as a submodule, not vendored

`zpwrcode` is its own public repository, and `zwire` consumes it through a git submodule at `extensions/zpwrcode` (declared in `.gitmodules`), not by copying its files into the tree. This is the same reuse-not-copy discipline `zgui-core` and the native host follow, and it has a concrete payoff: a fix or feature that lands in `zpwrcode` upstream reaches `zwire` with a submodule bump, and there is exactly one canonical copy of the extension rather than a frozen fork that drifts. `zwire` is a *host* for `zpwrcode`, not a second home for it.

The cost of a submodule is that it can be absent — a clone without `--recurse-submodules`, or a checkout where the submodule was not initialized, leaves `extensions/zpwrcode` empty. `zwire` handles this with the same graceful-skip rule the whole launcher uses: `bin/zwire` includes an extension directory in `--load-extension` only if it contains a `manifest.json`. An empty `zpwrcode` directory has no manifest, so it is silently skipped and `zwire` runs with just the new-tab and internal-HUD extensions. The power-tool is additive, not required; its absence degrades the browser, it does not break it. This is why the README can honestly say a fresh clone still runs.

Because it is a submodule and a public, separately-versioned project, this book does not re-document `zpwrcode`'s own feature surface in depth — that belongs to `zpwrcode`'s own documentation, and duplicating it here would go stale the moment upstream moves. What matters for `zwire` is the *seam*: how the power-tool is loaded (submodule, skipped-if-absent), how it stays color-synced with the rest of the HUD (the background worker pushes the scheme to it over `runtime.sendMessage`, and it can pull the current scheme or push a user pick back through the same bridge, since separate extensions cannot share storage — Chapter 8), and what it needs from the machine.

Native messaging and the developer-mode banner

Two operational notes close the chapter, both from the README's own notes section.

Native messaging. `zpwrcode`'s `pass` integration (the standard Unix password store) and its segmented-download feature both need `zpwrcode`'s native host installed — a separate step, documented in the `zpwrcode` project's setup, not something `zwire`'s launcher does for it. `zwire` wires *its own* HUD host (`com.zwire.hud`, for the terminal, system stats, and scheme file) automatically on launch; `zpwrcode`'s host is `zpwrcode`'s to install. The two hosts are independent, and the `allowed_origins` in the HUD host's manifest list all three extension ids (HUD, `zpwrcode`, new-tab) so the scheme bridge can be reached from any of them.

The developer-mode banner. Because the launcher loads the extensions unpacked via `--load-extension`,

Chromium shows its “disable developer-mode extensions” notice on startup. It is cosmetic. The extensions run fully — the banner is Chromium telling you that unpacked extensions are present, which is precisely what *zwire* intends. There is no way to load unpacked extensions on launch *without* the notice on current Chromium; *zwire* accepts it rather than working around it, and this book flags it so it is expected rather than alarming.

What the power-tool declares

This book does not re-document *zpwchrome*'s feature surface — that belongs to its own project and would go stale the moment upstream moved — but its manifest is worth reading once, because it is the concrete proof of Chapter 2's argument that only a genuine Chromium engine will do. *zpwchrome* is Manifest V3 with `minimum_chrome_version: 127`, and it declares nineteen permissions, among them the four that rule out every non-Chromium base: `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, and `webRequest` (with `webRequestAuthProvider`). Alongside those sit `tabs`, `tabGroups`, `sessions`, `bookmarks`, `history`, `scripting`, `webNavigation`, `cookies`, `downloads`, `contextMenus`, `notifications`, `clipboardWrite`, `alarms`, and `storage`, over `<all_urls>` host access, with a module service-worker background and six content scripts. It binds fifty-four keyboard commands (its action opens on `Alt+T`), which is a scale of its own — and part of why *zwire* loads it as a submodule and lets it own its own shortcuts rather than trying to re-carry any of it.

The feature summary, from the power-tool's own README, is a browser toolkit behind one toolbar icon: a browserpass-compatible Unix `pass` integration (fill, copy, OTP, profile and credit-card autofill, through a vendored Rust native host that walks `~/.password-store`), a segmented multi-connection download manager that takes over Chrome's default via a HEAD probe and parallel range requests, a JetBrains-style most-recently-used tab switcher with scenes and an opener tree and fzf history over thousands of entries, a Tampermonkey-equivalent userscript engine, full-page screenshots, a technology detector, a theme injector, a lights-off dimmer, JSON and XML viewers, a user-agent switcher, and find-in-all-tabs. It needs its own native host for the `pass` integration and the segmented downloads — a separate install, independent of *zwire*'s HUD host — and the `allowed_origins` in *zwire*'s own host manifest lists all three extension ids so the scheme bridge is reachable from any of them. What matters to *zwire* is only the seam of Chapter 18: loaded as a submodule, skipped if absent, color-synced over `runtime.sendMessage`, and speaking to the machine through its own host.

The seam, stated as a contract

Because the power-tool is a separately-versioned project, the durable thing to write down is not its feature list but the *contract* between it and *zwire* — the small set of facts each side may rely on. Stated as four clauses, it is short enough to check against the code and stable enough to survive upstream moving:

zwire will load the power-tool from its submodule directory if and only if that directory contains a manifest, and will do so unpacked, against the dedicated profile, with the same elevated flags the other extensions get. That is the loading clause, and its second half is what makes the first half safe: the power-tool is trusted with those flags because it is running in a profile that holds nothing.

zwire will push the active scheme to the power-tool over cross-extension messaging, and will accept a scheme the power-tool pushes back, with an echo guard on the *zwire* side so a value that originated in the power-tool is not relayed back to it. That is the color clause, and the guard is on *zwire*'s side because *zwire* is the component that can distinguish its own writes.

zwire's own native host manifest lists the power-tool's extension id in its allowed origins, so the power-tool may open a port to the HUD host if it wants the scheme directly rather than through a message. *zwire* does *not* install the power-tool's own native host, which is a separate binary for separate features and a separate install step. That is the host clause, and its asymmetry is deliberate: *zwire* ensures what *zwire* needs and never reaches into another project's setup.

Everything else is out of scope in both directions. `zwire` does not read the power-tool's storage, does not depend on any of its fifty-odd keyboard commands existing, and does not assume any particular feature is present; the power-tool, for its part, is a complete extension that runs in an ordinary Chrome and does not require `zwire`. That mutual independence is what makes the submodule model honest rather than a fork with extra steps.

Why the surface is not re-documented here

There is a real temptation, in a book this thorough, to enumerate the power-tool's features as carefully as the HUD's. Resisting it is a documentation-engineering decision, not laziness, and it is the same reasoning that governs the code. A book that described a separately-released project's feature surface would be a copy of that project's documentation, and a copy is stale the moment upstream ships — with the particular hazard that a *book* is a durable artifact a reader trusts more than a README. Describing the seam instead means describing something `zwire` actually controls and can therefore keep true.

The one part of the power-tool this book does document in detail is its manifest, and that is because the manifest is not a feature list — it is the evidence for the argument the entire project rests on. The declaration of a manifest-version-three extension with a minimum Chromium floor and the four permissions that exist only on a genuine Chromium engine is what makes “`zwire` must be Chromium” a checkable claim rather than a preference. That belongs in this book permanently, because it is an argument about `zwire`; the download manager's segment count does not, because it is a fact about another project.

The practical guidance that falls out for a reader: when something about the power-tool behaves unexpectedly, the question to ask first is which side of the seam it is on. A color that does not follow the scheme is a seam question and this book covers it. A missing extension after a fresh clone is a seam question and this book covers it. A password-store integration that cannot find its native host is not a seam question at all — it is the power-tool's own setup, deliberately independent of `zwire`'s, and no amount of `zwire` configuration will affect it.

Chapter 19 – The Launcher and State Layout

`bin/zwire` is the single entry point: it starts the base against `~/.zwire/profile` with `newtab`, `zpwchrome`, and `hud-internal` loaded and `--extensions-on-chrome-urls` set, passing any extra Chromium flag straight through. This chapter maps the launcher's flag assembly and the `$ZWIRE_STATE` directory layout.

What the launcher assembles

`bin/zwire` is a short, `set -euo pipefail` shell script, and reading it top to bottom is reading the entire runtime path:

1. **Resolve the base.** It checks `ZWIRE_BASE` (an executable override) first, then the path recorded in `~/.zwire/base.path`. If neither yields an executable binary it prints `zwire: no base browser installed – run ... fetch-base.sh` and exits non-zero. No base, no launch, with a message that names the fix.
2. **Build the extension list.** It iterates the three extension directories — `newtab`, `extensions/zpwchrome`, `extensions/hud-internal` — and includes each only if it contains a `manifest.json`, joining the survivors with commas. This is the graceful-skip rule: a missing submodule or deleted directory drops out of the list instead of breaking the launch. Note what is *not* here: `theme/` is deliberately excluded, for the palette-ordering reason Chapter 3 gave.
3. **Ensure the native host.** It looks for the HUD host manifest at `~/.zwire/profile/NativeMessagingHosts/com.zwire.hud.json` and reads the binary path out of it; if the manifest is missing or its target binary is gone, it runs `scripts/setup-native-host.sh` to build the Rust `zwire-host` and (re)write the manifest. This is what makes a bare `zwire` self-sufficient on Linux and in dev, not just the packaged macOS `.app`. It is never fatal — if the host cannot be built, the browser still launches, with the terminal and system stats disabled and a one-line note on stderr.
4. **exec the base** with a fixed flag set plus your arguments:

```
exec "$BASE" \
  --user-data-dir="$PROFILE" \
  --load-extension="$LOAD" \
  --extensions-on-chrome-urls \
  --test-type \
  --no-first-run \
  --no-default-browser-check \
  --homepage="chrome://newtab" \
  --disable-features=NtpFooter \
  --enable-features=SplitViewHorizontal,SplitViewTabRestore \
  "$@"
```

Each flag earns its place. `--user-data-dir` is the profile isolation the whole design rests on. `--load-extension` is the switch unbranded Chromium keeps and branded Chrome removed — the reason for the whole base choice. `--extensions-on-chrome-urls` lets the internal-HUD content scripts run on `chrome://` pages. `--test-type` suppresses the automation-restriction warnings that would otherwise fire with these flags. `--no-first-run` and `--no-default-browser-check` skip the onboarding

and the default-browser nag. `--homepage=chrome://newtab` points the home button at the HUD new tab. `--disable-features=NtpFooter` removes the new-tab footer that would sit under the HUD page. `--enable-features=SplitViewHorizontal,SplitViewTabRestore` turns on Chromium's side-by-side split view and its restore-on-reopen. And `"$@"` passes *everything you typed* straight through — so `zwire https://github.com` opens a URL, `zwire --incognito` starts incognito, and, relevant to Chapter 20, `zwire --remote-debugging-port=9222` opens the CDP endpoint the verification probes attach to. The launcher owns the HUD-defining flags and gets out of the way of every other Chromium flag.

Reading the launcher line by line

`bin/zwire` is short enough to hold in the head, and a few details below the summary are worth naming. It resolves its state directory by sourcing `state-dir.sh` and honoring `$ZWIRE_STATE`; then it runs the one-time legacy migration described in the next section. Base resolution has an exact precedence — an executable `$ZWIRE_BASE` first, then the path recorded in `$ZWIRE_STATE/base.path`, then nothing — and if nothing resolves to an executable it prints, verbatim, `zwire: no base browser installed - run: .../scripts/fetch-base.sh` and exits non-zero. The extension list is the three-directory loop of Chapter 3, filtered to directories that actually contain a `manifest.json` and joined with commas, with `theme/` conspicuously absent.

The native-host ensure step is more careful than “run the setup script.” The launcher reads the `"path"` field out of the existing `com.zwire.hud.json` with a one-line `sed`, and it re-runs `setup-native-host.sh` only when the manifest is missing or its recorded binary is no longer executable — a dangling manifest, not just an absent one, triggers a rebuild. And it is never fatal: if the host cannot be built, the launcher prints `zwire: native host unavailable (terminal/sysinfo disabled) - see scripts/setup-native-host.sh` to stderr and launches the browser anyway, minus the terminal and the system stats. That single property — the browser runs even when the host, a submodule, or a whole extension is missing — is the graceful-degradation rule made concrete in the one script every launch goes through.

The `$ZWIRE_STATE` directory

All of `zwire`'s state lives under `$ZWIRE_STATE`, and — a correction the earlier printings of this book owe the reader — its *default* is not a dotdir in `$HOME`. `scripts/state-dir.sh` resolves the OS's regular application-data location: `~/Library/Application Support/com.menketotechnologies.zwire` on macOS — named by the reverse-DNS bundle id, matching the `.app`'s `CFBundleIdentifier` and the platform convention that Application Support folders carry the bundle id — and `${XDG_CONFIG_HOME:-~/.config}/zwire` on Linux, with `$ZWIRE_STATE` overriding on every platform. There are two *legacy* locations earlier builds used: the original `~/.zwire` dotdir, and a brief intermediate macOS build that used the bare `~/Library/Application Support/zwire` name before it moved to the bundle id. `bin/zwire` does a one-time `mv` of whichever leftover it finds into the new state dir when no explicit `$ZWIRE_STATE` is set, so an upgrade keeps your profile, base, and scheme intact and then never touches the old dir again. The theme is the one exception to this per-install dir: the shared colour scheme and light/fx prefs live in a *fleet-wide* location, `~/.zwire` (the host's `global.toml`, overridable via `$ZWIRE_GLOBAL_DIR`), read and written by every MenkeTechnologies app so they all follow one look, and the C++ color patch (fork 0002) resolves that same `~/.zwire` natively — so the scheme file the host writes is the file the browser reads. `~/.zwire` is therefore a real, distinct directory now, not merely shorthand for `$ZWIRE_STATE`. The per-install tree, rooted at `$ZWIRE_STATE`, is a small, legible directory:

```

$ZWIRE_STATE/      (macOS: ~/Library/Application Support/com.menketotechnologies.zwire ·
└─ Linux: ~/.config/zwire)
  base/             the extracted Chromium (or the packaged fork)
  base.path         absolute path to the resolved binary (the launcher reads this)
  base.version      the pinned revision, e.g. "r1656770" or "fork-150.0.7871.46"
  profile/          the dedicated --user-data-dir (bookmarks, history, sessions)
    NativeMessagingHosts/com.zwire.hud.json  the HUD native host manifest
    Default/NativeMessagingHosts/com.zwire.hud.json  second location Chromium checks
  ext/             per-user copy of the loaded extensions, staged at launch

```

The theme is deliberately *not* in this tree. The shared scheme, light/fx prefs, and any custom schemes live in the fleet-wide `~/.zwire/global.toml` (with the host's plain-text `hud-scheme/hud-light` projections beside it for the fork's C++ mixer), so every zwire-host client on the machine — the browser, the new tab, `zpwchrome`, and the wider app suite — reconciles to one theme file rather than a per-install copy.

Two of the entries above repay a second look. `profile/` is the isolation boundary — a complete, separate browser profile that never shares a byte with system Chrome, holding your zwire bookmarks, history, sessions, and the extensions' own stored state (`zb_scheme`, `zb_keys`, `zb_custom_cmds`, `zb_term_open`, and the rest). And `ext/` exists because Chromium writes extension `_metadata/` indexes *into the extension directory*, which means loading extensions straight out of a read-only `.app` bundle fails and, on a multi-user machine, two users writing the same directory would collide. So the loaded extensions are staged into a per-user writable copy under `ext/` at launch, and each user gets their own. The `base.version` file is the one place that records whether you are on a snapshot (`r...`) or a compiled fork (`fork-...`), which is the single bit that distinguishes the two build paths on a running machine.

`ZWIRE_STATE` being an environment variable means the whole tree relocates: point it at a different directory and you get a second, independent zwire — a different profile, a different base, a different native host manifest — without touching the first. That is the same isolation principle one level up: as zwire keeps its state out of system Chrome, `ZWIRE_STATE` lets you keep one zwire's state out of another's.

Chapter 20 – The CDP Verification Layer

The top-level `v*.mjs` scripts — `vkeys`, `vovl`, `vpl`, `vpop`, `vpos`, `vsys`, `vterm`, `vwin` — are **verification probes**. Each connects to a running zwire's Chrome DevTools Protocol endpoint on `127.0.0.1:9222`, attaches to a live page or extension page, and asserts that a specific HUD feature is actually wired and rendering. They are not the overlay itself — the vim keys, the tmux tiling, the statusbar, the palette, and the terminal all live in the `hud-internal` content scripts. The `v*.mjs` scripts are how you *check*, from the outside, that those content scripts did their job in a real browser.

Talking to a live browser over CDP

Every probe shares the same eight-line bootstrap, and understanding it once explains all of them. The launcher does **not** open a debugging port by default; you opt in by passing it through:

```
zwire --remote-debugging-port=9222
node vterm.mjs
```

With the port open, a probe does this: `fetch http://127.0.0.1:9222/json/version` to get the browser's `webSocketDebuggerUrl`, open a `WebSocket` to it, and speak CDP over that socket. Commands are JSON objects with a monotonic `id`, a `method`, `params`, and (once attached to a target) a `sessionId`; responses come back tagged with the same `id`, and a small map of pending resolvers matches each reply to its request. To reach a page the probe calls `Target.attachToTarget` with `flatten:true` for a session id, enables `Runtime`, and then evaluates expressions in the page with `Runtime.evaluate (returnByValue:true)`, reading back whatever the HUD put in the DOM. Some probes open a fresh target first (`Target.createTarget` at an extension page URL) and some find an existing page in `/json/list` (matching a stable site like `github/amazon/wikipedia`) to attach to. `9222` is the wire because that is the conventional CDP port and what the probes hardcode; the browser only listens on it when you passed the flag.

That these are external CDP probes rather than in-page code is the point: they verify the HUD the way a black-box test would, driving a genuinely running browser and asserting on its real DOM, which is the only way to prove the multi-hop paths (page → content script → service worker → native host → PTY → shell) actually connect end to end.

The modules and what each verifies

Each probe targets one HUD surface and prints a short list of assertions:

- `vkeys.mjs` opens the Keyboard settings page (`pages/keys.html`), confirms `ZWIRE_KEYMAP` is present, counts the editable key chips and category headers, then *exercises a remap*: it writes `zb_keys.scrollDown='s'` into `chrome.storage.local` and reads it back, proving the remap round-trips through storage where `zvim` would pick it up.
- `vterm.mjs` opens the standalone terminal page (`pages/terminal.html`), checks that xterm's `Terminal` global loaded and the pane rendered, waits several seconds for a heavy zsh login to boot over the native-host PTY, and reports whether rows of real shell text appeared.

- `vovl.mjs` and `vpop.mjs` attach to an ordinary web page, confirm the terminal content script injected a (hidden) `#terminalPane`, synthesize a `Ctrl+`` keystroke via `Input.dispatchEvent`, and verify the overlay becomes `active` and xterm renders — `vpop` waits longer and reads back the booted shell's visible text.
- `vwin.mjs` triggers the terminal from a page's `Ctrl+``, then looks in `/json/list` for the terminal popup window that opened, attaches to it, and confirms xterm rendered and `chrome.runtime.connectNative` is reachable from it.
- `vsys.mjs` waits for the native host's system-stats stream to fill `zb_sys`, then reads each statusbar segment — CPU, MEM, NET, HOST, LAN, UP, BATT, the powerline chevron count, and the clock — proving the real-machine stats path is live.
- `vpos.mjs` and `vpl.mjs` verify statusbar *layout*: `vpos` reads the left-to-right segment order and checks the `C-b` prefix sits left of the scheme segment; `vpl` inspects the powerline chevron pseudo-elements (the `::after` border that draws each chevron) and the prefix block's background, confirming the powerline geometry renders as intended.

Taken together the eight probes cover the runtime HUD's riskiest surfaces — the remap round-trip, the terminal's full native-PTY path, the overlay toggle, the system-stats stream, and the statusbar's powerline geometry — from outside the browser, over the same protocol any external tool would use. They are the runtime path's answer to the fork path's `git apply --check`: a concrete, re-runnable proof that the thing is actually wired, not merely written.

Two probe shapes, and no test framework

The probes come in two shapes that reflect what they need to reach. `vkeys` and `vterm` create a fresh target at an extension page URL (`chrome-extension://<hud-id>/pages/keys.html` or `pages/terminal.html`) and assert against it, because the keymap page and the standalone terminal are extension pages you can open in isolation. The other six *attach to an existing tab* — they scan `/json/list` for a real page whose URL matches `github`, `amazon`, or `wikipedia`, because the overlay, statusbar, and terminal content scripts only inject into ordinary web pages, and a probe needs a genuine one to inject into. When no such tab is open, those probes exit cleanly rather than failing, since “no stable page” is a setup gap, not a broken feature.

There is no assertion framework: each probe evaluates an expression in the page and `console.logs` the boolean or the count, and pass or fail is read off the printed lines by a human or a CI grep. The overlay probes synthesize the real toggle by dispatching a `Ctrl+`` key event through `Input.dispatchEvent` (with the Ctrl modifier) and then wait — five seconds for `vovl`, seven for `vpop`, six for `vwin` — for a heavy zsh login to boot over the full page → content-script → worker → native host → PTY path before reading back the shell's visible text, which is the only way to prove that multi-hop chain end to end. Two probe lines are honestly dead weight, and this book flags them rather than pretend the suite is airtight: `vterm` reaches for a `window.__zbTermInstance` global that does not exist (the real instance is module-scoped), and `vovl` prints a placeholder string for the `zb_term_open` check instead of actually reading storage. They do not weaken the probes that matter, but they are there, and a book grounded in the code says so.

What a probe cannot see

The probes are the runtime path's strongest evidence, and knowing their limits is part of reading them honestly. Four categories of failure are outside what an external CDP observer can assert.

The first is **timing that is correct but slow**. A probe waits a fixed number of seconds for a heavy login shell to boot and then reads the visible text; a session that would have appeared a second later reads as a failure, and one that appeared instantly reads the same as one that took the full wait. The probes prove that the path connects, not that it connects promptly, and the fixed waits are calibrated for the slow case rather than measuring anything.

The second is **state the probe cannot reach**. A probe evaluates expressions in a page and reads back what the HUD put in the DOM, which means anything held only in a module's closure is invisible to it. Two probe lines in the suite are honest casualties of this and the book has already flagged them: one reaches for a terminal instance global that does not exist because the real instance is module-scoped, and one prints a placeholder instead of reading a storage flag. Both are dead weight rather than false confidence — they assert nothing — but they illustrate the boundary exactly.

The third is **the negative case**. Every probe asserts that something is present; none asserts that something is *absent*. There is no probe that a content script did not double-inject, that an unclaimed key was left alone for the page, or that a feature turned off is actually off. Those are the assertions that would catch the regressions the earlier chapters worry about most — the idempotence guard failing, a listener claiming a key it does not own — and they are simply not in the suite. Naming that gap is more useful than implying the eight probes are a complete safety net.

The fourth is **anything on the fork path**. The probes attach over the debugging protocol to a running browser and read the page's DOM; the fork's contributions are native chrome — tab shapes, the toolbar line, menu colors, the audio engine — none of which is reachable from a page context at all. The fork's equivalent proof is `git apply --check` against the pinned tree, which is a completely different kind of claim: it proves the diffs match the source, not that the result behaves. The two verification stories cover the two tiers and neither reaches the other's.

Why no assertion framework

The probes print booleans and counts and let a human or a grep decide, and it is worth defending that rather than treating it as an omission. A test framework buys you three things — structured pass/fail, setup and teardown, and a runner — and each of them is either unavailable or actively unhelpful here.

Structured pass/fail requires a definition of failure, and several probes have a third outcome that is neither: when no ordinary page is open to attach to, the correct behavior is to exit cleanly, because “no stable page” is a setup gap rather than a broken feature. A framework would have to model that as a skip, which is a concept the probes get for free by printing what they found and stopping.

Setup and teardown are impossible in the useful sense. The probes attach to a browser the operator launched with a debugging port they opted into, holding whatever profile state they have; the probes deliberately do not construct that environment, because constructing it would mean testing a browser that no user runs. The realism *is* the value.

And a runner would add a dependency to a project whose entire testing posture is that the base is prebuilt and there is nothing to compile. A probe is a single script with no imports beyond the platform's own fetch and web socket. That property is what lets one be written in an afternoon when a new HUD surface needs covering, which matters more, for a suite this size, than the ergonomics a framework would add.

What it would take to assert the negative

The gap the previous section names — every probe asserts presence, none asserts absence — deserves more than an acknowledgement, because the assertions that are missing are exactly the ones guarding the invariants the rest of the book leans on hardest. Three of them are worth designing on paper, both because they are achievable over the same protocol and because working out how you would test an invariant is the fastest way to find out whether you have actually stated it.

Single injection. The invariant is that a content script injected twice into the same document is a no-op the second time, which Chapter 7 records as the load guard and Chapter 26 records as the thing standing between a long-lived tab and a slow accumulation of stacked handlers. Asserting it externally requires a countable consequence of injection. Counting listeners is not available to a page-context expression, so the assertion has to be phrased against something the script produces exactly once — the count of a HUD element it inserts,

or the count of a marker it sets. The probe shape is: attach to a page, read the count, drive a same-document navigation, read it again, assert it did not move. That is a genuine regression test for the exact failure Chapter 26 says the guards prevent, and it is buildable with the machinery the existing probes already use.

The unclaimed key space. The invariant is Chapter 28's: a HUD listener calls `preventDefault` only when the key maps to a real action, so an unknown key reaches the page untouched. This one is testable and the test is elegant, because the debugging protocol can synthesize a key event and the page can observe whether it arrived. Install a listener in the page context that records the keys it sees, dispatch a key that is deliberately absent from the merged keymap, and assert the page's listener recorded it and that the default was not prevented. Then dispatch a key that *is* mapped and assert the opposite. Two dispatches and two assertions cover the invariant in both directions, which is more than any presence-only probe can claim.

A feature that is off. The invariant is that a toggle actually toggles — that vim disabled for a tab means the raw keys go back to the page, that a hidden statusbar is absent rather than transparent, that a terminal whose flag was reset does not re-open. Each of these is an assertion that some element is not in the DOM or some key is not claimed, and each is the exact assertion that would catch the class of bug where a feature's disable path was never wired. Chapter 31's real-boolean rule — a flag compared to `true` rather than for truthiness, with a named reset owner — is a rule with no automated enforcement anywhere, and a probe asserting “after reset, absent” is that enforcement.

Two things make these harder than they look, and they are the reason the suite does not already have them. The first is that a negative assertion has a failure mode a positive one does not: it passes when the feature is broken in a way that removes the thing entirely. A probe asserting “the statusbar is absent after hiding it” passes on a build where the statusbar never injected at all. Every negative assertion therefore needs a positive precondition — assert it is present, toggle, assert it is gone — which doubles the choreography and makes the probe stateful in a way the current ones are not.

The second is timing, which the section above already names as the first category of failure a probe cannot see. A negative assertion evaluated too early passes for the wrong reason, because the thing has not appeared yet. The existing probes handle timing with fixed waits calibrated for the slow case, which is adequate for “did it appear” and actively wrong for “is it absent” — a fixed wait before a negative assertion is a coin flip. The correct shape is a poll with a deadline in the positive direction and a settle-then-assert in the negative, which is more machinery than the current probes carry and is precisely the machinery a test framework would have supplied.

That last observation is the honest counterweight to the previous section's defense of having no framework. The defense holds for the assertions the suite makes today: presence checks against a live browser, printed as booleans, read by a human or a grep, with no dependency beyond the platform. It holds less well for the assertions the suite is missing, because negative assertions need setup, teardown, and a retry policy — which are three of the things a framework exists to provide. The complete position, then, is not “a framework would add nothing” but “a framework would add nothing yet,” and the day the suite grows its first negative assertion is the day that calculation changes.

Chapter 21 – The Source Fork

`fork/` compiles a patched Chromium so the *whole* browser — tab shapes, neon toolbar, HUD-colored frame, native menus and dialogs, and a browser-wide audio engine — lives in the HUD, at the cost of a ~100 GB checkout and a one-to-four-hour first build. All twenty-five HUD patches are authored against the pinned tag `150.0.7871.46` and verified apply-clean (58 hunks across 31 Chromium files). This chapter is the pipeline and the patch series, with one real hunk quoted so the level of the work is clear.

The pipeline: fetch, apply, build, package

The fork is four scripts run in order, and each is a thin, honest wrapper around the Chromium build tools:

1. `fork/fetch.sh` installs `depot_tools` if missing, `fetch --no-history chromium` into `$ZWIRE_SRC` (default `~/zwire-chromium`), checks out the exact tag from `fork/CHROMIUM_VERSION` onto a `zwire-<tag>` branch, and `gclient syncs` the deps for that tag. `--no-history` keeps the checkout to “only” ~100 GB rather than far more. This is the long, network-bound step.
2. `fork/apply-patches.sh SRC [--reverse]` reads `fork/patches/series` (an ordered list with in-line comments stripped) and applies each patch with `git apply --3way` from the Chromium `src/` dir; `--reverse` unapplies them in reverse order, which is how you back the patches out before rebasing onto a new Chromium tag.
3. `fork/build.sh SRC [OUT]` copies `fork/args.gn` into `out/<OUT>` (default `zwire`), runs `gn gen`, and then `autoninja -C out/<OUT> chrome`. That last command is the long compile — one to four hours the first time, minutes for an incremental rebuild after a single patch.
4. `fork/package.sh OUTDIR` installs the freshly built app as the `zwire` base: on macOS it finds the built `.app` (named `zwire.app` if the branding patch ran, else `Chromium.app`), copies it into `~/./zwire/base` under its own name, derives the real executable from the bundle's `CFBundleExecutable`, writes `base.path` and a `fork-<tag>base.version`, and then runs the same `scripts/rebrand-macos.sh` the runtime path uses. After this the same `bin/zwire` launches the HUD-chromed fork with the extensions preloaded — same launcher, forked base.

`fork/args.gn` configures the build: an *unbranded* Chromium (`is_chrome_branded=false`), an official (fully optimized) release build (`is_official_build=true`), with the slow parts turned off for a workstation build time — `symbol_level=0` and `blink_symbol_level=0` (no debug symbols, faster link), `chrome_pgo_phase=0` (skip profile-guided optimization), `enable_nacl=false` (drop legacy Native Client), and `use_thin_lto=false` (ThinLTO is slow for iteration). You set `target_cpu` to `arm64` or `x64` for your host. These are the knobs that turn a multi-hour Chromium build into merely a long one.

What “apply-clean” proves, and what it does not

The fork’s own docs state the boundary plainly, and the book repeats it because it is the honest status of the fork. Each patch was authored by reading its target file at the exact tag on the Chromium GitHub mirror (`raw.githubusercontent.com/chromium/chromium/150.0.7871.46/...`) and is verified with `git apply --check` against that tree. That proves the diff *context matches the pinned source* — the patch will apply without a reject. It does **not** prove the C++ *compiles*; `fork/build.sh` is that gate. So the series’ status is “all twenty-five authored and apply-clean against `150.0.7871.46`,” which is a real, checkable claim, and not the

stronger “compiled and shipped” claim, which the fork does not make. A CI check, `scripts/check-patch-series.sh`, keeps the `series` file and the patch files on disk consistent so the two never drift.

The twenty-five HUD patches

The patches live in `fork/patches/`, are listed in `series`, and each targets one native surface — a look a theme extension cannot restyle, or a behavior it cannot override. They fall into six groups: **nine styling** patches (0001–0009), **six behavior** patches that force zwire’s defaults over Chromium’s (0010–0015), **three native-color** patches that bind the native Views surface to the HUD palette (0020–0021, 0025), **three extension-compatible** patches that keep the HUD’s own pages and the Chrome Web Store working (0016, 0018, 0019), **one quit** patch (0017), and the **three audio** patches that compile a browser-wide DSP engine into the audio service (0022–0024). Read against the target-file map in `fork/patches/README.md`:

0001 — Tab shape (`chrome/browser/ui/views/tabs/tab_style_views.cc`, in `TabStyleViewsImpl::GetPath`). The patch is more than a corner-radius tweak: it replaces the rounded-rect tab path with a *hard trapezoid* — the classic-Chrome tab silhouette with straight diagonal sides — while keeping the interior clip and hit-test paths rectangular so the favicon and title are never clipped by the sloped edges. The real hunk:

```
- path.lineTo(tab_left, tab_top + top_left_corner_radius);
- path.arcTo(SkVector(top_left_corner_radius, top_left_corner_radius), 0,
-           SkPathBuilder::kSmall_ArcSize, SkPathDirection::kCW,
-           SkPoint(tab_left + top_left_corner_radius, tab_top));
+ // zwire HUD: diagonal left side up to the inset top-left corner.
+ path.lineTo(tab_left + hud_slope, tab_top);
```

The slope is $(\text{tab_bottom} - \text{tab_top}) * 0.28$, applied on each side, so the tab is a trapezoid whose top edge is inset from the bottom. The patch also nudges the favicon and title 8px to the right so they sit clear of the hard trapezoid’s sloped left edge rather than crowding into it. This is a real Views-layer restyle, not a color change — exactly the kind of thing the runtime path cannot do.

0002 — UI colors (`chrome/browser/ui/color/chrome_color_mixer.cc`, in `AddChromeColorMixer`, before the high-contrast early-return). Forces the cyberpunk palette onto frame, toolbar, toolbar text and icons, tab background and foreground, and omnibox background and border, and recolors the under-toolbar separator neon cyan, before the high-contrast early-return. The patch has since grown to seed the eight HUD schemes as Sys-token palettes and to end with a dead-last `AddZwireHudFinalMixer` that re-asserts the tab colors so the Material/native tab-strip mixers can’t leave the active tab a scheme-independent default. This is the patch that reaches everything the theme extension cannot, and the one the unloaded `theme/` extension would fight if it were loaded. Two later refinements live in the same patch. First, the native chrome now follows **light mode**: the mixer reads the `$ZWIRE_STATE/hud-light` projection the host writes when `[theme.ui].light` flips (Chapter 8), swaps the dark scheme values for the light variant, and repaints at startup so the choice applies on launch rather than only after a toggle. Second, the **focus ring follows the scheme** — the patch overrides Chromium’s stock GM3 red focus ring with the active scheme’s accent, so the ring matches the HUD instead of clashing with it.

0003 — UI font. The target is `ui/base/resource/resource_bundle.cc`, in the `InitDefaultFontList` method. It sets `gfx::FontList::SetDefaultFontDescription("Share Tech Mono, Monaco, 13px")` so all native chrome UI text renders in the HUD mono face, with Monaco as a guaranteed-present fallback before Share Tech Mono is installed.

0004 — Toolbar neon border (`chrome/browser/ui/views/toolbar/toolbar_view.cc`, in `Init`, the `kNormal` branch). Adds a 2px cyan bottom border via `views::CreateSolidSidedBorder(..., kColorToolbarContentAreaSeparator)`; the separator color itself is set neon cyan back in 0002, so the two patches together make the under-toolbar line glow.

0005 — Omnibox field. The target is `chrome/browser/ui/views/location_bar/location_bar_view.cc`, in the `ComputeBorderRadius` method. It clamps the field radius to 2px so the omnibox is the

sharp HUD field instead of the rounded pill; its background and border colors come from 0002.

0006 — Product strings (`chrome/app/theme/chromium/BRANDING`). Rewrites the product names `Chromium` → `zwire`, the company to `MenkeTechnologies`, and the bundle id to `com.menketechologies.zwire`. This is the file the unbranded (`is_chrome_branded=false`) build reads, which is why the fork's `package.sh` looks for a `zwire.app` and only falls back to `Chromium.app`.

0007 — DevTools theme (three DevTools front-end files: `design_system_tokens.css` appended, `main-meta.ts` for the `ui-theme` setting, and `ThemeSupport.ts` for `applyTheme`). Adds the eight HUD schemes to the DevTools Theme dropdown, default `zbhud-cyberpunk`. `ThemeSupport` maps `zbhud-<scheme>` onto the `dark` base and stamps `[data-zbhud=<scheme>]` on `<html>`; the appended CSS maps the `--sys-color-*` tokens onto per-scheme `--zb-*` key colors. Without `data-zbhud`, stock Light/Dark/Auto are untouched — the patch adds schemes rather than replacing the existing ones.

0008 — Allow framing (`content/browser/renderer_host/ancestor_throttle.cc`). Bypasses `X-Frame-Options` and frame-ancestors CSP so the tmux overlay (`ztmux.js`) can iframe any site into its N-pane tiling. This patch *replaces* an earlier `0008-tmux-split-prefix` patch, whose native 2-pane split was superseded by the JS overlay — the old patch consumed the `Ctrl-b` prefix and the next key before the page saw them (the “C-b %%” double-tap bug), so the native split was removed and the real N-pane tiling moved entirely into the content script, which just needs framing allowed.

0009 — Ext-command focus (`chrome/browser/ui/views/extensions/extension_keybinding_registry_views.cc`). When an extension command fires while a chrome UI surface (the omnibox or search bar) has focus, this hands focus to the page web contents so the `⌘K` palette overlay is typeable from the omnibox — closing the same focus gap the new-tab page solves in JavaScript, but at the engine level for ordinary pages.

0010 — Force zwire new-tab (`chrome/browser/search/search.cc`, in `HandleNewTabURLRewrite`). When the zwire new-tab extension is loaded, `chrome://newtab` always resolves to it instead of the default Google NTP, so the new tab is always the HUD — even the startup tab. This is the engine-level partner to the extension's own `chrome_url_overrides.newtab`: the override alone loses races on the very first tab, so the rewrite pins it.

0011 — Pinned extensions always show (`chrome/browser/ui/views/toolbar/toolbar_view.cc`). Holds the extensions container at its preferred width so pinned actions never collapse into the puzzle overflow when an extension is re-pinned at runtime — the zwire toolbar keeps its pinned tools visible instead of hiding them behind the generic extensions menu.

0012 — Allow frame-src (`content/browser/renderer_host/navigation_request.cc`). A host page's `frame-src` / `child-src` CSP never blocks a sub-frame navigation, so the tmux overlay can embed any site into a pane even when the top page's CSP would forbid it. This complements 0008: 0008 bypasses the *framed* site's frame-ancestors defense, and 0012 bypasses the *framing* page's frame-src restriction — both directions of the CSP handshake have to yield for arbitrary panes to work.

0013 — Restore last session (`chrome/browser/prefs/session_startup_pref.cc`). Sets the default startup preference to restore the last session, so tabs auto-restore on launch rather than opening a single fresh new-tab.

0014 — No session-crashed bubble (`chrome/browser/ui/views/session_crashed_bubble_view.cc`). Never shows the “Restore pages?” session-crashed bubble — the restore-last-session default (0013) already brings the tabs back, so the prompt is redundant noise the HUD suppresses.

0015 — No navigation block (`extensions/browser/extension_navigation_throttle.cc`). Never blocks a `chrome-extension://` navigation to a not-yet-registered extension, so the new-tab override page always loads at startup even in the window before its extension has finished registering — the engine-side race fix that makes 0010's forced new-tab reliable on the very first paint.

0016 — Ext private-API allowlist (`chrome/common/extensions/api/_permission_features.json`). Allowlists the `hud-internal` extension (by hash) for `developerPrivate` — widening the API's `extension_`

`types` to include `"extension"` — and `settingsPrivate`, so the HUD's own Extensions and Settings pages can drive `chrome.developerPrivate` and `chrome.settingsPrivate` the way the built-in `chrome://extensions` and `chrome://settings` do. Without it the reimplemented pages have no engine to talk to.

0017 — No hold-to-quit (`chrome/browser/app_controller_mac.mm`, in `runConfirmQuitPanel`). Forces the hold-⌘Q-to-quit gesture *off* — immediate quit — regardless of the `kConfirmToQuitEnabled` pref, which a synced profile can otherwise flip back on. A daily driver quits when you press quit.

0018 — Null policy-container crash fix (`extensions/browser/extension_function_dispatcher.cc`). Gates the `is_sandboxed` UMA probe on `RenderFrameHost::IsActive()` so an extension-function IPC arriving from a non-active frame (with a null `policy_container_host_`) cannot reach the `active_sandbox_flags()` `NOTREACHED`. This fixes a real SIGTRAP: a content script calling a `chrome.*` API on `chromewebstore.google.com` mid-navigation would crash the browser without it.

0019 — Web Store scripting (`chrome/common/extensions/chrome_extensions_client.cc`, in `IsScriptableURL`). Drops the gallery special-case (`kCannotScriptGallery`) so extension content scripts and `tabs.executeScript` run on the Chrome Web Store domains like any other page — store-privileged JavaScript bindings stay gated by `IsWebstoreOrigin` in `permissions_data.cc`, so this widens *scripting*, not *privilege*.

0020 — Native menu colors (`chrome/browser/ui/color/chrome_color_mixer.cc`, in `AddZwireHudFinalMixer`). Binds the `ui::kColorMenu*` family — the app/three-dot menu, context menus, and dropdowns — to the HUD Sys tokens (`kColorSysBaseContainer` and friends) so native Views menus follow the scheme and the light/dark toggle instead of the `NativeTheme` default that left light text on a light background. References, not literals: it depends on 0002's Sys-token block.

0021 — Native primitive colors (same file and mixer). Generalizes 0020 to the whole native surface. HUD light mode recolors the Sys tokens but never flips `NativeTheme`, so `core_default_color_mixer` still builds every native-Views *primitive* (`kColorPrimary/Secondary`, the background and midground endpoints) from the OS color mode — which inverts dialogs, bubbles, textfields, and tooltips when the HUD and OS modes disagree. Rebinding those primitives to the HUD-mode-aware Sys tokens makes the entire native tree track HUD light/dark + scheme in one place.

0022 — Output audio EQ (`services/audio/output_controller.{cc,h}` plus `chrome/browser/chrome_content_browser_client.cc`). The first of three patches that give zwire a **browser-wide equalizer**. A preamp + RBJ-biquad cascade (`ZwireAudioEq`) is applied in `OutputController::OnMoreData()` — the audio service's universal chokepoint that every output stream passes through before the OS device — so all tab audio (media element, MSE/YouTube, Web Audio, WebRTC) is equalized, not just the subset the renderer mixer sees. The chain is preamp → EQ cascade → gain → `tanh` soft-clip drive per channel, then an equal-power pan / mono-fold across the stereo pair. It is always on: an empty config is unity gain (audio still routed through it), and only `"off"` bypasses. Because the re-signed macOS audio utility is sandboxed and does not inherit the environment, `ChromeContentBrowserClient::AppendExtraCommandLineSwitches` forwards the config as the `--zwire-audio-eq` switch (switches survive the sandbox; env does not).

0023 — Audio tab-capture (`chrome/browser/extensions/api/tab_capture/tab_capture_api.cc` plus `chrome/common/extensions/api/_api_features.json`). Lets the HUD Audio page pull a playing tab's *real* output into its analyzer: it exposes `tabCapture` to `hud-internal` without the manifest permission (a second allowlisted feature variant, mirroring 0016) and defaults the allowlisted extension id so `getMediaStreamId` bypasses the `activeTab` picker and prompt. The grant is scoped to that one id.

0024 — Live audio config + meters (`services/audio/public/mojom/audio_service.mojom`, `services/audio/service.{cc,h}`, `content/browser/audio/audio_service.cc`). Makes the 0022 EQ **live** with nothing open and no relaunch. The audio sandbox cannot read the config file, so the unsandboxed browser process polls it off-thread and, on change, pushes the spec over a new `mojom AudioService::SetZwireEqConfig`, which atomically swaps the process-global config the cascade reads each block. The same patch carries the **meters** back-channel: the service computes a compact post-DSP

frame (a Goertzel spectrum, peak/RMS per channel, phase correlation, a decimated stereo scope) of the real output and pumps it — on a dedicated ThreadPool sequence, off the UI thread so trackpad gestures cannot starve it — to `$ZWIRE_STATE/meters`, which the HUD Audio page reads over the native host. Because the page reads the engine's true output rather than a `tabCapture` of the source, watching the meters never mutes or drops the audio.

0025 — Native custom palette (`chrome/browser/ui/color/chrome_color_mixer.cc`, again in `AddZwireHudFinalMixer`). This one closes the last gap between the HUD and the native chrome: **custom** schemes. Patch 0002 paints the native surface by matching the active scheme's *name* against a baked eight-entry table (`kHudSchemes`), which works for the eight built-ins and fails silently for anything else — a user-authored scheme is named `custom-N`, that name is in no table, and the native chrome stays stuck on whichever built-in it painted last while the extension surface follows the new colors. 0025 removes the name lookup from the decision. The mixer reads a `hud-palette` base-color projection — a plain text file of `--var=#rrggbb` lines that `zwire-host` writes out of `[theme.palette]` whenever the scheme changes, the same projection mechanism Chapter 8's sync bus uses for `hud-light` — and, when the file is present and complete, paints tab labels and accent, toolbar, omnibox, and frame from those **resolved** colors instead of the table. A name is no longer evidence of anything; the colors are. Two implementation details carry the change: `ActiveHudScheme()` now returns by value (call sites bind a `const&`) because the palette can be rebuilt under them, and `ReadHudPaletteFile()` parses under a lock and repaints on change through the existing `~/.zwire` directory watcher, so a scheme switch repaints the native chrome without a relaunch. It depends on 0002 — it replaces that patch's color *source*, not its wiring — and it is the reason the scheme picker no longer needs its entries to exist in a C++ table to be usable.

The rebase cost

The rebase story is the fork's ongoing cost and the docs do not hide it: the `radius-calc`, `color-mixer`, `Init`, and `ComputeBorderRadius` anchors shift between Chromium milestones, so most releases you re-anchor most patches. The procedure is in `fork/patches/README.md` — bump `CHROMIUM_VERSION`, reverse the series off the old tree, fetch the new tag, re-apply and fix any `.rej`, and re-build. You can re-verify an anchor without a full checkout by reading the target file at the new tag from `raw.githubusercontent.com/chromium/chromium/<TAG>/<path>` and `git apply --check`-ing against it. That is the solo-fork commitment, and it is the price of putting the native chrome in the HUD.

The build config, and the series discipline

`fork/args.gn` is the whole build configuration, and every knob is a deliberate trade between build quality and build time on a single workstation:

```
is_chrome_branded      = false    # ships as unbranded Chromium, not Google Chrome
is_official_build       = true      # full release-quality optimization
proprietary_codecs      = true      # H.264 / AAC / MPEG in <video>/MSE
ffmpeg_branding         = "Chrome"  # bundle the ffmpeg build that carries them
enable_platform_hevc    = true      # allow HEVC where the OS provides it
enable_platform_hevc_decoding = true
is_debug               = false
is_component_build      = false
symbol_level           = 0          # no debug symbols -> much faster link, smaller
blink_symbol_level      = 0
chrome_pgo_phase        = 0          # skip profile-guided optimization (a large time sink)
enable_nacl             = false      # drop legacy Native Client
use_thin_lto            = false      # ThinLTO is slow; off for iteration builds
target_cpu              = "arm64"    # set to your host: "arm64" | "x64"
dcheck_always_on        = false
```

The first two lines are the identity — unbranded so there is no Google logo and the `BRANDING` strings of patch

0006 take effect, official so the result is a real release build; the last of the trailing block are the knobs that turn a multi-hour Chromium compile into merely a long one by dropping symbols, PGO, NaCl, and ThinLTO.

The codec block is the one place the fork's *behavior* differs from a stock Chromium build. Stock Chromium ships `ffmpeg_branding="Chromium"` — open codecs only (VP8/VP9, AV1, Opus, Vorbis) and no H.264 or AAC — so an H.264 stream that plays in Google Chrome plays in the zwire fork's stock base only as silence and a black frame. Setting `proprietary_codecs=true` with `ffmpeg_branding="Chrome"` bundles the ffmpeg build that carries H.264/AAC/MPEG, so the fork decodes what Chrome does: Synology SurveillanceStation web live-view, most IP cameras, and ordinary MP4/AAC media. HEVC (H.265) is *not* software-decoded even here — Chromium routes it to the platform hardware decoder (VideoToolbox on macOS), which the two `enable_platform_hevc[_decoding]` flags wire up. This is a fork-only capability by construction: the runtime rebrand runs the prebuilt open-codec snapshot and cannot add codecs without recompiling.

The patch series has its own discipline, enforced in CI. `fork/patches/series` is the ordered list `apply-patches.sh` reads, and `scripts/check-patch-series.sh` asserts a strict one-to-one correspondence between the entries in that file and the `*.patch` files on disk — a patch on disk but missing from `series` would silently never apply, and a `series` entry with no file would fail to apply, and the check catches both drifts before they ship. Each series entry carries an inline note recording that it was verified apply-clean against `150.0.7871.46`, and one entry is instructive by its absence: an earlier `0008-tmux-split-prefix.patch` is commented out with an explanation, because its native two-pane split “consumed the prefix + the next key before the page saw them (the ‘C-b %%’ double-tap)” and was superseded by the JS overlay — its `0008` number reused by the framing patch. The series file is not just a manifest; it is a small changelog of what the fork tried and backed out.

The economics of the rebase

The runtime rebrand tracks upstream Chromium for free; the fork does not, and the difference is the single most important thing to understand before choosing the compiled path. A stock snapshot is a security fix one `fetch-base.sh` away because it is not a fork of anything — it *is* the upstream artifact, re-signed and renamed. The fork, by contrast, is twenty-five patches pinned to one tag, and every time Chromium moves, those patches must be carried forward across a codebase that changes under them without warning. That carrying-forward is the recurring cost, and it is worth being precise about where it falls, because it does not fall evenly across the series.

The patches sort into three risk tiers by *what* they touch. The cheapest are the string and resource patches — the `BRANDING` file, the product name, the icon — which live in files that change slowly and rarely conflict; a rebase moves them with a trivial context shift or none at all. The middle tier is the CSS-and-color work: the trapezoid tab shapes, the neon under-toolbar line, the Share Tech Mono UI font, the frame color wired to the scheme. These live in the Views layer, which Google refactors on its own schedule, so they conflict *sometimes* — a renamed method here, a moved constant there — but the conflicts are mechanical, because the patch's intent is legible from its diff and the new upstream shape is usually obvious. The expensive tier is the behavioral C++: the `is_sandboxed` gate in the function dispatcher, the three audio-service patches that reach into `OutputController::OnMoreData` and add a new `mojom` method, the codec branding. These touch code that carries real invariants, and a rebase there is not “fix the context” but “re-understand the surrounding code well enough to know the patch is still correct” — because upstream may have changed the very assumption the patch depended on. The audio EQ that hooks the universal mixing callback is the canonical example: if Chromium reworks how the audio service routes output, the patch does not fail to apply — it applies and is *wrong*, which is the worst outcome a rebase can produce.

This asymmetry is why the fork is framed throughout the book as the tier for whoever will *pay the build-and-rebase cost*, not merely the build cost. The build is a one-time multi-hour compile made merely long by the `symbol_level=0` and no-PGO knobs; the rebase is a recurring tax that scales with how far the pin has fallen behind. The disciplines that make it survivable are exactly the ones this chapter described: the `series` file as a manifest CI keeps honest, the inline apply-clean notes that record which tag each patch was last verified against, and the willingness — visible in the commented-out `0008` — to back a patch out rather than carry

a fragile one forward. The honest reading is that the fork is a maintained thing, not a finished one, and the runtime rebrand exists precisely so that the daily driver never has to be. The line between the two tiers is not a wart; it is the boundary between what upstream will keep working for you and what you have chosen to keep working yourself.

Chapter 22 – Dev Loop, Testing, and CI

zwire is a browser you assemble, not compile, so its “build” tooling is a set of small shell scripts that fetch, rebrand, cache-bust, and reset the base, plus a JS syntax gate and a CI that lints every script and validates every JSON. This chapter is that loop.

The script set

The `scripts/` directory is a family of dev-loop commands, all sharing one cyberpunk banner wrapper (`cyberpunk.sh`, which just sets ANSI color variables and a banner from `APP_TITLE/APP_SUB`). Reading them as a set:

```
dev.sh      launch the browser from the checkout (extensions unpacked,
            foreground; edits to content scripts hot-reload)
build.sh    "build" == make sure a rebranded base bundle exists + is current
            (fetch the snapshot if missing, then rebrand) – no compile step
clean.sh    remove the /Applications wrapper + staged dist/, keep base + profile
rebuild.sh  clean + build
bust.sh     re-seal + re-register the bundle with LaunchServices (Chapter 5)
nuke.sh     wipe base + wrapper, re-fetch from scratch, rebrand, reinstall
            (preserves the profile)
test.sh     a JS syntax gate over the front-end + each extension's own tests
```

The design point is that none of these compile the browser — the base is a prebuilt snapshot — so “build” and “rebuild” are really “make sure a current, rebranded base exists,” and the destructive ones are careful about what they keep: `clean` and `nuke` never touch your profile, and `clean` does not even touch the 325 MB base (that is `nuke`’s job). The npm `scripts` in `package.json` alias these (`pnpm dev`, `pnpm build`, `pnpm nuke`, and so on) for anyone who reaches for the package manager.

Testing

`scripts/test.sh` is a JS syntax gate over the repo’s own front-end (the new-tab page, the theme, the root `v*.mjs` scripts) plus a delegation to each bundled extension’s own test script. There is no Rust or compile step at the browser level, because the base is a prebuilt snapshot; the tests that matter for behavior are the CDP verification probes of Chapter 20, which drive a running browser. The syntax gate catches the cheap failures (a broken content script would silently not load in Chromium) before they reach a launch.

CI

The GitHub Actions `ci.yml` runs three checks on every push and PR, on a plain Ubuntu runner, with the `zpwchrome` submodule deliberately *not* checked out (nothing CI checks needs it):

- `shellcheck` over `bin/zwire`, `scripts/*.sh`, and `fork/*.sh` at warning level — every shell script the project ships is linted.

- **JSON validation** over every tracked `*.json` outside the `zpwchrome` submodule, discovered with `git ls-files` (never hardcoded), each parsed with `python3 -c json.load`.
- **patch-series consistency** via `scripts/check-patch-series.sh`, which asserts the `fork/patches/series` list and the patch files on disk agree.

A second workflow, `pages.yml`, publishes `docs/` to GitHub Pages — the site the README's “docs · online” badge points at — on any push that touches `docs/`, with a single-deployment concurrency guard so an in-flight publish is never cancelled. The CI is intentionally lightweight: lint, validate, check consistency. It does not build the browser (there is nothing to build) or compile the fork (that needs the 100 GB checkout); those are local, deliberate operations.

Inside the script family

The dev-loop scripts share one banner library (`cyberpunk.sh`, sourced for the ANSI banner keyed off `APP_TITLE`) and each is careful about what it keeps. `dev.sh` runs the browser from the checkout in the foreground — its comment notes that edits to the content scripts hot-reload while a `background.js` change needs the reload glyph on `chrome://extensions`. `build.sh` states outright that “‘build’ == ‘make sure a rebranded base bundle exists and is current’”: it fetches the snapshot if missing and runs the rebrand on Darwin only, with no compile step because the base is prebuilt. `nuke.sh` is the heavy reset — it removes the `/Applications` wrapper and the 325 MB base, re-fetches, rebrands, and reinstalls, and its comment is emphatic that “your profile ... is preserved.” `bust.sh` is the macOS-only icon-cache buster and exits early on any other platform.

`test.sh` is a JS syntax gate, not a behavior suite, and its comment says why: “there is no Rust/compile step at the browser level — the base is a prebuilt snapshot.” It runs `node --check` over the repo's own front-end (the new-tab page, the theme, the root scripts), deliberately excluding `node_modules` and the vendored `lib/zgui-core`, and then delegates to each bundled extension's own `pnpm test`. A broken content script would fail silently in Chromium rather than throw, so catching the parse errors cheaply, before a launch, is the point; the behavioral proofs are the CDP probes of Chapter 20, which drive a running browser rather than lint a file.

What the syntax gate catches, and what it structurally cannot

The test script's own comment explains why it is a syntax gate rather than a behavior suite, and it is worth taking the argument seriously rather than reading it as an apology. In an ordinary application a parse error is loud: the program refuses to start. In a browser extension it is not. A content script with a syntax error does not throw into anything the user sees; Chromium declines to inject it and the feature is simply absent, on every page, with no error and no visible difference from a feature that was never written. That failure mode — silent, total, and invisible — is exactly the one a cheap parse check eliminates completely, which is why the cheapest possible gate is disproportionately valuable here.

The gate's scope is drawn to match that reasoning. It parses the repository's own front-end and the root probe scripts, deliberately excluding vendored trees and the shared toolkit submodule, because a submodule's syntax is the submodule's CI's problem and re-checking it here would only produce failures nobody in this repository can fix. Then it delegates to each bundled extension's own test script, which is the same reuse-not-duplicate discipline the rest of the project follows: the extension knows how to test itself.

What it cannot catch is everything above the parser. A script that parses perfectly but registers a listener on the wrong event, writes a storage key nobody reads, or claims a key it should have left alone will pass the gate and fail in the browser. There is no type checking, no linting of the JavaScript, and no execution. This is a deliberate floor rather than an aspiration: the behavioral proofs live in the verification probes, which drive a running browser, and the gate exists to make sure the code they are driving actually loaded.

What CI checks, and the one thing it protects that nothing else does

The three CI checks look modest and two of them are: linting every shell script the project ships and parsing every tracked JSON file are hygiene, valuable mostly because this project *is* shell scripts and JSON manifests to an unusual degree. A malformed manifest is another silent failure — an extension that does not load, with the developer banner still appearing because the other two did — so parsing every JSON file is the manifest-level analogue of the syntax gate.

The third check is different in kind. The patch-series consistency check asserts a one-to-one correspondence between the entries in the fork's series file and the patch files on disk, and it is the only automated statement anyone makes about the fork without a hundred-gigabyte checkout. Both directions of the drift it catches are genuinely dangerous and neither is visible by inspection. A patch file on disk that is missing from the series never applies, so a fix you believe is in the build is not; the symptom is a fork that behaves like an older one for no discoverable reason. A series entry with no file fails at apply time, which is at least loud, but it fails after the long fetch step rather than before it. Catching both on every push, on a runner that never touches Chromium, is a large amount of value for a script that compares two lists.

It is worth naming what CI deliberately does not do, since the omissions are principled. It does not check out the power-tool submodule, because nothing it checks needs it — which also means CI runs on a clone shaped exactly like the reduced one a user gets without recursive initialization, incidentally exercising the graceful-skip path. It does not build the browser, because there is nothing to build. And it does not compile the fork, because that requires the full source tree and hours of compute; the fork's CI story stops at “the patches are consistent and were verified apply-clean against the pinned tag,” and the chapter on the fork is explicit that this is a weaker claim than “compiles.”

The dev loop's real shape

Read as a workflow rather than a list, the scripts form a ladder of increasingly expensive resets, and knowing which rung to reach for is most of the value. Running the browser from the checkout is the inner loop, and its important property is that content script edits hot-reload while a background worker change needs an explicit reload on the extensions page — a distinction that costs people minutes until they know it, because a worker edit that appears not to have taken is indistinguishable from a worker edit that was wrong.

Above that, “build” means only that a rebranded base exists and is current, so it is cheap and safe to run at any time. “Clean” removes the installed wrapper and the staged distribution while leaving the base and the profile alone, which makes it the right first move when the *packaging* looks wrong. The cache-buster addresses the one macOS failure that no amount of correct code fixes — a stale LaunchServices icon — and does nothing anywhere else. And the full reset re-fetches the base from scratch, which is the expensive rung and the one that costs a large download, while still preserving the profile. That preservation is the load-bearing property of the whole ladder: every rung, including the most destructive, leaves the user's bookmarks, history, sessions, and extension state intact, which is what makes reaching for a reset a reasonable troubleshooting step rather than a last resort.

Chapter 23 — Packaging and Self-Contained Installs

`scripts/install.sh` is the developer install — a PATH symlink to the checkout. But `zwire` also ships three *self-contained* installers that bundle everything the browser needs into one location, so you can delete the repo and the base and the app still runs. This chapter is those three, and the one substitution they all make.

The developer install vs the self-contained install

`scripts/install.sh` (Chapter 5) is the light path: it fetches the base, symlinks `bin/zwire` into `~/.local/bin`, and rebrands on macOS. The launcher it links still runs *from the checkout* — delete the repo and `zwire` breaks. That is right for development, where the checkout is where you are editing.

The self-contained installers are the opposite: they copy everything into one owned location so the checkout becomes disposable. On macOS, `localinstall.sh` builds `/Applications/zwire.app` containing the Chromium base (`Contents/Resources/browser/`), the three extensions (`Contents/Resources/ext/`), the native host (`Contents/Resources/native/`), and a bundle-relative launcher (`Contents/MacOS/zwire`). Delete the repo and `~/.zwire/base` afterward and the app still runs, with no system dependencies. Only the per-user profile (`~/.zwire/profile`) and the per-user extension copy (`~/.zwire/ext`, staged from the bundle at launch) live outside it.

The Linux and Windows installers mirror this shape:

- `localinstall-linux.sh` assembles everything under `~/.local/opt/zwire/` (browser + extensions + host), drops a `zwire` launcher on PATH and a `.desktop` entry for the app menu and icon, and needs no root — a pure user install. Delete the repo afterward and it still runs.
- `localinstall-windows.ps1` assembles everything under `%LOCALAPPDATA%\zwire` (the `Win_x64` snapshot, the extensions, `zwire-host.exe`), and wires the native host through the *registry* — because Windows, unlike macOS and Linux, does not read native-messaging host manifests from the profile directory. It drops a `zwire.cmd` launcher.

The native host in the packaged install

The one component that cannot ship as source is the native host, because it is a compiled binary. `scripts/setup-native-host.sh` builds `zwire-host` with `cargo build --release` (its own submodule, `extensions/hud-internal/native/zwire-host`) and writes the native-messaging manifest pointing at it; the launcher calls that on first run. Because the binary is a single static Rust artifact with no runtime dependencies, it drops straight into the app bundle — there is nothing for the deployed machine to provide, no interpreter and no `psutil` to be missing. On the scheme-bridge / sysinfo / PTY path `zwire`'s browser actually drives, the wire format is exactly Chapter 15's: the 4-byte-length framing, the eight scheme names, the atomic tmp-and-rename writes to the HUD scheme/ui files, the base64 `output/exit` PTY frames. On Windows the manifest becomes a `.exe` path and lives in the registry; on macOS and Linux it is the profile-relative JSON of Chapter 15.

What the host really is: a full local agent

zwire's browser uses only a small corner of `zwire-host`. The scheme bridge, the sysinfo stream, and the PTY relay are three of roughly fifty commands across fifteen modules, reachable over two transports — the host is a local-machine agent that the HUD happens to be one client of.

The extra surface, module by module: a namespaced key-value store (`kv_get/kv_set/kv_merge/kv_del/kv_keys`, each an atomic JSON file under `~/.<app>/kv/`); a filesystem API (`fs_read/fs_write/fs_append/fs_list/fs_walk/fs_stat/fs_mkdir/fs_rm`, with a 64-MiB read cap and a 50k-entry bounded recursive walk); file observers (`fs_watch` and a rotation-aware `fs_tail`); an arbitrary command runner (`exec`, returning base64 stdout/stderr and an exit code); background jobs (`job_start/job_poll/job_list/job_result`, which fire a desktop notification on completion); process tools (`ps/kill/which`); a process- global publish/subscribe bus (`sub/unsub/pub`); OS integration (`open/clipboard_get/clipboard_set/notify/hostinfo`); and a TCP peer mesh (`auth/peer_hello/peer_connect/peers/remotes`) that federates the bus and runs a request on another host. On the shared capabilities it is also richer: PTY sessions are multiplexed by id (one connection can drive many terminals) and take a shell, args, cwd, and env; the sysinfo interval is configurable and there are one-shot (`sysinfo_once`) and explicit `sysinfo_stop` variants, so a client tunes the cadence or halts the stream without closing the port. A `hello` reply advertises exactly which of these capabilities were compiled in, so a client can feature-test rather than guess.

The second transport is the reason the Rust host exists as its own project rather than a script. Besides Chrome native-messaging over stdio, it runs a line-delimited JSON daemon over a Unix-domain socket (a named pipe on Windows), created owner-only under an owner-only directory precisely because it exposes `exec`, the filesystem, and PTYs — a local agent any app on the machine can call, not only the browser. That is the whole point of the README's framing: one self-contained binary means the consuming bundle has “zero runtime dependencies,” and an embedder that only wants the light capabilities can compile it without the system-stats and PTY features entirely. “No runtime dependencies” is about the deployed artifact needing no Python and no `psutil`, not about the crate graph — it builds on `serde_json`, `base64`, and the optional `sysinfo` and `portable-pty`.

Two sysinfo details worth pinning

Two fields of the sysinfo frame carry more care than the rest, because a naive implementation would drop or mangle them. First, the disk-I/O rate (`io: {r, w}`, read and write bytes per second): the host keeps a `Disks` handle alive across ticks and reads `Disk::usage()` — bytes since the last refresh — summed over every disk and divided by the elapsed interval, so the streaming rate is real rather than a lifetime total. (The one-shot `sysinfo_once` has only a single instant to work with, so it double-refreshes to report a near-zero delta rather than a lifetime total mislabelled as a rate — an honest “no rate from one sample.”) Second, battery, which has no single cross-platform source: the host reads it natively on all three platforms — `pmset` on macOS, `/sys/class/power_supply` on Linux (capacity plus a charging/`Mains`-online check), and `GetSystemPowerStatus` on Windows — so the statusbar renders identically everywhere. The only remaining absence is honest and physical: machines with no battery — desktops and VMs — omit the segment, exactly as they should.

The whole packaging story is one idea: make the app own everything it needs so the checkout is disposable, and the native host is part of that — a single self-contained Rust binary bundled into the app with nothing for the deployed machine to provide. What the user gets is a normal app — an `.app`, a `~/.local` install, a `%LOCALAPPDATA%` install — that happens to be Chromium wearing the HUD, with that host doing the scheme, stats, and terminal work.

What the self-contained installer actually assembles

The macOS `localinstall.sh` copies four things into `/Applications/zwire.app`: the whole Chromium base bundle into `Contents/Resources/browser/`, the three extensions into `Contents/Resources/ext/` (via `rsync` excluding `node_modules`, `.git`, `tests`, and `target`), the freshly built `zwire-host` into `Contents/Resources/native/`, and a bundle-relative launcher at `Contents/MacOS/zwire`. That launcher is a quoted heredoc, so every `$var` is resolved at *runtime*, not install time; it defaults its state to `~/Library/Application Support/com.menketechologies.zwire`, resolves the browser executable from the bundle's own `CFBundleExecutable` via `PlistBuddy`, and passes the same nine flags `bin/zwire` does. The Info.plist it writes sets the bundle id to `com.menketechologies.zwire`, and the whole app is ad-hoc signed and LaunchServices-registered at the end.

The one subtlety that repays reading is the per-user extension staging, and its comment spells out the failure it avoids: Chromium writes each unpacked extension's compiled declarativeNetRequest ruleset and content hashes into `<ext>/_metadata/`, so the browser needs *write* access to the extension directory, but a shared `/Applications` bundle has a single owner — so a second user would fail to load the extensions with an “Internal error while parsing rules.” The launcher therefore `rsyncs` the bundled extensions into a per-user `$ZWIRE_STATE/ext` (excluding `_metadata`) at every launch and loads from there. The Linux installer mirrors the shape into `~/local/opt/zwire` with a `.desktop` entry, and its launcher adds a tenth flag the macOS one omits — `--no-sandbox`. The Windows installer assembles under `%LOCALAPPDATA%\zwire` and, because Windows does not read host manifests from the profile directory, wires the native host through the registry under both the Chromium and Chrome hives so it resolves regardless of the snapshot's product name.

Cutting a macOS release: `release-macos.sh`

Past macOS releases were built and uploaded by hand, so each cut risked drifting from the last; `scripts/release-macos.sh` captures the recipe as a repeatable script that turns a `v<version>` tag into published artifacts. It reuses `localinstall.sh`'s self-contained bundle assembly — the *same* `/Applications/zwire.app` of the section above — but redirects it to a writable staging dir through a `$ZWIRE_DEST` override (default `/Applications`), so the release build needs no `sudo`: `localinstall.sh` learned to honor `$ZWIRE_DEST` precisely so CI and this script can assemble the self-contained app into a directory they own. It then packages the staged app two ways and uploads both to the GitHub release for the tag: a `zwire-<v>.pkg` (a `pkgbuild` component, identifier `com.menketechologies.zwire`, install-location `/Applications`, forced non-relocatable to match prior pkgs) and a `zwire-<v>-macos.zip` (a `ditto --keepParent` archive with `zwire.app` at its root). The guards are what make it safe to run repeatedly: it refuses to overwrite an existing release, requires the `v<version>` tag (pushing it to origin if missing), and warns on a dirty tree; `--dry-run` builds the artifacts without publishing and `--notes-file` overrides the auto-generated changelog (commit subjects since the previous `v*` tag). Signing is unchanged — the bundle stays ad-hoc signed with no notarization, exactly as every prior release, so first launch still needs a right-click → Open. Artifacts land in `dist/` (gitignored), and the whole thing is wired as the `release-macos package.json` script.

Chapter 24 – Cross-Platform, Updating, and Operations

The operational chapter: what works on macOS (aarch64/x64) versus Linux (x86_64) versus Windows, how to roll the base forward, and the day-to-day notes — native messaging setup, the cosmetic developer banner, and re-running the rebrand after an upgrade.

Platform matrix

The `zwire` launcher itself is portable — it is POSIX shell that reads a base path, builds an extension list, ensures a native host, and `execs` Chromium — and it runs on macOS (aarch64 and x64) and Linux (x86_64), with the PowerShell installer covering Windows. What is shared and what differs, at a glance:

	macOS	Linux	Windows
snapshot platform	Mac_Arm / Mac	Linux_x64	Win_x64
state dir default	~/Library/App...	~/.config/zwire	%APPDATA%\zwire
identity rebrand	bundle patch	launcher + .desktop	launcher name
native host wiring	profile dir	profile dir	registry (both hives)
extra launch flag	–	--no-sandbox	–
self-contained to	/Applications	~/.local/opt	%LOCALAPPDATA%

The parts that differ per platform are three:

- **The base download.** `fetch-base.sh` maps the host to the right snapshot: `Mac_Arm` and `Mac` for the two macOS architectures, `Linux_x64` for Linux, and `Win_x64` for Git-Bash/MSYS on Windows. The launcher and the native host are cross-platform; the base is per-platform because Chromium's snapshots are.
- **The rebrand.** As Chapter 5 established, the in-place Dock/icon rebrand is macOS-only, because only macOS carries app identity as bundle metadata. On Linux the launcher name is the brand, and a `.desktop` entry gives it a menu icon; on Windows the installer places a named launcher. `install.sh` only calls `rebrand-macos.sh` under `Darwin`.
- **The native host install.** On macOS and Linux the HUD host is installed *profile-relative* into `<user-data-dir>/NativeMessagingHosts/`, which Chromium reads on every platform, so there is no per-OS `~/Library` vs `~/.config` split — `setup-native-host.sh` writes the same manifest in the same place. On Windows the manifest is wired through the registry instead, because Windows Chromium does not read host manifests from the profile dir. `bin/zwire` invokes the setup on launch when the manifest is missing or dangling, which is what makes a bare `zwire` self-sufficient in dev on macOS and Linux.

The host binary is `zwire-host`, a cross-platform Rust program built with `cargo build --release` on first need (or when `FORCE=1`) — the sole native host, detailed in Chapter 15. On Windows/WSL the manifest points at a `.exe` because the browser wants a Windows path even though the host source is portable Rust.

Updating the base and re-applying the rebrand

Rolling the base forward is two commands on Linux and three on macOS:

```

scripts/fetch-base.sh           # latest Chromium snapshot (or a pinned revision)
scripts/fetch-base.sh 1656770  # pin an exact revision instead
scripts/rebrand-macos.sh       # macOS only: re-apply the rebrand after the swap

```

The rebrand must be re-run after every base swap for the mechanical reason Chapter 5 gave: `fetch-base.sh` unzips a fresh, stock, Chromium-signed bundle over the patched one, so the name, the icon, the deleted `CFBundleIconName`, and the ad-hoc re-sign are all gone and must be reapplied. `install.sh` chains the two on macOS so a first install is one command; an in-place upgrade is where you run them separately and must not forget the rebrand, or the Dock reverts to “Chromium.” If the Dock icon lingers even after the rebrand, `scripts/bust.sh` forces the LaunchServices cache to refresh (Chapter 5).

The runbook notes

Two operational notes carry over from earlier chapters and belong in the runbook:

- **Native messaging setup.** `zwire`’s own HUD host (`com.zwire.hud`) is wired automatically on launch. `zpwrcrome`’s native host — needed for its `pass` integration and segmented downloads — is a separate install documented by `zpwrcrome` itself; `zwire` does not install it for you.
- **The developer-mode banner.** Loading unpacked extensions via `--load-extension` raises Chromium’s developer-extensions notice on startup. It is cosmetic; the extensions run fully. It is expected, not a fault.

That is the whole operational surface: fetch (and pin) the base, rebrand on macOS, let the launcher wire the native host, and expect one cosmetic banner. Everything else — the profile, the extension staging, the scheme file, the terminal PTY — the launcher and the host manage on their own, which is the point of a browser that is mostly a shell script wrapped around a stock engine.

First run, migration, and the degradation ladder

A first launch on a clean machine does more than open a window, and knowing the order helps when something is missing. The launcher resolves its state directory (the OS app-data location unless `$ZWIRE_STATE` says otherwise), performs the one-time migration of any leftover `~/zwire` into it, and then, if no base is recorded, exits with the exact `run: .../scripts/fetch-base.sh` message — so a fresh clone’s first `zwire` tells you the single missing step rather than failing opaquely. Once the base is present, the launcher ensures the native host by building the Rust binary if its manifest is missing or its target is gone, and only then execs the browser.

The value of that sequence is a *degradation ladder* rather than a set of hard requirements. No base is the only fatal condition. A missing native host is not fatal — the browser launches with the terminal and system stats disabled and one stderr line. A missing submodule (`zpwrcrome`, `zgui-core`) is not fatal — its extension directory has no `manifest.json`, so the launcher skips it and loads what remains. A stale LaunchServices icon is a cosmetic annoyance with a dedicated fix (`bust.sh`), not a functional fault. A base upgrade on macOS silently reverts the rebrand, which is why the update recipe ends with `rebrand-macos.sh` — the one manual step the launcher cannot infer. Read as a whole, the operational model is that the browser tries to run with whatever is present and tells you, tersely and on stderr, only about the one gap that actually stops it: no base. Everything above the base is best-effort, and the launcher’s job is to assemble as much of the HUD as the machine currently has.

Three platforms, one port, and where the asymmetry actually lives

The platform matrix reads like three ports of the same program, and that framing is misleading in a way worth correcting, because it makes the Windows column look like an unfinished version of the macOS one rather

than a structurally different situation.

The launcher is not ported at all. It is one POSIX shell script that resolves a base path, filters a list of directories, ensures a native host, and `execs` a binary, and every one of those operations is platform-neutral. The native host is one Rust program compiled per target. The extensions are JavaScript loaded by a Chromium that behaves identically everywhere. The overwhelming majority of this book describes code that has no platform-specific behavior whatsoever, and that is a direct consequence of the design premise: a browser that is mostly a stock engine plus a shell script plus web code has almost no surface on which platforms can differ.

The asymmetry is concentrated in exactly three places, and each is a place where the *operating system*, not *zwire*, defines the mechanism.

Where an application's identity lives. Chapter 5 established this and it bears restating as a platform statement rather than a macOS one. macOS carries identity as metadata inside a bundle, which is why the rebrand is a bundle patch and why it must be re-applied after a base swap that replaces the bundle. Linux carries identity in how a process was invoked plus a desktop-entry file, which is why the launcher's name *is* the brand and no rebrand step exists. Windows carries it in the launcher the installer places. Three mechanisms, three amounts of work, and the amount of work is proportional to how much the OS insists on knowing about an application before it will display its name — which is a property of the OS, not a measure of how well *zwire* supports it.

Where the browser looks for a native-host manifest. Chromium reads host manifests from the profile directory on macOS and Linux, which is what lets `setup-native-host.sh` write one file in one place and have it work on both — and, more importantly, is what makes the host *profile-relative*, so it is wired for *zwire* and invisible to system Chrome. Windows Chromium does not read manifests from the profile directory; the wiring goes through the registry. That single difference propagates: the install step is different, the uninstall step is different, and the failure mode is different — Chapter 36 records that a silent host on Windows is a registry problem rather than a file one, which means the whole diagnostic path diverges from the one a macOS or Linux user follows. It also means the isolation property is weaker in shape: a registry key is a machine-or-user-scoped fact rather than a profile-scoped one, so the neat statement that the host is wired to *zwire*'s profile and nowhere else is a statement about the two Unix-like platforms specifically.

The sandbox. The matrix carries one extra launch flag on Linux, and it is the one entry in the table with a security consequence rather than a packaging one. Chapter 25's residual risk section already names it: on Linux the renderer sandbox is off, and the protection is the dedicated profile and the fact that the user chose the pages. It belongs in the platform chapter as well, because it is the clearest case of a platform difference that is not merely mechanical. macOS and Windows run with the engine's own sandbox intact; Linux does not, so the profile boundary is carrying more weight there than elsewhere. A reader deciding which machine to run *zwire* on should know that the containment story is not identical across the three.

What falls out of this is a better way to read the matrix. The rows are not a support-tier ranking. `install.sh` chains the rebrand under Darwin because Darwin is the only platform with a bundle to patch; the Windows installer is PowerShell because Windows is the platform where a shell script is not the native idiom; the Linux column has an extra flag because Linux is where the sandbox needs it. In every case the platform-specific code exists at the exact boundary where an OS-defined mechanism begins, and nowhere else — which is the property that keeps a project this size cross-platform at all. The generalizable rule is that portability is not achieved by abstracting over platform differences but by arranging for almost nothing to touch them, and then writing the small amount that does in each platform's own idiom rather than in a lowest-common-denominator one.

Chapter 25 – The Trust Boundaries and the Threat Model

Every capability the earlier chapters celebrated is also a boundary widened. `zwire` loads unpacked extensions with elevated flags, runs a native host that hands out `exec`, `fs`, and `pty`, injects content scripts into privileged `chrome://` pages, and — on the fork — bypasses `X-Frame-Options` and frame-ancestors CSP so the tmux overlay can pane any site. Each of those is a hole in a wall Chromium built on purpose, and a browser that opens holes without accounting for them is a liability, not a tool. This chapter is the accounting: who can do what inside a running `zwire`, where the widenings actually land, and why the ones that look alarming are contained by the ones that do not.

The profile is the blast radius

The first and most important boundary is the one Chapter 1 opened with: `zwire` runs against `~/.zwire/profile`, a dedicated `--user-data-dir`, and never the profile that holds your real logins. This is the containment on which everything else rests. The elevated flags, the unpacked extensions, the developer-mode banner, the native host wired into `NativeMessagingHosts/` — all of it is scoped to a profile that started empty and holds only what you did *inside* `zwire`. If every extension in the load set were malicious, the worst it could reach through the browser is the cookies, history, and saved passwords of the `zwire` profile, which is to say the ones you chose to create there. Your system Chrome, its logins, and its saved payment methods live in a different `--user-data-dir` the `zwire` process never names and cannot see. The isolation is not a courtesy to Chrome; it is the design's primary security property, and it is why loading unpacked, elevated-flag extensions is a defensible thing to do at all. You would never point those flags at the profile with your bank in it — so `zwire` never does.

The launch flags and what they actually relax

Three flags in `bin/zwire` change Chromium's posture, and it is worth being exact about what each one loosens, because the reputational weight of the flag names exceeds what they do. `--load-extension` loads the unpacked extension directories; its real cost is that unpacked extensions are not Web Store-reviewed, so the trust you extend is trust in *this author's* code, staged from a copy you can read on disk. `--extensions-on-chrome-urls` is the one that lets the internal-HUD content scripts run on `chrome://` pages at all — the single relaxation that makes the skinned internal pages possible — and it is narrow: it widens *where a content script may run*, not *what the script may then do*, which stays governed by the manifest's declared permissions. `--test-type` suppresses the automation-restriction warnings Chromium raises when it detects it is being driven, which is cosmetic in effect but carries a real caveat: it also relaxes a handful of automation guardrails, and it is the reason the developer-mode banner is merely annoying rather than blocking. On Linux a fourth flag, `--no-sandbox`, appears where the macOS launcher omits it — a genuine reduction in the renderer sandbox, forced by the way the snapshot ships on that platform, and the one flag in the set the book does not wave away: it is the place where `zwire` on Linux is measurably less defended than `zwire` on macOS, and a reader who runs untrusted pages should know it.

The native host is the largest boundary, and its gates

The native host is the widest hole zwire opens, by a wide margin: `exec` runs arbitrary programs, `fs_*` reads and writes the filesystem, and `pty_spawn` forks a login shell. Anything that can send that host a command owns the machine to the extent your user account does. So the gates around *who can send it a command* are the load-bearing security in the whole system, and they are worth stating plainly. On the native-messaging path, Chromium itself is the gate: the host's manifest lists `allowed_origins`, and only the extension origins named there can `connectNative` to it — a page you visit cannot, because a page is not an extension origin. On the daemon path, where the same router is reachable over a socket, the gate is the filesystem: the Unix socket is created `0600` inside a `0700` directory, so only your own user account can open it, and a different user on the same machine simply cannot see it. The one path that crosses a machine boundary — TCP peering — is the one path that demands a shared `--token` before anything privileged runs, because it is the only path where the “same user, same machine” assumption the local gates rely on no longer holds. The design is coherent: local callers are trusted because the OS already authenticated them by letting them open an owner-only socket, and remote callers are trusted only after presenting a secret. The place to be careful is exactly the place the host is careful — the moment a capability leaves the machine.

What the fork widens, and what it deliberately does not

The fork's patches include two that a security reader should look at directly, because both loosen a Chromium invariant. The first is the `is_sandboxed` gate in `extension_function_dispatcher.cc` (patch 0018), which fixes a real SIGTRAP by letting a content script's extension function run in a context Chromium's assertion would otherwise reject. The important detail is the one the chapter on the fork already drew: the patch widens *scripting*, not *privilege*. The store-privileged JavaScript bindings — the APIs Chromium reserves for Web Store extensions — stay gated by `IsWebstoreOrigin` in `permissions_data.cc`, untouched by the patch. zwire's extensions gain the ability to run where the assertion used to fire; they do not gain the ability to call APIs they were never granted. The second is the CSP and `X-Frame-Options` bypass that lets the tmux overlay embed a site whose headers forbid framing. This is a genuine relaxation of a defense-in-depth mechanism, and it is scoped as tightly as the goal allows: it yields for *sub-frame navigation* so a pane can load an otherwise-unframeable site, not for the top-level page, and the sandboxed pane iframe (Chapter 10's exact token set) is the compensating control — the embedded site runs, but in a frame whose sandbox attributes the overlay chose, not with the ambient authority of the host page. The pattern in both patches is the same, and it is the pattern to hold the fork to: widen the narrowest thing that unblocks the feature, and leave the adjacent invariant standing.

The honest residual risk

No account of a threat model is complete if it only lists the mitigations, so here is what remains. You are running unpacked extensions you did not review line by line; your trust is in the author, backed by the fact that the code is readable on disk and staged into a copy you can inspect. You are running a native host that can own your account; your protection is that only your own extensions and your own user can reach it. On Linux you are running with the renderer sandbox off; your protection is the dedicated profile and the fact that you, not an adversary, chose the pages. And you are, on the fork, running with two Chromium invariants relaxed; your protection is that each relaxation is scoped to the exact surface the feature needs. None of these is zero risk, and the book does not pretend otherwise. The design's claim is narrower and defensible: every widening is contained by the profile boundary, and the profile boundary is the one wall zwire never touches. Keep your real life in your real browser, keep zwire's power scoped to zwire's profile, and the residual risk is the risk you can actually reason about.

Chapter 26 – The Performance Budget

A browser that injects a suite of content scripts on every page, streams system stats continuously, holds live PTYs, and repaints every `chrome://` page is spending your machine's time whether you asked it to or not. If that spending is not accounted for, the HUD becomes a tax on every navigation and every idle minute, and the neon is not worth a browser that stutters. This chapter is the budget: where the cycles and bytes go, what keeps the always-on machinery cheap, and where the cost could bite if the disciplines slipped.

The per-page injection cost

The largest recurring expense is the one that runs on literally every page load: the internal-HUD content-script suite injects vim keys, the tmux overlay, the powerline statusbar, the palette, and the fuzzy find into each document. The suite is kept cheap by three habits. First, the scripts run in a fixed order the manifest declares, so shared state (the keybinding registry, the scheme) is resolved once and read by the rest rather than each script recomputing it. Second, every injectable is idempotent and guards its own re-entry — a content script that has already installed its listeners on a document detects that and returns, which matters because same-document navigations and the overlay's own re-injection would otherwise stack duplicate handlers. Third, the heavy surfaces are lazy: the terminal's xterm bundle, the palette's list machinery, and the fuzzy matcher's index are not built at injection time but on first use, so a page you open and immediately leave paid only for the key listeners, not for the UI behind them. The statusbar is the one visible always-on surface, and it reads its data from a storage key the worker fills, so painting it is a read and a layout, not a computation.

The always-on streams

Two things run when no page asked them to, and both are deliberately throttled. The system-stats stream samples the machine on its own background thread in the native host, at an interval floored at 250 ms — fast enough that the statusbar's CPU and memory readouts feel live, slow enough that the sampling itself never shows up in the number it reports. Running it on a dedicated thread is what keeps a held stream from blocking the connection's other RPCs, so a HUD can carry a stats stream, a PTY, and a file watch on one pipe without any of them starving the others. The rates in the sample — network and disk throughput — are computed as deltas over the elapsed wall-clock interval, which means the host does no work to *maintain* a counter between ticks; it subtracts two readings when a tick fires and is otherwise idle. The second always-on cost is the CI poll, and it is deliberately slow: a `chrome.alarms` heartbeat every ten minutes, tuned to stay under GitHub's anonymous request cap. Ten minutes is not a monitoring interval anyone would choose for urgency; it is the interval chosen so the feature is free — a handful of requests an hour against a public API, invisible in the budget.

The native host's footprint

The bridge that does the most is the smallest thing in the system. `zwire-host` is a single static Rust binary around half a megabyte, built with `opt-level="z"` for size and `panic = "abort"` to drop the unwinding machinery, with no Python interpreter, no `psutil`, and no runtime dependencies to load. This is not incidental thrift; it is what makes the host affordable to start on demand. Chromium spawns it per connection on the native-messaging path, so its startup time is paid every time the browser opens a port to it, and a half-megabyte static binary starts in the noise. The alternative the book explicitly rejected — a Python reference host — would

have paid an interpreter startup and a dependency import on every spawn, and would have failed outright on a machine without the right Python and packages. The Rust host's budget is: near-zero to ship (it is packaged inside the bundle), near-zero to start, and, while running, the cost of whatever it was asked to do — a sysinfo sample, a PTY read loop, a filesystem crawl — and nothing while idle.

The base and the staging

The one genuinely large number is the base itself: the extracted Chromium snapshot, or the packaged fork, is around 325 MB on disk. That is the price of a real Blink engine and it is non-negotiable — it is the same disk a Chrome install spends, and it is exactly what a WebView wrapper would have saved by giving up the capabilities Chapter 2 said zwire cannot give up. The one place zwire adds to it is the per-user extension staging: at launch the loaded extensions are copied into a writable `~/.zwire/ext/` tree rather than run in place, because Chromium writes `_metadata/` indexes into the extension directory and every user needs a writable copy. That copy is small — the extensions are JavaScript and CSS, not the engine — and it is paid once per launch, fast enough to be invisible against the browser's own startup. The honest summary of the disk budget is that zwire costs one Chromium plus a rounding error, and the rounding error is the price of never running an extension in place.

Where it could bite, and the mitigations

The failure modes are the mirror image of the disciplines. If a content script dropped its idempotence guard, the overlay's re-injection would stack handlers and a long-lived tab would slow with every same-document navigation — which is precisely why the guards exist and why the CDP probes assert single-injection. If the stats interval floor were removed, a HUD asking for a tighter stream could spin the host's sample thread hot enough to perturb its own CPU reading — which is why the floor is in the host, not left to the caller. If the terminal built its xterm bundle at injection instead of first use, every page load would pay for a terminal most pages never open — which is why it is lazy. The pattern across all of them is that the budget is enforced at the cheapest layer that can see the whole picture: the host floors the stream, the manifest orders the injection, the guards live in the scripts themselves. A HUD is only worth running if it is nearly free when you are not looking at it, and the whole point of the accounting is that, when you are reading a page and not touching the HUD, almost nothing in this chapter is spending anything at all.

What the budget does not price

The accounting above covers what the HUD *executes*. Three costs sit outside it because they are not executed by any of zwire's own code — they are paid by the browser on the HUD's behalf, in the compositor, the network stack, and the process model — and they are the ones most likely to be underestimated because no profiler attributes them to a script.

Compositing the always-on surfaces. The statusbar is fixed to the bottom of every top-level page with a translucent, blurred background. A translucent blur is not a paint; it is a read-back of what is behind the element followed by a filter, and it is among the most expensive common CSS effects there is. Worse, it is not a one-time cost: an element whose backdrop is blurred must be recomposited whenever the content behind it changes, which on a scrolling page is every frame. A fixed-position element also promotes itself and its neighbourhood into their own compositing layers, which costs GPU memory proportional to the painted area, per tab, for as long as the tab exists. None of this shows up as JavaScript time and none of it appears in the “painting it is a read and a layout” accounting, which is accurate about the script and silent about the pixels.

The mitigations are ones the design already has, and it is worth recognizing them as mitigations rather than as features. The bar is top-frame only, so a page with twenty iframes pays for one bar and not twenty. It is a fixed small height rather than a proportional one, so the recomposited area is bounded regardless of viewport size. And it is toggleable from the palette, which converts an unconditional per-tab GPU cost into an opt-out. The

honest addition to the budget is that of everything the HUD does on an ordinary page, the blurred bar is the item whose cost scales with the *user's* behavior — scrolling — rather than with the HUD's, and it is therefore the one worth measuring on a low-end machine before assuming the accounting above is complete.

The rule engine. Chapter 7 records a declarative-net-request rule set carrying the frame-busting rule. Declarative rules are evaluated by the browser's own network stack, per request, on every navigation and subresource load in the browser — which is precisely why the declarative form exists rather than a JavaScript listener, and why it is fast. But fast is not free, and the cost is structurally different from every other cost in this chapter: it is paid per network request rather than per page or per keystroke, it is paid whether or not any HUD surface is open, and it is paid inside a component the extension cannot instrument. A single small rule set is genuinely negligible here. The reason to name it is that this is the one budget line with no natural back-pressure — nothing about adding a second, tenth, or hundredth rule would feel expensive at the moment it was added, and the cost would appear as a uniform slight slowdown of all browsing that no profiler would attribute to the extension.

Per-tab multiplication. The per-page injection cost is described above as the price of a page load, which is the right frame for CPU. It is the wrong frame for memory. A content script suite is instantiated per document and lives as long as the document does, so a browser with a hundred open tabs holds a hundred instances of the keybinding table, the scheme variables, the palette's structures, and the statusbar's DOM. The laziness discipline is what keeps this bounded — the third habit above is that the heavy surfaces build on first use rather than at injection, so a tab you opened and never touched holds listeners and a small table rather than a terminal bundle and a fuzzy index — and it is worth seeing that this discipline does double duty. It was introduced as a page-load latency argument, and the larger payoff is a memory one: laziness means the resident cost of a background tab is a floor rather than the full feature set. A long-lived browser with many tabs is the workload where an eager implementation of the same features would become visibly expensive, and it is the workload nobody tests.

The unifying observation is that all three of these costs are invisible to the tools that would measure the costs this chapter already prices. Script execution shows in a profiler; compositing shows in a frame timeline nobody opens; rule evaluation shows nowhere; and per-tab residency shows only in a process-level memory figure that mixes the page's own usage with the extension's. A performance budget that counts only what a profiler attributes to your code is a budget for the costs you would have found anyway. The ones worth writing down are the ones nothing will tell you about.

Chapter 27 — Extending the HUD

zwire is built to be extended by its author, and the seams it exposes to him are the same seams open to you. This chapter is the practical one: how to add a ninth scheme, a new HUD internal page, a new content script, a palette command, and a native-host capability — each as a walk down the exact files the earlier chapters mapped, so that extending zwire is a matter of adding a row to a table the system already reads, rather than inventing a new mechanism.

Adding a ninth scheme

A scheme is a set of role colors under a name, and adding one means writing that name into every table that enumerates the eight. There are more of those than there look to be, and forgetting one is the classic way a new scheme half-works. The color table lives in `schemes.js`, and it lives there *twice* — once for the new tab and once for the internal HUD — because the two run in different worlds and each carries its own copy; both must gain the new entry, dark and light. The native host validates incoming scheme names against its list of eight allowed names before it will write the scheme file, so a ninth name the host does not know will be rejected at the bridge and never persist — the allowed-names list in the host is the third place to edit. If you want the scheme reachable from the palette, it wants a row in the palette's scheme commands; if you run the fork, the DevTools Theme dropdown that Chapter 21 wired enumerates the same names and wants the ninth too. The rule to internalize is that the eight is not a constant in one place; it is a consensus across the content scripts, the new tab, the host's validator, and the fork's native mixer, and a new scheme exists only once all of them agree it does.

Adding a HUD internal page

The internal pages are the most self-similar surface to extend, because they are all built from the same shell. A new page is four moves. First, add the page under `pages/` and boot it with `zg-boot.js`, the shared shell that gives every internal page the scheme, the fonts, the fuzzy-filter behavior, and the HUD chrome for free — you write the page's content, not its frame. If the page is a list of things, build it on `zg-list.js`, the list factory the history, bookmarks, and downloads pages already share, and you inherit the fuzzy filter and the row rendering rather than re-implementing them. Second, if the page is meant to *replace* a `chrome://` page, add its redirect to `redirect.js`, which is what turns a navigation to the native page into a load of your HUD replacement. Third, if the page draws on a privileged Chrome API — the settings page's `settingsPrivate`, the extensions page's `developerPrivate` — declare that permission in the manifest, because the redirect gives you the surface but not the authority. Fourth, if the page should be reachable by keyword, seed a palette command for it. The whole design of the internal pages is that they are the same page with different content, so the cost of a new one is the content and the wiring, never the chrome.

Adding a content script

A new always-on behavior — the class vim, the overlay, and the statusbar belong to — is a content script, and the two things that make it a good citizen are the two the suite already enforces. First, place it in the manifest's injection order relative to the scripts it depends on: if it reads the keybinding registry or the scheme, it runs after the scripts that establish them, because the order is the contract. Second, make it idempotent — guard the moment it installs its listeners so that a second injection into the same document detects the first and returns

— because the overlay’s re-injection across same-document navigations *will* run it twice and a script that stacks its handlers is a slow leak. If the behavior needs a key, register it in the `zkeys.js` keybinding registry rather than attaching a raw listener, so it appears in the keybinding editor, respects the user’s remaps in `zb_keys`, and does not collide silently with an existing binding. A content script that reads the registry, honors the injection order, and guards its own re-entry is indistinguishable from the ones that shipped — which is the point of having a suite rather than a pile.

Adding a palette command

The command palette is a registry, not a switch statement, so a new command is a new entry, not a new branch. Seed it in `cmd-defaults.js`, which is where the built-in keyword commands live, and give it a command *type* the palette already knows how to execute — a URL to open, a `shell` one-liner to run in the terminal, an internal page to navigate to. The type is what matters: because the palette dispatches on type, a new command that reuses an existing type needs no new execution code at all, and a command that genuinely needs a new behavior adds one type handler that every future command of that kind reuses. Commands that must reach across into the service worker — anything touching tabs or windows — go through the `zb_cmd` storage bus rather than calling into the worker directly, because a content script cannot call the worker’s privileged APIs and the bus is the seam that wakes the worker to do the privileged work. User-defined commands land in `zb_custom_cmds` through the command editor; the seeded ones in `cmd-defaults.js` are the same shape, which is why the editor can show and edit both.

Adding a host capability

The native host is the most disciplined thing to extend, because its whole architecture is one router in front of many small modules. A new capability is a new module under `src/` and a new `cmd` the router in `session.rs` dispatches to — and because all three transports sit behind that one router, the capability is immediately reachable over native messaging, over the local socket, and from the in-process library, without touching any of them. The module’s shape is set by the fifteen that precede it: it takes the parsed request, does its work, and returns a JSON reply, echoing the request’s `id` if it carries one so it can be multiplexed against other in-flight work. If the capability streams — like `sysinfo_start` or `fs_watch` — it runs on its own thread and emits keyed frames the caller cancels by `id`, following the pattern the stats stream and the file watcher already set. And if the capability is privileged in a way the socket’s owner-only gate does not cover, it inherits the token requirement on the TCP path for free, because that gate lives in the transport, not in the module. The router is the extension point; the modules are leaves; and a new leaf is a new file and a new line in the dispatch, which is exactly as much ceremony as a capability of the machine deserves.

Chapter 28 – The Keyboard Arbitration Model

Every feature chapter so far has claimed a key. Vim claims `j`, the overlay claims `Ctrl-b` and then a second key after it, the palette claims `⌘K`, the find bar claims `⌘F`, the terminal claims `Ctrl+`` and then every `Ctrl+<letter>` while it has focus, and the pages underneath all of them claim whatever they like. Read one chapter at a time this looks like a set of independent features; read together it is a contention problem, and the HUD only works because the contention was designed rather than discovered. This chapter is the arbitration model: who can claim a keystroke, in what order, with what mechanism, and what the invariant is that lets six claimants share one keyboard without a table of special cases.

Seven claimants, in dispatch order

A key pressed inside `zwire` passes through seven potential owners before anything happens, and only the last four are `zwire`'s code:

1	the OS / window manager	⌘-Tab, ⌘-Space, screenshot chords
2	Chromium's own accelerators	⌘T, ⌘W, ⌘Q, ⌘L, ⌘K on the new tab
3	<code>chrome.commands</code> shortcuts	the worker's open-palette (<code>⌘K</code> / <code>Ctrl+K</code>)
4	the top frame's capture list	<code>zvim</code> , <code>zpalette</code> , <code>zfind</code> , the overlay
5	the pane forwarder	<code>ztmux-pane.js</code> , inside a framed document
6	the page's own listeners	a site's single-key <code>j</code> , its own / shortcut
7	the focused field / xterm	text entry, readline, control codes

The first three are not negotiable from JavaScript. The OS wins outright. Chromium's accelerators win over any page listener on the surfaces Chromium chooses to guard, which is exactly the situation the new-tab page runs into: Chrome reserves `⌘K` at the browser level there, and no amount of capture-phase cleverness in the page will see it. And `chrome.commands` — layer three — is the only mechanism that fires *regardless of focus*, which is why the background worker, not any content script, is the browser-wide owner of `⌘K`.

That ownership has a price the worker's own comment names: registering `⌘K` as a browser command consumes it on every tab, including ones where a page listener would have been perfectly capable of catching it. `zwire` pays that price deliberately, because the alternative — a page listener on ordinary pages and nothing on the new tab — would make the palette's most-used chord work in most places and mysteriously fail in one. A key that works everywhere is worth more than a key that is cheap to bind. The worker then routes the press by inspecting the active tab: the content-script palette on an ordinary page, the new-tab extension's own palette through a cross-extension message, a HUD page's palette through a runtime broadcast. Three destinations, one owner, one chord.

The four arbitration devices

Below layer three, `zwire`'s scripts arbitrate among themselves with exactly four mechanisms, and every keyboard feature in the book is built from some combination of them.

Capture-phase listening is the first. Every HUD key listener registers with the capture flag, on `document`, which puts it ahead of the page's own handlers in the dispatch order. This is what lets vim's `j` beat a site's own

`j` without knowing anything about that site. Capture is the cheap half of the mechanism; the expensive half is knowing when *not* to use it.

`stopImmediatePropagation` is the second, and it is the one that distinguishes a claim from a suggestion. `preventDefault` alone stops the browser's default action but lets other listeners — including the page's, and including other HUD scripts — still run. When vim decides it owns a key it calls both, so the site's single-key shortcut never also fires. The comment in `zvim.js` on that call reads “win over site single-key shortcuts,” and it is the entire reason vim on a page feels like vim rather than like vim plus whatever the site thought that letter meant.

The **prefix state machine** is the third, and it is how the overlay gets a whole command vocabulary out of keys that are otherwise unowned. Arming `Ctrl-b` puts the top frame into a state where the *next* key is interpreted against the tmux table and then the state clears; a 900-millisecond timeout clears it anyway if nothing follows. The same device appears at smaller scale inside vim, where `g`, `z`, `y`, `m`, and ``` each arm a one-key prefix under the same timeout. A prefix is the standard answer to “I need forty commands and I have twenty-six unclaimed letters,” and it is also an *ownership* device: while a prefix is armed, one claimant owns the next key unconditionally, which resolves the contention for that keystroke by fiat instead of by rule.

Delegation across frames is the fourth. A pane iframe is a different document with a different listener set, and the top frame cannot listen inside it. So the pane runs a thin forwarder that catches only the keys the overlay needs — the prefix, the key that follows it, the palette chord, a copy-mode yank, a synchronized keystroke — and `postMessages` them up with a tagged envelope. The overlay's logic stays in one place; only the *sensing* is duplicated. This is the difference between a distributed feature and a duplicated one, and it is why the tmux prefix works while your cursor is inside a framed site.

The invariant: claim only what is mapped

The rule that makes all of this compose is stated in one sentence and enforced in several places: **a HUD listener calls `preventDefault` only when the key maps to a real action**. An unknown key reaches the page untouched.

This sounds obvious and is not. The natural way to write a keyboard layer is a switch statement that swallows everything and falls through to a default that also swallows, because that guarantees your feature works. It also guarantees that every site's own keyboard interface breaks, and — worse — that the *next* HUD feature to want a key finds that key already consumed by a script that was not using it. The claim-only-what-is-mapped rule keeps the unclaimed keyspace genuinely unclaimed, which is what allows six independent scripts to be added over time without an arbitration table.

The rule has three corollaries visible in the code. First, the editable guard: inside an `input`, `textarea`, `select`, or `contentEditable`, vim claims nothing except `Escape` — because in a text field every letter is already mapped, by the field. Second, the modifier bailout: a key held with `Alt`, `Meta`, or `Ctrl` is left alone by the motion layer, because those chords belong to the palette, the find bar, and the vim toggle, and a motion handler that grabbed them would silently break the features above it. Third, the ordering of guards inside a single listener matters as much as the ordering of listeners: `zvim.js` checks the per-tab disable flag, then hint mode, then the editable guard, then the modifier bailout, and only then consults the keymap. Hint mode has to come *before* the editable guard, because typing a hint label is not typing into a field even when a field happens to be focused; getting that order wrong would make link hints unusable on any page with a search box.

Focus as a claim modifier

Two features change what the keyboard means depending on where focus sits, and both are worth naming because they are the cases the layered model does not handle by itself.

The terminal is the sharper one. When the terminal overlay has focus, `Ctrl+<letter>` must reach the PTY as a control code — `Ctrl-A` to the start of the line, `Ctrl-W` to kill a word, `Ctrl-U` to kill to the start — because

that is what those keys mean in a shell. But on macOS the web view would otherwise swallow several of them for its own Emacs-style text bindings, which would leave the browser terminal missing exactly the editing keys that make a terminal usable. So the terminal installs a capture-phase handler that, while it has focus, forwards those combinations straight through as control bytes. This is a focus-scoped claim: the same chord is claimed by the terminal when the terminal is focused and by nobody in particular otherwise.

The second is the tmux prefix's alternate. The prefix is `Ctrl-b`, the real tmux one, and it is also `⌘-b`. The alternate exists for a historical reason the fork's series file records: an earlier native split patch consumed `Ctrl-b` and the key after it before the page ever saw them — the “C-b %%” double-tap — and while that patch is gone, the alternate chord it forced remains, because a second prefix that nothing else intercepts is a genuinely useful escape hatch on a page or a platform where the first one is contested.

Remapping makes the model dynamic

The last complication is that none of the bindings above are constants. Every default key lives in `zkeys.js` as data, and a user's remaps live in `chrome.storage.local` under `zb_keys` as `{action: key}` pairs merged over the defaults. That merge happens at load *and* again whenever the key changes, because the settings page writes it live.

This has a real consequence for how the listeners are written. A keyboard layer that bound a handler per key would need to unbind and rebind on every remap, and would race with a page that was already open. So the HUD listeners are written the other way around: one listener per concern, which consults a `key → action` table built from the registry at dispatch time. Rebinding `scrollDown` from `j` to `s` rewrites a table entry, and the very next keystroke on every open page dispatches through the new table with no reload and no re-registration. The same registry and the same storage key drive the overlay's post-prefix keys, which is why one settings page remaps both layers and why the overlay's help card — built from the same registry — can never drift from what the keys actually do.

The whole model reduces to four rules a new feature must follow to be a good citizen: register in the shared registry rather than binding a raw key; listen once, in capture, and dispatch through the table; claim only what is mapped and claim it completely with both `preventDefault` and `stopImmediatePropagation`; and if the feature spans frames, forward the sensing upward rather than duplicating the logic. A keyboard layer built that way can be added to the five that already exist without anyone auditing the other five — which is the only property that matters when the layers were written years apart.

Chapter 29 – The Six Boundaries and Their Transports

zwire is not one program. It is a content script in an isolated world, a service worker with its own lifecycle, two other extensions with their own storage, a set of cross-origin iframes, a native process, and — on the fork path — a C++ color mixer inside the browser itself. Every feature in this book that feels like one coherent behavior is actually a message crossing at least one boundary between those, and the choice of transport at each crossing is the single most consequential design decision in the runtime path. This chapter is the catalog: six boundaries, the transport each one uses, what that transport can and cannot carry, and the rule that decides.

The six crossings

Page and content script. A content script runs in an isolated world: it shares the DOM with the page and shares nothing else — not the page's JavaScript globals, not its prototypes, not its variables. The transport across this boundary is the DOM itself, plus synthesized events. zwire uses it deliberately little. The find bar registers highlight ranges rather than wrapping matches in elements; the synchronize-panes receiver splices text into a field and then dispatches a real `input` event so the page's own handlers see the change. Both are the same discipline: touch the DOM the way a user would, not the way a script would, so the page cannot tell the difference and cannot be broken by the difference. The one thing this boundary is *good* for is that it is free and synchronous, and the one thing it cannot do is reach anything outside the document.

Content script and service worker. This is the most-crossed boundary in the system and the one with the most careful transport, because the receiver may not be running. The obvious mechanism, `runtime.sendMessage`, is not reliable against a suspended MV3 worker. The mechanism zwire actually uses for anything that must happen is the storage command bus: a content script writes a `zb_cmd` object into `chrome.storage.local`, and `storage.onChanged` is a reliable wake. The worker's own comment states the contract — storage wakes it where a message to a sleeping worker would not, “so palette navigation / tab-switching always executes.” In the other direction the worker does not wait to be asked at all: it publishes the tab list, the extension list, the shortcut table, the frequency list, and the system-stats sample eagerly, so a page reads a value rather than issuing a query that might never be answered.

Extension and extension. Three extensions run in a zwire window and they share nothing. `chrome.storage.local` is per-extension. There are exactly two channels. The first is messaging with explicit consent on both ends: the worker pushes the scheme to the power-tool over `runtime.sendMessage`, and the new-tab page is reachable because its manifest lists the HUD worker in `externally_connectable`. That channel is asymmetric — it works when the sender knows the receiver's id and the receiver has opted in — and it is push-only, so it cannot serve as a source of truth. The second is the native host's files, which every extension can reach through its own native-messaging port, and which is the only *symmetric* channel the three share. That is the entire reason the scheme lives in a file rather than in storage.

Top frame and pane iframe. A pane holds an arbitrary cross-origin site, so the two documents share nothing at all, not even the DOM. The transport is `postMessage` with a tagged envelope and a deliberately small vocabulary: upward, the prefix, a post-prefix command key, a palette request, a yank, a synchronized keystroke; downward, a sync-group change, a copy-mode toggle, paste text, and a keystroke to apply. The vocabulary is small on purpose — every message shape is a piece of API surface between two documents that cannot be updated atomically, since one of them is a page from the internet running the forwarder that was injected into it.

Browser and native host. Length-prefixed JSON over stdio: a four-byte little-endian length, then that many

bytes of UTF-8 body. It is the only transport in the system that crosses a process boundary, and it is the only one with a real multiplexing story — a request may carry an `id` that is echoed on its reply, so one connection can hold a system-stats stream, several PTY sessions, and a file watch at once, each keyed and each cancellable independently. The cost of this boundary is that only an extension origin listed in the host manifest's `allowed_origins` may open it at all, which is also, from the threat-model chapter's point of view, its best property.

Browser and the compiled fork. On the fork path there is a consumer of the scheme that is not a web context at all: the native color mixer, compiled into Chromium's Views layer. There is no message channel between a content script and a C++ mixer, and there will never be one. The transport is a file — and specifically, a *plain-text projection* of a file. The host owns `global.toml` as the real store, and writes a trivial `hud-scheme` (and, for light mode, `hud-light`) beside it purely so a C++ process that cannot parse TOML at paint time has something it can read in a line. The projection is not redundancy; it is a deliberate narrowing of a format to fit the weakest consumer.

The durability ladder

Laid out together, the six transports sort into a ladder by how much they require of the receiver, and the ladder is the design rule:

transport	receiver must be	survives	notifies?
file (+ projection)	nothing	everything	no (poll/pub)
chrome.storage.local	the same extension	worker eviction	yes (onChanged)
long-lived port	alive now	navigation (if held)	yes
postMessage	a live frame	nothing	yes
one-shot sendMessage	awake now	nothing	yes

The rule that falls out is: **pick the transport with the weakest liveness requirement that can carry the payload.** Every hard-won behavior in the runtime path is an application of it. The scheme is a file because one of its consumers is a C++ process and another is a different extension, and neither can be assumed to be running. The command bus is storage because the worker may be asleep. The PTY is a long-lived port because it is a stream and a stream has no meaning without an attached receiver. The pane protocol is `postMessage` because there is no weaker option across an origin boundary. And one-shot `sendMessage` is used only where the receiver's liveness is already established — the worker pushing to the power-tool, which is either loaded or irrelevant.

The corollary is equally practical: when a path stops working, the first question is whether it used a transport stronger than its receiver could guarantee. That was exactly the automation bus's original failure. A one-shot native message issued from a page's content script lost its reply, because the worker was torn down during the native round trip, and external pages saw a zero-percent success rate. The fix was not to add a keepalive timer; it was to move the reply onto a channel that was already open for another reason — the persistent sysinfo port — and correlate it with a request id. The path did not need more liveness; it needed a transport whose liveness was already paid for.

What a file costs

The bottom rung deserves its own accounting, because “just write a file” hides real engineering. A file has no lifecycle, no receiver, and no version skew, and it can be read by a content script through the host, by a separate extension through its own host connection, by a C++ mixer at paint time, and by a shell script. That is an enormous amount of reach for a transport with no protocol.

It buys that reach by giving up three things. It does not notify — which is why the new tab polls the host every 1.5 seconds and why the newer design adds a publish/subscribe topic on the host so subscribers get a push instead. It has no atomicity of its own — which is why every write is a write to a temp file followed by a rename, so a concurrent reader sees the old contents or the new ones and never a half-written file. And it has no serialization across writers — which is the failure that actually bit: two concurrent writes, one carrying a

scheme and one carrying a UI preference, could each read the old state, modify their own field, and write back, so the second write silently reverted the first. The fix is a cross-process lock file around the read-modify-write of `global.toml`. That sequence — reach for the file because it is the only universal channel, then re-add atomicity, notification, and serialization one at a time — is the honest cost of the bottom rung, and it is worth remembering before reaching for a file as though it were free.

Why the boundaries are not consolidated

A reasonable reader asks why six. Could the HUD not run one transport everywhere and be simpler? It could not, and the reason is that the boundaries are not *zwire*'s; they are Chromium's, and each exists to enforce something. The isolated world exists so a page cannot tamper with an extension. The per-extension storage partition exists so one extension cannot read another's data. The service worker's eviction exists so background extensions cannot hold memory forever. The cross-origin frame boundary exists so a framed site cannot read its framer. The native-messaging origin allowlist exists so an arbitrary page cannot spawn a local process. Every one of these is a security property the threat-model chapter relies on, and consolidating the transports would mean defeating them.

So the design is the opposite of consolidation: accept all six boundaries as given, choose the correct transport at each, and make the *feature* the thing that spans them. The scheme picker looks like one control and is several renderers reconciling to one file. The palette looks like one list and is four implementations sharing one item source. The terminal looks like one window and is a page, a worker, a process, and a PTY. In each case the coherence the user sees is the sum of correct transport choices, and none of it survives a single wrong one.

Chapter 30 — Living Inside MV3

Half the non-obvious engineering in the internal HUD exists because of one fact: the background service worker can be destroyed at any moment, and there is nothing the extension can do about it. Manifest V3 replaced MV2's persistent background page — a document with a stable heap, long-lived timers, real DOM, and a lifetime equal to the browser's — with an event-driven worker the browser wakes on demand and suspends when idle. Everything that used to be a variable is now a question about where state lives. This chapter takes the MV3 constraint seriously as a design input and walks the five patterns zwire uses to live inside it, because those patterns generalize well past this browser.

What was actually taken away

It is worth being precise about the loss, because the popular framing (“MV3 constrained blocking `webRequest`”) is not the part that shaped this codebase. Three things went.

The **heap** went. A persistent page could hold a map of tab id to session, a cached list, a debounce timer, an open connection — and rely on all of it still being there the next time an event fired. A worker cannot. Every in-memory structure is a cache of something that must exist elsewhere, or it is data you have agreed to lose.

The **guaranteed reachability** went. `runtime.sendMessage` to a persistent page always landed because the page was always there. To a suspended worker it is a race: the browser may wake the worker and deliver, or the send may fail in a way the sender is not well positioned to detect or retry. Any feature whose correctness depends on the message arriving needs a different mechanism.

The **timer** went, in the sense that mattered. A `setInterval` in a persistent page is a heartbeat for the browser's lifetime; a `setInterval` in a worker lasts until the worker is suspended, which may be seconds. Periodic work has to move to `chrome.alarms`, which survives suspension by waking the worker — and which, by design, is unsuitable for anything fine-grained. The HUD's only alarm is the CI poll, on a ten-minute heartbeat, which is precisely the kind of coarse periodic work alarms are for.

Pattern 1: storage as the wake mechanism

The single most load-bearing MV3 pattern in the HUD is that `storage.onChange` is a reliable wake and a message is not. So anything a page needs the worker to actually do is written into `chrome.storage.local` as a `zb_cmd` object rather than sent as a message. The worker's `onChange` handler dispatches on the action field across the whole tab-and-window vocabulary — open, activate, new tab, new window, duplicate, reopen, pin, mute, close, close others, next, previous, the tiling engine, and a bare ping that re-runs every mirror.

There is a subtlety in the bus that is easy to miss and painful to debug: a storage event fires on a *change*, and writing the same value twice is not a change. A user who presses `x` to close a tab and then presses `x` again would write an identical `zb_cmd` and the second press would silently do nothing. So the bus carries a monotonic tag alongside the action, and every write bumps it. The tag has no meaning to the handler; its only job is to guarantee the write is a change. Any storage-as-a-bus design needs this, and every one that omits it has an intermittent bug that reproduces only on repeated identical commands.

Pattern 2: publish eagerly, never query

The second pattern inverts the usual client-server shape. A content-script palette cannot call `chrome.tabs` or `chrome.management`; those APIs are the worker's. The naive design is a request-response: the palette asks the worker for the tab list when it opens. Under MV3 that request may be issued to a sleeping worker at the exact moment the user is waiting on a UI, which is the worst possible time to lose a message.

So the worker does not wait to be asked. It keeps the tab list, the enabled-extension list, the extension shortcut table, and the frequency list continuously fresh, each wired to the Chromium event that would change it — tab updates, extension installs, new visits — and the palette simply *reads storage*. The read always succeeds and never depends on the worker being awake. The cost is that the mirrors are maintained even when nothing is reading them, which is a small, bounded, event-driven cost; the benefit is that the most latency-sensitive surface in the HUD never waits on a process that might not exist.

The same reasoning explains why the scheme is mirrored into `zb_scheme` rather than fetched on demand. The worker's comment is explicit that a `sendResponse` across the native host is unreliable because the worker can suspend mid-round-trip, while a storage write always lands. Publish the answer before the question is asked, and the question never has to survive the worker's lifecycle.

Pattern 3: keepalive by useful work

The third pattern is the one most likely to be got wrong elsewhere. A worker that must stay alive across a slow operation needs a keepalive, and the tempting implementation is a timer or a dummy port whose only purpose is to prevent suspension. `zwire` does not have one. What it has is a port that was already open for a real reason: the persistent native connection that streams system stats. The worker's own comment names the second motive for holding it open — it keeps the worker itself alive.

That port keeps the worker alive as a *side effect* of doing something the HUD wanted anyway, and the automation bus deliberately reuses it. The bus's original failure was precisely a lifecycle failure — a one-shot native message from a page lost its reply because the worker was torn down during the native round trip, and external pages saw a zero-percent success rate. The fix routes the reply over the already-open `sysinfo` port, correlated by a request id so the browser action fires exactly once. The general principle: if you need the worker alive, find the work that legitimately keeps it alive and put your dependency on that, rather than adding a lifecycle hack whose only justification is itself.

Pattern 4: idempotent seeding with tombstones

A worker's startup handlers run more often than a persistent page's ever did — on browser start, on extension install, on extension reload, and effectively any time the browser felt like evicting and re-waking it. Anything those handlers do must therefore be safe to do repeatedly.

The keyword-command seeder is the clean example. The worker seeds a default set of palette commands on first run, guarded by a seeded flag so it does not re-run, and merges defaults with any existing user entries so edits are never clobbered. That covers the first-run case. It does not cover the *deletion* case: a user who deletes a seeded command would find it resurrected the next time the seeder decided it had not run. So deletions are recorded in a tombstone list the seeder consults. Guard plus merge plus tombstone is the full shape of an idempotent seed, and any two of the three is a bug that only shows up after a browser restart.

The native seed follows the same discipline from the other direction: on startup the worker asks the host for the current scheme and mirrors whatever the file already held, because — as its comment says — the file may already carry a scheme picked in a prior session or written before launch. Startup is not initialization; it is reconciliation with state that outlived the process.

Pattern 5: state outside the heap, with a reconcile step

The last pattern is what happens to structures that genuinely cannot be recomputed. The worker's window-tiling engine holds a session of named window groups and their panes — a real data structure with no source of truth anywhere else. It survives eviction by mirroring itself to `zb_tmux_state`, and it rebuilds from that mirror when the worker comes back, defaulting to “the current window is the sole pane” if there is nothing to rebuild from.

Mirroring alone is not enough, though, and the engine's second half is the interesting part. The world it describes — real browser windows — can change while the worker is not running. Windows get closed, focus moves, groups empty out, and none of those events reached a process that was asleep. So the engine carries a `reconcile` step that drops panes whose windows no longer exist and prunes empty groups, wired to `windows.onRemoved` and `onFocusChanged`. The mirror restores what the worker believed; the reconcile step corrects it against what is actually true. Any persisted model of external state needs both, and the reconcile step is the one people forget, because it only matters after an eviction they did not observe.

The per-tab PTY relay shows the same shape with different tradeoffs. Sessions are keyed by tab id so a shell survives page navigation; output produced while no page is attached is buffered and flushed on reconnect, capped at 500 messages; a spawn is honored once per tab so a re-injected content script re-attaches instead of forking a second shell; and a content-script disconnect nulls the attachment rather than killing the native port, while an actual tab removal tears it down. Here the state is genuinely unrecoverable — a live process — so the design does not try to persist it; it tries to keep it attached to the one thing with a comparable lifetime, the tab.

What is deliberately allowed to be lost

The honest end of this chapter is the list of state `zwire` does *not* try to save, because deciding what to lose is as much a part of the pattern as deciding what to persist. Vim's marks are per-page scroll positions in memory, and they are gone on reload — they are bookmarks for the page you are reading, not durable locations, and persisting them would create a synchronization problem far larger than the feature. The per-tab vim disable flag is in memory too, deliberately transient, so a page where you turned vim off gets its raw keys back and the next page starts clean. The PTY's detached-output buffer is capped and will drop the oldest messages rather than grow without bound. Each of these is a decision that the state's value does not justify a persistence mechanism, and stating them as decisions is what distinguishes a design from an oversight.

The rule that ties the chapter together: treat the worker as a pure function from events plus persistent storage to effects. Anything it remembers between two events is either mirrored, reconstructible, or expendable — and you must be able to say, for every piece of state, which of the three it is.

Chapter 31 – The Storage Schema and Its Ownership Rules

Appendix I lists the `zb_*` keys. A list is not a schema. What makes the storage layer work is not the set of keys but the discipline around them: for each key, exactly one writer, a stated set of readers, a defined lifetime, and a rule for what happens when its shape changes under a page that is already running. A shared key-value store with no ownership rules is a mutable global namespace shared by a dozen scripts across three execution contexts, which is the fastest way known to make a feature that works in isolation fail in combination. This chapter is the discipline.

Five roles, not two dozen keys

Every key in the store falls into one of five roles, and the role determines the rules.

Mirrors are worker-written and page-read: the active scheme, the system-stats sample, the open-tab list, the enabled extensions, the frequency list, the extension shortcut table, the CI tally. The worker maintains each against the Chromium event that would invalidate it, and a reader treats the value as a fact about the world. The rule is absolute: **a reader of a mirror never writes it**. A page that wants a different scheme does not write `zb_scheme`; it asks the owner, which relays to the host, which writes the file, which comes back as a new mirror value. Writing a mirror directly would produce a value that disagrees with its source and would be silently overwritten at the owner's next update — the worst kind of bug, because it works exactly once.

Buses are page-written and worker-read, and consumed rather than held: the command bus and the session-load trigger. A bus value is an *edge*, not a state. It carries a monotonic tag so a repeated identical command still registers as a change, and nothing reads it for its value after the handler has run. The rule here is that a bus key must never be treated as a source of truth for anything — its content is meaningful for exactly one dispatch.

Preferences are written by a settings surface and read everywhere: the keybinding remaps, the keyword commands, the statusbar visibility, the tmux prefix chord and its timeout, the CI configuration. These have a real multi-writer story — the Keyboard page, the Commands page, and the palette can all touch them — and the rule that keeps it safe is that preferences are *user intent*, so a later write legitimately supersedes an earlier one, and every reader re-reads on change rather than caching at load. That last half is why rebinding a key changes behavior on already-open pages with no reload: the readers subscribed rather than sampled.

Session state is written by either side and exists to survive an eviction or a navigation: the window-tiling engine's state, the named tmux layouts, the terminal's open flag. These are the keys where lifetime is the hard part, and the rule is that anything describing transient UI must have a defined reset. The terminal's flag is the clearest case: it re-opens the overlay only when the value is the boolean `true` — not merely truthy — and the worker resets it to false on browser startup and on extension reload, so a stale flag from a previous session cannot make the terminal pop up on every page of a fresh one. A session key without a reset owner is a haunting.

Guards and tombstones are one-shot markers: the seeded flag and the removed-defaults list. Their only rule is that they are written before the thing they guard is considered done, and never cleared as a shortcut — clearing a tombstone list to “start fresh” resurrects every command the user deliberately deleted.

The rules, stated

Five roles produce six rules, and they are worth stating flatly because every one of them corresponds to a class of bug:

```
1 single writer      each mirror has exactly one owner; readers never write
2 nonce on buses     a bus write carries a monotonic tag so repeats fire
3 subscribe, don't cache readers re-read on change rather than sampling at load
4 real booleans      a flag is compared to true, never merely for truthiness
5 reset owner        transient UI state names who clears it and when
6 tombstone before seed a seeder checks deletions before it merges defaults
```

Rule three is the one that most shapes how the scripts are written. The HUD's content scripts do not read their configuration once at injection; they read it and then listen for changes to it. That is why a scheme change repaints an open page, why a remap takes effect on a tab that was already loaded, why the statusbar re-renders when the stats sample updates without any polling of its own, and why the tmux overlay notices a session-load trigger written by a page in a different tab. Subscription is what turns a key-value store into a coordination mechanism; sampling would turn it into a configuration file that everyone reads at slightly different times.

Schema evolution under a running page

The awkward case in any shared store is a key whose shape changes while old readers are still alive. In a browser extension this is not hypothetical: a user updates the extension, and pages loaded before the update keep running their old content scripts against a store the new worker is writing.

The overlay's own history supplies the worked example. Its per-tab session snapshot originally recorded a per-window sync boolean — is this window broadcasting? — and later needed a list of which panes were members of the sync group. The snapshot restore path migrates the old boolean to the new member-id array on the way *in*. That direction matters. Migrating on read means an old value is understood by new code, and the store keeps whatever the newest writer put there. Migrating on write — rewriting the store to the new shape when the new code loads — would break the older content script still running in another tab, which would read a shape it does not understand. The rule generalizes: **migrate on read, never on write**, and accept that the store may contain several generations of a value at once.

The corollary is that a reader must tolerate a missing key as well as an old one. The tiling engine defaults to “the current window is the sole pane” when there is nothing to rebuild from; the palette treats an absent mirror as an empty list rather than an error. A store that is read by code older and newer than itself has to be read defensively at every site, and the cheapest way to make that consistent is to have exactly one reader per key inside each script rather than scattered direct reads.

The two escapes from the schema

Two categories of state deliberately do not live in this store, and knowing why is knowing the boundary the schema draws.

The first escape is per-origin `localStorage`: the new tab's quick-launch tiles and its search-engine template, and the visual-effect toggles the shared effects module persists under its own keys. These are page-local by nature — they belong to one document's own presentation — and using extension storage for them would mean the worker mediating a preference no other context needs. The cost is exactly the isolation that makes them cheap: `localStorage` does not cross origins, so a preference stored there is invisible to every other surface. This is precisely why the effects toggles exist in *two* places. The page keeps its own local copy so it can apply the effect without a round-trip, and the Settings page pushes the same four values through to the native prefs so the separate new-tab extension follows. The duplication is not sloppiness; it is a local cache plus a shared source, and the settings page is the bridge that keeps them agreeing.

The second escape is the native store — the host's `global.toml` and its plain-text projections. Anything that must be seen by a second extension, or by a C++ mixer, or by another application in the suite, cannot live in `chrome.storage` at all, because that store is partitioned per extension by design. The scheme, the light-mode flag, the effect preferences, and any user-defined custom schemes are therefore native-side, and `zb_scheme` is a *mirror* of the native value rather than the value itself. The schema's top-level rule follows from that: for any piece of state, ask how many extensions and how many processes must see it, and let the answer choose the store. One script, `localStorage`. One extension, `chrome.storage.local`. More than one extension, or any non-web consumer, the native file — with a storage mirror in front of it for the scripts that need a fast local read.

Why this is a schema and not a convention

The reason to write these rules down rather than leave them as habits is that the store is the one part of the system every context touches. A content script, the worker, four palette implementations, a dozen HUD pages, and the settings surfaces all read and write it, and they were written at different times for different features. There is no compiler enforcing any of the six rules; there is no type on any key. The only thing standing between this design and a mutable global namespace is that each key has a stated owner, a stated lifetime, and a stated read discipline — and that a new feature adds a *row*, with all three declared, rather than a new access pattern to an existing row. A schema, in a system with no schema enforcement, is exactly the set of invariants everyone agreed to maintain by hand, and the value of writing it down is that the next person to add a key knows which five roles they are choosing among.

Chapter 32 — When a Hop Fails

Chapter 36 is the field guide for the symptoms a user sees. This chapter is the layer underneath it: what happens *inside* a multi-hop path when one hop does not do its job, which failures the design absorbs silently, which ones surface, and why the ones that surface were chosen to surface. Every headline feature in this book is a chain of crossings — surface to worker to host to file to renderers, or page to worker to process to PTY — and a chain is only as trustworthy as its account of partial execution. Appendix AA traces three of these paths when everything works. This chapter traces the same shapes when something does not.

Path A: a scheme change with a hop missing

The scheme fan-out is a write followed by an uncoordinated set of reads: a surface relays the pick, the worker writes the mirror and relays to the host, the host takes the lock and rewrites the store and its projections, and the renderers reconcile on their own schedules — content scripts through the observed mirror, the new tab through its own poll of the host, the power-tool through a push, the fork's mixer through the projection file at paint time. There is no transaction anywhere in that sentence, and the interesting question is what each missing hop actually costs.

If the **host is not running**, the mirror still updates and the file does not. The symptom is the most-reported desync in the book: the internal pages and the ordinary page skins recolor immediately, and the new tab does not, because the new tab's only channel is the file. This is a good failure in one specific sense — it fails toward *visible* disagreement rather than silent partial persistence, and the disagreement points directly at the missing component. It is a bad failure in that nothing announces it; the surfaces that worked give no hint that a consumer was left behind. The compensating design is that the file is the source of truth, so as soon as the host returns and any surface writes a scheme again, every renderer converges. Nothing needs repair; the system only needs one more correct write.

If the **write lands but a renderer has not read yet**, there is a window in which two surfaces disagree. For the new tab that window is bounded by its poll interval; for a content script it is bounded by the storage change event, which is prompt; for the fork's mixer it is bounded by a repaint. Transient disagreement across a fan-out with per-consumer refresh is not a bug and should not be engineered away — the alternative is a coordination protocol whose failure modes would be far worse than a fraction of a second of mismatched neon.

If **two writers race**, the failure is real and was fixed rather than tolerated. Two concurrent writes — one carrying a scheme, one carrying a UI preference — could each read the current state, modify their own field, and write the whole thing back, so the second write silently reverted the first and a scheme change appeared to “not take.” This is the classic lost update, and it is the one failure in the fan-out that the design refuses to absorb, because its symptom is indistinguishable from the feature not working. The host serializes the read-modify-write behind a cross-process lock.

If the **push loops**, the guard catches it. The worker pushes the scheme to the power-tool, and the power-tool can push a user's pick back; without a guard, each push would trigger the other. A record of the last pushed value breaks the cycle. This is worth naming as a category: any bidirectional sync between two stores that both emit change notifications needs an echo guard, and the guard belongs on the side that can distinguish “this value came from me” — which is the pusher.

Path B: a browser action that must run exactly once

The automation bus has a stricter correctness requirement than the scheme fan-out, because its effects are not idempotent. Recoloring twice is recoloring; closing a tab twice is closing two tabs. The path is a trigger running stryke code, the host stamping the action into a file-backed key-value entry, and the service worker's single executor reading it and issuing the Chrome API call.

The failure that shaped the design was a **lifecycle failure**: a one-shot native message issued from a page's content script lost its reply because the worker was torn down during the native round trip. The symptom was total — a zero-percent success rate from external pages — which is, perversely, the easiest kind of failure to diagnose, because nothing worked at all. The fix delivers the reply over the persistent port that already streams system stats and correlates it with a request id so the action fires exactly once. Two properties are doing the work there: the port is alive because it was alive for another reason, and the id makes a duplicated delivery detectable rather than merely unlikely.

The failure the design deliberately does *not* guard is an **unknown verb**. The host strips the `browser.` prefix and stamps whatever verb it was given; it never consults a list, so discovery is advisory and execution is open. A verb the executor has no branch for is a no-op. That is a real gap and it is a chosen one: the alternative is that the host carries a copy of the verb list and every new capability requires changing two components in lockstep, with a version-skew failure mode of its own — a host one release behind would reject verbs the executor could handle perfectly well. The trade is that a typo fails silently instead of loudly, which is the right side of the trade for a surface where the executor is the single place a browser action is ever run.

The concentration of execution in one function is itself a failure-containment decision. Four surfaces can ask for a browser action — a page's palette, the new-tab palette, a HUD page, a script from a terminal — and the new-tab page in particular is a different extension with isolated storage, bridging through `externally_connectable`. Because all four converge on one executor, there is exactly one place where an action can fire, one place to add a capability, and one place a bug in dispatch can live.

Path C: a keystroke broadcast to eight panes

Synchronize-panes is the least reliable path in the system and is the one where that is the correct design. A key typed in a pane is forwarded up as a semantic token, relayed down to every other pane in the group, and applied by each pane against its own focused field, dispatching a real `input` event so the framed page's handlers see the change. There is no acknowledgement anywhere in that chain.

Consider what the alternative would cost. To make the broadcast reliable you would need each pane to confirm application, the top frame to track outstanding confirmations, and some policy for a pane that never answers — a cross-origin iframe running an arbitrary site, which may be slow, may have navigated, may have lost its focused field, or may have been closed between the keydown and the relay. The policy for a non-answering pane would have to be either “block the typist,” which makes the feature unusable, or “proceed anyway,” which is what fire-and-forget already does with none of the machinery.

The design instead makes each hop *locally correct* so that a failure is confined to one pane. The most important instance is why **tokens travel instead of results**. When you press `Ctrl-w`, the top frame does not compute the resulting text and send it; it sends the kill-word token, and each receiving pane walks back over trailing whitespace and a run of non-whitespace *in its own field* and deletes that span. If the panes' fields hold different text — which they will, the moment one site autocompleted and another did not — a broadcast of resulting text would corrupt seven fields to match one. A broadcast of intent leaves each field correct with respect to its own contents. The same reasoning explains the last-focused fallback: a pane whose field lost focus applies the edit to the field it most recently had, because the alternative is dropping the keystroke in exactly the pane the user was not looking at.

The residual failure is honest and small: a pane that is genuinely unable to apply an edit silently does not, and the typist finds out by looking. For a feature whose purpose is typing one command into eight terminals at

once, that is the right failure — the same one you would get typing into eight real terminals where one had scrolled away.

The one path that does not tolerate partial execution

Three paths above are safe under partial execution because their writes are idempotent, their truth is single-sourced, or their failures are confined. The terminal is none of those. A PTY is a stateful byte stream, and a dropped write is not a missing repaint; it is a corrupted command line, possibly a half-typed destructive command. There is no reconciliation step that fixes it after the fact.

So the terminal is the one place the design pays for ordering and delivery instead of routing around them. The bytes travel over a long-lived port rather than a bus, because a stream needs an attached receiver by definition. Output produced while no page is attached is buffered and flushed on reconnect rather than discarded, so navigating does not lose the shell's output. The spawn is honored once per tab, so a re-injected content script re-attaches to the running shell instead of forking a second one behind the same UI — which would produce the genuinely confusing failure of two shells sharing a screen. A content-script disconnect leaves the native port alive, because the page's lifetime is not the session's lifetime; only the tab's removal ends it. And the palette's shell one-liner waits — polling for up to ten seconds until the PTY is actually alive — before writing the command, rather than writing optimistically into a session that may not exist yet.

Every one of those is a mechanism the other three paths do without. That is the chapter's point: partial-failure tolerance is not a property you apply uniformly, it is a property you buy where the cost of failure justifies it. The scheme fan-out gets idempotence and a single source of truth and nothing else, because a stale color for a second costs nothing. The automation bus gets exactly-once delivery and an open verb surface, because double execution is unacceptable and an unknown verb is merely useless. The broadcast gets local correctness and no delivery guarantee, because confirmation would cost more than the failure. The terminal gets ordering, buffering, a spawn guard, and a readiness poll, because a stream cannot be repaired. Reading the four together is reading the actual engineering judgment in the system, which is not “make everything reliable” but “know what each path costs when it half-runs.”

Chapter 33 – The Cost of Reuse

The book has said “extracted verbatim,” “consumed as a submodule,” and “promoted into the shared toolkit” in a dozen places, always approvingly. That approval is earned but it is not free, and a chapter that only celebrated reuse would be leaving out the half of the story a maintainer actually lives with. `zwire` consumes shared code three different ways, and the differences between them are not stylistic — each is the correct answer to a different constraint, and choosing the wrong one produces a specific, predictable class of failure. This chapter is the taxonomy and the tax.

Three consumption models

The submodule is the default: `zgui-core`, `zpwchrome`, and the native `zwire-host` are all git submodules with their own repositories, their own CI, and their own release cadence. The condition that makes a submodule right is that the shared thing is a *project* — it has consumers other than this one, tests of its own, and a version history worth tracking separately. The payoff is that an improvement upstream reaches `zwire` with a pointer bump instead of a hand-merge, and that there is exactly one canonical copy rather than a frozen fork drifting quietly. `zgui-core`’s own guidance is emphatic on this point: consume it as a submodule and load modules directly from the submodule path, never copy the component tree, because copies go stale and partial-sync.

The verbatim copy is used exactly where a submodule cannot reach, and the constraint is an extension boundary. The palette’s backend-agnostic item source is vendored into each extension that carries a palette, and the scheme table exists in more than one place — once for the new tab and once for the internal HUD — because the two run in different worlds. This is not a lapse in discipline; it is forced. A Chrome extension cannot load a script file out of a *different* extension’s directory at content-script injection time. The new-tab extension and the internal HUD are separate extensions with separate packages, so a file both need is a file both must contain. The discipline that makes it survivable is that the copies are *verbatim* and declared as such — the point of vendoring the palette’s item source into each extension is precisely so the four palette implementations cannot drift, and the value of saying so in the file is that the next maintainer edits the source rather than the copy.

The shim-adapted shared component is the most interesting of the three, because it is what happens when a shared component’s assumptions do not match the consumer’s environment. The terminal front end is the suite’s shared embedded-terminal component, and its own header records that the only `zwire`-specific change is a transport shim that picks a transport at load. The tiling overlay is the shared window manager plus a provider that supplies iframe panes and a pane-side forwarder. The statusbar is the shared powerline bar plus a thin adapter that feeds it from storage keys. In each case the *component* is shared unchanged and the *difference* is isolated into a named adapter, which is the pattern that lets one implementation serve a browser extension and a desktop application at the same time.

The promotion migration, as a case study

The two largest content scripts in the HUD — the tiling overlay and the powerline statusbar — both began as bespoke `zwire` code and were later promoted into the shared toolkit. That migration is worth walking, because “move the file into the shared repo” describes almost none of the actual work.

The overlay’s original form assumed it owned its panes. Once it became a shared window manager consumed by desktop applications as well, that assumption no longer held for `zwire` specifically: `zwire`’s panes are

cross-origin iframes, so the shared manager cannot listen for keys inside a pane, cannot read a pane's selection, and cannot apply a keystroke to a pane's field. The shared component therefore had to grow an *external-feed* surface — entry points through which a consumer outside the component can inject a prefix, a command key, a relayed sync keystroke, and a yank. That API exists in the toolkit at a specific version precisely so a cross-origin-iframe consumer can feed the manager from outside. The migration was not a move; it was an API design exercise driven by the one consumer whose environment was hardest.

On *zwire*'s side the single script split into two. The top frame loads the shared manager, its widget dependencies, and a provider that initializes it with iframe-backed panes — an address bar, the pane creation and render hooks, and bridges for applying keys, entering copy mode, and pasting, all over `postMessage`. Every frame loads the pane-side forwarder extracted from the old script, which relays the prefix, synced keystrokes, and yanks upward and runs copy mode locally over the pane's own document. The model, the command set, the copy-mode behavior, and the persistence are all unchanged; what moved is where the code lives. The retired original stays in the tree for reference and is no longer injected — which is itself a small honest decision worth naming, because deleting it would erase the record of what the shared component was derived from.

The tax

Three costs come with all of this, and they are real.

The first is **version coupling**. Before the promotion, the overlay's behavior was determined entirely by a file in this repository. After it, the overlay's behavior is determined by the toolkit submodule's pointer, and the external-feed API the provider depends on exists only from a particular toolkit version onward. A submodule bump is therefore no longer a routine hygiene commit; it is a semantic event that can change how the browser's tiling behaves, and a bump *backward* — or a checkout that resolves the submodule to an older pointer — produces a provider calling an API that is not there. The failure signature is a feature that is present in the manifest and dead at runtime, which is a distinctly worse debugging experience than a file that is simply missing.

The second is **the absent-submodule case**, which *zwire* handles well precisely because it planned for it. A clone without recursive submodule initialization leaves the *power-tool*'s directory without a manifest, and the launcher's rule — include an extension directory only if it contains a manifest — silently skips it. The browser runs with what remains. This turns a missing dependency into a missing feature, which is the right conversion for an additive component, and it is why a fresh clone runs at all. The cost is that the failure is *quiet*: nothing errors, the capability is just not there, and a user who does not know the submodule exists has no way to discover that they are running a reduced browser. The failure-mode chapter has to name this explicitly for exactly that reason.

The third is **the copy that gets edited**. The verbatim-copy model works only as long as every copy is genuinely verbatim, and there is no mechanism enforcing that — no generator, no checksum gate, nothing in CI comparing the vendored item source across the extensions that carry it. It survives on discipline and on the files saying what they are. This is the weakest link in the reuse story and the honest thing to say about it is that its safety is a convention rather than a property. A divergence would show up as a palette that behaves subtly differently on the new tab than on an ordinary page, which is precisely the kind of bug that gets attributed to the surrounding surface rather than to the shared file.

The rule

The three models sort cleanly by constraint, and the decision is nearly mechanical:

constraint	model	zwire example
own repo, own tests, other consumers	submodule	zgui-core, zwire-host
crosses an extension package boundary	verbatim copy	palette items, scheme table
shared component, other environment	shim + adapter	terminal, tmux, powerline

Never copy what can be submoduled. Never submodule across a boundary that cannot load across it. When you must copy, copy verbatim, say so in the file, and edit the source. And when a shared component nearly fits, add an adapter rather than a fork — because a fork is a copy that has stopped announcing itself, and every cost above gets worse when the divergence is invisible.

The net verdict on the reuse discipline is favorable and should be stated as a measurement rather than a virtue: promoting the overlay and the statusbar into the shared toolkit made the HUD's own code smaller and its dependency graph *sharper* — fewer lines here, more explicit versions there. The adapters are the price, and they are cheap. But the price is paid in a currency the original bespoke scripts did not use at all: version management. That is the trade, and it is worth making, as long as everyone reading the manifest understands that a submodule pointer is now part of the browser's behavior.

Chapter 34 – The Lineage: The Cyber-punk Idiom and the zpwr Fleet

zwire did not arrive from nowhere, and it is not a browser that happened to be painted with neon. It is the browser-shaped member of a larger family of tools that share an idiom — a keyboard grammar borrowed from tmux and vim, a powerline aesthetic, a neon palette — and, more concretely, share code. Understanding the lineage explains things the feature chapters can only assert: why the terminal is “extracted verbatim from Audio-Haxor,” why the fuzzy matcher is the same one in six other apps, and why a browser was, in a sense, the family’s missing member.

The idiom: tmux, vim, and powerline as a grammar

The HUD’s controls are not invented from scratch; they are a dialect of a grammar the author already spoke everywhere else. The tmux overlay uses tmux’s prefix and tmux’s split, navigate, and zoom verbs, because a person who lives in tmux should not have to learn a second tiling vocabulary to tile browser panes. Vim on every page uses vim’s motions for the same reason: the cost of the feature is near zero for someone who already has the motions in their fingers, and the whole value of putting them in the browser is that the browser stops being the one place the grammar does not work. The powerline statusbar is the same aesthetic that terminal prompts and editor status lines in this world already wear — the segmented, arrow-jointed bar that encodes state as colored runs. None of these is a novelty; each is a *continuation*, and that is the point. The HUD is legible on first sight to anyone who came from tmux, vim, and a powerline prompt, because it is those three grammars wearing a browser instead of a terminal.

The fleet: shared submodules across the house

The lineage is not only aesthetic; it is literal shared code, tracked as git submodules. `zgui-core` is the shared toolkit — the fuzzy matcher, the palette, the visual effects, the fonts — consumed by Audio-Haxor, `zmax`, `zpwr-daw`, `traderview`, `ztranslator`, `zpdf`, `zemail`, `zoffice`, and more, and by `zwire`’s internal HUD among them. `zpwr-i18n` is a shared translation catalog. The native `zwire-host` is itself a submodule with its own CI and README, written to be embedded by sibling hosts. Because these are submodules rather than copied snapshots, a fix to the fuzzy matcher or a new palette behavior propagates to every consumer through the same pointer bump, which is why the book can describe a feature as behaving “the way it does in the other apps” and mean it exactly: it is the same code. The fleet is not a set of apps that resemble each other; it is a set of apps built from a shared kit, and `zwire` draws on that kit the way the others do.

Verbatim extraction as a design value

The most distinctive habit in the lineage is the refusal to re-implement. The terminal front end is not `zwire`-original — `terminal.js` is the shared `zpwr-embed-terminal` component, extracted verbatim from Audio-Haxor, with its header recording that the only `zwire`-specific change is a transport shim. The palette and the find bar are not `zwire`-original UI. The fuzzy matcher is the shared one. This is a value, not an accident: when a component is *extracted verbatim* rather than forked and edited, every app that carries it benefits from every fix, and no app accumulates a private divergence that has to be maintained alone. The transport shim pattern is

how this survives real differences between hosts — the terminal component auto-detects whether it is talking to a Tauri backend, a JUCE backend, or, in zwire, a native messaging port through a service worker, and the same component runs over all of them because the *difference* is isolated to a shim while the *component* stays shared. The discipline is the same one the native host follows with its library crate: write the capability once, embed it many times, and let the seam absorb the platform.

Why a browser was the missing member

Seen against the fleet, zwire fills an obvious gap. The family had an audio workstation, an editor, a translator, a PDF tool, a mail client, an office suite — tools that each put the tmux/vim/powerline grammar on a specific task. The one application everyone uses all day and nobody in the family had reshaped was the browser, and the browser is the hardest one, because you cannot reshape it with a Tauri window: as Chapter 2 argued at length, the power the fleet's idiom wants — the extensions, the native chrome — exists only inside a real Blink engine. So the browser waited until the approach was clear: not a from-scratch engine and not a WebView shell, but a real Chromium owned at the seams a rebrand and a patch series can reach. zwire is the fleet's grammar finally applied to the application that resisted it longest, which is why it reads as continuous with the others despite being, mechanically, the least like them.

The neon palette as house style

The eight schemes are house style, not zwire style. The neon-on-near-black palette, the rationing of high chroma to edges and signal, the Share Tech Mono and Orbitron display faces — these are the fleet's look, and zwire wears them so that a person moving from the audio workstation to the browser to the editor sees one continuous world rather than three apps that happen to be dark. This is why the palette is a closed color *system* (Chapter 8) rather than a set of pretty colors: a shared look across many apps has to be defined as roles and derivations, so that a new app can adopt it correctly and an old one can gain a light mode without every surface being hand-tuned. The cyberpunk idiom is the family's signature, and zwire's contribution to it is the hardest surface the family had ever had to paint — a browser's chrome, reached on the runtime path through content scripts and on the fork path through C++ — proving the house style survives all the way down to the native frame.

Chapter 35 – Positioning: zwire Among the Custom Browsers

“Custom browser” is a crowded and imprecise category, and zwire is easy to misfile in it. This chapter draws the map of the neighborhood — qutebrowser, Vivaldi, Arc, plain theme extensions, Electron wrappers, and the from-scratch engine — and places zwire precisely against each, because the value of the project is clearest when you can see exactly which trade every neighbor made that zwire refused, and which one it made that they did not.

The five other answers

There are five well-trodden ways to get a browser that is not stock Chrome, and each is a coherent answer to a different question. **qutebrowser** is keyboard-first and vim-modal like zwire’s page layer, but it is built on Qt WebEngine and its own Python application, so it is a different application wearing a Chromium *render* engine rather than a Chromium *browser* — the extension ecosystem is not there. **Vivaldi** is a heavily customized Chromium with tiling, sessions, and deep UI configuration, shipped as a closed product on top of the engine; it is the closest in *features* and the furthest in *model*, because it is a company’s fork you consume, not a rebrand you own. **Arc** reshaped the browser’s information architecture — spaces, a sidebar, a different notion of a tab — on a Chromium base, as a designed product. **Theme extensions** are the floor: they repaint the frame and nothing else, and Chapter 1 spent its length on exactly why that floor is where a rebrand that only themed would stop. **Electron and Tauri wrappers** are the other pole: they give you total control of a window’s contents at the cost of not being a browser at all — no address bar you did not build, no extension host, and, for Tauri, not even a Blink engine.

What zwire shares with each, and where it diverges

zwire shares qutebrowser’s conviction that the browser should be driven from the keyboard, and diverges by keeping the full Chrome extension host — zwire’s vim is a content script layered onto a real Chromium, not a modal application that replaced it, so **zpwirchrome** and its MV3 permissions still run. It shares Vivaldi’s and Arc’s ambition to reshape the whole browser, and diverges on ownership: Vivaldi and Arc are products you receive, while zwire is a rebrand and a patch series you apply to a snapshot Google builds, so nothing sits between you and the upstream engine except a shell script and, optionally, twenty-five readable patches. It shares the theme extension’s mechanism on the runtime path — it is an extension-and-flags rebrand — but pushes that mechanism far past where a theme stops, into content, behavior, and the internal pages. And it shares nothing important with the Electron wrapper, because the wrapper gave up the one thing zwire’s whole justification rests on: it is a real browser, with a real address bar, a real extension host, and a real Blink engine, not a single web view in a native window.

The runtime rebrand versus everyone who forks

The sharpest divergence is structural, and it is the two-tier design itself. Every neighbor that reshapes the native chrome — Vivaldi, Arc — does it by *maintaining a fork*, which means they own a divergence from upstream permanently and pay to carry it forward forever. zwire’s insight is that most of what people want from

a custom browser does not actually require the fork: colors, content, behavior, the new tab, the internal pages, the vim layer, the tiling, the terminal — all of it lives on the runtime path, on top of a stock snapshot that tracks upstream for free. The fork exists only for the residue that genuinely needs C++ — tab shape, the UI font, the native frame, the audio engine — and it is offered as an *option*, not imposed as the price of entry. No neighbor draws this line. The theme extension stops short of it; Vivaldi and Arc pay past it for everyone. *zwire* is the only one that put the line exactly where the capability boundary is and let you choose which side to stand on.

The cost *zwire* chose not to pay

Positioning is as much about refusals as features, and *zwire*'s defining refusal is the one Chapter 1 named: it does not write a browser. A from-scratch engine is a decade of work nobody finishes, and it buys independence from Google's engine at a price no small project can afford. *zwire* refuses that cost and inherits Blink's engine, security model, and update cadence instead — which is why it exists in kilobytes of shell and JavaScript rather than millions of lines of C++. It also refuses the Electron cost, which is the opposite trade: total window control in exchange for not being a browser. And on the daily-driver path it refuses the fork's cost — the recurring rebase tax of Chapter 21 — by making the fork optional. The through-line is that *zwire* is unusually clear-eyed about which costs buy something and which do not: it pays for a dedicated profile, an extension set, a native host, and (optionally) a patch series, and it declines to pay for an engine, a runtime, or a permanent fork it does not need.

Who *zwire* is for

The honest answer to “who is this for” falls out of the positioning. It is for someone who already lives in *tmux*, *vim*, and a powerline prompt and wants the browser to stop being the exception — who values the keyboard grammar enough to want it on every page. It is for someone who wants a real Chromium with real extensions, not a web view in a native shell, so the *WebView* wrappers are out. It is for someone who would rather own a rebrand of an upstream snapshot than consume a company's fork, so the update cadence stays Google's and the customization stays readable. And it is for someone willing to run a separate profile for their power tools, keeping their real logins in their real browser. It is not for someone who wants a supported product with a company behind it, or who needs the native-chrome HUD without being willing to build and rebase, or who wants their everyday and their experimental browser to be the same profile. The category “custom browser” holds all of these people; *zwire* is the answer for the specific one who wants the fleet's grammar on a Chromium they own.

Chapter 36 — Failure Modes and Recovery

Chapter 24 gave the degradation ladder — the principled account of what is fatal and what is best-effort. This chapter is the field guide beside it: the symptoms you will actually see, the cause behind each, and the fix, ordered roughly by how often you hit them. It is the chapter you turn to when `zwire` is misbehaving and you want the one command that sets it right, and it closes with the recovery kit — the small set of scripts that resolve almost everything.

The base is gone or wrong

The one fatal condition is a missing base, and it announces itself clearly: a fresh `zwire` on a machine with no base recorded exits with the exact `run: .../scripts/fetch-base.sh` message rather than failing opaquely. The fix is the command it printed. The subtler version is a *wrong* base — a snapshot that is too old for the `minimum_chrome_version: 127` the power-tool declares, or one that lost the `--load-extension` switch. The tell is that the browser launches but the extensions do not load or the power-tool refuses to; the fix is `fetch-base.sh` to a current snapshot, or `fetch-base.sh <revision>` to pin a known-good one. The base is the floor of the whole stack, so almost any “nothing works” symptom is worth checking against the base first: `base.path` and `base.version` at the top of the state directory are the entire contract, and if `base.path` points at a binary that is gone, everything above it fails in confusing ways until you re-fetch.

The rebrand reverted (macOS)

The most common cosmetic failure, and the one Chapter 24 warned would happen, is the Dock reverting to “Chromium” after a base upgrade. The cause is mechanical: `fetch-base.sh` unzips a fresh stock bundle over the patched one, so the name, the icon, the deleted `CFBundleIconName`, and the ad-hoc re-sign are all gone. The fix is `rebrand-macos.sh`, the one manual step an in-place upgrade must not forget — which is why `install.sh` chains it on a first install but leaves it to you on an upgrade. If the rebrand ran but the old icon lingers, the cause is one level further out: macOS’s LaunchServices cache is still serving the previous icon, and `bust.sh` forces it to refresh. The order to try is: re-run the rebrand first, and only reach for `bust.sh` if the name is right but the icon is stale.

The native host is silent

If the terminal will not open and the statusbar’s system stats are blank, the native host is not answering. This is not fatal — the browser runs fine without it, with one stderr line — but it disables the two features that depend on it. The launcher builds the host on launch when its manifest is missing or its target binary is gone, so the first fix is to relaunch and let it rebuild, or to force it with `FORCE=1`. If it still will not connect, the manifest is the thing to check: `com.zwire.hud.json` under the profile’s `NativeMessagingHosts/` must point `path` at a binary that exists and must list the extension’s origin in `allowed_origins`. On Windows the wiring is the registry rather than the profile dir, so a Windows host that is silent is a registry problem, not a file one. The daemon path has its own failure — a socket that will not open — and the cause there is almost always

permissions: the socket is `0600` inside a `0700` dir, so a socket owned by another user, or a stale socket file from a crashed daemon, blocks a new one until it is removed.

A submodule is missing

If a feature is simply absent — no power-tool, no shared toolkit behaviors — the likely cause is an uninitialized submodule. `zpwchrome` and `zgui-core` are submodules, and a clone without `--recurse-submodules` leaves their directories without a `manifest.json`, so the launcher skips them silently and loads what remains. The symptom is a browser that works but is missing a whole capability; the fix is `git submodule update --init --recursive`. This is best-effort by design — a missing submodule degrades rather than crashes — which is exactly why it can be easy to miss: nothing errors, the feature is just not there. If a submodule is present but at the wrong commit, the tell is subtler still, a feature that half-works or behaves like an older version; the fix is to sync the submodule to the pointer the superproject records.

Extensions did not load, or the wrong ones did

If the developer-mode banner appears but a specific extension is not running, the staging is the place to look. The launcher copies the loaded extensions into `~/.zwire/ext/` at launch rather than running them in place, so a stale copy there can shadow a change you made to the source. The fix is to clear the staged copy and let the launcher restage — the dev-loop scripts (Chapter 22) do this as part of a rebuild. If the *wrong* extensions loaded, or system Chrome's extensions somehow appeared, the cause is a profile confusion: `zwire` must be running against `~/.zwire/profile`, and if `$ZWIRE_STATE` or the profile path were overridden to point elsewhere, the browser is loading a different world. The `state-dir.sh` resolution and the `base.path` contract are the two things to verify when the browser is running but is not the browser you expected.

Schemes desynced

If the new tab is one scheme and the internal pages are another, the sync bus has a stale reader. The scheme lives in a native file the host owns, written atomically, and every surface reads it — the new-tab poller, the internal HUD, the fork's color mixer. A desync means one reader did not see the last write: usually a page that was open across the change and did not re-read, which a reload fixes, or a host that is not running so the pub/sub push never fired, which folds back into the silent-host case above. The mirrored `zb_scheme` storage key is the cross-script copy; if it and the native file disagree, the native file is the source of truth and the fix is to re-trigger a scheme write (set the scheme again) so every subscriber gets the push.

The fork will not rebase

On the fork path, the recurring failure is a patch that no longer applies after upstream moved, and Chapter 21's economics section is the frame for it. The `check-patch-series.sh` gate catches the structural drift — a patch on disk missing from `series`, or a `series` entry with no file — before anything applies. A patch that applies but is *wrong* after a rebase is the dangerous case the CI cannot catch, and the audio patches are the ones to re-verify by hand. The pragmatic recovery is the one the series file already models: if a patch has become fragile, back it out with a commented-out `series` line and a note, as `0008` was, rather than carrying a broken one forward. And the ultimate fallback is the two-tier design itself — if the fork will not build, the runtime rebrand still runs on a stock snapshot, so a broken rebase costs you the native-chrome HUD, not a working browser.

The recovery kit

Almost everything above resolves to one of a small set of scripts, and it is worth holding them as a kit. `fetch-base.sh` re-establishes the base, pinned or latest, and fixes the fatal case and the wrong-base case. `rebrand-macos.sh` re-applies the identity after any base swap, and `bust.sh` clears the LaunchServices cache when the icon is the only thing stale. `setup-native-host.sh` (or a relaunch, or `FORCE=1`) rebuilds and rewires the host when the terminal and stats go quiet. The dev-loop rebuild restages the extensions when a stale copy shadows a change. `git submodule update --init --recursive` restores a missing capability. And `nuke.sh` is the bottom of the ladder — a full reset of the state directory back to nothing, after which a single `install.sh` (or `fetch-base.sh` plus `rebrand-macos.sh`) rebuilds the world from the base up. The reason the kit is this small is the reason the whole browser is this small: zwire is mostly a shell script wrapped around a stock engine, so almost every failure is a missing file, a stale cache, or a reverted patch — and each has one command that puts it back.

Chapter 37 – Text Input, Composition, and the Keyboard That Is Not Yours

Chapter 28 settled who owns a keystroke. It did not settle what a keystroke *is*, and that turns out to be the harder question. A keyboard layer built on single-character bindings — `j` scrolls, `x` closes, `f` starts link hints — is making an assumption about the relationship between a physical key, a character, and an intention, and that assumption holds cleanly for exactly one configuration: a US QWERTY layout, no input method engine, no dead keys, no modifier that composes. Every other configuration in the world stresses it somewhere. This chapter is about those stresses: which of them the model the book has described already absorbs, which ones it absorbs by accident, and which are genuinely open.

It is worth saying plainly at the start that this chapter reasons *forward* from the design the earlier chapters established rather than backward from a file. Where the book has stated a mechanism — the capture-phase listener, the editable guard, the key-to-action table rebuilt from the registry at dispatch time, the prefix state machine and its timeout — this chapter takes it as given and works out what follows. Where the consequence depends on an implementation detail the book has not stated, it says so rather than guessing.

The character, the key, and the physical position

A browser hands a keyboard event three different descriptions of the same press, and they answer three different questions. One is the *character produced*, which is what the user believes they typed and which already accounts for the active layout, the modifiers held, and any transformation the OS applied. One is the *physical position*, which identifies the key by where it sits on a notional US keyboard regardless of what the user's layout prints on it. And one is the legacy numeric code, which is neither and which exists because it predates the other two.

The choice between the first two is not a style preference; it is a decision about what a binding *means*. A binding expressed as a character means “the key that types this letter,” which is stable across layouts in the sense the user cares about: a Dvorak typist who has spent a decade reaching for `j` with the index finger of the right hand will find `j` wherever their layout puts it, and the vim binding follows their fingers because their fingers follow the letter. A binding expressed as a physical position means “this key,” which is stable in a different sense: the same piece of plastic in the same place, whatever it prints. Games choose position. Text-adjacent tools choose character, because their users learned the binding as a letter in a mnemonic — `f` for follow, `x` for close, `d` for down.

The registry model the book describes is a character model. Chapter 28 states the mechanism directly: defaults live in `zkeys.js` as data, user remaps live in storage as `{action: key}` pairs merged over the defaults, and rebinding an action from `j` to `s` rewrites a table entry that the next keystroke on every open page dispatches through. A table whose keys are single characters and whose remap surface lets a user type the character they want is a character table, and the mnemonic vocabulary the HUD inherits from vim and tmux is a mnemonic vocabulary, so this is the correct choice. It is worth naming as a choice, though, because it has two consequences that only appear on layouts the author does not use.

The first is that a layout where the mnemonic letter is unreachable makes the binding unreachable. On a layout where a character requires a modifier that the motion layer bails out on — Chapter 28's modifier bailout leaves anything held with `Alt`, `Meta`, or `Ctrl` alone — a binding on that character cannot fire at all, because the layer that would claim it has already decided the press belongs to someone else. This is not a bug in the bailout; the bailout is what keeps the motion layer from eating the palette's and the find bar's chords. It is a genuine

interaction between two correct rules, and the remedy is the remedy the design already provides: the binding is data, so a user on such a layout rebinds it to a character their layout produces unmodified, and the shared registry means the keybinding editor and the overlay's help card both immediately describe the new reality.

The second consequence is subtler. A character table is only as stable as the layout is, and the layout can change *while a page is open*. Switching input source is an OS gesture, not a browser one, and nothing in the page is notified. The design absorbs this without doing anything, because the table is consulted at dispatch time rather than bound at registration time — the same property Chapter 28 introduced for a quite different reason. A listener that had bound one handler per key at injection would hold a set of handlers keyed to the layout that was active when the page loaded; a listener that consults a table on every press simply looks up whatever character the new layout produced. Dispatch-time lookup was chosen so remaps take effect without a reload; it pays for layout switching as a side effect. That is the pleasant kind of design accident, and it is worth recording precisely because nobody planned it.

Composition: the case where a keystroke is not an intention

The genuinely hard case is an input method engine. When a user is composing text in Japanese, Chinese, Korean, or any of the other scripts that require an IME, the keystrokes they type are not characters they intend to insert. They are input to a composition session that the IME owns, and which will eventually produce a completely different string. During that session, a press of `j` is not the letter `j` in any sense the user would recognize; it is a syllable fragment, a candidate selection, or a navigation within a candidate list.

A capture-phase listener on `document` sees these presses. That is the whole point of capture-phase listening — it runs ahead of the page's handlers — and the IME's composition is happening in a field the page owns. So the arrangement that makes vim beat a site's single-key shortcut also puts the HUD's listener in front of an in-flight composition, and a layer that claimed `j` there would break text entry in text entry for every script that requires one.

What saves the common case is the guard that was written for a different reason. Chapter 28's editable guard states that inside an `input`, `textarea`, `select`, or `contentEditable`, vim claims nothing except `Escape`, with the justification that in a text field every letter is already mapped by the field. Composition almost always happens inside exactly such an element, so the guard that exists to stop vim eating your search query also stops it eating your composition. This is the same shape as the layout-switch case: a rule written for the obvious reason covering a non-obvious one, and it is worth being explicit that the coverage is *incidental*, because incidental coverage is fragile in a specific way. The guard is scoped to a node type. Composition is scoped to a composition session. Those two scopes coincide in ordinary use and diverge in the cases that are hardest to test.

Where they diverge is worth enumerating honestly, because the book has been clear elsewhere that naming a gap is more useful than implying completeness. The clearest divergence is hint mode. Chapter 28 records that the guard ordering inside the vim listener puts hint mode *before* the editable guard, and gives the reason: typing a hint label is not typing into a field even when a field happens to be focused, and reversing the order would make link hints unusable on any page with a search box. That ordering is correct for its stated purpose and it is exactly the ordering that takes hint mode outside the guard that would have protected a composition. A user who arms link hints while an IME session is live is in a state where two systems both believe they own the next keystroke, and the model as described has no arbitration for it. The practical impact is small — arming hints mid-composition is an unusual thing to do — but “unusual” is not “impossible,” and the honest statement is that this is an unarbitrated corner rather than a solved one.

The second divergence is the platform's own signal for the condition. Browsers do expose whether a key event belongs to an in-flight composition, and they emit explicit start and end events around the session. A layer that consulted that signal directly would be scoped to the actual condition rather than to a proxy for it, and would cover the hint-mode corner and any future feature that claims keys outside a field. The book does not claim the HUD does this, because the book has not read a file that says so. What the book can say is what the

correct shape would be if it does not: a composition check belongs at the very top of the guard ladder, above hint mode, because composition is the one state in which the user's keystrokes provably do not mean what they appear to mean.

Dead keys and the two-press character

A dead key is a smaller version of the same problem, and it stresses a different mechanism. On layouts with dead keys, a character like `é` is produced by pressing an accent key that produces nothing visible and then a letter key that combines with it. Two physical presses, one resulting character, and an intermediate state the user is in for the duration.

The interesting collision is with the prefix state machine. Chapter 28 describes the overlay's `Ctrl-b` prefix and vim's one-key prefixes — `g`, `z`, `y`, `m`, and the backtick — as a state in which “one claimant owns the next key unconditionally, which resolves the contention for that keystroke by fiat instead of by rule.” A dead key is structurally identical: the OS also owns the next key unconditionally, for its own reasons, and it armed first because it sits below the browser entirely. Two prefix machines stacked, one in the OS and one in the page, both consuming the same next press.

The resolution is layering, and it favors the OS: a dead key's combination happens before the browser produces a character event at all, so the page sees the composed result and never the intermediate. That means a page-level prefix armed *before* a dead-key sequence receives the composed character as its post-prefix key — an accented letter that almost certainly maps to nothing in the tmux table, which clears the prefix harmlessly. It also means the reverse ordering is invisible: a dead key armed while nothing else is pending simply produces a composed character the HUD sees as one press. The stacking is benign, but only because the timeout exists. Without the 900-millisecond prefix timeout Chapter 28 records, a prefix armed and then followed by a sequence the table does not recognize would leave the overlay waiting indefinitely for a command key that is never coming. The timeout was written as a courtesy to a user who pressed the prefix by accident; it is also the thing that keeps a prefix from being stranded by an input path it knows nothing about.

The macOS Option key, and why the alternate prefix is a hazard as well as a help

Chapter 28 records the tmux prefix as `Ctrl-b` with `⌘-b` as an alternate, and gives the historical reason the alternate exists: an earlier native split patch consumed `Ctrl-b` before the page saw it, and the alternate outlived the patch because a second prefix that nothing else intercepts is a useful escape hatch.

It is worth naming the cost of that particular alternate, because it lands exactly on this chapter's subject. On macOS the Option key is not only a modifier; it is a character-producing modifier. Option plus a letter yields a different character, and Option plus certain letters is itself a dead key that arms an accent. So `⌘-b` on a US layout is a chord that also produces a character, and on some layouts it is a chord that also arms a dead key. A binding on that chord is therefore doing something the `Ctrl-b` binding is not: it is claiming a keystroke that the text-input system would otherwise have turned into content.

Chapter 28's invariant handles this correctly and it is worth tracing why. The rule is claim only what is mapped, claimed completely with both `preventDefault` and `stopImmediatePropagation`. Because `⌘-b` is mapped, the claim is legitimate and the suppression of the character it would have produced is intentional rather than collateral. And because the editable guard fires first, the claim does not apply inside a text field, which is the only place the character would have been wanted. The chord is claimed where it means “arm the prefix” and released where it means “type a character,” which is the correct resolution and is produced by rules that were written before anyone thought about Option-as-a-character-modifier. That is the third instance in this chapter of a general rule covering a case it was not designed for, and the pattern is worth extracting: a keyboard layer that arbitrates by *condition* — is this mapped, is this field editable, is a prefix armed — degrades gracefully on inputs its author never saw, while one that arbitrates by *enumeration* fails on the first input not in

the list.

The terminal is the exception, and it is exempt for the right reason

Everything above is about a layer that must share the keyboard with text entry. The terminal shares nothing: while it has focus, Chapter 28 records that it installs a capture-phase handler forwarding `Ctrl+<letter>` combinations straight through as control bytes, because those chords mean line editing in a shell and the web view would otherwise consume several of them for its own text bindings.

That is a total claim over a range of chords, and it is safe precisely because it is focus-scoped. When the terminal has focus, the user is not composing text into a web page; they are typing into a shell, and the shell's own line editor is the thing that should interpret their keystrokes. The exemption is not “the terminal is special”; it is “the terminal is a different text-entry system, so the browser’s text-entry rules do not apply inside it.”

The consequence worth stating is that this makes the terminal the one surface where the IME story is genuinely out of the HUD’s hands. Composition inside a terminal emulator is a hard problem in every terminal emulator, native ones included, because the composition UI wants to render inline over a grid the shell owns. Whatever the embedded terminal component does about it is what zwire does about it, and that is a correct place for the responsibility to sit: the terminal is a shared component consumed from the toolkit, and the right fix for a text-input gap in a shared component is a fix in the shared component, where every consumer in the fleet gets it. Chapter 33 made the general version of this argument about the cost of reuse; this is the case where the reuse discipline pays, because a text-input problem solved once in the toolkit is solved for the browser, the editor, and the audio workstation at once.

What a reader should take from this

Three things generalize past zwire.

Bind by character when the vocabulary is mnemonic, by position when it is spatial, and never mix the two in one table — a table with some entries meaning “the letter” and some meaning “the key” is unexplainable to a user on any layout but the author’s.

Guard by condition rather than by enumeration. Every case in this chapter that the design absorbed, it absorbed because a rule tested a property of the current state — is this element editable, is this key mapped, is a prefix armed — rather than checking membership in a list. Every case that remains open is one where the tested property is a proxy for the real condition rather than the condition itself, which is exactly the relationship between “the focus is in a text field” and “a composition is in flight.”

And put the timeout in. A prefix machine without one is a state that some input path you have never heard of will eventually strand, and the input paths you have never heard of outnumber the ones you have.

Chapter 38 — Living on the Open Web

Every chapter so far has looked outward from the HUD: what it does, how it is wired, what it costs zwire to run. This one looks the other way. The internal-HUD suite injects into `<all_urls>`, which means every design decision in this book is also a decision imposed on every document on the web, written by people who have never heard of it. A browser extension that runs everywhere is a guest in every page, and the difference between a good guest and a bad one is not politeness — it is a small set of concrete disciplines about namespace, layout, event synthesis, and the platform defenses it chooses to relax. This chapter is those disciplines, the hazards they address, and the ones they do not.

The namespace problem, and why it is worse than it looks

The HUD styles itself entirely in terms of CSS custom properties. Chapter 8 names them explicitly — the accent, the cyan, the magenta, the green, the red, the text weights, the background depths, the borders — as an ordered vocabulary that every HUD surface reads, so that swapping which scheme's values are written to `:root` recolors the entire browser. Chapter 11 records that the statusbar adapter injects the scheme onto `:root` so the shared powerline bar inherits the live theme.

Writing to `:root` is writing to the document's own root element, in the document's own cascade. Custom properties inherit, and they inherit into shadow trees, which is precisely why they are the standard mechanism for theming web components. So a value written at `:root` by a content script is visible to every element in the page, including elements inside shadow roots the script cannot otherwise touch. That reach is what makes the mechanism work for the HUD's own surfaces. It is also what makes it a namespace collision waiting to happen, because the names in question are exactly the names a site would pick. A design system that defines an accent color and a border color and a primary background is not being unimaginative when it calls them accent, border, and primary background; those are the words. If a site defines its own property with a colliding name at a lower-specificity origin, and the HUD writes the same name at `:root`, the site's components silently render in cyberpunk colors.

The mitigation available to any injecting extension is a prefix, and the tension is that a prefix costs exactly the reach that made the mechanism attractive. Chapter 8's color-science section names the HUD's roles in prefixed form when it describes the skinning layer painting roles rather than pixels — the page asks for the accent role, and the scheme table decides what the accent is this minute. The general rule for anyone building this is: a custom property written into a document you do not own must carry a namespace nobody else would choose, and the HUD surfaces that consume it must read the namespaced name. The cost is that a shared component promoted into the toolkit — the powerline bar, the tiling manager — has to be fed the namespaced values by its adapter, which is precisely the job Chapter 33 assigns to adapters. The cost is real and it is the correct place to pay it.

The same reasoning applies one layer up, to element identifiers and class names. Chapter 20 records the terminal content script injecting a `#terminalPane` into ordinary web pages, and Chapter 11 records the statusbar's segments carrying `data-*` hooks. An id is a document-global name. A page that already has an element with that id has a duplicate-id document, and every selector either party writes becomes ambiguous. This class of bug does not announce itself: the page keeps working, one of the two parties occasionally addresses the other's element, and the failure surfaces as an inexplicable misbehavior on one specific site. The discipline is the same as for properties — a namespace, applied consistently — and the reason to state it

as a rule rather than a habit is that ids are added casually, one at a time, by whoever is building a feature, which is exactly how a namespace erodes.

Layout: the browser's viewport is not the HUD's

The statusbar is fixed to the bottom of every top-level page at a defined height, with a translucent blurred background. That is a deliberate, permanent occupation of the bottom strip of every document in the browser, and it collides with the open web in three distinct ways.

The first is a site's own fixed bottom furniture. Cookie banners, chat launchers, audio players, sticky action bars, and mobile-style bottom navigation all live in the same strip. Two fixed elements at `bottom: 0` do not negotiate; they stack, and which one is on top is decided by the z-index ladder. Chapter 10 records that the HUD has a z-index ladder for exactly this reason on the overlay side, and the statusbar's presence in the same contested band means the ladder has to account for a value the HUD does not control — sites routinely use very large z-index values, and a HUD element that wanted to be reliably on top would need to be above all of them. Being above all of them has its own cost: the HUD is then also above the site's modal dialogs, its video controls, and its own overlays, which is the wrong answer for a bar that is decorative most of the time.

The second is viewport-relative sizing. A layout built on full-viewport-height units computes against the viewport, not against the space the HUD left. A page that positions its own footer at the bottom of a full-height column now has that footer under a statusbar. There is no mechanism by which a content script can shrink the viewport a page computes against; the only options are to overlay and accept the occlusion, or to displace the document by offsetting the body, which breaks any element positioned relative to the viewport rather than the document. Overlaying is the lesser evil and it is the choice the design makes, and the honest statement is that the bottom strip of every page is a place where a site's content can be hidden. That the bar can be toggled from the palette — which Chapter 11 records — is not a workaround; it is the actual remedy, and it is why a toggle for an always-on decorative surface is not optional polish.

The third is scroll containers. A site with its own internal scrolling regions, or with a scroll-jacking implementation, interacts with vim's motions in ways neither party anticipated: the motion layer scrolls a document that the site believes it is managing. Chapter 28's invariant limits the damage — the motion layer claims only what is mapped, and a site that handled the key itself would have received it if the HUD had not — but it does not eliminate it, because the HUD's claim on `j` is precisely a claim to scroll in place of whatever the site would have done. This is the one place where the HUD's whole value proposition is in direct tension with web compatibility, and the resolution is the per-tab disable flag Chapter 30 lists among the state deliberately allowed to be lost: on a page where the site's own scrolling is the right behavior, you turn vim off for that tab and the raw keys go back to the page.

Touching the DOM the way a user would

Chapter 29 states a discipline in one sentence that deserves more than one: the find bar registers highlight ranges rather than wrapping matches in elements, and the synchronize-panes receiver splices text into a field and then dispatches a real input event so the page's own handlers see the change — “touch the DOM the way a user would, not the way a script would, so the page cannot tell the difference and cannot be broken by the difference.”

Both halves of that are load-bearing and they fail in opposite directions.

Wrapping matches in elements is the classic implementation of an in-page find, and it is a structural edit to a document the extension does not own. It changes the node tree under the site's own scripts, which may be holding references, observing mutations, running a diffing renderer that will overwrite the wrappers on its next pass, or computing text content that now contains element boundaries where it expected a text run. On a modern application framework, injecting wrapper elements into rendered output is an invitation for the framework to either erase them or crash trying to reconcile them. Registering ranges with the platform's own highlight mechanism — which Chapter 13 covers — sidesteps all of it: the document is not modified, the

highlight is a presentation-layer concern, and a framework re-render simply invalidates ranges rather than fighting with foreign nodes.

Splicing text into a field is the opposite situation: here the extension *must* modify the document, because the whole point is to change the field's value. The hazard is that setting a value programmatically does not, by itself, tell the page anything happened. A controlled input in a modern framework holds its value in application state and re-renders from it; a programmatic write that produces no event leaves the DOM and the application state disagreeing, and the next render silently reverts the write. Hence the synthesized input event, which is not a formality but the actual mechanism by which the page learns. Chapter 32 adds the second half of the discipline for the broadcast case: tokens travel instead of results, so each pane applies the *intent* against its own field rather than receiving a computed string, because the panes' fields hold different text and a broadcast of results would corrupt seven fields to match one.

The generalizable rule is: read without modifying wherever the platform provides a non-destructive mechanism, and when you must modify, modify through the same events the user's own input would have generated. An extension that follows both is nearly invisible to the pages it runs on. An extension that follows neither is the reason users blame websites for extension bugs.

The defenses the HUD relaxes, and the blast radius of relaxing them

The frame-busting rule is the sharpest thing in this chapter, because it is the one place the HUD does not merely coexist with a site's intentions but overrides them. Chapter 7 records the `frame_bust` declarative-net-request rule that lets the tmux overlay iframe any site, and Chapter 25 records the fork's corresponding relaxation of `X-Frame-Options` and frame-ancestors CSP.

Those headers exist for a specific reason: they are a site's declaration that it must not be embedded, and the attack they prevent is clickjacking — an attacker frames a sensitive page, overlays invisible or misleading UI, and induces the user to click a control they cannot see. Every bank, every payment flow, and every account-settings page that sets those headers is defending against exactly that.

The HUD's justification is real and it is worth stating precisely rather than waving at. The threat model those headers address is *an attacker's page framing a victim's page*. The overlay's situation is *the user's own browser, at the user's own instruction, framing a page the user navigated to*. The party doing the framing is the user, and the party being defended against is the user, which is not a coherent threat. This is the same reasoning that lets a browser's own reader mode, print preview, or developer tools show a page in ways the page did not sanction: the user's agent acts for the user.

What keeps that argument honest is scope, and Chapter 25 names the compensating controls: the relaxation yields for sub-frame navigation so a pane can load an otherwise-unframeable site, not for the top-level page, and the pane iframe carries a sandbox attribute set the overlay chose, so the embedded site runs with the overlay's chosen authority rather than ambient authority. The residual risk is not zero and the shape of it is specific: any page that runs in a zwire tab can, in principle, frame any other site, because the rule that strips the header is not scoped to the overlay's own frames in a way a page cannot reach. The containment for that is the containment for everything else in Chapter 25 — the dedicated profile — and the honest summary is that this relaxation is defensible because of the profile boundary, not independently of it.

The re-injection problem, which is the web's fault and the HUD's problem

A single-page application navigates without a document load. The URL changes, the view changes, and from the extension's point of view nothing happened — or, depending on the navigation event, something happened that looks enough like a load to trigger another injection. Chapter 7 records that every script carries a load

guard “because content scripts can be injected more than once across an SPA’s frame and navigation events, and a second injection must be a no-op,” and Chapter 26 records that dropping the guard would stack handlers and make a long-lived tab slow with every same-document navigation.

The part worth adding is what a *correct* guard has to do beyond returning early. Returning early is right for a script whose only effect is to install listeners on a document that still exists. It is not sufficient for a script that installed UI into a subtree the application has since replaced, which is the ordinary outcome of an SPA route change: the guard says “already loaded,” the script returns, and the UI it installed is gone because the framework unmounted the container it lived in. The full shape of an idempotent injectable on the modern web is therefore a guard plus a re-attach path — detect that a prior installation exists, verify its DOM is still present, and re-mount if it is not — and the reason the HUD mostly does not need the second half is that its surfaces attach to the document body or to fixed-position containers of its own creation rather than into the application’s subtree. That is a design property worth naming as a defense rather than an accident: injecting *beside* the application rather than *into* it is what makes the guard sufficient.

What good citizenship actually costs

Collected, the disciplines are five, and they are cheap individually and invisible collectively:

```
namespace    every property, id, and class carries a prefix nobody else picks
overlay      occupy fixed strips, never displace the document; offer a toggle
read-only    prefer non-destructive platform mechanisms over structural edits
synthesize   when you must write, write through the events a user would generate
attach-beside inject next to the application, not inside its subtree
```

The rule underneath all five is that the page is a hostile-by-default environment in the specific sense that it was written by someone who did not know you would be there, will change without telling you, and is entitled to every name and every pixel you did not explicitly claim. That framing produces the right instinct at every decision point, and it is the same instinct Chapter 28’s keyboard invariant encodes for keys: claim only what you are actually using, claim it completely, and leave the rest of the space genuinely unclaimed for whoever needs it next.

Chapter 39 – Accessibility and the Neon Surface

A HUD built on color, motion, glyph density, and keyboard chords is not obviously an accessible interface, and a book that spent a chapter on the color science owes a chapter on who the colors do not work for. This one is deliberately uncomfortable in places. Some of what follows is a description of properties the design already has, some is an argument for why certain choices are better than they look, and some is a plain statement that a capability is absent. Distinguishing the three is the whole value of writing it down.

Two accessibility stories, and only one is usually told

The word covers two quite different populations, and a keyboard-driven HUD sits at opposite ends of the spectrum for each.

For a user with a motor impairment, or repetitive strain injury, or anyone for whom the mouse is a cost, *zwire* is *unusually* accessible, and the feature that makes it so is the one the earlier chapters treated as a power-user convenience. Link hints — Chapter 9’s **f** — convert every actionable element on a page into a short typed label. That is the single most effective mouse-elimination technique that exists on the web, and it works on pages whose authors did nothing to support keyboard navigation, because it does not depend on the page’s tab order being sane. The vim motion layer means scrolling, paging, and jumping to the top or bottom of a document never require a pointing device. The palette means navigation, tab management, and window operations are reachable by typing a few characters. A browser where every operation has a keyboard path is a browser a person can use without a mouse at all, and that is a genuine accessibility property that *zwire* has by construction rather than by retrofit.

For a user with a visual impairment, the story is harder, and the honest version has both good and bad in it.

Contrast: the work that was already done, and the gap that remains

Chapter 8 records a real piece of accessibility engineering without calling it one. The light schemes, it says, are the harder problem: inverting a dark cyberpunk palette does not produce a usable light one, so each role is re-derived against a light ground, and the dimmed foreground — “the one role most likely to fail a contrast check” — is pinned to a value that clears the readable threshold rather than being computed as a fixed alpha over the background, “because a fixed alpha is exactly what fails when the background changes from black to white.”

That is a contrast-first derivation, and it is the right method. Computing a dim foreground as a translucency over an unknown background is the single most common way a theme fails a contrast requirement, because the resulting contrast depends on a value the theme author did not control. Pinning the value instead means the ratio is a property of the palette rather than a property of the page.

The discipline Chapter 8 describes for the accent colors is also, incidentally, an accessibility discipline: the accent is rationed to borders, the active tab, the statusbar segments, the cursor, and the focus ring, and never used for body text, with the body foreground held as a high-value near-white well below full saturation. Saturated neon as body text on near-black is unreadable for everyone and disproportionately unreadable for

users with low vision or astigmatism, where high-chroma text on a dark ground produces visible halation. A palette that never sets prose in the accent has avoided its worst failure mode as a side effect of a design preference.

The gap is that none of this is verified anywhere in the build. Chapter 22 covers the dev loop and CI; the checks it describes are a JavaScript syntax gate, shell linting, JSON validation, and a patch-series gate. A contrast audit — computing the ratio of every foreground role against every background role in all sixteen palette derivations, and failing the build on a regression — is a mechanical check that the palette’s own structure makes easy, because Chapter 8 established that the palette is a small closed set of roles rather than a pile of colors. The table exists, the roles are named, the pairings that matter are enumerable. That such a check does not exist is a real gap, and it is the kind of gap that a book grounded in the tree should name rather than imply away.

Motion, and the preference the platform already knows

The HUD ships visual effects — scanlines, a vignette, neon glow, and animation — and Chapter 8 records that they are toggled through the shared preferences, with the notable inversion that an effect is *on* when its class is *absent*, and that the four toggles travel through the fleet-wide store so every surface follows.

That toggle set is, functionally, an accessibility control, and it is a better one than most applications ship because it is granular: a user who is fine with glow but for whom animation is a problem can turn off animation alone. Vestibular disorders, migraine triggers, and photosensitivity are distinct conditions with distinct triggers, and a single “reduce effects” switch conflates them.

What the design has that it does not obviously use is that the platform already knows the answer for one of them. Operating systems expose a reduced-motion preference, browsers surface it to CSS, and a user who has set it has already told every application on their machine that animation is a problem. The correct default for the animation toggle is therefore not “on” but “on unless the platform says otherwise,” which costs a media query and removes the need for the user to discover a setting in a browser they just installed. The same argument applies to the other three effects with less force — scanlines and vignette are not motion — but a photosensitivity-driven reduced-motion preference is a reasonable signal for damping a full-screen glow as well.

The book cannot claim the HUD does this, because the book has not read a file that says so. It can say that the preference plumbing already exists, that Chapter 8’s store is the correct place for a platform-derived default to land, and that the fleet-wide nature of the store means solving it once solves it for every application in the family — which is the strongest argument for putting the work there rather than in the browser.

Screen readers and the overlay problem

This is where the honest answer is least comfortable.

A screen reader presents a document as a linear traversal of an accessibility tree derived from the DOM. Anything a content script adds to the page is part of that tree unless it is explicitly excluded, and anything the script overlays visually is *not* thereby overlaid in the traversal — a screen reader user encounters the statusbar’s segments as content in the document, in whatever position the elements occupy, regardless of the fact that they are painted as a fixed bar.

That produces a specific, predictable annoyance: a system monitor’s worth of continuously changing numbers, inserted into every page in the browser, traversable as text. The correct handling is that decorative chrome is marked as hidden from assistive technology, so it paints for sighted users and does not exist for the traversal. The statusbar is the clearest candidate — it is a status display, not content, and its numbers change on the stream’s cadence, which if announced would be intolerable. Chapter 11’s `data-*` hooks mean the values are already written into spans by a refresh function, so the exclusion would be a property of the container rather than something to maintain per segment.

The palette and the find bar are the opposite case: they are genuinely interactive, they take focus, and they must be *reachable*. A modal overlay that takes focus needs to announce itself as such, needs its input labeled, needs its results list to expose the active item so a screen reader can announce the selection as the user moves through it, and needs focus to return where it came from on dismissal. These are not exotic requirements; they are the standard obligations of a combobox pattern, and the palette is exactly a combobox. Chapter 17 records that the palette shell and the readline behavior are shared from the toolkit, which means — again — that this is work that belongs in `zgui-core` and that doing it once fixes it across the whole fleet, including the palettes Chapter 33 enumerates as verbatim copies in the other extensions.

The tiling overlay is the hardest surface and there is no comfortable answer for it. Panes are cross-origin iframes; a screen reader traverses each frame's document, and the visual arrangement — which pane is where, which is zoomed, which is active — is conveyed entirely by geometry and border color, neither of which reaches the traversal. Making a tiling window manager legible to a linear traversal is a genuinely unsolved interface problem and not one a browser extension is going to solve in passing. The defensible position is that the overlay is an *optional* surface, not part of the browsing baseline: a user for whom it is unusable does not turn it on, and everything else in the browser continues to work. That is a real answer, and it is only a real answer because the overlay genuinely is optional.

Color as the only channel

Three signals in the HUD are carried by color alone, and each is worth checking against a user with a color vision deficiency — a population large enough that any interface shipping a hue-only signal will be used by someone who cannot read it.

The prefix lamp is the clearest case. Chapter 11 records that pressing the tmux prefix flips the block's background from dim to the scheme accent for a period and then fades back. That is a brightness change as well as a hue change, which is what makes it survive: a luminance difference is perceptible regardless of hue discrimination, and a signal built on “dim becomes bright” works where one built on “green becomes red” would not. The design got this right, and it got it right by accident of the palette's structure — Chapter 8's roles include dimmed and full-strength variants of the same hue, so the natural way to express “active” in this palette is already a luminance step.

The CI indicator is the case that needs the most care. Chapter 8 records the worker tallying failing, running, and passing counts into a status object “the statusbar can show as a dot.” A three-state indicator distinguished only by hue is the textbook color-only signal, and the three states in question are conventionally red, yellow, and green — the exact pairing that the most common deficiency makes hardest. The remedy is cheap and standard: carry a glyph or a count alongside the hue, so the state is legible without color discrimination. The statusbar already does this elsewhere — Chapter 11 records the battery segment prefixing a charging glyph before the percent rather than relying on color to distinguish charging from discharging, and the network and IO segments using directional arrows rather than colored runs. The convention exists in the bar; the CI dot is where it would need to be applied.

The third is the active-pane indication in the overlay, which is a border in the accent color. Here the mitigating factor is that the accent is the *only* high-chroma element in a near-monochrome field, so it is distinguished from its surroundings by chroma and luminance rather than by hue against another hue — which is the discrimination that survives. A user who cannot distinguish the accent's hue can still distinguish “the saturated one.”

The summary a maintainer can act on

The honest ledger has three columns.

Already good, by construction: complete keyboard reachability, link hints as a mouse eliminator, a contrast-derived light palette, accent rationed away from body text, granular effect toggles, and signals that mostly use luminance rather than hue.

Cheap and missing: a contrast check in CI over the palette's role pairings, a platform-reduced-motion default for the animation toggle, the CI dot carrying a glyph, and marking the statusbar as decorative so it leaves the accessibility tree.

Hard and honestly unsolved: the combobox obligations of the palette and find bar, which belong in the shared toolkit rather than here; and the tiling overlay, which is a genuinely unsolved interface problem and whose defense is that it is optional.

The reason to lay it out this way rather than as a general statement of intent is that two of the three columns are actionable lists, and the third is a boundary. An accessibility chapter that only described the good column would be marketing; one that only described the gaps would be flagellation. The useful artifact is the one that says which is which.

Chapter 40 – Two Browsers, One Host, One Theme File

Chapter 19 ends with a sentence that opens a whole subject and then closes the chapter: point `ZWIRE_STATE` at a different directory and you get a second, independent zwire — a different profile, a different base, a different native host manifest — without touching the first. That is true, and it is a genuinely useful property, and it is also only half of the picture, because the same chapter records that the theme deliberately does *not* live in that tree. The scheme, the light flag, the effect preferences, and any custom schemes live in the fleet-wide store the host owns, read and written by every zwire-host client on the machine.

So the two-instance case is not “two isolated browsers.” It is two isolated browsers sharing one theme, and possibly sharing it with an audio workstation, an editor, and a PDF tool at the same time. This chapter is about what is shared, what is not, what the sharing costs, and what the design does about it.

The partition, stated precisely

The state a running zwire touches divides into three tiers, and the tier determines the concurrency story.

Per-install state lives under the state directory and is isolated by construction. The base and its two contract files, the profile with its bookmarks and history and sessions, and the staged extension copy are all rooted there, so two state directories are two independent worlds. Chapter 19’s `ext/` note makes the multi-user version of this explicit: the staged copy exists because Chromium writes extension metadata indexes into the extension directory, so every user needs their own writable copy. The same reasoning applies to two instances of one user — two state directories, two staged copies, no contention.

Per-extension state lives in `chrome.storage.local` inside a profile, so it inherits the profile’s isolation completely. Every `zb_*` key Chapter 31 catalogs — the mirrors, the buses, the preferences, the session state, the guards — exists once per profile. Two instances have two independent command buses, two independent tiling engine states, two independent keybinding remap sets. This is worth stating because it is the surprise in the other direction: a user who sets up their keybindings in one instance does *not* get them in the second, because keybindings are extension storage and extension storage follows the profile. The theme travels; the keymap does not.

Fleet state lives in the host’s store, outside every state directory, shared by every client on the machine. Chapter 19 names it as a real distinct directory rather than shorthand, and Chapter 8 enumerates what it holds: the scheme, the light flag, the effect preferences, and user-defined custom schemes. This is the only tier with a genuine concurrency problem, and it is the only tier that has a lock.

The partition is defensible in each direction. Theme is fleet state because Chapter 8’s whole argument is that a scheme picked in any application repaints all of them, and an isolated theme would defeat that for the second browser instance as thoroughly as for the audio workstation. Keybindings are per-profile because a remap is a property of how you drive *this* browser, and the fleet’s other applications do not have a vim layer to remap. What is worth noticing is that the partition was not chosen by asking “should this be shared” per key; it falls out of which store each piece of state was already in, for reasons that had nothing to do with multi-instance. That it lands somewhere sensible is fortunate rather than designed, and it is the kind of thing worth checking rather than assuming the next time a piece of state is added.

The lock, and what it is actually protecting

Chapter 8 records the failure that motivated the lock and Chapter 32 records its shape: two concurrent writes, one carrying a scheme and one carrying a UI preference, could each read the current state, modify their own field, and write the whole thing back, so the second write silently reverted the first. The fix serializes the read-modify-write behind a cross-process lock file.

The detail worth drawing out is *why the write is a read-modify-write in the first place*, because that is what makes the lock necessary and it is not inevitable. The store is a single document holding several independent values. A writer that wants to change one value has to read the document, change its field, and write the whole document back, because the format has no notion of a partial update. That is the classic lost-update shape, and it appears the moment two writers exist.

There were two other available designs and it is worth seeing why the lock wins. One is to split the document into one file per value, so a scheme write and a light-mode write touch different files and cannot conflict. That eliminates the lost update entirely and it is why the plain-text projections Chapter 29 describes — the ones the fork’s C++ mixer reads — are immune to it: a projection holds exactly one value, so a write is a whole-file replace with no read step. The cost is that a consumer wanting a consistent view of several values has to read several files with no guarantee they were written together, which trades a lost update for a torn read. The other design is a last-writer-wins document with a version counter, where a writer that finds the version has changed re-reads and retries. That is a lock in everything but name, with the retry loop exposed to the caller.

The lock is the right answer here for a reason specific to this workload: the writes are human-initiated and rare. A user picks a scheme, toggles light mode, or changes an effect — these are events measured in seconds apart at their fastest, and the contention window is the duration of a small file rewrite. A lock whose contention is effectively never is a lock with no cost, and it buys a consistent multi-value document that every consumer can read in one operation. The projections exist alongside it precisely because one consumer — a C++ paint path — cannot afford even that, and Chapter 29 is right to call the projection “a deliberate narrowing of a format to fit the weakest consumer.”

How many host processes are there, actually

The native-messaging model spawns a host process per connection. Chapter 26 makes this explicit as a performance argument: Chromium spawns the host per connection on the native-messaging path, so its startup time is paid every time the browser opens a port to it, and a half-megabyte static binary starts in the noise.

Follow that through the multi-instance case. Each browser instance has its own service worker. Chapter 8 records the worker holding a persistent native port for the system-stats stream, and Chapter 30 records that the same port is the worker’s keepalive. So each running instance holds at least one long-lived host process. Chapter 6 records the new-tab extension polling the host on its own interval through its own native-messaging connection, which is a second origin with its own port. Chapter 8 records the power-tool subscribing the host bus directly. Multiply by instances and the machine is running several host processes at once, each with its own sysinfo sampling thread on the interval Chapter 26 floors.

This is fine, and it is worth explaining why rather than treating it as obviously fine. The host is stateless with respect to everything except the files it owns. A sysinfo sample is a read of the machine, and N processes reading the machine produce N identical answers at slightly different instants — wasteful but not incorrect. A PTY session belongs to the connection that spawned it, and Chapter 8’s per-tab keying means sessions are already partitioned by tab within an instance, so partitioning further by instance changes nothing. The file store is the only shared mutable resource, and it is the one thing behind a lock.

The daemon path is where this stops being free, and Chapter 25 already names its gate: a Unix socket created `0600` inside a `0700` directory, so only the owning user can open it. A socket is a single rendezvous point, which means the daemon path is the one where two instances genuinely share a process rather than each having their own — and where a stale socket file from a crashed daemon blocks a new one until it is removed, which

Chapter 36 lists as a real failure mode. The tradeoff between the two transports is therefore also a concurrency tradeoff: per-connection spawn gives every client its own process and no rendezvous problem at the cost of N processes, and the daemon gives one process at the cost of a rendezvous that can go stale.

The shared log is the interesting case

Chapter 8 records that the host keeps a shared command log recording every message it sends and receives, the sysinfo stream included, which the HOST internal page reads back through a command.

A shared append-only log written by several concurrent processes is the one piece of fleet state that is not behind the lock and does not need to be, provided one property holds: that each record is written in a single append and is small enough that the operating system does not interleave two writers' bytes. Line-delimited records are the standard way to get this, and the log's format is line-delimited by construction. The consequence is that the log is *interleaved* rather than corrupted — records from several processes in timestamp-ish order, each intact.

That interleaving is a feature for the purpose the log serves. A user debugging why a scheme change did not reach the new tab wants to see, in one file, the write from one client and the absence of a read by another. A per-process log would put those two facts in two files and make the correlation manual. Chapter 41 takes this up as the central observability argument; the concurrency point here is narrower: the log is the one shared mutable resource whose semantics *improve* under concurrency, and it is the one that correctly has no lock.

The open question the book cannot answer from the text is growth. A log that records every message including a continuously streaming sysinfo feed, written by several processes, grows at a rate proportional to the number of running clients and never shrinks on its own unless something rotates it. Whether something does is not stated anywhere the book has read, and the honest thing is to flag it as the question to ask rather than to assert an answer in either direction.

The rule that falls out

The multi-instance story reduces to a single question to ask of every new piece of state, and it is the question Chapter 31 already asks in a different form: how many processes must see this?

One profile, and it belongs in extension storage, where it is isolated for free and needs no concurrency thought at all. One machine, and it belongs in the host's store, where it needs a lock the moment there are two writers and a projection the moment there is a consumer that cannot parse the format. One process, and it belongs in memory, with Chapter 30's discipline about whether it is mirrored, reconstructible, or expendable.

The failure mode to watch for is a piece of state that answers “one profile” today and “one machine” after a feature is added — a preference that starts in extension storage and later needs to be seen by the new-tab extension, which Chapter 29 establishes cannot read it. That migration is the one that arrives without warning, because the code that needs the value is in a different extension and simply finds nothing there. Knowing the three tiers in advance is what makes that a five-minute decision rather than a surprise.

Chapter 41 – Debugging a System With Six Boundaries

Chapter 36 tells you what to do when a symptom matches a known failure. Chapter 32 tells you what happens inside a path when one hop fails. Neither tells you how to find out which hop failed when the symptom is new, and that is the everyday work. A feature in this browser is a message crossing several of Chapter 29's six boundaries, each with a different transport, a different observability surface, and a different failure signature — and the single most useful skill for working on it is knowing, for any symptom, which boundary to interrogate first and what tool sees across it.

This chapter is that method. It is deliberately concrete: the instruments available at each boundary, what each can and cannot see, and three worked bisections against the paths Chapter 32 already described.

The instruments, and what each one can see

There are five ways to observe a running zwire, and they have almost no overlap.

The service worker's own inspector. A Chromium extensions page exposes an inspect link for a running service worker, opening devtools attached to it. This is the only view of the worker's console, its exceptions, and its heap. Its defining limitation is the one Chapter 30 is entirely about: the worker may not be running when you go looking, and attaching an inspector to a worker is itself an event that affects its lifecycle. Anything you learn from the worker's console is about the worker *now*, not about the worker at the moment your bug happened.

A page's devtools, on the content script's context. Devtools opened on an ordinary page can switch its console context to the extension's isolated world, which is where every content script in Chapter 7's injection stack runs. This sees the content script's globals, its listeners, and the DOM as the script sees it. It does not see the worker, does not see another extension's scripts, and — because Chapter 29's first boundary is real — does not see the page's own JavaScript globals.

The storage viewer. Every `zb_*` key Chapter 31 catalogs is inspectable, and this is the most underrated instrument in the set, because Chapter 30's second pattern means the store holds the worker's answers rather than the worker's questions. If the tab list mirror is stale, the worker has not run its update; if the command bus key holds an action with a tag that never incremented, the page never wrote; if the scheme mirror disagrees with the host's file, the fan-out lost a hop. The store is where most of the system's state is *already* published for reasons that have nothing to do with debugging, which makes reading it the cheapest possible first move.

The host's shared log. Chapter 8 records the host recording every message it sends and receives, the sysinfo stream included, readable back through the HOST internal page. This is the only instrument that sees across the process boundary, and — per Chapter 40 — the only one that sees several clients at once. It answers the question no browser-side tool can: did the request leave the browser, and did a reply come back?

The CDP probes. Chapter 20's `v*.mjs` scripts attach to a running browser over the debugging protocol and assert on real DOM in a real page. Their designed use is verification, but their debugging use is different and arguably more valuable: each probe is a pre-written, known-good traversal of one multi-hop path, so running one tells you whether that entire path is intact without you constructing the test. Chapter 20 is also explicit about their limits — they cannot see module-scoped state, cannot assert absence, cannot distinguish slow from broken, and cannot reach the fork's native chrome at all.

The bisection order

The instruments sort into a natural order, and the order is a bisection: start at the end of the path with the cheapest observation and walk backward.

symptom	first look	then	then
UI wrong	the store	content script ctx	worker inspector
nothing happens	the store's bus tag	worker inspector	host log
stale value	the store vs host	host log	the writing surface
cross-extension	host log	the other ext's ctx	the manifest wiring
process silent	host log	the manifest	the binary path

The reason to start at the store rather than at the code is that the store is a *published answer*. Chapter 30's publish-eagerly pattern means the worker writes its state into storage as a matter of course, so the store already contains a record of whether the worker did its job, with no instrumentation required and no dependence on the worker being awake when you look. That is an enormous debugging advantage that fell out of a design decision made for latency reasons, and it is worth naming because the general lesson transfers: a system that publishes its intermediate state for correctness reasons is a system that is observable for free.

Bisection one: the scheme did not follow

The symptom Chapter 32 calls the most-reported desync: internal pages and page skins recolor, the new tab does not.

Chapter 29 gives the path shape — a surface relays the pick, the worker writes the mirror and relays to the host, the host takes the lock and rewrites the store and its projections, and the renderers reconcile on their own schedules. Four hops, four places to look, and the bisection takes two observations.

Read the scheme mirror in storage. If it holds the new scheme, the surface and the worker both did their jobs and the failure is downstream — the host did not write, or a renderer did not read. If it holds the old scheme, the failure is upstream and the worker never ran the relay, which means either the surface did not ask or the worker was asleep and the ask used a transport that does not wake it.

Then read the host's log. A relay that reached the host appears there as a message in; a write that succeeded appears as its reply. Between those two observations every hop in the path is localized, and the two most common outcomes have distinct signatures. If the mirror updated and the log shows nothing, the host is not running — which Chapter 36 tells you produces exactly this symptom, because the new tab's only channel is the file. If the mirror updated and the log shows the write, the failure is a renderer that has not re-read, and Chapter 32's second case applies: for the new tab that window is bounded by its poll interval, so waiting one interval distinguishes a slow renderer from a stuck one.

The instructive part is what you did *not* have to do. You did not attach an inspector to the worker, did not open devtools on the new tab, and did not read any code. Two published records — one in extension storage because the worker publishes eagerly, one in a log because the host records everything — localized a four-hop failure to one hop.

Bisection two: a browser action did not fire

Chapter 32's Path B: a script asks for a browser action and nothing happens.

This path has a property the scheme fan-out does not, and it is the one that makes it debuggable: the action carries a correlation id. Chapter 7 records the host stamping the action with a nonce into the file-backed key-value entry, and Chapter 32 records the reply travelling over the persistent sysinfo port correlated by a request id so the action fires exactly once. A correlation id is a debugging affordance as much as a correctness one — it means every record of this action, in every log and every store, is findable by one string.

The bisection follows the id. Does the host log show the stamp? If not, the trigger never reached the host and the problem is upstream of the browser entirely — the script, the subprocess, the invocation. If the stamp is there, does the store hold the action entry? If not, the write failed. If it is there and nothing happened, the executor did not run it, and there are exactly two reasons: the worker never woke, or the worker woke and had no branch for the verb.

Those two are distinguishable, and Chapter 32 explains why the second is silent by design: the host accepts any verb and never consults a list, so discovery is advisory and execution is open, and a verb the executor has no branch for is a no-op. That is a chosen trade with a chosen cost, and the cost is paid precisely here, in a debugging session, by a person who has to rule out a typo. The mitigation is not a change to the design — Chapter 32 defends the openness convincingly — but a debugging habit: check the verb against the discovery list before checking anything else, because the failure mode that produces silence at the executor with a clean record everywhere upstream is overwhelmingly a name that does not match.

Bisection three: a key does nothing

This is the hardest symptom in the system to localize, because Chapter 28 establishes seven potential owners of a keystroke and the first three are outside JavaScript entirely.

The bisection is a ladder and it runs top-down rather than bottom-up, because the upper layers are the ones that consume a key invisibly. Does the key produce an event in the page at all? A listener in the content script's console context answers this, and a *no* means one of the first three layers took it — the OS, a Chromium accelerator, or a registered browser command. Chapter 28 names the specific case that trips people: the new-tab page reserves the palette chord at the browser level, so a page listener there will never see it, which is exactly why the worker owns that chord as a browser command rather than a page listener.

If the event arrives, the next question is whether the HUD claimed it. Chapter 28's invariant says a listener calls `preventDefault` only when the key maps to a real action, so an unmapped key reaching the page untouched is *correct behavior* and the symptom “my binding does nothing” most often means the binding is not in the table. Read the merged keymap — defaults from the registry, overlaid with the stored remaps — rather than reading the defaults file, because Chapter 28's whole point is that the table is dynamic and a remap the user forgot about is indistinguishable from a missing binding.

If the key is in the table and the event arrives and nothing happens, the remaining candidates are the guard ladder and the transport. The guards are enumerable — Chapter 28 lists them in order: the per-tab disable flag, hint mode, the editable guard, the modifier bailout — and each has a visible state. And if the action is one that crosses to the worker, the failure may not be in the keyboard layer at all but in the bus, which puts you back in bisection one's territory: read the command bus key and see whether the write happened and whether its tag incremented.

That last point is the most useful thing in this chapter for a new maintainer. A large fraction of “the keyboard is broken” reports are not keyboard failures; they are bus failures wearing a keyboard symptom, because the keystroke's whole job was to write a bus entry. Chapter 30's monotonic tag is the tell: an entry whose tag did not move on the second press means the write path is dead, and an entry whose tag moved while nothing happened means the write path is fine and the worker is the problem. One field distinguishes two entirely different investigations.

What is genuinely unobservable

Four things resist every instrument above, and knowing them prevents hours spent looking for a view that does not exist.

The worker's past. Chapter 30 establishes that the worker can be destroyed at any moment, and a destroyed worker's console history goes with it. There is no retrospective view of what the worker did before an eviction.

The compensating design is, again, the store: anything the worker *published* survives the worker, which is why a debugging session reads state rather than logs on the browser side.

A pane's interior. Chapter 29's fourth boundary is a cross-origin frame, and it is opaque by design. The top frame cannot read what happened inside a pane, and neither can you from the top frame's context. The forwarder protocol's deliberately small vocabulary — which Chapter 29 justifies as API-surface minimization — is also the complete list of what is observable about a pane from outside it. Debugging inside a pane means attaching to the pane's own context, which is a different devtools target.

The fork's native chrome. Chapter 20 says this outright: the fork's contributions are native chrome, none of which is reachable from a page context at all, and its verification story is a patch-application check rather than a behavioral one. There is no CDP view of a color mixer.

The moment of a race. The lost-update failure Chapter 8 records — two concurrent writes each reverting the other — is invisible after the fact by construction, because the store holds one consistent-looking value and nothing records that it was written twice. The shared log is the only instrument that would show both writes, which is a strong argument for the log recording everything rather than only what seemed interesting at the time.

The general principle

Every instrument in this chapter exists because a design decision made for another reason left a record behind. The store is readable because the worker publishes eagerly for latency. The correlation ids exist because exactly-once delivery needed them. The shared log exists because a host with several clients needed one place to see them. The CDP probes exist because the runtime path needed a verification story.

That is not a coincidence; it is the strongest available argument for a design habit. A system whose components communicate through *durable, inspectable intermediates* — a key in a store, a line in a log, a file on disk — is debuggable with a text editor. A system whose components communicate through ephemeral in-memory calls is debuggable only with a debugger attached at the moment of failure, which for a service worker that may not exist when you look is not a workable proposition at all. Chapter 29's durability ladder was presented as a reliability argument. It is an observability argument in exactly the same order, and for exactly the same reason: a transport that survives its receiver also survives your investigation.

Chapter 42 — Incognito, and the Extension That Is Not There

Appendix E ends with the observation that `zwire --incognito` “just works,” which is true in the narrow sense the appendix means: the launcher passes user arguments straight through, Chromium receives the flag, and an incognito window opens. What opens, though, is not a `zwire` window with private browsing turned on. In the default configuration it is a stock Chromium window with none of this book in it, and that is worth understanding precisely, because the two ways to react to it — leave it as it is, or turn the HUD on in incognito — have consequences that run through every chapter of the runtime path.

The default is a browser with no HUD at all

Two independent facts decide what runs in an incognito window, and neither of them is under `zwire`'s control.

The first is a per-extension user setting. Chrome's own documentation for the `incognito` manifest key lists three values — `"spanning"`, `"split"`, and `"not_allowed"` — with `"spanning"` as the default. What the key does *not* do is enable the extension in incognito; that is the user-facing “Allow in incognito” toggle, which is off by default and which the extension author cannot set. The reasoning behind that default is exactly the reasoning this chapter is about: an extension enabled in incognito can write incognito-derived data to disk, and the browser has no way to verify that it does not, so the decision is handed to the person who can.

The second fact is that Appendix O's three manifest summaries record no `incognito` declaration at all — not on the new-tab extension, not on `hud-internal`, not on the power-tool. No declaration means the default, and the default is `"spanning"` *if the user has allowed the extension at all*.

Put together: on a fresh profile, an incognito window in `zwire` has no statusbar, no vim motions, no `%K` palette on ordinary pages, no tmux overlay, no fuzzy find bar, no terminal, and no HUD replacements for the five redirected `chrome://` pages. The new tab is Chromium's, not the HUD's, because `chrome_url_overrides.newtab` is a capability of an extension that is not running. The neon is gone. Every keystroke goes to the page.

That is a startling amount of absence, and it is worth naming the one place it is genuinely useful. Chapter 41 lays out an instrument set and a bisection order for a system with six boundaries, and the instruments it names are all *additive* — read the store, read the log, attach a probe. Incognito is the one subtractive instrument available, and it is the fastest possible answer to the question “is this the site or is this us?” A page that misbehaves in a normal window and behaves in an incognito window has told you, in one keystroke and with no extension reload, that the entire injected suite is implicated. A page that misbehaves in both has exonerated it. No other single action in the system removes all three extensions at once while leaving the browser, the profile, and the base identical.

If you turn it on: spanning means one store, not two

The HUD's own extensions page can flip the incognito toggle — Chapter 14 lists it among the per-extension controls that page exposes through `chrome.developerPrivate` — so this is a decision a `zwire` user can make from inside `zwire`. If you make it, the default `"spanning"` mode has one architectural consequence that dominates all the others: `chrome.storage.local` is a single store, shared by the incognito and non-incognito contexts, and Chapter 31's five roles all become channels between them.

Walk the roles with that substitution in place.

The **mirrors** now describe both contexts at once. `zb_tabs` is the worker's live list of open tabs, read by the palette; with the HUD spanning, an incognito tab is a tab, and its title and URL are visible in the `%%K` palette of a normal window. `zb_frecent` is frequency-scored history, top thirty, read by both the palette and the new tab — so an incognito visit competes for a slot in a list that renders on a durable surface.

The **buses** cross contexts too. `zb_cmd` is written by a content script and executed by the worker; in spanning mode there is one worker, so a command written from an incognito page is executed by the same executor that serves normal windows, against a tab id that may belong to either.

Session state is the sharpest case, because the role is *defined* by durability. `zb_tmux_sessions` holds named layouts written by the Sessions page, and Chapter 14 records that a layout is a set of windows and panes with a rendered preview — which is to say a list of URLs. Saving a session from an incognito window writes those URLs into a durable key that the Sessions page will happily export as JSON. `zb_term_open` is a flag that re-opens the terminal overlay on the next page; set in an incognito window, it re-opens the terminal in a normal one, because the worker resets it on browser startup and extension reload and nothing about incognito is a reset event.

The alternative, "`split`", gives incognito its own store — and pays for it in exactly the places Chapter 31 says to look. The one-shot guards are the clearest: `zb_cmds_seeded` is a first-run flag and `zb_cmds_removed` is a tombstone list, and in split mode each exists twice. The seeder runs again against the incognito store, and it runs without the tombstones the user accumulated in the normal one, so every default keyword command the user deliberately deleted comes back in incognito. Split mode does not remove the coupling; it converts a leakage problem into a divergence problem.

The host has never heard of incognito

The deeper issue is below the extension entirely. `chrome.storage` at least has a mode distinction to argue about; the native host has none.

A native-messaging port opened from an incognito context reaches the same `com.zwire.hud` manifest, spawns the same binary, and talks the same protocol as one opened from a normal window. Appendix G's legacy path carries a scheme read, a scheme write, a UI-prefs merge, a sysinfo stream, and the PTY relay, and not one of those messages has a field that could carry "this request came from a private context" even if the host wanted to honor it. The host writes `hud-scheme` and `hud-ui.json`, appends to the shared command log Chapter 40 describes, and — through `pty_spawn` — forks a login shell.

That last one deserves to be stated without any softening. The terminal in an incognito window is a login shell on your machine. It reads your shell configuration, it writes your shell's history file, and every command you run from it is recorded by the shell exactly as if you had opened a terminal application. The browser's incognito contract covers cookies, history, and cache in a profile; it does not and cannot cover a process the browser spawned on your behalf. This is not a defect in the host — the host is doing precisely what Chapter 15 says it does — but it is the concrete form of the hazard that the "Allow in incognito" default-off setting exists to prevent, and in zwire it is not hypothetical.

The rule, and why zwire needs incognito less than most browsers

Chapter 41 closes on a design principle: a system whose components communicate through durable, inspectable intermediates is debuggable with a text editor. That principle is the whole reason the runtime path works — the store survives worker eviction, the file survives everything, the log survives the process. It is also, read from this chapter's angle, a statement that the system is built out of things that persist by construction. An architecture whose central virtue is durability cannot be made ephemeral by a toggle. The toggle governs

whether the extensions run; it does not and could not reach into a storage key, a TOML file, an append-only log, and a shell history file and make them forget.

So the honest posture is a choice between two coherent configurations, not a slider:

```
toggle off (default)  incognito is stock Chromium; the HUD is absent;
                      the incognito contract holds exactly as Chromium
                      defines it; you also get the best subtractive
                      debugging instrument in the system for free
toggle on (spanning)  the HUD is present and incognito means "no profile
                      cookies," not "no record" - mirrors, buses, session
                      keys, the native files, the log, and the shell's own
                      history all see the session
```

What makes this less painful in *zwire* than it would be in a general-purpose browser is Chapter 25's containment. The reason a person reaches for incognito in their daily browser is usually that the daily browser is signed into everything and every navigation lands in a profile full of their real life. *zwire*'s normal windows already run against a dedicated `--user-data-dir` that started empty and holds only what you did inside *zwire*. The distance between "a *zwire* window" and "an incognito window" is therefore much smaller than the distance between those two things in a browser with your real profile behind it, and the correct move for most *zwire* users is to leave the toggle off, keep the HUD out of private windows entirely, and get a clean bisection tool in the bargain.

Chapter 43 – The Dead Zones

Appendix P's table is usually read as a description of where the HUD runs. Read the other way — as a description of where it does not — it is a map of the surfaces on which every keyboard model, every visual affordance, and every assumption in this book quietly stops applying. Those surfaces are not rare. A user hits several of them in an ordinary hour, and because the HUD's absence is silent rather than an error, the experience is “the keybinding didn't work” rather than “you are somewhere the keybinding does not exist.” This chapter is the map, and the design consequences of each region on it.

The regions

The new-tab URLs. Every block in Appendix P that matches broadly excludes the new-tab URLs, and the reason is ownership: the new-tab extension serves that page. The consequence is that on the HUD's own front door, the statusbar does not exist, the tmux overlay is not loaded, the fuzzy find bar is not installed, the terminal boot script has not run, and the vim motion layer is absent. The palette is present, but it is the new tab's own — which is why the extension declares `externally_connectable.ids: [ hud-internal ]`, so the HUD worker can reach in and open it. Pressing the tmux prefix on the new tab does nothing at all, and nothing indicates why.

The five redirected `chrome://` pages. Block 1 runs `redirect.js` and only `redirect.js`. It navigates you to an extension-origin document under `pages/`, and every other block's match set — `http`, `https`, `file`, `chrome://` — excludes `chrome-extension://` by construction. So the destination of the redirect is a dead zone for the entire content-script suite. This is the structural reason `zg-boot.js` exists in the shape Chapter 14 describes: the HUD pages do not *inherit* the palette, the effects, and the scheme bridge from the injected stack, because the injected stack does not reach them. They mount their own. Every HUD page is a re-implementation of the ambient environment, and the reason it is not duplication is that it is the same shared modules mounted from a different entry point.

Another extension's pages. Appendix O records the power-tool's `options_ui` as `scripts-manager/manager.html`. That page lives on the power-tool's own extension origin, and no host permission zwire can declare will let `hud-internal` inject into it. Chromium forbids one extension from scripting another, and this is one of the boundaries Chapter 29 correctly says exists to enforce something. The design consequence is direct: the cross-extension scheme bridge is not an optimization. It is the *only* mechanism by which the power-tool's own surfaces can follow the active scheme, because the mechanism used everywhere else — injecting a stylesheet — is unavailable on principle.

DevTools. A DevTools window is not a document any block matches. When focus is in DevTools, there is no HUD code running in that surface at all: no vim, no palette handler, no prefix machine. This is the largest routinely-visited dead zone for a developer, and it has a pleasant property — the keyboard model reverts wholesale rather than partially, so there is no ambiguity about who owns a key. It also means that the one place a developer is most likely to want the fuzzy find bar is the one place it is guaranteed absent.

Subframes on ordinary pages. Block 6 is the only `all_frames` block, and it carries `ztmux-pane.js` alone. Every other block is top-frame only. That is correct — one statusbar per page, not one per iframe — but it produces a behavior that looks like a bug from the outside: click into an embedded video player, a comment widget, or a framed editor on an ordinary page, and the vim motions stop responding, the palette hotkey stops working, and the find bar does not open, because the focused document has none of those scripts in it. The pane forwarder is loaded, but Chapter 10's forwarder protocol is a conversation with the *overlay's* top frame,

which is not present on an ordinary page. The user's model is "I am on this page"; the platform's model is "you are in that document," and the keyboard follows the platform.

view-source: and the viewer surfaces. The match patterns are scheme-based. A **view-source:** URL is its own scheme and appears in no block, so it is a dead zone by construction rather than by policy. The same reasoning applies to any surface where the top-level document is not an ordinary HTML page the site authored — a built-in viewer renders content, but the assumptions the suite makes about a document (a body to attach a fixed bar beside, a scrollable page for motions to move) are assumptions about ordinary documents.

The browser's own chrome. The omnibox, the tab strip, native menus and dialogs, and the download surfaces are not documents at all. Chapter 1 draws this line as the boundary between what a runtime rebrand can reach and what only the compiled fork can, and Chapter 3 lays out the seven layers. The point to add here is a *keyboard* one, and it is the reason this chapter matters more than a list of URLs would suggest.

Why the palette needs two doors

Chapter 28 enumerates seven claimants for a keystroke, in dispatch order, all of them running inside a document. When focus is in the omnibox, none of them is running. Nothing in the content-script stack can see that keystroke, arbitrate it, or claim it, because the stack is not there.

That is why Appendix O records a **commands** entry on **hud-internal** — **open-palette** bound to **%K / Ctrl+K** — *in addition to* the palette's own key handling inside the page. The two are not redundant paths to the same thing. A manifest command is registered with the browser and fires regardless of which surface has focus, including surfaces where no content script exists; a page-level handler is precise, arbitrable against the other six claimants, and able to consider what the page is doing. Neither can do the other's job. The command entry cannot participate in Chapter 28's dispatch order because it fires above it; the page handler cannot fire in the omnibox because it does not exist there.

The generalizable rule is the one worth taking out of this chapter: **an affordance that must work everywhere needs a browser-level door as well as a page-level one, and the two doors have different arbitration models.** The palette has both. The vim layer, the find bar, the terminal hotkey, and the tmux prefix have only the page-level door, which is a deliberate and correct choice — they are all page-directed operations, and an operation that acts on a page has nothing to do when there is no page. The affordance that breaks this rule is the one that feels global to the user but is implemented page-side only, and the test for it is simple: put focus in the omnibox and press the key.

The map, collected

surface	suite present?	what fills the gap
new-tab URLs	no	the new-tab extension's own page
redirected chrome:// pages	redirect only	zg-boot.js re-mounts on the HUD page
other chrome:// pages	skin only	theme.js token override
another extension's pages	never possible	the cross-extension scheme bridge
DevTools	no	nothing; the model reverts wholesale
subframes on ordinary pages	pane forwarder	nothing, unless it is an overlay pane
view-source / viewers	no	nothing
omnibox, tabs, native menus	no	the manifest `commands` entry (%K)

Read down the right column and the architecture of the HUD's edges becomes visible. There are exactly three mechanisms for covering a dead zone — mount the same shared modules from a different entry point, push state to a surface you cannot inject into, or register at the browser level instead of the page level — and every place the HUD feels seamless across a boundary is one of the three. Everywhere else, the correct behavior is the honest one: nothing happens, the key belongs to whoever is there, and the user is briefly in a plain browser.

Chapter 44 – Zoom, Density, and the Geometry That Is Not Yours

The HUD draws in three coordinate systems and mostly does not say which one it is in. Chapter 11 gives the statusbar a 22-pixel height with 11-pixel transparent borders forming the chevrons; Chapter 10 gives the overlay a z-index ladder from 2 to 2147483647; Chapter 8's window tiler computes pixel rectangles from a display work area. Those are three different kinds of pixel, and the differences surface only under conditions a developer's own machine usually does not reproduce: a non-default page zoom, a fractional device pixel ratio, a second monitor with a different scale factor, and a fullscreen element. This chapter is those four conditions and what each one does to geometry the rest of the book states as fixed.

Three coordinate systems

space	set by	affected by page zoom?
CSS px in a page	the document's own layout	yes – everything scales
device px	the display's scale factor	no – physical pixels
DIPs in window API	chrome.windows / system.display	no – browser-level coords

Every content-script surface — the statusbar, the find bar, the palette, the overlay's panes and dividers — lives in the first space. Every measurement the window-tiling engine makes lives in the third. Nothing in the HUD lives in the second, which is exactly why the second is where the visual artifacts appear: it is the space nobody wrote code against.

Page zoom scales the HUD, including the parts that were meant to be fixed

A content script's stylesheet is part of the document's cascade, so a 22-pixel bar is 22 CSS pixels, and page zoom multiplies CSS pixels. At 150% zoom the statusbar is half again as tall and occludes half again as much of the page; at 200% it is twice as tall. There is no mechanism by which a stylesheet injected into a document can opt out of the document's zoom factor, because zoom is applied below the level the stylesheet describes.

This interacts with two things the book states elsewhere as bounded. Chapter 26's addendum argues the recomposited area of the blurred bar is bounded because the bar is a fixed small height rather than a proportional one. That argument holds at 100% and degrades linearly with zoom: the bar's area scales, and so does the cost of recompositing it on every scroll frame. And Chapter 38's argument that the bar occupies a strip of the page which the toggle exists to reclaim becomes more pressing at high zoom, because a user who zooms is usually a user who needs the pixels.

The design options are worth naming honestly rather than pretending the issue is settled. A surface can be left in the document's zoom space, which is what happens today and which has the virtue that the bar scales with the text a user chose to make bigger — an accessibility argument, and not a weak one, given Chapter 39's subject. Or a surface can counter-scale, which requires knowing the current zoom factor and re-deriving it on every change, and which produces a bar rendered at 22 physical-ish pixels next to text the user deliberately enlarged. The first is defensible; the second is a decision to override a user preference. What is not defensible is having neither answer written down, because then the behavior is whatever the cascade happened to do.

Fractional zoom and the chevrons

Chapter 11's chevrons are geometry, not glyphs: an 11-pixel transparent top border and an 11-pixel transparent bottom border on a 22-pixel bar, with a solid side border drawing the arrow. That arithmetic is exact — 11 and 11 sum to 22 — and it stays exact only while both numbers stay integers.

At a zoom of 110%, 11 becomes 12.1 and 22 becomes 24.2. At a device pixel ratio of 1.5, integers in CSS space land on half-pixels in device space. In both cases the engine rounds, and the two halves of a triangle can round in different directions, which puts the arrow's tip a device pixel off the vertical center and leaves a hairline seam between two interlocking chevrons that are supposed to tessellate. The two shades that Chapter 11 uses to make consecutive blocks read as distinct steps are what make the seam visible: a seam between two identical colors is nothing, and a seam between `--bg-card` and `--bg-secondary` is a line.

This is the class of visual bug that never reproduces on the machine it was written on and always reproduces on someone's external monitor. The structural fix is to make the height even in the target space — which is why 22 with 11-pixel halves was a good choice to begin with — and the structural hazard is that any future change to the bar height must preserve divisibility by two, in a codebase where nothing enforces that and nothing documents it. Naming it is the enforcement mechanism available.

The window tiler computes in a third space, on one display

Chapter 8's window-tiling engine takes its tiling area from `chrome.system.display`'s *primary* work area, with a documented fallback of 1440×820 when the display API is unavailable, and `rectsFor(n, layout)` produces pixel rectangles — a 60% main pane for `main-v`, a $\lceil\sqrt{n}\rceil$ -column grid otherwise.

Two things follow that a single-monitor developer never sees.

The first is that the tiling area is the primary display's. On a multi-monitor desktop, the rectangles are computed against the primary work area regardless of which display the browser windows are currently on, so `retile` gathers the panes onto the primary monitor. That is a coherent behavior — a tmux-style tiler that owns a screen — but it is a decision about *which* screen that the vocabulary does not express: tmux's `select-layout` has no notion of a second monitor because a terminal has one. A window manager that tiles real OS windows inherits a question a terminal multiplexer never had to answer, and the current answer is “the primary one.”

The second is the fallback. 1440×820 is a plausible-looking rectangle, which is what makes it dangerous. When the display API is unavailable the engine does not fail; it tiles against a fictional screen, and on a large display the result is a set of windows crowded into the upper-left region with a large empty area to the right and below. The failure presents as a layout the user considers ugly rather than as an error they would report. The general lesson is one worth applying to every fallback constant in a geometry path: a fallback that produces a *wrong* result silently is worse than one that produces an obviously broken result, and a fallback whose value looks like a real device is the worst of both.

A third, subtler hazard belongs to the same space. Display work areas are reported in device-independent pixels, and a mixed-DPI desktop — a high-density laptop panel plus a standard external monitor — has different scale factors per display in one global coordinate space. A rectangle computed from one display's work area and applied to a window that ends up on the other is correct in units and wrong in appearance. Nothing in the tiler's model distinguishes the two, because the model is a single work area and a grid inside it.

Fullscreen outranks the entire ladder

Chapter 10 records that the copy-mode indicator inside a pane climbs to z-index 2147483647 “so it is never hidden by page content.” That is true of page content and it is not true in general, because there is a rendering surface above every z-index there is.

MDN's definition of the top layer is explicit: it “sits on top of all other layers in a web document,” it “renders above all other content regardless of z-index and stacking context,” and it cannot be manipulated from code. Three things are promoted into it — an element made fullscreen through `requestFullscreen()`, a `<dialog>` shown with `showModal()`, and an element shown with `showPopover()`.

All three are reachable from an ordinary page, and one of them is reachable from inside a pane on purpose: Chapter 10 records that pane iframes carry `allow='clipboard-read; clipboard-write; fullscreen'` precisely so fullscreen works inside them. So a video player in a pane, put into fullscreen by the user, renders above the copy-mode indicator, above the pane badges, above the drag dividers, and above the overlay itself at 2147483640. The maximum integer is not the top of the stack; the top layer is, and no z-index reaches it.

The same mechanism explains a smaller behavior on ordinary pages: a fullscreen video hides the statusbar, which is the desired outcome, but the HUD does not learn that it happened. `zb_status` records whether the bar is *shown*, and a bar that is shown-but-covered is a state the key cannot represent. That is a benign disagreement today. It stops being benign the moment any logic conditions on `zb_status` to decide whether a surface is visible to the user, which is the kind of inference that gets added later by someone who reads the key's name and not this paragraph.

The rule

Each of the four conditions in this chapter is a case of code written in one coordinate system being read as though it were in another. The discipline that prevents all four is a documentation discipline rather than a code one: **every geometric constant should record the space it is in**. A 22-pixel bar in document space, a 60% main pane in display space, an 11-pixel border that must stay an integer after zoom and DPR, a fallback rectangle in DIPs that is a fiction rather than a device. None of those facts is expensive to write down, and each of them is a bug that otherwise reproduces only on hardware the author does not own.

Chapter 45 – The Long Session

Chapter 26 prices what the HUD spends: per page load, per sample interval, per network request, per open tab. It is an accounting of rates, and a rate can be small forever. This chapter asks a different question of the same system — which quantities never go down — and it asks it because the workload `zwire` is built for is not a browsing session. It is a browser left open across a week of work, with the terminal attached, the stats stream running, the CI poll firing every ten minutes, and a tab count that goes up on Monday and comes down on Friday.

For every durable thing in the system, there is exactly one interesting property: what bounds it. Chapter 31's schema gives each key an owner, a reader set, and a lifetime in principle. The column it does not have is the bound in practice, and the absence of that column is where long-session problems live.

The quantities and their bounds

Bounded by the platform: the service worker. The one component that cannot leak across days is the one most people would suspect first. MV3 evicts the worker, and Chapter 30's patterns are all written around that fact. An eviction frees the worker's entire heap, so whatever the worker accumulated between wakeups is discarded on a cadence the platform enforces and the extension cannot defeat. The worker is structurally incapable of a slow leak.

Bounded by nothing: a content script in a tab you left open. The inverse case, and the one the intuition gets backwards. A tab open since Monday is running the same content-script instances since Monday. Every listener registered at injection is still registered; every structure the palette or the find bar built on first use is still resident; every subscription Chapter 31's rule three requires — readers re-read on change rather than caching at load — is still live and still firing on every storage change from every other tab. The “background” process has a platform-enforced lifetime and the “foreground” script does not, which is the opposite of how the two are usually described.

Bounded by the process model: the native host on the messaging path. Chapter 26 records that Chromium spawns the host per connection on the native-messaging path, which makes the host's lifetime a function of the port's. Close the port, the process goes.

Bounded by nothing: the daemon. The same binary run as a socket daemon is the longest-lived thing in the system. It holds the PTYs, the file observers Appendix G lists under `fs_watch` and `watch_list`, the pub/sub subscriptions, and the peer connections — all of them keyed and individually cancellable, which is the right design, and all of them held until something cancels them. `watch_list` exists so you can see what accumulated; that it exists is a sign the accumulation was anticipated.

Bounded by user intent: the durable overlay keys. `zb_tmux_sessions` grows when the user creates or duplicates a session and shrinks when they delete one. This is the healthy case: growth is proportional to deliberate action, and the Sessions page is both the thing that grows it and the thing that prunes it.

Bounded by an explicit cap, or not bounded at all: the mirrors. Appendix I is unusually informative here, because it records a cap on exactly one mirror: `zb_frecent` is “frequency-scored history, top 30.” That is the pattern every mirror needs, and it is the question to ask of each of the others. `zb_tabs` is bounded by the tab count, which is bounded by the user. `zb_shortcuts` is bounded by the extension count. `zb_ci_cache` is described as “cached CI run data,” and a cache of run data for a repo list whose size is set by `repolimit` in `zb_ci` is the entry on the list whose bound is a configuration value rather than a structural one — which is fine, and which is worth knowing when someone raises `repolimit`.

Already flagged, not re-litigated here: the shared log. Chapter 40 states the growth question and correctly declines to assert an answer. It stays there.

The quota failure nobody expects

The reason the bounds matter more in this system than in most is a property of `chrome.storage.local` that the manifest makes checkable. Appendix O gives `hud-internal` fifteen permissions: `storage`, `nativeMessaging`, `management`, `tabs`, `webNavigation`, `history`, `downloads`, `bookmarks`, `sessions`, `alarms`, `notifications`, `developerPrivate`, `settingsPrivate`, `system.display`, and `declarativeNetRequest`. Chapter 7’s walk through the same permission set names four more that the appendix’s list does not carry — `browsingData`, `tabGroups`, `readingList`, and `power`, backing the automation bus’s data-clearing, tab-grouping, reading-list, and keep-awake verbs — so the appendix’s fifteen is a summary rather than the whole declaration, and a reader reconstructing the manifest should take the union. What matters here survives the discrepancy: `unlimitedStorage` appears in neither list.

Without it, `storage.local` has a fixed quota, and the failure mode when a store approaches its quota is the one that makes unbounded growth genuinely dangerous rather than merely untidy: **the write that fails is not the write that grew**. A store filled by an ever-growing cache does not break the cache. It breaks whichever key is written next, which in this system is whichever feature the user touched most recently — a remap they just saved, a session they just named, a command the palette just put on the bus. Chapter 32’s hop-failure analysis assumes a hop that does not arrive; this is a hop that arrives and cannot land, and it lands on an innocent key.

That converts an abstract question (“is `zb_ci_cache` bounded?”) into a concrete one with a blast radius that has nothing to do with CI. It is also why the single-writer rule from Chapter 31 is load-bearing for capacity and not only for correctness: with one writer per key, there is one place to add a cap, and the cap is enforceable.

The maintainer’s column

Everything above reduces to one addition to the storage schema — a sixth column recording, for each durable key, what bounds it:

bound type	example	what to check
platform	worker heap	nothing; eviction handles it
user action	<code>zb_tmux_sessions</code>	that a delete path exists
structural	<code>zb_tabs</code> , <code>zb_shortcuts</code>	that the underlying set is bounded
explicit cap	<code>zb_frecent</code> (top 30)	that the cap is enforced on write
configuration	<code>zb_ci_cache</code> (<code>repolimit</code>)	what happens when the config is raised
unbounded	a new key with none of these	add one before shipping it

The rule is that a durable key with no entry in the last column is a key whose growth is someone’s future incident, and the cheapest moment to give it a bound is the moment it is added — the same moment Chapter 31 says to declare its owner, its readers, and its lifetime. Three of those four are already the convention. The fourth is the one that only matters after the software has been running longer than anyone tested it, which is precisely the workload this browser is built for.

Chapter 46 — What Leaves the Machine

Chapter 25 answers “what can reach my data” and answers it with the profile boundary. This chapter answers a different question with a different answer: what does this browser talk to, and what can talk to it. The two are not the same question, and the profile boundary — the strongest containment in the system — has nothing to say about the second one. A dedicated `--user-data-dir` contains *data*. It does not contain *packets*.

An owner of a browser should be able to enumerate its network surface on one page. Here is that page, split into what zwire’s own code initiates, what it deliberately does not do, what listens, and what is inherited rather than chosen.

Egress that zwire’s own code initiates

The base download. Chapter 2 and Chapter 4 name the host directly: `storage.googleapis.com/chromium-browser-snapshots`, from which `fetch-base.sh` reads a sentinel to learn the current revision for the platform and then downloads the archive. Chapter 26 puts the archive at around 325 MB. This is unauthenticated, it happens on install and on each deliberate base upgrade, and it is the single largest transfer the project makes. It is also the one with the clearest integrity question, which Chapter 4’s discussion of pinning is the right place to reason about.

The CI poll. The worker’s `chrome.alarms` heartbeat fires every ten minutes and queries the GitHub REST API. Chapter 26 records the interval as chosen to stay under GitHub’s anonymous request cap — which tells you the default posture is anonymous — and Chapter 14 records what changes when it is not: a token in `zb_ci` “raises the repo budget and switches the repo query from the public account to your full affiliations.”

That sentence deserves to be read as a privacy statement, because it is one. Configuring a token converts a periodic anonymous request into a periodic authenticated one that identifies an account and enumerates private affiliations, every ten minutes, for as long as the browser runs. That may be exactly what the user wants; the point is that it is a different network posture, entered through a configuration field rather than a prompt.

It also has a storage consequence that belongs to Chapter 25’s accounting and is not in it. Appendix I records `zb_ci` as `{ user, token, repolimit }` — the token lives in `chrome.storage.local`, in cleartext, in the same store as the keybinding remaps. That does not change the blast radius, which is still the profile, but it changes what is *in* the blast radius: a credential to a service outside the profile. The threat model’s claim is that the worst an attacker reaches through the browser is what you created inside zwire, and a token you pasted into the CI config is now one of those things.

Search. The new tab’s `zb.engine` is a search-engine template with `%s`, and Chapter 6 records that anything typed into the omnibox that is not a URL goes to the search engine. This is ordinary browser behavior and it is listed here for completeness, with one distinction worth drawing: the palette’s own matching, the fuzzy find bar, and the frequency ranking are entirely local computations over local data. The palette is not a search box that queries anything. The fallthrough to a search engine is, and it is the only part of the typing experience that leaves the machine.

The power-tool’s own surfaces. Chapter 18 and Chapter 24 both draw the line: `zpwchrome` has its own native host, installed separately, documented by `zpwchrome` itself, and zwire does not install it. Whatever

egress its features carry is outside zwire's inventory and should be read from its own documentation rather than inferred from this one.

Egress that is absent by construction

Three absences are worth stating as positive properties, because each one was a decision and each one is checkable.

The new tab makes no network requests of its own. Chapter 6 records that its fonts are vendored locally under `newtab/fonts/` rather than linked from a CDN, and Chapter 3 states the consequence plainly: it renders offline. A new-tab page with no remote font, no remote stylesheet, and no analytics is a page that reveals nothing by being opened — and a new tab is the most frequently opened page in any browser.

There is no extension-update channel. The three extensions are loaded unpacked from disk by `--load-extension`, and Appendix O's manifest summaries record no `update_url` on any of them. Nothing periodically checks a server on their behalf.

The host has no network path of its own on the browser's route. Appendix G's legacy path — the only path zwire's front end invokes — is a scheme read, a scheme write, a UI-prefs merge, a sysinfo stream, and the PTY relay. None of them is a network operation. The extended agent includes a TCP peer mesh, and it is off unless a daemon is running with peering configured. The one field in the sysinfo sample that involves the network is `pip`, the public IP, which Appendix G notes is “refreshed sparingly” — an admission that determining a public IP requires asking someone, and a design choice to ask rarely.

Ingress: two local channels with very different gates

Two things in this system listen, and comparing their gates is instructive.

Chapter 25 describes the daemon's socket carefully: a Unix socket created `0600` inside a `0700` directory, so the operating system's own permission check is the authentication, and a different user on the machine cannot see it. Remote callers over the peer mesh must present a shared `--token`. That is a considered design with a gate at each tier.

The CDP endpoint is the other one. Chapter 20 records that `--remote-debugging-port=9222` opens the DevTools protocol on `127.0.0.1:9222`, and Appendix L records that the endpoint is “opened only when you pass `--remote-debugging-port=9222`.” What the chapters do not say, and what a reader should hold alongside them, is that CDP is a full-control channel — a client that connects can read pages, inject script, and drive the browser — and that a loopback listener has no per-user gate of the kind the daemon socket has. The entire security of this channel is the fact that the flag is opt-in and short-lived.

The two channels sit at opposite ends of a spectrum in the same system:

channel	gate	lifetime
daemon socket	filesystem mode 0600 in a 0700 dir	as long as the daemon runs
peer mesh (TCP)	shared --token	as long as peering is on
CDP :9222	the flag was passed	as long as that browser runs

The practical guidance follows from the table rather than from any claim about relative risk: the flag is a verification tool, Chapter 20's probes are what it is for, and a browser left running all week with the port open is a decision to leave a full-control channel open all week. Nothing about the profile boundary constrains it, because the profile constrains what an attacker *reaches* and this channel is about who is *driving*.

What is inherited rather than chosen

The honest limit of this inventory is the base. `zwire`'s contribution is nine flags in Appendix E, and none of them changes the engine's own network behavior. A stock unbranded Chromium snapshot arrives with whatever component-updater, safe-browsing, and field-trial behavior that build carries, and the launcher neither enables nor disables any of it.

So the accurate summary is two-part, and the second part is the one usually left out of documents like this: **`zwire` adds a small, enumerable egress surface — a base download, a ten-minute API poll, and the search fallback — and inherits an unenumerated one from the engine underneath it.** An owner who cares about the second part has the same tools any Chromium user has, and Chapter 14 notes the settings replacement renders every preference `chrome.settingsPrivate` exposes, which is the surface where those behaviors are configured. What this book can say with confidence is limited to `zwire`'s own code, and pretending otherwise would be the kind of rounding-up the introduction promised not to do.

Chapter 47 — Teardown, Portability, and the Second Machine

Appendix B ends with a sentence that is precise about what it describes and easy to over-read: delete the state tree and “there is no hidden state anywhere else.” That is a statement about where *zwire writes at runtime*, and as such it is exactly right — the appendix itself names the one runtime file that lives outside the tree, and the tree accounts for everything else the browser produces while running.

It is not a statement about *zwire’s footprint on a machine*, and the difference matters twice: when removing *zwire*, and when trying to reproduce a working *zwire* somewhere else. Installation artifacts, platform registrations, and the fleet-wide theme file all live outside the tree by design. This chapter is the complete footprint, the order in which to take it apart, and the considerably more interesting question of what actually has to travel to a second machine.

The footprint, inside and out

```
inside $ZWIRE_STATE
base/           the extracted snapshot or packaged fork
base.path       absolute path to the resolved binary
base.version    "r<REV>" or "fork-<TAG>"
profile/       the dedicated --user-data-dir
  NativeMessagingHosts/com.zwire.hud.json (path + allowed_origins)
  Default/NativeMessagingHosts/          (second location)
ext/         the per-launch writable extension staging

outside $ZWIRE_STATE
~/.zwire/global.toml    the fleet-wide theme store
~/.zwire/hud-scheme     plain-text projection for the fork's mixer
~/.zwire/hud-light      light-mode projection
~/.zwire/hud-ui.json    shared visual-effect + light-mode prefs
the PATH symlink       placed by install.sh
the .desktop entry     Linux
the registry entries   Windows, both hives
the self-contained target /Applications, ~/.local/opt, %LOCALAPPDATA%
the checkout           including submodules and the host build output
legacy dirs            ~/.zwire and the bare-zwire macOS folder
zpwrcrome's native host installed separately, removed separately
```

Three of those deserve comment.

`~/.zwire/global.toml` and its projections are outside the per-install tree on purpose: Appendix B calls them the one channel through which the new-tab extension, the internal HUD, the power-tool, the wider app suite, and — on the fork — the native chrome agree on one look. A file shared across a fleet of applications cannot live inside one application’s state directory. The consequence for teardown is that removing *zwire* leaves the theme behind, which is correct behavior if the other applications in the suite are staying and surprising if the user believed the tree was everything.

The native-host manifest inside `profile/` holds an absolute path to the host binary plus the `allowed_origins` list. That is what makes the host profile-relative on macOS and Linux, and it is also what makes the

profile directory non-relocatable: move the tree and the manifest's path is stale. On Windows the same wiring is registry state, which is why Chapter 24 says the uninstall step differs there — a file that vanishes with a directory versus a key that does not.

The legacy locations are a teardown detail with a lifespan. Appendix B records that the old `~/.zwire` dotdir and a brief bare-`zwire` macOS folder are auto-migrated once. A migration that runs once leaves the question of whether the source was removed or merely copied, and a thorough teardown checks both paths rather than assuming.

Order matters, because the launcher rebuilds

Teardown is not “delete the tree,” because the tree is partly self-healing. Chapter 24's first-run sequence has the launcher ensuring the native host — building the Rust binary if the manifest is missing or its target is gone — and Chapter 26 records that extensions are staged into a writable `ext/` copy at launch. Both of those run every time. So deleting the tree while a launcher can still be invoked produces a partial rebuild on the next accidental `zwire`, and deleting `profile/` while a `zwire` is *running* leaves a browser writing into a directory that no longer exists.

The correct order is the reverse of install:

1. stop everything (browser and any host daemon)
2. remove the platform registration:
the `PATH` symlink, the `.desktop` entry, or the registry keys
3. remove the state tree (`$ZWIRE_STATE`) and any legacy dirs
4. decide about `~/.zwire/global.toml` and its projections —
keep them if any other suite application remains
5. remove the self-contained install target, if one was used
6. remove `zpwchrome`'s native host per its own documentation

Step two before step three is the load-bearing ordering. Once nothing on `PATH` can invoke the launcher, nothing rebuilds what step three removes.

What cannot travel, and why

The instinct for a second machine is to copy the state tree. Most of it cannot go.

`base/` is per-platform by construction — Chapter 24's matrix maps the host to `Mac_Arm`, `Mac`, `Linux_x64`, or `Win_x64`, and a snapshot is a binary for one of them. `base.path` is an absolute path. The native-host manifest holds another absolute path, to a binary compiled for the local target. On Windows the wiring is not in the tree at all. Copying any of these produces a configuration that is wrong in a way that presents as “the host is silent,” which Chapter 36 catalogs as a symptom with several causes and which now has one more.

What is portable is short: `global.toml` and its projections. A theme is not machine-specific, and the fleet file is a plain text store outside the profile. Copy it and the second machine looks right immediately, on every application in the suite.

The portability problem the design did not choose

Everything else the user authored is in the wrong place to move, and it is worth being exact about why, because this is a structural constraint rather than an oversight.

The HUD's user-authored configuration is a set of `chrome.storage.local` keys. Appendix I lists them, and the ones a user actually creates by hand are the remaps in `zb_keys`, the keyword commands in `zb_custom_cmds` with their `zb_cmds_removed` tombstones, the tmux prefix in `zb_tmux_prefix` and its options

in `zb_tmux_opts`, the named layouts in `zb_tmux_sessions`, the CI configuration in `zb_ci`, and — in page `localStorage` rather than extension storage — the new tab's `zb.tiles` and `zb.engine`.

`chrome.storage.local` lives inside the Chromium profile, in Chromium's own on-disk format. There is no supported way to copy one key out of it, and copying the whole profile means copying cookies, history, cache, and sessions along with a format that has no cross-version or cross-platform portability guarantee. The `localStorage` keys are worse still: Chapter 31 records that they are per-origin by nature, which is the property that made them the right choice for a page's own presentation and the property that makes them invisible to any exporter that is not that page.

So the state a user would most want to move is the state stored in the least portable place in the system, and the remedy is not a better backup — it is an export path per key. The book records exactly one: Chapter 14 notes the Sessions page “exports and imports the whole set as JSON” for `zb_tmux_sessions`. That is the model. Every other user-authored key group listed above has an editor — the Keyboard page, the Commands page, the CI configuration, the new tab itself — and an editor that can write a key can serialize it.

The smallest set that reconstitutes a workstation

The useful inversion of Chapter 24's degradation ladder is a reconstitution ladder: rather than asking what still works when a piece is missing, ask what is the minimum that has to arrive for everything else to rebuild itself.

must be carried	the checkout with its submodule pointers ~/.zwire/global.toml (+ hud-ui.json) a JSON export of the tmux sessions whatever the other editors can be made to export
rebuilds itself	base/ + base.path + base.version (fetch-base.sh) the host binary (built on first launch) the native-host manifest (written on launch) ext/ (staged on launch) profile/ (created by Chromium)

The left column is small, and every item on it is a plain file. The right column is everything the launcher and the browser reconstruct from nothing, which is the same property that makes the degradation ladder work — a system that assembles as much of itself as the machine currently has is also a system that rebuilds most of itself on a new machine.

The rule to carry away is the one the storage schema does not currently state: **any state a user authored by hand needs an export path owned by the surface that edits it**. Not a backup of the store — an export of the value, in a format the user can read, from the page that created it. One page in the HUD already does this. The gap between that page and the others is the whole distance between “my configuration is portable” and “my configuration is in a profile.”

Chapter 48 – Version Numbers as a Contract

There are a lot of numbers in this book. `v0.6.56` on the cover, `1.0.13` and `0.6.29` and `0.22.36` on three manifests, `minimum_chrome_version: 127`, `r<REV>` or `fork-<TAG>` in `base.version`, `v1.0.17` and `v1.0.18` on the shared toolkit, a pinned Chromium tag under twenty-five patches. They look like one kind of thing and they are three kinds, and confusing them is how a system acquires a failure that nothing reports.

Every version number in a system is one of three things. It is a **gate**, enforced by something that refuses to proceed on a mismatch. It is a **resolution input**, consumed by tooling to decide what to fetch or load. Or it is a **label**, displayed to humans and read by nothing. The dangerous ones are the labels that look like gates and the gates that are enforced somewhere the operator cannot see.

The inventory, classified

number	kind	enforced by
<code>minimum_chrome_version 127</code>	gate	Chromium's extension loader
<code>base.version / base.path</code>	resolution input	the launcher
<code>zgui-core submodule pointer</code>	de-facto gate	nothing at load time
Chromium tag (fork)	resolution input	the patch series
<code>package.json v0.6.56</code>	label	the cover; the System page
extension manifest versions	label	the System page
native protocol (legacy)	no version at all	–
storage keys	no version at all	per-key shape sniffing

Three rows on that table are worth the rest of the chapter.

The gate nobody local can check

`minimum_chrome_version: 127` is a real gate. Chapter 2 states what it means without hedging: a base older than Chromium 127 refuses to load the power-tool, and the browser enforces it.

Now put that next to how a base is chosen. `fetch-base.sh` reads a sentinel from the snapshot bucket to learn the current revision for the platform, or accepts a pinned revision directly, and Appendix B records the result as `base.version`, in the form `r<REV>`. A Chromium revision is a monotonically increasing build number. It is not a milestone. Nothing in the launcher, the fetch script, or the state directory holds a milestone number for the installed base at all.

So the gate is enforced by the browser and unverifiable by the tooling around it. Pin an old enough revision and the failure appears as Chapter 36 describes it — an extension that did not load — with the diagnosis living in the browser's own extensions surface rather than in anything zwire wrote. This is not a defect to be fixed by inventing a version check; it is a property to know about, because it is the one constraint in the system that the resolution input cannot express. The practical form of knowing it: a base chosen by “latest snapshot” satisfies the gate trivially and a base chosen by pinning an exact revision is the case where the constraint is live.

The de-facto gate with nothing behind it

The submodule story is the sharpest item in this chapter, because the two submodules fail in completely different ways and the project's stated safety net covers only one of them.

Two chapters make the same statement. Chapter 24's degradation ladder says "a missing submodule (`zpwchrome`, `zgui-core`) is not fatal — its extension directory has no `manifest.json`, so the launcher skips it and loads what remains," and Chapter 36 repeats it: a clone without `--recurse-submodules` "leaves their directories without a `manifest.json`, so the launcher skips them silently and loads what remains." That is precisely true of `zpwchrome`, which is an extension directory in the launcher's load list. It is not true of `zgui-core`, which Chapter 7's map places at `lib/zgui-core/` — inside `hud-internal`, as a library. The launcher's `manifest.json` filter never examines it, because it is not a candidate for loading; it is a set of files a content script expects to have been injected before it runs.

The failure modes diverge accordingly. A missing `zpwchrome` is one absent extension and a browser that otherwise works — the ladder's promise, kept. A missing `zgui-core` is a set of content scripts that load in the order Appendix P fixes and then reference a global that was never defined. Chapter 7 is explicit that the order in block 4 is load-bearing because later scripts read globals earlier ones set, and `ZGui` is one of those globals.

Version skew is the same story one degree subtler, and here the book supplies the exact case. Chapter 10 records that `zgui-core` v1.0.18 "added the external-feed API (`prefix/key/relaySync/yank`) precisely so a cross-origin-iframe consumer like this one can feed the shared WM from outside," and that `ztmux-config.js` is the provider that uses it. A checkout whose `zgui-core` pointer is behind v1.0.18 therefore has an overlay whose top frame loads, whose panes load, and whose forwarder relays prefix and keystrokes into a shared window manager that does not expose the entry points to receive them.

Chapter 36 already names the symptom class — "if a submodule is present but at the wrong commit, the tell is subtler still, a feature that half-works or behaves like an older version" — and this is the mechanism under that sentence. The submodule pointer is functioning as a version gate, and nothing enforces it at load time. The enforcement is git's, at checkout time, and only if someone runs the update.

The channel with a version handshake is the one nobody uses

Appendix G splits the native protocol into two paths, and the split is the interesting part.

The legacy path — "what the browser uses," in the appendix's own words — is the scheme read, the scheme write, the UI-prefs merge, the sysinfo stream, and the PTY relay. Not one of those messages carries a protocol version, on either side. The browser sends `{cmd:"get"}` and the host answers `{scheme, ui}`, and if the two were built from different generations of the source there is no field in which they could discover it.

The extended agent, which the appendix says `zwire`'s front end does not call, opens with `hello`, and `hello` returns `{ok,host,version,os,arch,pid,caps:[...]}` — a version *and* a capability list, which is the strongest form of the handshake, because a caller can ask what is supported rather than infer it from a number.

That asymmetry is worth stating plainly: the one channel in the system with a real version negotiation is the one the browser never opens, and the channel the browser depends on for the scheme, the statusbar, and the terminal has none. What makes it work anyway is that the host binary is built from the same checkout by the launcher on first need, so the two sides are versioned together by construction. That is a legitimate answer — co-deployment is a real compatibility strategy — and its boundary is exactly where co-deployment stops holding: a packaged install (Chapter 23) ships a host inside the bundle, and a self-contained install and a developer checkout on the same machine are two hosts built at two times. The `caps` list already exists in the binary. Nothing about the legacy path is stopping a future scheme read from learning what it is talking to.

Migration without a schema version

The last row is the storage schema. Chapter 31 establishes the discipline — migrate on read, never on write, because an old content script in another tab may still be running against the same store — and the worked example is the overlay's per-window sync boolean becoming a list of member ids.

What the schema deliberately does not have is a version number. Migration is therefore *shape sniffing*: the restore path looks at the value it found and decides what generation it belongs to. That is the correct choice for the stated constraint, because a store-wide version number would have to be written by someone, and the only writer available is exactly the new code whose write would break the old readers.

The limit is worth naming, because it is where this strategy stops. Shape sniffing scales per-key and does not compose: it works when a value's generations are structurally distinguishable (a boolean and an array are not confusable), and it has no answer when two generations of a key are the same shape with different meaning, or when a change spans several keys that must move together. Neither has happened. When one does, the migration cannot be per-key and per-read, and that is the moment the schema needs a version — introduced, per the chapter's own rule, in a way that old readers tolerate rather than one they must understand.

The rule

Write the classification down. For every version number in a system, record which of the three it is, and for every gate, record what enforces it and where the failure appears.

Applied here, that exercise produces four sentences that are more useful than any of the numbers individually. The power-tool's minimum is enforced by the browser and invisible to zwire's tooling, so pinning a base is the operation that can violate it. The toolkit submodule pointer is a version dependency with no runtime check, so a stale pointer presents as a half-working feature rather than an error. The native protocol's compatibility rests on co-deployment, which holds for a developer checkout and is worth re-examining for a packaged install. And the storage schema has migration without versioning, which is correct today and has a known boundary. Every one of those is a sentence about where a mismatch would *appear*, which is the only thing anyone actually needs from a version number at three in the morning.

Chapter 49 – Lifecycle Hooks: stryke on Browser Events

Every chapter so far describes a browser that responds to a person. This one describes the browser responding to itself. A **hook** binds a lifecycle event — a tab opened, a download finished, the scheme changed, the audio engine reshaped — to a **stryke** script that runs outside the browser and can act back on it. It is the seam where the HUD stops being an interface and becomes an automation substrate.

The interesting property is that neither side can do the job alone. The MV3 service worker sees every browser event and cannot run a scripting language: it is JavaScript in a sandbox with no process spawn, no filesystem, and a lifetime measured in seconds. **zwire-host** can run anything the machine can run and sees no browser events at all. The hook is the wire between them, and the whole design falls out of that split.

The firing side

background.js defines **fireHook** at the top of the worker, above every listener, so any handler in the file can call it:

```
chrome.runtime.sendMessage(HOST, { cmd: 'hook_fire', event: event, payload: payload
  || {} },
```

It is deliberately best-effort. There is no reply to wait for, no error surfaced to the user, and no check for whether a hook exists — the host no-ops when nothing is bound to the event. A browser event must never be slowed down, or worse blocked, by an automation layer nobody has configured.

Forty-nine **fireHook** call sites are registered as listeners in one IIFE near the end of the worker, each wrapped so a missing API or permission on one platform cannot break the rest:

```
var on = function (obj, ev, fn) { try { if (obj && obj[ev] && obj[ev].addListener)
  obj[ev].addListener(fn); } catch (e) {} };
```

The registration is at top level for an MV3 reason rather than a stylistic one. A listener attached inside a callback does not wake a torn-down worker; a listener attached during the worker's initial evaluation does. Every hookable event is therefore also a wake reason, which is what makes hooks fire at all in a browser that keeps its extension worker dead most of the time.

Alongside the specific events there is **act()**, the catch-all:

```
var act = function (command, extra) { var p = { command: command }; if (extra) for (var k
  in extra) p[k] = extra[k]; fireHook('action', p); };
```

action fires for every command, menu item, palette entry, and toolbar invocation, carrying the command name in **\$.command**. One hook bound to **action** sees everything the HUD does; the specific events still fire independently, so a script can be as narrow or as broad as its author wants. The pattern is carried over from Audio-Haxor, where the same catch-all lets one script observe an entire application.

The event catalogue

The authoritative list is `hooks::events()` in the host — 53 named events in twelve groups: the `action` catch-all, runtime and browser lifecycle, tabs, windows, top-frame navigation, downloads, bookmarks, history, sessions, management of other extensions, input and system, and HUD lifecycle. Each entry carries three fields:

```
{ "name": "tab-updated", "desc": "A tab finished loading", "sample": { "tabId": 12, "url":  
  "https://example.com", "status": "complete" } },
```

The `desc` is what the page's searchable picker filters on. The `sample` is the more useful field: it is the shape of the payload a script will actually receive, so the editor can show an author what `$_` looks like before the event has ever fired. A catalogue with samples is a documentation surface that cannot drift from the code, because the code is the catalogue.

The worker's listeners and the host's list are two halves of one contract, and the source says so — “*Event names match `hooks::events()` in the native host (keep in sync).*” Four catalogued events do not come from a worker listener at all: `palette-command`, `session-saved`, `pane-split`, and `audio-eq-changed` are fired by HUD pages through a `zbFireHook` relay message, so that every hook firing still passes through the one native-host seam rather than opening a second path. The contract has one loose end in the other direction: the worker's global error handler fires `zerror` with the failing site, message, and stack, and that name is not in the catalogue — a hook can be bound to it by hand, but the picker will not offer it.

The running side

Hooks live in `~/.zwire/hooks`: a `hooks.json` manifest of `{ id, name, event, enabled, timeout_ms }` records, and one `<id>.st` stryke file per hook next to it. The host reads the manifest per operation rather than caching it in memory. That is a deliberate departure from the Audio-Haxor original, noted in the source: `zwire-host` has no long-lived per-app state object to hang a cache on, and a manifest this small is cheaper to re-read than to invalidate.

When an event arrives, `fire` matches it against the enabled hooks, spawns each one through `stryke_runner::run_script` with the hook's timeout (default 10 seconds), and pipes the envelope in on stdin:

```
{ "event": "download-completed", "payload": { ... } } → stdin  
{ "actions": [ { "notify": { ... } }, ... ] } ← stdout
```

The script's stdout is where the loop closes. `dispatch_stdout` scans the output for an `{"actions": [...]}` object — tolerating log noise around it, which the unit tests pin explicitly — and `dispatch_action` validates and runs each recognized action against the host's own capabilities:

action	dispatches to
<code>notify</code>	<code>osops::handle("notify", ...)</code>
<code>open</code>	<code>osops::handle("open", ...)</code>
<code>exec</code>	<code>exec::run(...)</code>
<code>pub</code>	<code>bus::publish(...)</code>

Unknown actions are ignored rather than rejected, so a script written against a newer host degrades to doing less on an older one instead of failing outright. This is the same four-verb set the Audio-Haxor port arrived with, minus that application's database-bound verbs (tag, favorite, trash) and re-hosted onto the modules `zwire-host` already had — the port's stated method throughout: carry the model verbatim, replace the seams.

The result of every run — success, failure, timeout, and the action count — is published on the host's own pub/sub bus under the `hook-result` topic. That is what makes the Hooks page a live view rather than a form:

it subscribes to the topic over a `connectNative` port, so a hook that fires because a tab opened in some other window shows its result in the editor immediately.

The editing side

`pages/hooks.js` is the CRUD surface, and it reaches the host through ten commands — `hooks_list`, `hooks_events`, `hooks_save`, `hooks_delete`, `hooks_set_enabled`, `hooks_get_script`, `hooks_script_path`, `hooks_set_script`, `hooks_test_run`, and `hook_fire`. The editor is Monaco with the stryke language server behind it (`stryke_lsp_start` / `stryke_lsp_send` / `stryke_lsp_stop` on the host side), carrying vim and emacs modes, which is the same editing surface the rest of the MenkeTechnologies stack uses rather than a bespoke textarea.

Two affordances matter more than the rest. The **event picker** is searchable over the 53-entry catalogue with its descriptions, so binding a hook does not require knowing the event's name in advance. The **test-run** button sends `hooks_test_run`, which executes the script against the event's `sample` payload without waiting for the real event — the difference between authoring a download hook in ten seconds and authoring it by downloading things.

The failure semantics are worth stating plainly, because an automation layer that hides them is worse than none. A hook that hangs is killed at its timeout. A hook whose script is missing, or whose stryke binary is absent, fails on the host and reports on the bus rather than anywhere the browser can see. And a hook is not a transaction: the browser event has already happened by the time the script runs, so a hook observes and reacts — it never vetoes.

Chapter 50 – Output Triggers: Regex on What the Page Renders

A terminal emulator can watch its own output. iTerm2 calls them triggers: a regex over the text scrolling past, and an action when it matches. The reason no browser ships the equivalent is not that nobody wants it — it is that a tab is not a text stream. A tab multiplexer never sees rendered text; it sees tab objects and URLs.

zwire sees rendered text, because the HUD is already a content script on every page. `ztriggers.js` is the terminal-trigger idea moved into the browser: a regex matched against page text *as it renders and streams*, and, on a match, a chain of typed steps.

The scan loop

The engine runs on real web pages only — `http`, `https`, `file`; the manifest's injection group excludes the HUD's own extension pages. It attaches one `MutationObserver` to `document.documentElement` with `subtree`, `childList`, and `characterData` set, plus a one-shot scan of what is already rendered when it starts:

```
try { enqueue((document.body || document.documentElement).innerText || ''); } catch (e) {}
```

Mutations do not scan directly; they enqueue text and schedule a flush. Three constants bound the work, and each one exists because the unbounded version is a page-stalling bug:

<code>FLUSH_MS</code>	<code>350</code>	coalesce a mutation burst into one scan
<code>MAX_SCAN_LINES</code>	<code>2000</code>	cap the lines examined per flush
<code>MAX_LINE_LEN</code>	<code>4000</code>	ignore minified blobs and base64 – not real output

A page that rewrites its DOM sixty times a second produces at most three scans a second, each bounded in both dimensions. That is the difference between a trigger engine and a jank generator.

Not scanning yourself

The single most important selector in the file is the exclusion list:

```
var ZWIRE_UI_SEL = '.palette-overlay,.zs-toast-container,.zs-toast,[data-zwire-ui]';
```

zwire injects its own UI into the page it is watching. Without the exclusion, two failures follow immediately. The ⌘K palette's command names are page text, so opening the palette to run a command can match a trigger looking for that command's own words. Worse, a trigger's result toast is page text too — a trigger that fires and reports would match itself on the next flush and re-fire forever. `isZwireUi` walks up from the mutated node with `closest()` and drops anything inside zwire's own surface, and the observer skips both the mutation target and each added node.

Compilation and the gate

Each stored trigger compiles to a small record: the regex, an optional URL-filter regex, a cooldown, the once flag, and the step chain. Two compilation details are load-bearing. Flags are normalized before use:

```
function normFlags(f) { var out = (f || 'i').replace(/[gy]/g, ''); return out || ''; }
```

`.test()` on a global or sticky regex advances `lastIndex`, so the same pattern would skip every other line on a multi-line scan. Stripping `g` and `y` removes a bug class that would otherwise present as “my trigger only matches sometimes.” The default is case-insensitive, because a user typing a pattern into a form is not thinking about flags at all.

An invalid pattern, or an invalid URL filter, drops the trigger rather than throwing into the reactive loop — fail-closed, in a loop that runs on every page.

The URL filter is applied at compile time, not match time: `recompile` keeps only the triggers whose filter matches the current page, and if none survive, the observer is never attached. A page that can match nothing pays nothing.

Firing is gated twice. Each trigger carries its own `cooldownMs` (default 1500), so a burst of matching output cannot spawn a process storm; and a `once` trigger records its id in a module-level `firedOnce` map, so it fires at most once per page load. The map is module-level for a subtle reason the source calls out — it must survive `recompile()`, because an unrelated storage edit (enabling some other trigger) must not re-arm a trigger that has already fired. It resets naturally on a full navigation, when the content script is reloaded. Within a flush, a matching trigger fires once and breaks out of the line loop, so one burst of output is one action.

The action

A trigger’s action is not a command string; it is a **chain of typed steps** — `shell`, `stryke`, `js`, `applescript`, `batch`, `action`, `scheme`, `host`, `url` — the identical step set a `⌘K` keyword command runs, edited by the same `ZwireStepWizard` and executed through the same exported runner:

```
window.ZWIRE_CMD_EXEC.runCustom({ steps: c.steps }, line);
```

The matched line is passed as the argument, so `{q}` in any step expands to it. This is the chapter’s real design point: triggers did not need an execution model, because the palette already had one. Adding a new *cause* to an existing *effect* system is a different amount of work from adding both, and it is why a trigger can do anything a command can — including running a stryke script, which puts Chapter 49’s automation surface behind a regex on page text.

`pages/triggers.js` is the CRUD surface over `chrome.storage.local`’s `zb_triggers` array, with a per-trigger enable toggle; the content script watches the same key through `chrome.storage.onChanged` and recompiles in place, so editing a trigger takes effect on already-open tabs without a reload.

Chapter 51 — Pane Pipelines: A Dataflow Edge Between Two Panes

Chapter 10 describes ztmux as a tiling window manager in the browser: recursive splits, a prefix, a status bar, and a live webview in every pane. This chapter describes what happens when the panes stop being independent.

A **pane pipeline** is a persisted, reactive dataflow *edge* between two tiled webviews:

```
source pane --[ extract ]--> [ filter ] --> sink pane
```

It is the browser-native analog of `curl ... | jq | xargs`, except the stages are live rendered pages rather than processes. The tiling overlay stops being a layout manager and becomes a graph.

A pure engine with three consumers

`zpipes-core.js` holds every decision and touches nothing:

THIS FILE IS PURE: no DOM, no `chrome.*`, no window side effects beyond hanging the API off a `window`-like global.

What a source emits, how a filter transforms it, whether an emission fires, the sink message to post, and whether an edge would close a cycle — all computed here. Three runtimes consume it: `ztmux-pane.js` (the pane-side forwarder that extracts and applies), `ztmux-config.js` (the top-frame relay that runs the edge), and `pages/pipes.js` (the CRUD page that validates one before saving). A fourth consumer is `tests/pipes.mjs`, which is the reason for the purity: a rule about cycles is worth far more when it can be asserted headlessly than when it can only be clicked.

Three call sites sharing one engine also means the page cannot accept an edge the runtime would reject, and the runtime cannot behave in a way the page's preview did not show.

Source, filter, sink

A **source** names a URL-filter regex and one of four extraction kinds — a CSS selector's text, a regex over rendered text (capture group 1 when present), the current selection, or the pane's live URL. Extraction re-runs reactively when the source pane's content changes, through the same throttled `MutationObserver` discipline Chapter 50 describes; `linesOf` is deliberately identical in semantics to the triggers engine's, so a pipeline and a trigger see page text the same way.

A **filter** is one of three kinds. `none` passes values through. `js` evaluates an expression with `lines` and `text` in scope. `ops` is a small stryke-flavoured chain written with `|>`:

```
trim |> nonempty |> uniq |> grep ^https? |> take 5 |> join ,
```

The verb table is closed and split by arity — `trim`, `upper`, `lower`, `uniq`, `sort`, `reverse`, `nonempty`, `first`, `last`, `count`, `collapse` take no argument; `nth`, `take`, `drop`, `join`, `grep`, `reject`, `replace`, `prepend`, `append` take one. `parseOps` rejects an unknown verb at *validation* time, so a typo surfaces in the editor rather than as an edge that silently never fires.

A **sink** names its own URL filter and one of five deliveries: **navigate** the pane, **fill** a selector-addressed field, **replace** or **append** a node's content, or **batch**-open every line. Sink writes are posted as messages through the pane forwarder — never a direct cross-origin DOM write from the top frame, which the browser would refuse and which would be the wrong trust boundary even if it did not.

The gate and the graph

Every emission passes **gate(edge, state, now, value)**: the per-edge cooldown (default 1500 ms), the **once** flag, and value **dedupe**, which is on by default so a re-render that produces the same text does not re-fire the edge. Dedupe is the flag that makes a reactive edge usable on a page that repaints constantly.

The structural risk is a loop. An edge from A to B and an edge from B to A is a livelock: each sink write mutates a page that is some other edge's source. **zwire** refuses it before it can exist. Edges are nodes-by-URL-pattern — **nodeId** maps an empty pattern to *****, the match-anything node — and **wouldCycle(existing, sourcePattern, sinkPattern)** runs a reachability check over the enabled edges' adjacency before the page saves a new one. **graphCycle(edges)** answers the same question for a whole set, which is what validates an imported configuration.

Because a cycle check is a *structural* guarantee and pages are not structures, there is a second, dumber defense underneath it:

```
var HOP_BUDGET = 8;
```

A single emission may traverse at most eight hops before the runtime drops it. The source comment is explicit that this is the belt to the cycle check's suspenders: even with $A \rightarrow B \rightarrow A$ rejected at creation, a long fan-out or a self-referential DOM write can re-arm an observer, and a budget bounds what that can cost.

Living with it

An edge is started from the overlay with the **tmux** prefix then **|**, which seeds the active pane as the source and opens the editor; **pages/pipes.js** is full CRUD over **chrome.storage.local**'s **zb_pipes** array with per-edge enable toggles. Every edge is persisted, so a saved session's panes reconnect with their dataflow intact.

The positioning claim in the project's README is narrow and worth restating in the book's own terms: tiling browsers tile, and command-chain features chain commands, but a persisted data relationship *between two tiled views* is not something the neighbouring tools model at all. Vivaldi's Command Chains are sequences of actions; Zen and Arc split without cross-split dataflow; qutebrowser has no tiling to connect. The pairing is the novel part, not either half.

Chapter 52 — The Audio Dashboard

Chapter 21 describes patches 0022 through 0024 — the browser-wide DSP engine compiled into Chromium's audio service, its live configuration path, and the meters back-channel. This chapter is the surface on top of them: [pages/audio.html](#), the page that makes an engine buried in a sandboxed utility process feel like a piece of studio hardware.

The page has two jobs, and the file says so up front: **control** the real browser-wide engine, and **preview and analyze** locally.

Control: a spec string on disk

The engine's entire configuration is one string. The EQ bands serialize first, followed by two-field directives — [gain](#), [pan](#), [mono](#), [drive](#), [thresh](#), [ratio](#), [width](#), [delay](#), [feedback](#), [delaymix](#), [reverb](#), [room](#), [damp](#), [ceiling](#) — and the whole thing is written to `$ZWIRE_STATE/audio-eq`. The browser process polls that file and pushes it into the audio service over the `mojom` interface patch 0024 adds; the engine rebuilds its cascade when the generation changes. Nothing in the path requires the page to be open, which is why an EQ configured once keeps shaping audio in a browser whose HUD is closed.

The interesting engineering is in how fast that write is. The first implementation sent a native message per knob move, which spawned a host process, which ran `/bin/sh`, which wrote the file — 30 to 80 milliseconds of process setup per event, on a control the user is *turning*. The current path writes over the port that is already open:

```
meterPort.postMessage({ id: 'eqw', cmd: 'fs_write', path: eqPath(), text: spec });
```

The persistent meter port is already connected for the meters feed, so a knob move costs one message on an existing pipe. A one-shot `sendNativeMessage` remains as the fallback for the window where the port is down or reconnecting.

On top of that sits a **leading-and-trailing throttle** rather than a debounce:

fire immediately on the first move, then at most every 30 ms DURING a continuous turn (plus a trailing write)

A pure debounce would be silent for the whole gesture and correct only at the end, which is exactly wrong for a control you are listening to. Thirty milliseconds of write cadence against the engine's 25 ms poll puts the audible change inside the same perceptual moment as the knob movement — the measured round trip dropped from roughly 160 ms to 40–50 ms.

One more detail closes a paint gap: the authoritative spec lives on disk and is read back asynchronously, so the page mirrors every persist into `localStorage` and paints from the mirror on load. Without it the controls would show defaults for the length of a host round-trip and then jump.

Saving explicitly — as opposed to turning a knob — also fires the `audio-eq-changed` lifecycle event, so Chapter 49's hooks can react to an EQ change like any other browser event.

Analyze: the real output, not a capture

The meters are the part that is easy to get wrong. The obvious implementation taps the tab with `tabCapture` and analyzes that stream; patch 0023 even makes `tabCapture` available to the HUD without a picker or prompt. But a capture mutes the source while it runs and drops audio when it is released, which makes “watch the meters” a destructive operation.

Instead the audio service computes the meter frame itself — a Goertzel spectrum, peak and RMS per channel, phase correlation, and a decimated stereo scope — from the **real post-DSP output**, the browser process pumps it to `$ZWIRE_STATE/meters` at roughly 30 fps, and the page reads that file over the native host. Closing the page touches nothing; the audio never notices it was being watched.

That pump is also the chapter’s best example of a bug that only exists at the seam. The first version ran on the UI thread, where the compositor’s gesture scheduler starved it: during a trackpad scroll the meter file froze, so the charts appeared to hang until the gesture ended. Moving the pump to a dedicated `ThreadPool` sequence took the file from 0.7 writes/second under gesture to 10+, and the page’s feed from 2/second to 27.5. The lesson generalizes past audio: a data path that is correct in isolation can still be starved by the thread it borrowed.

The local graph

Alongside the real engine, the page runs its own Web Audio chain so a curve can be auditioned against a file, the microphone, pink noise, or a tone:

```
source → inputGain → [5-band biquad EQ] → compressor → pan → master → analyser →  
~ out
```

That local graph drives the instrument wall — spectrum, spectrogram, oscilloscope, goniometer, correlation, LUFS, peak, VU, gain reduction — with every widget a `ZGui.*` component from the shared toolkit, per the `zgui-core-only` rule Chapter 17 sets out. The same rule is why the audio page looks like the rest of the HUD despite being, in function, a mixing console.

Correctness

DSP is the one part of the fork where “it looks right” is not evidence. `fork/tests/run_dsp_tests.sh` extracts the engine’s source **straight out of patch 0022** and asserts per-effect invariants against it, so the tested code is literally the patched code rather than a reimplement that agrees with it today. That is the same port-verification instinct the rest of this book documents — pin the thing itself, not a copy of it.

Chapter 53 — The Later HUD Pages

Chapter 14 covers the pages that *replace* a `chrome://` surface. The pages in this chapter replace nothing: they are features the browser does not have, added as HUD pages because a HUD page is the cheapest surface in the system — an extension page, the shared shell, the shared palette, the shared fuzzy filter, and the scheme applied for free.

One pattern runs through all of them and is worth naming before the list. Each page exposes its pure logic on a `window.ZB*` global — `ZBNotes`, `ZBFeeds`, `ZBTranslate`, `ZBReadingList`, and `ZWIRE_PIPES` for Chapter 51's engine — so the parsing, sorting, and tree logic can be exercised headlessly by the `tests/*.mjs` suite without a browser. A page whose logic is only reachable through a click is a page whose logic is untested.

Dashboard

`dashboard.html` is the launcher: one searchable, categorized tile grid that opens every zwire HUD page *and* every `chrome://` internal page. The chrome is `ZGui.tiles` — the launcher component the toolkit already carried from traderview and Audio-Haxor — and per-category tile order is drag-reorderable and persisted through `ZGui.drag`. The shell header's search box drives the filter and the tiles' own search is switched off, so the page has exactly one filter box rather than two that disagree.

Its `chrome://` catalogue is transcribed verbatim from this build's own `chrome://chrome-urls`, in six sections: browser destinations, curated `settings` sub-pages (which `chrome-urls` does not enumerate), WebUI and dialogs, side panels and top-chrome, internals and diagnostics, and debug internals. Two classes are deliberately excluded, and the source says why: the “Command URLs for Debug” — `chrome://crash`, `kill`, `hang`, `gpubcrash`, `memory-exhaust`, `quit` — crash or hang the renderer on purpose, and `chrome-untrusted://` origins are component and iframe origins rather than navigable top-level pages. The maintenance rule is stated in the file rather than left implicit: when the browser gains or loses a WebUI, re-dump `chrome://chrome-urls` and reconcile.

The zwire section carries 23 tiles: one per HUD page, plus the two native destinations (`chrome://downloads`, `chrome://newtab`) the HUD intentionally does not replace with a page of its own.

Notes

`notes.html` ports Vivaldi's Notes: a folder tree of Markdown notes over a flat array of `{id, parentId, type, title, content, url, ts}` nodes in `chrome.storage`'s `zb_notes`. Left pane is folders and notes with search; right pane is a Markdown editor with live preview through `ZGui.markdown`, plus attach-current-page, move-to-folder, and delete. It autosaves as you type — there is no save button to forget. The store helpers are pure functions over the node array, which is what `tests/notes.mjs` drives.

Feeds

`feeds.html` ports Vivaldi's built-in feed reader. Feed URLs persist in `zb_feeds`; each is fetched directly from the extension page under `<all_urls>` host access — no proxy, no service — and parsed by a regex-based RSS and Atom parser that handles CDATA and entity decoding. The parser is regex-based specifically so it runs outside a browser: `window.ZBFeeds` is what `tests/feeds.mjs` exercises, with no DOM parser to stub.

Translate

`translate.html` ports Vivaldi's Translate panel: type text, choose source (or auto-detect) and target, get the translation live. It calls the keyless Google `translate_a/single` endpoint directly from the extension page — no API key to store, no external command to shell out to — and `parseTranslation` folds that endpoint's nested array response into text plus detected language. Swap languages, copy the result; nothing is stored.

Reading list

`readinglist.html` surfaces `chrome.readingList`, which has no first-class UI of its own worth the name: add the current tab, mark read or unread, remove, open, with two filters (Unread, All). The ordering helper is the page's whole logic — newest first, unread before read when both are shown — and it is exported for tests like the rest.

App Store

`store.html` is the HUD's shop window: the MenkeTechnologies product catalogue rendered as `ZGui.productCard` tiles from a static `store-catalog.js`, grouped by category and filterable, each linking to its live product page. Cover images are vendored under `pages/store-assets/` so the tab renders with no network at all, with a glyph fallback when an image is missing.

It is also the one page `zwire` opens by itself. On `onInstalled`, `background.js` opens `store.html?welcome=1` once, with a welcome modal — the entire first-run experience, deliberately placed on a page the user can close rather than on the new-tab page, which stays exactly as configured. `zwire` itself is free and open source; the catalogue is the rest of the stack.

Host console

`host.html` is documented with the native host in Chapter 15 rather than here, but it belongs to the same family: a HUD page whose only job is to make an invisible subsystem inspectable — every command of the one JSON protocol, issued by hand, with the reply rendered.

Appendix A – File and Directory Map

A single reference for where each thing this book discussed actually lives in the [zwire](#) repository.

```

bin/zwire                                the launcher - resolve base, build ext list, ensure
├─ host, exec                             edition source (version), npm-script aliases
package.json                             zwire.icns, zwire.ico, icon.svg, icon-1024.png
branding/

scripts/
  fetch-base.sh                          download + pin the Chromium snapshot base
  install.sh                              PATH symlink + fetch + (macOS) rebrand
  rebrand-macos.sh                        in-place Dock name + icon + re-sign
  setup-native-host.sh                    build zwire-host + write profile-relative manifest
  localinstall.sh                         self-contained /Applications/zwire.app
  localinstall-linux.sh                   self-contained ~/.local/opt/zwire + .desktop
  localinstall-windows.ps1                self-contained %LOCALAPPDATA%\zwire + registry host
  dev / build / rebuild / clean           dev-loop helpers (no compile - assemble base)
  bust / nuke / test / cyberpunk          cache-bust · full reset · syntax gate · banner lib
  check-patch-series.sh                   CI: series file <-> patch files consistency

newtab/
  newtab.{html,css,js}                   the HUD new-tab extension (chrome_url_overrides)
  palette.js schemes.js                  page, style, logic (clock/tiles/omnibox/focus)
  scheme-sync.js tmux-pane.js             new-tab command palette + the 8-scheme table
  fonts/                                  scheme follow via native file + tiling surface
                                          locally-vendored display faces (offline-correct)

extensions/
  hud-internal/                           the internal-HUD content-script suite
    manifest.json                         MV3: content-script order, commands, permissions
    background.js                         scheme relay + tab bus + term-open state + cmd seed
    theme.js schemes.js                   chrome:// WebUI skin + the 8-scheme table
    redirect.js rules/frame_bust.json     redirect chrome:// pages · strip X-Frame-Options
    zvim.js ztmux-config.js zpowerline.js vim keys · tmux overlay (ZGui.tmux) ·
├─ statusbar (ZGui.powerline)
  zpalette.js zfind.js                   global %%K palette · fuzzy find (Custom Highlight API)
  zkeys.js cmd-defaults.js               keybinding registry · seeded %%K keyword commands
  pages/
    extensions.js                         extension manager (management + developerPrivate)
    settings.js                           all Chrome prefs (settingsPrivate) + scheme/effects
    history/bookmarks/downloads            the three list pages (shared zg-list.js)
    sessions.js                            durable tmux-session CRUD (zb_tmux_sessions)
    version.js                             HUD chrome://version + live native-host probe
    keys.js commands.js                   keybinding editor · %%K keyword-command editor
    extshortcuts.js ci.js                 extension shortcuts · GitHub Actions dashboard
    zg-boot.js zg-list.js fzf.js          shared shell · list factory · fuzzy matcher
  lib/term/
    native/zwire-host                     xterm.js terminal (native-PTY bridged)
├─ (Rust)
  native/zwire-host/src/                  scheme + ui + sysinfo + PTY + fs/exec native host
├─
  lib/zgui-core/                           the Rust host: transport/proto/session/store/
  zpwrchrome/                             sysmon/pty/exec/fsops/jobs/watch/procs/osops/
                                          bus/peer/api (submodule, packaged path)
                                          shared fuzzy/palette/fx/fonts toolkit (submodule)
                                          the MV3 power-tool (submodule)

theme/                                    a colors-only Chrome theme - present, NOT loaded

fork/
  CHROMIUM_VERSION                       the source-build path (optional alternate base)
  args.gn                                 pinned tag: 150.0.7871.46
  fetch.sh apply-patches.sh               unbranded release GN config
  build.sh package.sh                     depot_tools + checkout · apply/reverse series
  patches/0001..0025 + series             gn gen + autoninja · install as base + rebrand
                                          the twenty-five HUD patches, apply-clean at the tag

v{keys,ovl,pl,pop,pos,sys,term,win}.mjs CDP verification probes (port 9222)
.github/workflows/ci.yml                  shellcheck + JSON validate + patch-series check
.github/workflows/pages.yml               publish docs/ to GitHub Pages

```

Appendix B — The State Directory (\$ZWIRE_STATE)

Everything zwire writes at runtime, in one place. The default is the OS application- data location — `~/Library/Application Support/com.menketotechnologies.zwire` on macOS (named by the reverse-DNS bundle id), `${XDG_CONFIG_HOME:~/.config}/zwire` on Linux — resolved by `scripts/state-dir.sh`; the old `~/.zwire` dotdir and the brief bare-zwire macOS folder are legacy locations, auto-migrated once. `$ZWIRE_STATE` overrides it on every platform.

```
$ZWIRE_STATE/                                (default: OS app-data dir; ~/.zwire + bare zwire are
├── legacy, migrated)                         the extracted Chromium snapshot, or the packaged fork
├── base/                                     the extracted Chromium snapshot, or the packaged fork
├── (~325MB)
├── base.path                                absolute path to the resolved binary (launcher reads
├── this)
├── base.version                             "r<REV>" for a snapshot, "fork-<TAG>" for a compiled
├── fork
├── profile/                                 the dedicated --user-data-dir
├── (bookmarks/history/sessions)
├── NativeMessagingHosts/
│   ├── com.zwire.hud.json                  the HUD native host manifest (path + allowed_origins)
│   └── Default/NativeMessagingHosts/      second location Chromium also checks
├── ext/                                     per-user writable copy of the loaded extensions (staged
├── at launch)
```

The extension-owned keys inside `profile/` that this book referenced:

```
zb_scheme      the mirrored active scheme (worker <-> content scripts)
zb_keys        user keybinding remaps { action: key }
zb_custom_cmds the %K keyword-command rules
zb_sys         the streamed system-stats sample (statusbar reads this)
zb_term_open   whether the terminal overlay should re-open on the next page
zb_cmd         the storage command bus (wakes the worker for tab ops)
zb.tiles/zb.engine new-tab tiles + search engine (page localStorage, not chrome.storage)
```

The two files at the top level — `base.path` and `base.version` — are the entire contract between the base layer and the launcher. The theme itself lives *outside* this per-install tree, in the fleet-wide `~/.zwire/global.toml` (with the host's `hud-scheme/hud-light` text projections beside it for the fork's native color mixer) — the one channel through which the new-tab extension, the internal HUD, `zpwichrome`, the wider app suite, and (on the fork) the native chrome all agree on one look. `profile/` and `ext/` are the isolation boundary that keeps zwire a guest on the machine. Delete the whole tree and the next `zwire` launch fails with “no base browser installed” until you re-fetch — there is no hidden state anywhere else.

Appendix C – Keybinding Reference

The full default keymap from [zkeys.js](#), the single source of truth both [zvim.js](#) and the Keyboard settings page read. Every key here is remappable in the settings page (which writes [zb_keys](#)), except the fixed native prefix.

```
— Scroll —
j scroll down      k scroll up
h scroll left      l scroll right
d half-page down  u half-page up
G bottom of page  H top of document
M middle of document L bottom of document

— Tabs & history —
J previous tab      K next tab
x close tab         t new tab      r reload
[ history back      ] history forward

— Jump & hints —
f link hints (click)    F link hints (new tab)
g g-prefix: gg top · gt/gT tab next/prev · gi focus input
z z-prefix: zt/zz/zb position ref element top/center/bottom
y y-prefix: yy yank URL
m set mark m<x>        ` jump to mark `<x> (also ')

— Palette & find —
o command palette      : command palette
/ find in page          \ toggle vim off (Ctrl/⌘+\\ re-enables)
⌘K / Ctrl+K open palette    ⌘F / Ctrl+F open find

— Tmux overlay (prefix Ctrl-b, or ⌘b) —
% split right      " split down      o next pane      ; last (prev) pane
h select pane ←    j select pane ↓    k select pane ↑    l select pane →
H resize pane ←    J resize pane ↓    K resize pane ↑    L resize pane →
z zoom             x close pane        { swap ←          } swap →
O rotate           ! break to window q pane numbers    space cycle layout
F grid 4           G grid 8           I grid 16
c new window       n next window      p prev window      Tab last window
, rename window    . move window      ' go to window     w window list
& kill window      m mark / swap    e sync panes       E toggle pane in sync
[ copy mode        ] paste buffer    = choose buffer
s sessions         S save session
r reload pane      t clock              : palette          d detach    ? help
0-9 select window  ↔↑↓ focus by direction
Ctrl/Alt + ↔↑↓ resize the active pane

— Fixed (native fork) —
Ctrl-b the tmux split prefix (rebindable in the Keyboard page)
```

Appendix D — The Eight-Scheme Palette

The full dark palettes from `schemes.js`. Every HUD surface styles itself in terms of these twenty CSS variables; each scheme also ships a matched `lightVars` set (omitted here for space) for light mode. Colors are the `vars` (dark) values.

```
cyberpunk  - Hot pink + cyan (default)
  accent #ff2a6d  cyan #05d9e8  magenta #d300c5  green #39ff14  red #ff073a
  text #e0f0ff / dim #7a8ba8 / muted #3d4f6a
  bg #05050a / #0a0a14 / card #0d0d1a / hover #12122a  border #1a1a3e

midnight   - Deep blue + electric purple
  accent #7c3aed  cyan #38bdf8  magenta #6366f1  green #34d399  red #f87171
  text #e2e8f0 / dim #94a3b8 / muted #475569
  bg #020617 / #0f172a / card #1e293b / hover #334155  border #334155

matrix     - Green phosphor terminal
  accent #00ff41  cyan #00ffcc  magenta #008f39  green #00ff41  red #ff3333
  text #d4ffd4 / dim #7ccc7c / muted #406640
  bg #000500 / #001a00 / card #002a00 / hover #003a00  border #004d00

ember      - Burnt orange + amber
  accent #f97316  cyan #fbbf24  magenta #dc2626  green #84cc16  red #ef4444
  text #fed7aa / dim #d97706 / muted #78350f
  bg #0a0503 / #1a0f05 / card #27150a / hover #3a1f0f  border #4a2c17

arctic     - Pale ice + steel blue
  accent #60a5fa  cyan #22d3ee  magenta #818cf8  green #2dd4bf  red #f87171
  text #e0f2fe / dim #7dd3fc / muted #0369a1
  bg #020617 / #0c1424 / card #0f1e33 / hover #1e2f4d  border #1e3a5f

crimson    - Rose-red accent + teal highlight
  accent #e11d48  cyan #2dd4bf  magenta #f43f5e  green #22c55e  red #ff073a
  text #ffe4e6 / dim #b08a92 / muted #6b4a52
  bg #0a0506 / #140a0c / card #1a0d10 / hover #2a1318  border #3e1a22

toxic      - Acid-lime accent + magenta
  accent #c6ff00  cyan #00e5ff  magenta #ff00aa  green #39ff14  red #ff073a
  text #e8ffd0 / dim #8a9a6a / muted #4a5a32
  bg #07090a / #0c0f0a / card #0f130c / hover #161b10  border #2a3a1a

vapor      - Vaporwave pastel pink + cyan
  accent #ff6ec7  cyan #72f1ff  magenta #c792ea  green #5af2b0  red #ff6b8b
  text #f0e6ff / dim #a99cc4 / muted #6a5f86
  bg #0d0814 / #140d1f / card #1a1228 / hover #241836  border #2e2142
```

The twenty keys in full order (`SCHEME_VAR_KEYS`): `--accent`, `--accent-light`, `--accent-glow`, `--cyan`, `--cyan-glow`, `--cyan-dim`, `--magenta`, `--magenta-glow`, `--green`, `--green-bg`, `--red`, `--text`, `--text-dim`, `--text-muted`, `--bg-primary`, `--bg-secondary`, `--bg-card`, `--bg-hover`, `--border`, `--border-glow`.

The light palettes (lightVars)

Each scheme ships a matched light variant under `lightVars`, and light mode swaps these in for the dark `vars` above. The light sets invert the depth relationship — `--bg-primary` becomes a near-white, `--bg-card` becomes `#ffffff`, the text darkens — while keeping each scheme’s hue identity, with accents deepened for contrast on a light ground. The full light key colors, in the same layout as the dark table:

```
cyberpunk  accent #d6196e  cyan #0891b2  magenta #a300a3  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #f0f2f5 / #e4e7ec / card #ffffff / hover #f7f8fa  border #cbd5e1

midnight   accent #6d28d9  cyan #0284c7  magenta #4f46e5  green #047857  red #dc2626
text #0f172a / dim #475569 / muted #94a3b8
bg #f1f5f9 / #e2e8f0 / card #ffffff / hover #f8fafc  border #cbd5e1

matrix     accent #16a34a  cyan #059669  magenta #15803d  green #16a34a  red #dc2626
text #14532d / dim #4d7c0f / muted #86efac
bg #f0fdf4 / #dcfce7 / card #ffffff / hover #f7fee7  border #bbf7d0

ember      accent #c2410c  cyan #ca8a04  magenta #b91c1c  green #65a30d  red #dc2626
text #431407 / dim #9a3412 / muted #fdb74
bg #fff7ed / #ffedd5 / card #ffffff / hover #fef3c7  border #fed7aa

arctic     accent #2563eb  cyan #0891b2  magenta #4f46e5  green #0f766e  red #dc2626
text #0c4a6e / dim #0369a1 / muted #7dd3fc
bg #f0f9ff / #e0f2fe / card #ffffff / hover #f0f9ff  border #bae6fd

crimson    accent #be123c  cyan #0d9488  magenta #be185d  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #faf0f1 / #f2e4e6 / card #ffffff / hover #fdf6f7  border #e0c5cb

toxic      accent #5c8a00  cyan #0891b2  magenta #a3006e  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #f3f7ec / #e7eedb / card #ffffff / hover #f8fbf2  border #d0dcc0

vapor      accent #d6469e  cyan #0e9bb0  magenta #8b5cf6  green #10a37f  red #e11d48
text #2a1e3a / dim #5a4a72 / muted #9a8ab0
bg #f7f0fb / #ede2f5 / card #ffffff / hover #faf4fd  border #ddc9ec
```

Two families collapse toward a shared light red: every scheme except vapor uses `#dc2626` for `--red` in light mode (vapor keeps a rose `#e11d48`), where the dark palettes each carried their own. That is the one place the light variants converge — a bright neutral red reads as “error” on any light background, so the schemes stop differentiating it.

Appendix E – Launch Flag Reference

Every flag `bin/zwire` passes to the base, and why. User arguments ("`$@"`") are appended after these, so any Chromium flag you pass overrides or adds to them.

<code>--user-data-dir="\$PROFILE"</code>	isolate to <code>~/.zwire/profile</code> (never system Chrome)
<code>--load-extension="\$LOAD"</code>	preload newtab + zpwrchrome + hud-internal (the switch unbranded Chromium keeps, branded Chrome dropped in 137) – only dirs with a <code>manifest.json</code>
<code>--extensions-on-chrome-urls</code>	let hud-internal content scripts run on <code>chrome://</code>
<code>--test-type</code>	suppress the automation-restriction warnings the above flags would otherwise raise
<code>--no-first-run</code>	skip Chromium's onboarding
<code>--no-default-browser-check</code>	skip the default-browser nag
<code>--homepage="chrome://newtab"</code>	the home button opens the HUD new tab
<code>--disable-features=NtpFooter</code>	drop the new-tab footer under the HUD page
<code>--enable-features=SplitViewHorizontal,SplitViewTabRestore</code>	side-by-side split view + restore-on-reopen
<code>"\$@"</code>	everything you typed, passed straight through (e.g. a URL, <code>--incognito</code> ,
<code>--remote-debugging-port=9222</code>)	

That is the whole launcher contract: nine HUD-defining flags, then your arguments. The launcher owns what makes `zwire` `zwire` and gets out of the way of every other Chromium flag — which is why `zwire --remote-debugging-port=9222` (Chapter 20), `zwire --incognito`, and `zwire https://github.com` all just work.

Appendix F – The Chrome WebUI Token Map

`theme.js` skins the `chrome://` pages that have no bespoke HUD replacement by overriding Chrome's own CSS custom properties, each with `!important`, so the WebUI components recolor without any markup change. It deliberately maps *only* palette and the mono font — dropping the grid lattice, scanlines, glow, and floating picker — so these pages inherit the theme with no HUD chrome on top. The full override map, token to HUD variable:

```

— Chrome design-system tokens (--color-sys-*) —————
--color-sys-surface                -> --bg-primary
--color-sys-surface-variant        -> --bg-card
--color-sys-surface-subtle         -> --bg-secondary
--color-sys-base                   -> --bg-primary
--color-sys-base-container         -> --bg-card
--color-sys-base-container-elevated -> --bg-hover
--color-sys-header                 -> --bg-secondary
--color-sys-header-container       -> --bg-secondary
--color-sys-on-surface             -> --text
--color-sys-on-surface-subtle      -> --text-dim
--color-sys-on-surface-variant     -> --text-dim
--color-sys-on-surface-secondary  -> --text-dim
--color-sys-primary                -> --cyan
--color-sys-on-primary             -> --bg-primary
--color-sys-primary-container      -> --bg-hover
--color-sys-on-primary-container   -> --accent
--color-sys-secondary             -> --accent
--color-sys-outline                -> --border
--color-sys-tonal-container        -> --bg-hover
--color-sys-tonal-outline          -> --border
--color-sys-neutral-container      -> --bg-card
--color-sys-inverse-surface        -> --text
--color-sys-state-hover-on-subtle  -> --bg-hover

— Fallback tokens (--cr-fallback-color-*) —————
--cr-fallback-color-base-container -> --bg-card
--cr-fallback-color-surface        -> --bg-primary
--cr-fallback-color-primary        -> --cyan
--cr-fallback-color-on-surface     -> --text
--cr-fallback-color-on-surface-subtle -> --text-dim
--cr-fallback-color-outline        -> --border
--cr-fallback-color-neutral-container -> --bg-card
--cr-fallback-color-tonal-container -> --bg-hover
--cr-fallback-color-divider        -> --border

— Component tokens (--cr-*) —————
--cr-card-background-color         -> --bg-card
--cr-primary-background-color      -> --bg-primary
--cr-secondary-background-color    -> --bg-secondary
--cr-primary-text-color            -> --text
--cr-secondary-text-color          -> --text-dim
--cr-toolbar-background-color      -> --bg-secondary
--cr-hover-background-color        -> --bg-hover
--cr-link-color                    -> --cyan
--cr-focus-outline-color           -> --accent
--cr-toggle-color                  -> --cyan
--cr-separator-line                -> 1px solid var(--border)

— Legacy Google tokens (--google-*) —————
--google-grey-900 -> --bg-primary  --google-grey-800 -> --bg-card
--google-grey-700 -> --bg-hover    --google-grey-500 -> --text-dim
--google-blue-300 -> --cyan         --google-blue-400 -> --cyan
--google-blue-500 -> --accent       --google-blue-600 -> --accent

— Also set —————
color-scheme: dark
font-family: "Share Tech Mono","Monaco",monospace (on the full element list)

```

The stylesheet is injected into `document.head` and recursively into every open shadow root, because Chrome's

WebUI is a tree of custom elements and a `:root`, `:host` rule must reach inside each shadow boundary to recolor the components within it. The scheme it reads is `zb_scheme` from `chrome.storage.local` (a content script cannot call the native host itself), falling back to a `zbhud-get` worker message, and it applies cyberpunk first to avoid a flash of the stock theme before the real scheme resolves.

Appendix G – The Native-Messaging Protocol

Every message the browser exchanges with the native host, framed as a 4-byte little-endian length followed by that many UTF-8 bytes of JSON. The **legacy path** is what zwire's browser side actually invokes — the scheme bridge, the sysinfo stream, and the PTY relay of Chapter 15. The **extended** commands are the larger local-machine agent **zwire-host** also exposes; zwire's front end does not call them, but they ship in the same binary and are reachable over its socket daemon.

```
— Legacy zwire path (what the browser uses) —————
{cmd:"get"}           -> {scheme, ui}   read the scheme + UI-pref files
{scheme:"<name>"}     validate vs 8 names, atomically write hud-scheme
{ui:{...}}            merge visual-effect prefs into hud-ui.json (atomic)
{cmd:"sysinfo_start"} stream {sys:{...}} per interval until the port closes
{cmd:"pty_spawn"}     fork a login shell in a PTY (becomes a terminal port)
  {cmd:"pty_write", ...} keystrokes to the shell
  {cmd:"pty_resize", ...} TIOCSWINSZ the window size
  {cmd:"pty_kill"}     end the shell
  <- {ev:"output", b64:...} shell output, base64, 64KB chunks
  <- {ev:"exit"}       the shell died

— Extended agent (same binary, not called by zwire UI) —
hello / ping          {ok,host,version,os,arch,pid,caps:[...]}
kv_get/set/merge/del/keys namespaced JSON key-value store (~/.<app>/kv/)
fs_read/write/append  file contents (64 MiB read cap)
fs_list/walk/stat     directory listing + bounded recursive crawl
fs_mkdir/rm           make (-p) / remove (recursive for dirs)
fs_watch / fs_tail    stream change events / follow a file (rotation-aware)
watch_stop / watch_list drop / list active observers
exec                 run a program to completion -> {ok,code,stdout,stderr}
job_start/poll/list/result background jobs (+ desktop notify on completion)
ps / kill / which     process list / signal / PATH resolve
sub / unsub / pub     process-global publish-subscribe bus
open / clipboard_get/set OS open + clipboard
notify / hostinfo     desktop notification / host facts
sysinfo_once / sysinfo_stop one-shot sample / stop a running stream
auth / peer_hello     token auth / peer handshake (TCP mesh)
peer_connect/peers/remote dial a peer / list peers / run a request remotely
```

The **sys** sample carries: **cpu**, **mem{u,t,p}**, **swap{u,t,p}**, **load[3]**, **uptime**, **disk{u,t,p}**, **net{up,down}**, **io{r,w}** (disk read/write rate), **temp**, **host**, **lip** (LAN IP), **pip** (public IP, refreshed sparingly), and **batt{p,c}** — the battery read natively per platform (**pmset** on macOS, **/sys/class/power_supply** on Linux, **GetSystemPowerStatus** on Windows), the disk-I/O and network figures as per-second rates over the elapsed interval. The stream cadence is a configurable **interval_ms**, with one-shot (**sysinfo_once**) and **sysinfo_stop** variants.

Appendix H – The Seeded Keyword Commands

The `⌘K` palette's keyword commands are seeded from `cmd-defaults.js` (`ZWIRE_CMD_DEFAULTS`) on first use and thereafter live in the user-editable `zb_custom_cmds` array, edited on the Commands page. New defaults added to the file appear automatically; a default you delete is remembered in `zb_cmds_removed` and never re-added. Each `url` entry substitutes your typed argument for `{q}`; the three `action` entries take no argument. The full seed set:

```
— AI & search —
cg ChatGPT          cl Claude          pp Perplexity
gm Gemini           wa WolframAlpha    kagi Kagi
bing Bing          def Define word    dd DevDocs

— Google —
gm Gmail            gd Google Drive    cal Google Calendar
tr Google Translate img Google Images  sch Google Scholar
fl Google Flights  wx Weather

— Code & dev —
gh GitHub           gist GitHub Gist    pr GitHub PRs
iss GitHub Issues  cop GitHub Copilot  gl GitLab
bb Bitbucket       grep grep.app       rex regex101
ciu Can I Use      bp Bundlephobia     npt npm trends
mvn Maven Central  hf Hugging Face     cp CodePen
csb CodeSandbox    repl Replit         aw Arch Wiki
uni Emoji / Unicode

— Cloud & tools —
aws AWS Console    gcp GCP Console     vc Vercel
cf Cloudflare      no Notion            fig Figma
lin Linear         trl Trello

— Comms —
li LinkedIn        sl Slack             dc Discord
tg Telegram        wsp WhatsApp

— Media & shopping —
imdb IMDb          nf Netflix           sp Spotify
yt YouTube Music   tw Twitch            sc SoundCloud
eb eBay            etsy Etsy            abnb Airbnb
yelp Yelp

— Reading —
hn Hacker News     nyt New York Times   arx arXiv

— Actions (no argument) —
rl Reload page     cu Copy page URL     cs Cycle color scheme
```

The list is intentionally a starting kit, not a fixed set: it is seeded once, yours to prune and extend, and the seeder's job is only to make sure a fresh profile has a sensible default without ever overwriting what you have

changed. (The worker carries a shorter first-run seed of the same shape; the `cmd-defaults.js` list is the fuller one that backfills the rest by id on the next load.)

Appendix I – The Storage Keys

The HUD's cross-script state lives in `chrome.storage.local` under a family of `zb_*` keys, plus a couple of page-local `localStorage` keys and the two native files. This is the whole set the book referenced, in one place — the shared memory the content scripts, the worker, and the HUD pages reconcile through:

```
— chrome.storage.local (shared across the extension) —
zb_scheme          the mirrored active scheme (worker <-> content scripts)
zb_sys             the streamed system-stats sample (statusbar reads this)
zb_tabs            live list of open tabs (palette reads it, worker writes it)
zb_exts            enabled extensions (palette Tweak/Manage entries)
zb_frecent         frequency-scored history, top 30 (palette + new tab)
zb_shortcuts       every extension's keyboard shortcuts (via developerPrivate)
zb_cmd             the command bus – content scripts write, worker executes
zb_keys            user keybinding remaps { action: key }
zb_custom_cmds     the %%K keyword-command rules
zb_cmds_seeded     one-shot guard: the worker's first-run seed ran
zb_cmds_removed    tombstones for deleted default commands (never re-seed)
zb_status          whether the powerline statusbar is shown
zb_term_open       whether the terminal overlay should re-open on the next page
zb_tmux            statusbar readout of the in-page overlay (counts, prefix armed)
zb_tmux_state      the worker window-tiling engine's state (eviction recovery)
zb_tmux_sessions   durable named tmux layouts (the Sessions page)
zb_tmux_load       trigger the live overlay watches to attach a saved session
zb_tmux_prefix     the tmux prefix chord (Keyboard page)
zb_tmux_opts       tmux options, e.g. { timeout } (Keyboard page)
zb_ci              CI-monitor config { user, token, repolimit }
zb_ci_status       the CI tally { fail, running, ok, total, at }
zb_ci_cache        cached CI run data (the CI page)

— page localStorage (per-origin, not shared) —
zb.tiles           new-tab quick-launch tiles [{ label, url }]
zb.engine          new-tab search engine template with %s

— native files (~/.zwire, the cross-extension bus) —
hud-scheme         the active scheme (fork mixer + new-tab both read it)
hud-ui.json        shared visual-effect + light-mode prefs
```

The distinction that matters: `chrome.storage.local` is shared among the internal HUD's own scripts but *not* across extensions, and `localStorage` is not shared even across origins — so anything the new-tab extension or the power-tool must also see travels through the two native files, which are the only channel two extensions share. That is the whole reason `hud-scheme` and `hud-ui.json` exist as files rather than storage keys.

Appendix J – The Rust Host Modules

`zwire-host` is organized into fifteen small modules under `src/`, and the split is the map of the superset described in Chapter 23. For the reader who opens the crate, each module and its one job:

<code>main.rs</code>	entry point: pick the transport, run the session loop
<code>lib.rs</code>	the crate root; re-exports the api facade for embedders
<code>transport.rs</code>	three entry points – Chrome stdio, the NDJSON socket daemon, and a one-shot client call
<code>proto.rs</code>	framing (4-byte LE length + JSON; NDJSON for the socket), the 128-MiB message guard, id-tagged replies
<code>session.rs</code>	the per-connection router: dispatch, held PTYs/watchers/subs, teardown on drop
<code>store.rs</code>	file-backed state under <code>~/.<app>/</code> – the kv store and the legacy scheme/ui files, all atomic tmp-and-rename (no SQLite)
<code>sysmon.rs</code>	the system sample via the <code>`sysinfo`</code> crate (not <code>psutil</code>), streamed every interval
<code>pty.rs</code>	PTY terminals via <code>`portable-pty`</code> , multiplexed by id, base64 I/O
<code>exec.rs</code>	run a program to completion (base64 stdout/stderr + exit code)
<code>fsops.rs</code>	read/write/append/list/walk/stat/mkdir/rm, with size + count caps
<code>jobs.rs</code>	background jobs + completion notifications
<code>watch.rs</code>	poll-based <code>fs_watch</code> and rotation-aware <code>fs_tail</code> observers
<code>procs.rs</code>	<code>ps</code> / <code>kill</code> / <code>which</code> (reusing the <code>sysinfo</code> crate)
<code>osops.rs</code>	<code>open</code> / <code>clipboard</code> / <code>notify</code> / <code>hostinfo</code> (shelling to platform tools)
<code>bus.rs</code>	the process-global publish-subscribe broker
<code>peer.rs</code>	the TCP host-to-host mesh: auth, dial, bus federation, remote
<code>api.rs</code>	an in-process library facade (<code>exec/walk/read/write</code>) for embedders

The release profile is tuned for a small artifact — `opt-level="z"`, LTO, symbols stripped, `panic="abort"` — which is how a host that carries a kv store, a filesystem API, a PTY multiplexer, a job runner, and a TCP mesh still fits in roughly half a megabyte with no external runtime. `zwire`'s browser uses only `store.rs`, `sysmon.rs`, and `pty.rs`; the rest is the larger local agent the same binary offers any app that speaks its socket.

Appendix K – The Launcher and Build Scripts

The runtime path is a handful of shell scripts. Their contracts, in one place:

```
— bin/zwire – the launcher, in sequence —————
1 source state-dir.sh; STATE = $ZWIRE_STATE or the OS app-data dir
2 one-time migrate a leftover ~/.zwire into STATE (if no $ZWIRE_STATE)
3 base = executable $ZWIRE_BASE, else $STATE/base.path, else FAIL
  "zwire: no base browser installed – run: ../scripts/fetch-base.sh"
4 ext list = { newtab, extensions/zpwrchrome, extensions/hud-internal }
  filtered to dirs with a manifest.json (theme/ deliberately excluded)
5 read "path" from com.zwire.hud.json; if missing/dangling run
  setup-native-host.sh (never fatal – browser runs without the host)
6 exec base with the nine flags + "$@"

— the nine launch flags (Appendix E has the why) —————
--user-data-dir --load-extension --extensions-on-chrome-urls --test-type
--no-first-run --no-default-browser-check --homepage=chrome://newtab
--disable-features=NtpFooter
--enable-features=SplitViewHorizontal,SplitViewTabRestore
  (the Linux self-contained launcher adds a tenth: --no-sandbox)

— fetch-base.sh – host -> snapshot —————
Darwin/arm64 -> Mac_Arm    chrome-mac.zip    chrome-mac
Darwin/x86_64 -> Mac       chrome-mac.zip    chrome-mac
Linux/x86_64  -> Linux_x64 chrome-linux.zip chrome-linux
MINGW/MSYS   -> Win_x64    chrome-win.zip    chrome-win
  revision = arg, else the platform's LAST_CHANGE
  writes base.path (abs binary) + base.version ("r<REV>")

— the self-contained install layouts —————
macOS    /Applications/zwire.app/Contents/{Resources/{browser,ext,native},MacOS/zwire}
Linux    ~/.local/opt/zwire/{browser,ext,native} + PATH link + .desktop
Windows  %LOCALAPPDATA%\zwire\{browser,ext,native} + zwire.cmd + registry host
  each stages extensions per-user (rsync, excluding _metadata) at launch
```

The three pinned extension ids appear identically in four scripts and each extension's own **key**: HUD [omcggnjfm b m p d l o f k l b p d d k h n f i b f h g g](#), zpwrchrome [h p p p d c h p n p h m i i j d e a n i b p c a d g k n m a j a](#), new-tab [g p o e p n e k o i p l h k e g j p o c n p e i j i e f g i e b](#). The native host name is `com.zwire.hud`, its manifest lists all three as `allowed_origins`, and the pinned fork tag is `150.0.7871.46`.

Appendix L – The CDP Verification Probes

The eight `v*.mjs` probes attach to a running `zwire` over the Chrome DevTools Protocol on `127.0.0.1:9222` (opened only when you pass `--remote-debugging-port=9222`). Each shares one bootstrap — `fetch /json/version`, open the `websocketDebuggerUrl`, `Target.attachToTarget` with `flatten:true`, `Runtime.enable`, then evaluate expressions with `returnByValue:true` — and prints booleans and counts that a human or a CI grep reads as pass or fail. What each verifies:

probe	target	asserts
<code>vkeys</code>	<code>createTarget keys.html</code>	<code>ZWIRE_KEYMAP</code> present · chip/category counts · writes <code>zb_keys.scrollDown='s'</code> + reads it back
<code>vterm</code>	<code>createTarget terminal.html</code>	<code>Terminal</code> global · <code>#terminalPane</code> · <code>.xterm</code> · waits 5s for <code>zsh</code> · rows of shell text appeared
<code>vovl</code>	<code>attach github/amazon/wiki</code>	<code>synth Ctrl+`</code> · <code>#terminalPane</code> becomes <code>.active</code> · <code>.xterm</code> rendered
<code>vpop</code>	<code>attach a stable page</code>	<code>Ctrl+`</code> · <code>.active</code> · waits 7s for the <code>SW->host PTY</code> shell · reads back visible shell text
<code>vwin</code>	<code>attach a stable page</code>	<code>Ctrl+`</code> opens <code>terminal.html</code> popup window · attach it · <code>.xterm</code> · <code>connectNative</code> reachable
<code>vsys</code>	<code>attach a stable page</code>	waits 5s for <code>zb_sys</code> · reads <code>CPU/MEM/NET/HOST/LAN/UP/BATT</code> segments · <code>.plr</code> chevron count · clock
<code>vpos</code>	<code>attach a stable page</code>	left-to-right segment order · <code>C-b</code> prefix sits left of the scheme segment
<code>vpl</code>	<code>attach a stable page</code>	the powerline <code>::after</code> chevron borders + the prefix block background render as intended

`vkeys` and `vterm` open an extension page directly; the other six attach to an existing `github/amazon/wikipedia` tab (and exit cleanly if none is open, since that is a setup gap, not a fault). Two probe lines are dead — `vterm` reaches for a non-existent `window.__zbTermInstance`, and `vovl` prints a placeholder instead of reading `zb_term_open` — noted here so the suite is not oversold.

Appendix M – The zpwrchrome Manifest

The power-tool's manifest, as the concrete proof of Chapter 2 (this book does not otherwise document `zpwrchrome`, which has its own):

```
manifest_version 3 · minimum_chrome_version 127 · module service worker
version 0.22.36 · <all_urls> host access · 6 content scripts · 55 commands

permissions (19):
  userScripts · declarativeNetRequestWithHostAccess · nativeMessaging ·
  webRequest · webRequestAuthProvider · tabs · tabGroups · sessions ·
  bookmarks · history · scripting · webNavigation · cookies · downloads ·
  contextMenus · notifications · clipboardWrite · alarms · storage

the four that require a genuine Chromium engine (Chapter 2):
  userScripts · declarativeNetRequestWithHostAccess · nativeMessaging · webRequest

feature surface (one toolbar icon): browserpass-compatible `pass` (fill/copy/
OTP/autofill via a Rust host over ~/.password-store) · segmented download
manager · MRU tab switcher (scenes/opener-tree/fzf history) · userscript
engine · full-page screenshot · tech detector · theme injector · lights-off
dimmer · JSON/XML viewers · UA switcher · find-in-all-tabs
```

Appendix N – The Fork Patch Targets

The twenty-five HUD patches, each apply-clean against `150.0.7871.46`, and the native surface each reaches that a theme extension cannot:

```
0001 tab shape      tab_style_views.cc (GetPath)      trapezoid tabs, hud_slope 0.28
0002 UI colors    chrome_color_mixer.{cc,h}+mixers.cc HUD palette + 8 schemes on frame/
                                toolbar/tabs/omnibox + separator

0003 UI font      resource_bundle.cc (InitDefault...) Share Tech Mono, Monaco fallback
0004 toolbar line toolbar_view.cc (Init, kNormal) 2px cyan under-toolbar border
0005 omnibox      location_bar_view.cc (ComputeBorderRadius) clamp field radius to 2px
0006 product str. chrome/app/theme/chromium/BRANDING Chromium -> zwire, bundle id
0007 DevTools     3 front_end files          8 HUD schemes in Theme dropdown
0008 allow framing ancestor_throttle.cc      bypass X-Frame-Options / CSP
0009 ext focus     extension_keybinding_registry_views.cc hand focus to page for %K
0010 force newtab  search.cc (HandleNewTabURLRewrite) chrome://newtab -> zwire
0011 pinned exts  toolbar_view.cc          pinned actions never overflow
0012 frame-src     navigation_request.cc     host CSP never blocks subframe
0013 restore sess. session_startup_pref.cc     default startup = last session
0014 no crash bubl session_crashed_bubble_view.cc     suppress "Restore pages?" bubble
0015 no nav block extension_navigation_throttle.cc allow NTP nav pre-registration
0016 ext priv API _permission_features.json hud-internal:
    ~ developer/settingsPrivate
0017 no hold-quit app_controller_mac.mm       immediate %Q (no hold-to-quit)
0018 null-pol fix extension_function_dispatcher.cc gate is_sandboxed on IsActive
    ~ (crash fix)
0019 store script chrome_extensions_client.cc   content scripts run on Web Store
0020 menu colors  chrome_color_mixer.cc (final mixer) native menus -> HUD Sys tokens
0021 primitive col chrome_color_mixer.cc (final mixer) native primitives -> HUD Sys
    ~ tokens
0022 audio EQ     output_controller.{cc,h}+client.cc browser-wide output EQ (every
    ~ stream)
0023 audio capture tab_capture_api.cc + _api_features Audio page taps a tab, no prompt
0024 audio live   audio_service.mojom + service.{cc,h} live EQ config + output meters
0025 custom palette chrome_color_mixer.cc (final mixer) paint native chrome from the
                                hud-palette base colours, not
                                the baked scheme-name table
```

The series file records each patch's apply-clean verification and keeps a commented-out `0008-tmux-split-prefix` tombstone (the native two-pane split that ate the `C-b` prefix, superseded by the JS overlay); `scripts/check-patch-series.sh` gates the file against the patches on disk. `fork/args.gn` builds it unbranded (`is_chrome_branded=false`), official (`is_official_build=true`), with symbols, PGO, NaCl, and ThinLTO all off for a workstation-tolerable build time.

Appendix 0 — The Three Extension Manifests

zwire loads three unpacked extensions, each pinned to a stable id by an embedded [key](#). Their manifests side by side:

```
— newtab — "zwire New Tab" —————
manifest_version 3 · version 1.0.13 · id gpoepnekoiplhkegjpcnpeijiefgieb
chrome_url_overrides.newtab -> newtab.html
permissions: nativeMessaging · tabs · history
externally_connectable.ids: [ hud-internal ] (so the worker can open its palette)
web_accessible_resources: newtab/palette/schemes/scheme-sync/tmux-pane/fonts, <all_urls>

— hud-internal — "zwire HUD Internal" —————
manifest_version 3 · version 0.6.29 · id omcggnjfbmpdlofklbpddkhnfibfhgg
permissions (20): storage · nativeMessaging · management · tabs · scripting ·
  webNavigation · history · downloads · bookmarks · sessions · alarms ·
  notifications · developerPrivate · settingsPrivate · system.display ·
  declarativeNetRequest · browsingData · tabGroups · readingList · power
host_permissions: <all_urls>
declarative_net_request: rules/frame_bust.json
background: background.js (classic service worker, not a module)
commands: open-palette = ⌘K / Ctrl+K
content_scripts: 11 blocks (Appendix / Chapter 7)
externally_connectable.ids: [ zpwrchrome ]
web_accessible_resources: pages/* · schemes.js · lib/term/*
  (no minimum_chrome_version, no chrome_url_overrides)

— zpwrchrome — the power-tool (submodule) —————
manifest_version 3 · version 0.22.36 · minimum_chrome_version 127
id hpppdchpnphmiijdeanibpcadgknmaja
permissions (19, incl. userScripts · declarativeNetRequestWithHostAccess ·
  nativeMessaging · webRequest) · host_permissions <all_urls>
background: module service worker · 6 content scripts · 55 commands (Alt+T)
options_ui: scripts-manager/manager.html · externally_connectable.ids: [ hud-internal ]
```

The asymmetry in [externally_connectable](#) is the cross-extension messaging graph: the new-tab extension accepts messages from the HUD worker (for ⌘K on the new tab), and both the HUD and the power-tool accept messages from each other for the scheme bridge — the only three extensions on the machine that speak to one another, wired by id, with the native host's [allowed_origins](#) completing the triangle so any of the three can reach [com.zwire.hud](#).

Appendix P – The Content-Script Injection Order

The `hud-internal` manifest registers seven content-script blocks; the order within a block is load-bearing because later scripts read globals earlier ones set. Each script also guards against double injection with a `window.__zb...Loaded` flag.

block	run_at	frames	scripts (in order)	defines
1	document_start	top	redirect.js	(redirect)
2	document_start	top	schemes.js, theme.js	window.ZWIRE_HUD
3	document_idle	top	schemes.js, fzf.js, zfind.js	ZBFzf, find bar
4	document_start	top	schemes.js, zgui util/fzf/command-palette, zkeys.js, cmd-defaults.js, zpalette.js, zvim.js	window.ZGui, ZWIRE_KEYMAP, palette, vim
5	document_idle	top	schemes.js, powerline.js, zpowerline.js	statusbar
6	document_idle	ALL	zterm-pane.js (+ top-frame ZGui.tmux & zterm-config.js)	tmux overlay
7	document_idle	top	xterm.js, terminal.js, zterm-boot.js	terminal

Block 1 matches only the five redirected `chrome://` pages; block 2 matches every other `chrome://` page (excluding the redirected set and the new-tab URLs); blocks 3–5 and 7 match `http/https/file/chrome://` (excluding the new-tab URLs); block 6 matches only `http/https/file` and is the sole `all_frames` block, so the tmux forwarder runs inside framed panes. `schemes.js` is re-listed in every block because each block is an isolated injection that needs the palette table present.

Appendix Q – The Fuzzy Matcher

`ZGui.fzf` (`fzf.js`) is the fzf-style matcher behind the palette, the find-bar fallback, and every fuzzy-filtered HUD page — extracted verbatim from Audio-Haxor so the whole suite ranks identically. Its scoring:

```
— weights (defaults, all tunable + localStorage-persisted) —
SCORE_MATCH          16    per matched character
SCORE_GAP_START      -3    first char of a gap between matches
SCORE_GAP_EXTENSION  -1    each further gap character
BONUS_BOUNDARY       9     match right after a non-word char (start of word)
BONUS_NON_WORD       8     match across a word/non-word class change
BONUS_CAMEL          7     lowercase -> UPPERCASE transition (camelCase)
BONUS_CONSECUTIVE    4     match adjacent to the previous match
BONUS_FIRST_CHAR     x2    the first matched char's bonus is doubled

— extended query sigils (space = AND, | = OR within a group) —
!term    negate (exclude matches)
'term    or term' exact substring
^term    prefix
term$    suffix
term     fuzzy (subsequence)
```

The matcher tries every start position of the needle's first character, greedily takes the earliest subsequent matches, scores each run with match + position bonus + consecutive/gap adjustment, and keeps the best. Matched characters are wrapped in `<mark class="fzf-hl">` for the cyan emphasis. The name field of a row gets a `+500` field bonus so a hit in the label outranks a hit in the detail.

Appendix R – Visual Effects and Readline

The cyberpunk look and the terminal-style input editing both come from `zgui-core`, on two clean conventions. Effects (`ZGui.fx`): an effect is *on* when its body class is *absent*.

```
— effects: pref -> body class -> CSS technique —————
scanlines  no-scanlines    repeating-linear-gradient on body::after (+ animated beams)
vignette    no-vignette    radial-gradient ellipse on body::before (transparent->dark)
glow        no-neon-glow   pervasive text-shadow / box-shadow (off = none !important)
anim        no-anim       all animation/transition (off = none !important)
light       [data-theme=light] flips the palette (a root attribute, not a body class)
grid        two linear-gradient hairlines, 40px fixed cell (always on)
           prefs persist to zg-fx-* localStorage and to native hud-ui.json (cross-extension)

— readline (installReadline, on every text input) —————
^A start-of-line  ^E end-of-line    ^B back char    ^F forward char
^H backspace      ^D delete char    ^K kill to EOL  ^U kill to BOL
^W kill word back ^Y yank           ^T transpose    M-b/M-f word back/fwd
M-d kill word fwd (^K/^U/^W/M-d feed a shared kill-ring that ^Y pastes)
  also: autocorrect/autocapitalize/spellcheck forced off; ⌘F/Ctrl+F focuses the filter
```

Appendix S – The System-Stats Sample

Every tick of the sysinfo stream writes one `zb_sys` object; the statusbar renders one field per segment. The sample, and which host produces each field:

field	meaning	shape	both / note
cpu	CPU percent	number	both
mem	memory used/total/percent	{u,t,p}	both
swap	swap used/total/percent	{u,t,p}	both
load	load averages (3)	[a,b,c]	both
uptime	seconds since boot	number	both
disk	root-fs used/total/percent	{u,t,p}	both
net	throughput (bytes/s)	{up,down}	both (delta / interval)
io	disk I/O (bytes/s)	{r,w}	both (delta / interval)
temp	max sensor temperature (°C)	number	both (hidden if absent)
batt	battery percent + charging	{p,c}	both (macOS/Linux/Windows)
host	short hostname	string	both
lip	LAN IP (UDP-connect trick)	string	both
pip	public IP (ipify, every 120s)	string	both

The stream cadence is a configurable `interval_ms` (floored at 250 ms), with one-shot (`sysinfo_once`) and `sysinfo_stop` variants and a background-thread pump that keeps the connection free for other RPCs. The one-shot `sysinfo_once` double-refreshes so its `net/io` rates report a near-zero delta rather than a lifetime total mislabelled as a per-second rate.

Appendix T – The Palette Destinations

Beyond your tabs, history, schemes, and keyword commands, the `⌘K` palette carries a static catalog of destinations. The HUD pages and the `chrome://` diagnostics tail:

```
— HUD replacement pages —
Extensions · Settings · History · Bookmarks · CI runs · Shortcuts (keys) ·
Extension shortcuts · Custom commands · System info (version)

— chrome:// destinations (≈44) —
newtab · downloads · history · bookmarks · extensions · settings · version ·
system · flags · discards · net-internals/#dns · gpu · net-internals ·
password-manager · inspect · net-export · policy · components · management ·
accessibility · omnibox · media-internals · webrtc-internals ·
serviceworker-internals · indexeddb-internals · quota-internals ·
blob-internals · histograms · tracing · crashes · device-log · gcm-internals ·
sync-internals · process-internals · autofill-internals · download-internals ·
signin-internals · translate-internals · user-actions · ukm · predictors ·
memory-internals · webui-gallery · chrome://about (all pages)

— chrome://settings sub-pages (19) —
You & Google · Appearance · Autofill · Payments · Addresses · Privacy ·
Security · Cookies · Site settings · Clear browsing data · Performance ·
Search engine · Default browser · On startup · Languages · Downloads ·
Accessibility · System · Reset

— external —
Chrome Web Store · zwire app store (menketechnologies.github.io/app-store/)
```

The new-tab palette carries the same static catalog (it is a separate extension and must re-carry it), which is why the two palettes list identical destinations even though they reach `chrome.tabs` by different paths (Chapter 12).

Appendix U – The Command Bus

Content scripts cannot call the privileged tab, window, and session APIs, so they write a `zb_cmd` object to `chrome.storage.local`; the `storage.onChanged` event reliably wakes the MV3 worker, which performs the action. Every `zb_cmd` carries an action field `a`, a monotonic tag `n` (so a repeated identical command still registers as a change), and any parameters. The worker's dispatch:

a	params	effect
ping	–	wake + refresh <code>zb_tabs/zb_exts/zb_frecent/zb_shortcuts</code>
open	url	navigate the active tab, else create a tab
openTab	url	create a new tab
activate	tabId	switch to a tab + focus its window
newTab	–	create an empty tab
newWindow	–	create a new window
duplicateTab	–	duplicate the active tab
reopenTab	–	<code>sessions.restore</code> (reopen last closed)
pinTab	–	toggle the active tab's pinned state
muteTab	–	toggle the active tab's muted state
closeTab	–	close the active tab
closeOthers	–	close every non-active, non-pinned tab in the window
nextTab	–	cycle to the next tab (last-focused window)
prevTab	–	cycle to the previous tab
tmux	sub, ...	drive the worker's OS-window tiling engine (Chapter 8)

The vim script issues `prevTab/nextTab/closeTab/newTab/openTab` this way; the palette adds `duplicateTab`, `reopenTab`, `pinTab`, `muteTab`, `closeOthers`, `newWindow`, and `activate`. A parallel path — `runtime.onMessage` — carries the scheme set (`zbhud-scheme`), the scheme get (`zbhud-get`), a content-script open (`zbopen`), and a wake ping (`zbping`); these are request/response where the bus is fire-and-forget, and the worker uses whichever suits the reliability the caller needs.

Appendix V – The Frecency Model

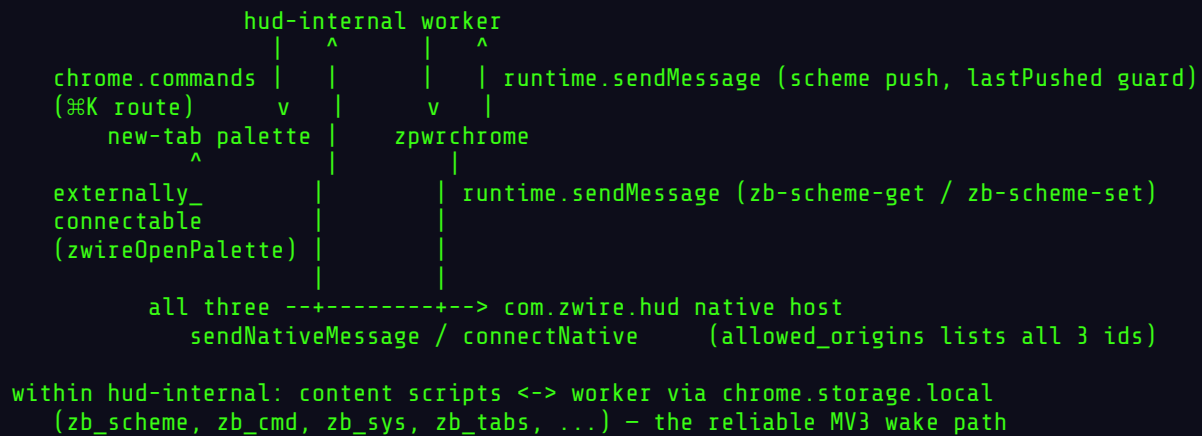
Three surfaces — the worker's `zb_frecent`, the new-tab palette, and the HUD pages' `zg-boot.js` — surface your “frequent + recent” history, and all three use the same formula, so a site ranks identically wherever it appears:

```
window  chrome.history.search over the last 90 days (startTime = now - 90d)
score    (visitCount + 2 * typedCount) / (1 + ageDays * 0.3)
        ageDays = (now - lastVisitTime) / 86_400_000
filter   drop URLs beginning with "chrome"
keep     the top 30 by score, descending
```

A typed visit counts double a clicked one (you meant to go there), and the divisor decays a site's score by roughly a third per day of age, so recency and frequency trade against each other rather than one dominating. The worker recomputes the list on every `history.onVisited`, so the palette's history rows are never stale.

Appendix W – The Extension Messaging Graph

Three extensions and one native host coordinate across boundaries that neither `chrome.storage` nor `localStorage` can cross. Who reaches whom:



The rule underneath the graph: storage is shared only *within* an extension, so every cross-extension value (the scheme, the UI prefs) travels through the native files, and every cross-extension *event* (open the palette, push a scheme) travels through `externally_connectable` or `runtime.sendMessage` between the specific ids each manifest allows.

Appendix X – The zgui-core Toolkit Surface

zwire loads a slice of the shared [zgui-core](#) toolkit; the modules and their public surface:

```
util.js  -> ZGui.util  escapeHtml · throttle · debounce · clamp · formatBytes ·
                                formatDuration · slugify · copyToClipboard · lineEdit ·
                                installReadline · focusFilterBar · dampenInput ·
                                installInputDefaults (auto-installs readline + input
                                defaults on load; binds ⌘F/Ctrl+F to the filter bar)
fzf.js   -> ZGui.fzf   fzfMatch · getMatchIndices · highlightWithIndices ·
                                searchScore · parseFzfQuery · setParam/defaults/bounds
command-palette.js -> ZGui.palette register · registerProvider · clear · open ·
                                close · isOpen · bindHotkey · create (three-tier ranking:
                                static+strong / secondary / fallback-last)
fx.js    -> ZGui.fx    set · toggle · get · all · load · buildToggles (the four
                                body.no-* effect classes)
crt.js   -> ZGui.crt / ZGui.neonGlow the legacy scanline + glow layers
fonts.css + webui/fonts/  Orbitron (display) + Share Tech Mono (body), SIL OFL
```

The guidance from [zgui-core](#)'s own docs is to consume it as a submodule and load the modules directly — never copy [webui/*](#) — which [zwire](#) follows for the content-script extension; the new-tab extension, a distinct origin, bundles its own copy of the [util/fzf/command-palette](#) trio under [lib/zgui/](#).

Appendix Y – The HUD Internal Pages

Each replacement page, the privileged API it stands on, and the state it reads or writes:

page	API	reads / writes
extensions	developerPrivate + management	enable/disable/reload/pack/site-access
settings	settingsPrivate	every Chrome pref + scheme/effects -> hud-ui
history	history.search	recency-sorted list (zg-list)
bookmarks	bookmarks.getTree	flattened folder-path table
downloads	downloads.search	open-on-click (native page owns the manager)
version	runtime + navigator + host	live com.zwire.hud probe
keys	ZWIRE_KEYMAP	writes zb_keys remaps + zb_tmux_prefix/opts
commands	-	writes zb_custom_cmds (+ zb_cmds_removed)
sessions	-	zb_tmux_sessions CRUD -> zb_tmux_load trigger
extshortcuts	developerPrivate	updateExtensionCommandKeybinding
ci	GitHub REST API	zb_ci config -> per-repo run accordions
readinglist	readinglist	add/read/remove, unread-first ordering
terminal	native host (pty_*)	a PTY per tab, keyed by tab id
host	native host (every cmd)	the console over the one JSON protocol
dashboard	-	tile order per category (ZGui.drag)
hooks	native host (hooks_*)	hooks.json + <id>.st, hook-result bus topic
triggers	-	zb_triggers CRUD (ztriggers.js compiles it)
pipes	-	zb_pipes CRUD (zpipes-core.js runs it)
audio	native host (fs_read/write)	\$STATE/audio-eq spec + \$STATE/meters frames
notes	-	zb_notes folder tree, autosaved
feeds	fetch (<all_urls>)	zb_feeds URLs; RSS/Atom parsed in-page
translate	fetch (<all_urls>)	keyless endpoint, no stored state
store	tabs.create	static catalog (store-catalog.js) + welcome

All twenty-three mount through `zg-boot.js` (`ZBHUD.mount`), share the `zb-*` shell chrome, carry the same `⌘K` palette, and filter through the same `fzf.js` — one surface, not twenty-three.

Appendix Z – The Dev-Loop Scripts

The `scripts/` family assembles and resets the base; none of them compile the browser (the base is a prebuilt snapshot). Each sources `cyberpunk.sh` for its banner:

```
dev.sh      launch from the checkout, foreground (content scripts hot-reload)
build.sh    ensure a current rebranded base exists (fetch if missing, then rebrand)
rebuild.sh  clean + build
clean.sh    remove the /Applications wrapper + staged dist/ (keeps base + profile)
bust.sh     re-seal + re-register the bundle with LaunchServices (macOS only)
nuke.sh     wipe base + wrapper, re-fetch, rebrand, reinstall (keeps the profile)
test.sh     node --check the front-end (excl. node_modules + lib/zgui-core),
            then each extension's own pnpm test
install.sh  PATH symlink + fetch + (macOS) rebrand – runs from the checkout
localinstall*.sh  self-contained app: browser + ext + Rust host in one location
setup-native-host.sh  cargo build --release zwire-host + write the host manifest
fetch-base.sh      download + pin the Chromium snapshot base
check-patch-series.sh  CI gate: fork/patches/series <-> the *.patch files on disk
```

The destructive ones are careful about what they keep: `clean` and `nuke` never touch the profile, and `clean` does not even touch the 325 MB base — that is `nuke`'s job. The npm scripts in `package.json` alias the common ones (`pnpm dev`, `pnpm build`, `pnpm nuke`) for anyone who reaches for the package manager instead.

Appendix AA – Three Paths, End to End

The HUD's design is a set of multi-hop paths across isolation boundaries. Three of them, traced in full, tie the book together.

Switching the color scheme:

```
pick a scheme (palette / picker / Settings page)
-> runtime.sendMessage {zbhud-scheme} to the worker + storage.local zb_scheme
-> worker: sendNativeMessage {scheme} to com.zwire.hud
    -> host writes ~/.zwire/hud-scheme (atomic tmp + rename)
-> worker: pushToZpwr (runtime.sendMessage, lastPushed guard)
renderers reconcile to the file / the mirror:
content scripts + chrome:// skin    <- storage.local zb_scheme (observed)
new-tab extension                  <- polls hud-scheme every 1.5s
zpwchrome                          <- the runtime.sendMessage push
fork native color mixer            <- reads hud-scheme natively
```

Opening the command palette with ⌘K:

```
ordinary page    zpalette.js catches ⌘K in a capture listener -> opens
new-tab page     Chrome reserves ⌘K -> worker owns it as a chrome.commands
                 shortcut -> sendMessage(newtab, zwireOpenPalette) -> palette opens
inside a pane    the forwarder postMessages {palette:1} up -> top frame opens,
                 navigates the active pane instead of a stray tab
```

A keystroke in the terminal overlay:

```
xterm onData -> content-script port {pty_write} -> worker (zwire-pty, per-tab)
-> connectNative com.zwire.hud -> host os.write(master, bytes)
shell output -> host reads 64KB -> base64 {ev:output,b64} -> worker
-> (buffer if detached, else) content-script port -> atob -> xterm.write
navigation: content script disconnects, worker keeps sess.nat alive,
next page re-injects, re-attaches to the same tab's shell – session survives
```

Each path crosses at least one boundary a single page cannot cross on its own — a suspended worker, a second extension, a native process — and each is routed around its guard by the mechanism the earlier chapters named: the storage bus for the worker, [externally_connectable](#) for the other extension, the per-tab port for the PTY. That routing, repeated for every feature, is the whole of what makes a stack of extensions behave like one browser.

Appendix AB – Glossary

The recurring names, in one place.

```
— native files + host —————
hud-scheme      the active scheme; the cross-extension source of truth (a file)
hud-ui.json     shared visual-effect + light-mode prefs (a file)
com.zwire.hud   the native-messaging host manifest -> the Rust zwire-host binary
zwire-pty       the runtime.connect port name for the terminal's PTY relay

— storage keys (chrome.storage.local, Appendix I) —————
zb_scheme       mirror of the active scheme (worker <-> content scripts)
zb_cmd          the command bus (content script -> worker, Appendix U)
zb_sys         the streamed system-stats sample (statusbar, Appendix S)
zb_tabs/_exts/_frecency/_shortcuts  worker-maintained mirrors for the palette
zb_keys         keybinding remaps { action: key }
zb_custom_cmds  the ⌘K keyword commands (Appendix H)
zb_term_open    whether the terminal re-opens on the next page
zb_tmux*        overlay readout, window-engine state, sessions, prefix, opts
zb_status       whether the powerline statusbar is shown
zb_ci*          CI-monitor config, status, cache

— globals (in the page / content scripts) —————
window.ZWIRE_HUD  the 8-scheme palette table (VAR_KEYS, ORDER, SCHEMES)
window.ZWIRE_KEYMAP the keybinding registry (Appendix C)
window.ZGui       the shared toolkit (fzf, palette, fx, util – Appendix X)
window.ZBHUD      the HUD-page shell (mount, palette, publishUi)

— the three extension ids —————
hud-internal      omcgnnjfbmpdlofklbpddkhnfibfhgg
zpwichrome        hpppdchpnphmiijdeanibpcadgknmaja
new-tab           gpoepnekoiplhkegjpcocnpeijiefgieb

— other terms —————
frecency          frequency+recency history score (Appendix V)
the fork          fork/, a compiled patched Chromium as an alternate base (Ch 21)
the rebrand       the runtime, no-compile skinning path (the default, Ch 5)
```

Appendix AC – The Statusbar Segments

The powerline statusbar (shared `ZGui.powerline` fed by `zpowerline.js`, top frame only, `position:fixed` bottom, 22px) is a left group of HUD state and a right group of live machine stats, split by a flex spacer, with true CSS-border powerline chevrons between blocks. Reading left to right:

```
— left: HUD state —————
ZW      the badge (cyan block, chevron)
C-b     the tmux prefix indicator (dim; flips to accent for 1.6s when armed)
<scheme> the active scheme's label
i·panes  per-window tmux readout (active emphasized, ± when syncing)
VIM     static mode marker
%K      clickable; opens the palette

— right: live machine (from zb_sys, Appendix 5) —————
seg    data-*      format
CPU    data-cpu    <n>%
MEM    data-mem    used/total  (humanized B/K/M/G/T, 1 decimal < 100)
SWP    data-swap    used/total
DSK    data-disk    <n>%
IO     data-io      R<rate> W<rate>          (bytes/s)
NET    data-net     ↑<rate> ↓<rate>          (bytes/s)
LD     data-load    a b c                    (load averages)
UP     data-up      Xd Yh Zm                (humanized uptime)
°      data-temp    <n>°C                    (segment hidden if no sensor)
BATT   data-batt    ⚡/battery + <n>%
LAN    data-lip     private IP
WAN    data-pip     public IP
@      data-host    short hostname
clock  data-clock   MM-DD HH:MM:SS          (own 1-second interval)
```

The chevrons are real triangles: an 11-pixel transparent top/bottom border plus a solid side border on a pseudo-element, left-of-spacer blocks pointing right and right-of-spacer blocks pointing left, consecutive blocks alternating two background shades. A stats refresh rewrites the `data-*` spans in place; a scheme change re-themes the bar from the same twenty variables as everything else.

Appendix AD — Grounding and Edition

Every claim in this book traces to a file in the `zwire` repository. Where the text says “the code comments say,” it is quoting the source verbatim; where it gives a number — 66 seeded commands, 44 `chrome://` destinations, a 0.28 tab slope, a 90-day frequency window, nine launch flags, twenty-five patches — that number was read out of the code, not recalled. Where the runtime path and the source fork differ, the book marks the boundary rather than rounding up; the honest gaps (the two dead CDP-probe lines) are named where they occur because a companion grounded in the code owes the reader its seams as much as its surfaces.

Two corrections this edition makes to earlier printings are worth flagging: the state directory default is the OS application-data location (with `~/.zwire` a legacy path that is migrated once, not the live one), and the Python reference host has been removed — the native host is now the Rust `zwire-host` alone, and where earlier printings speak of a “Python reference,” read that as the historical first implementation of the same scheme/stats/PTY path `zwire-host` now owns outright.

This edition also grows with the code. The patch series is twenty-five rather than twenty-four (0025 paints the native chrome from the `hud-palette` base colours, so custom schemes reach the native surface at all); the HUD is twenty-three pages rather than eleven; the automation bus is 109 `browser.*` verbs; and five new chapters — 49 through 53 — document the layers that did not exist when the earlier printings were set: lifecycle hooks, output triggers, pane pipelines, the audio dashboard, and the later HUD pages. Where an earlier printing gave a smaller number for any of these, the smaller number was true when it was printed.

The edition string on the cover tracks `zwire/package.json`. `zwire` is fast-moving software, a MenkeTechnologies project built on the Chromium project's engine; this book will move with the code, and where the two ever disagree, the code is right and the book is the bug.